

// ZNATIVE //

znative

first edition

THE MANAGER

the published ABI · the content-addressed store · native code as a package



MENKE
TECHNOLOGIES

Copyright and Attribution

znative — the zshrs package manager for compiled native code

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the Zsh project, the Rust Foundation, or any other project named in these pages.

The software this book describes

`znative` is the package manager built into `zshrs`, and it lives in the `zshrs` tree under `znative/`. `zshrs` is released under the MIT License, Copyright © 2026 MenkeTechnologies, and its source is at github.com/MenkeTechnologies/zshrs.

`znative` is an original design, not a port: its unit of installation is compiled native code behind a published, versioned ABI rather than shell text. Where this book names another package manager it is drawing a comparison, not citing a source.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
Before You Begin	6
A Package-Manager Primer	9
Chapter 1 — The Gap	15
Chapter 2 — The Published ABI	16
Chapter 3 — What znative Is	18
Chapter 4 — The Command Surface	21
Chapter 5 — Sources and Resolution	23
Chapter 6 — The Content-Addressed Store	25
Chapter 7 — Kind Detection	28
Chapter 8 — The Manifest: <code>znative.toml</code>	30
Chapter 9 — The <code>.zshrc</code> Idiom	32
Chapter 10 — Loading and Unloading: <code>zmodload -R</code>	34
Chapter 11 — The Three Extension Models	36
Chapter 12 — Writing a Native Plugin	38
Chapter 13 — The Host API	40
Chapter 14 — Native Completions	42
Chapter 15 — ABI Versioning and Safety	45
Chapter 16 — Porting a zsh Plugin: the Method	47
Chapter 17 — Self-Reentrant <code>fzf</code> Tools	50
Chapter 18 — The Worked Ports	51
Chapter 19 — Designing a Plugin's Command Surface	53
Chapter 20 — Threads, Not Forks	56
Chapter 21 — Evolving the ABI	58
Chapter 22 — The Install Pipeline, Step by Step	61

Chapter 23 — The Build Step	63
Chapter 24 — Cross-Architecture Native Plugins	65
Chapter 25 — When It Fails	67
Chapter 26 — Reconciliation: When the Index and the Store Disagree	69
Chapter 27 — Living With the Store	71
Chapter 28 — The Cost of a Load	74
Chapter 29 — Trust and Integrity	76
Chapter 30 — Coming From oh-my-zsh and zinit	81
Chapter 31 — The Landscape	83
Chapter 32 — Lineage: the Stryke Dependency Manager	85
Chapter 33 — Concurrent Shells and the Index	87
Chapter 34 — What <code>remove</code> Cannot Undo	90
Chapter 35 — Testing a Native Plugin	92
Chapter 36 — Debugging Across the Boundary	94
Chapter 37 — Plugin State and Configuration	97
Chapter 38 — The <code>.zshrc</code> as the Manifest	99
Chapter 39 — Publishing a Plugin: Versions, Tags, and the Release	101
Chapter 40 — The Day the ABI Moves	104
Chapter 41 — Re-entering the Shell	107
Chapter 42 — Completion Ordering in Both Directions	109
Chapter 43 — One Kind Per Plugin	112
Chapter 44 — The Store’s Address, and the Per-Machine Assumption	115
Chapter 45 — Driving <code>znative</code> From a Script	118
Chapter 46 — What <code>znative</code> Is Not	121
Chapter 47 — The Crate, Line by Line	122
Chapter 48 — The Host Side of the Table	127
Chapter 49 — Who Owns the Bytes	131
Chapter 50 — There Is No Panic Handler	134
Chapter 51 — Four Version Numbers That Do Not Agree	136
Chapter 52 — Six Slots Nobody Calls	138

Chapter 53 — The Fleet, Measured	140
Chapter 54 — <code>zshrs-git-fuzzy</code> : The Plugin That Loads Itself	142
Chapter 55 — Two Ways to Mutate a Shell: <code>zshrs-fasd</code> and <code>zshrs-zsh-z</code>	145
Chapter 56 — Resident State and Real Threads: <code>zshrs-revolver</code> and <code>zshrs-git-repos</code>	148
Chapter 57 — The Thin Ones: <code>forget</code> , <code>reveal</code> , <code>kubectl-completion</code>	150
Chapter 58 — Eleven Writes That Bypass <code>host.print</code>	152
Chapter 59 — What the Fleet’s Documentation Says, and Where the Source Disagrees	154
Chapter 60 — The Completion Path, End to End	156
Chapter 61 — The Only Plugin That Uses ABI v4	159
Chapter 62 — What Version 5 Would Have to Add	162
Chapter 63 — Twenty-Five Names in a Shared Namespace	165
Chapter 64 — The Example Tree, and How Far Behind It Is	168
Chapter 65 — What Building a Plugin Actually Involves	172
Chapter 66 — Thirty-Eight Ways to Fail	175
Chapter 67 — The Resolver, Read Literally	181
Chapter 68 — What the Integrity Hash Covers	186
Chapter 69 — Nine Locks in the Host, None in the Manager	189
Chapter 70 — The Store and the Bytecode Archives	193
Chapter 71 — One Store, Two Suffix Rules	196
Chapter 72 — What <code>update</code> Actually Does	200
Chapter 73 — The Index Is a Path Interpreter	204
Chapter 74 — A Promise With No Expiry Date	208
Chapter 75 — What Cross-Architecture Would Actually Cost	211
Chapter 76 — The Store and the Caches That Never Meet It	214
Chapter 77 — Uninstall and <code>gc</code> , Measured	218
Chapter 78 — What a Registry Would Have to Provide	223
Chapter 79 — The Failure Taxonomy by Durable State	226
Chapter 80 — Two Managers From One Port	231
Appendix A — Command Reference	234
Appendix B — Source-Spec Grammar	235

Appendix C — Store Layout	236
Appendix D — <code>znative.toml</code> Schema	237
Appendix E — Host API Reference	238
Appendix F — <code>zmodload -R</code> Reference	239
Appendix G — Porting Cheat-Sheet	240
Appendix H — The Doctrine	241
Appendix I — Inside the <code>znative</code> Crate	242
Appendix J — The Build Decision	244
Appendix K — Cross-Architecture Rules	245
Appendix L — Trust and Integrity at a Glance	246
Appendix M — Reconciliation Matrix	247
Appendix N — <code>HostApi</code> Field Reference	248
Appendix O — The Fleet at a Glance	251
Appendix P — Reproducing the Measurements	253
Colophon	256
Reproducing the Book’s Claims	257
Also from MenkeTechnologies	258

Before You Begin

This is the book about `znative` — the one you read in a chair, front to back, to understand not only *what* the tool does but *why* it is shaped the way it is. It is not the README, which enumerates every subcommand, path, and flag and which you reach for when you need one fact fast. This book is the other thing: the continuous account of a package manager whose unit of installation is compiled native code rather than shell text, and of the one guarantee — a published, versioned ABI — that makes installing that native code safe.

`znative` is the `zshrs` package manager, and the first shell package manager whose unit of installation can be native machine code rather than shell text. It is not a fork of an existing plugin manager and it is not a wrapper around `git clone`. It is a small, global-only manager built into `zshrs` — the first compiled Unix shell — and it installs two kinds of plugin from one command surface: the script plugins every `zsh` user already knows (oh-my-zsh, zinit bundles, loose `.zsh` files) and compiled Rust `cdylib` plugins that load at native speed through a stable ABI. Every prior shell plugin manager does exactly one thing — clone a git repository and source its `.zsh` files. `znative` does that too, and it also does the thing none of them can: install a compiled plugin.

The reason no shell package manager before `znative` installed native code is not an oversight; it is a consequence. Until `zshrs` published its native-plugin interface as an independently-versioned ABI, there was nothing safe to install — a native shell module compiled against a shell's private internal headers is welded to one exact build and can crash a mismatched one. This book spends its early chapters on that problem, because the manager only makes sense once the ABI underneath it does, and the two ship together as one invention in two halves: a published ABI makes a native plugin *buildable* by a stranger; a package manager makes it *installable* by one.

The whole workflow collapses to a single self-installing line in a `.zshrc`:

```
znative load MenkeTechnologies/zshrs-forgit
```

On the first shell start that line resolves the source, installs the plugin into a content-addressed store, and loads it. On every start after, it loads from the store with zero network. There is no separate install step, no second tool, and no second mental model layered on top of the plugin workflow `zsh` users already have.

This is fast-moving software. `znative` is its own crate — the `zshrs` package manager — so the edition string on the cover tracks the `znative` crate version. Where a behaviour depends on the compiled-plugin path — the ABI, the `cdylib` build, the native registry — this book marks it, because the line between the script path and the native path *is* the design.

Where the figures come from. Every line number, count and transcript in this book was read from the vendored checkout at `src/zshrs`, commit `03e614a032` — `znative/` for the crate itself, `src/extensions/plugin_host.rs` for the host side. The wider tree carries a second `zshrs` checkout at a later commit; both `znative/` and `plugin_host.rs` are byte-identical between the two, so every `lib.rs:NNN` and `plugin_host.rs:NNN` here resolves in either, but the commit named above is the one the counts were taken at.

How to read this book

The book runs bottom-up, in seven movements. **Chapters 1–3** are the pitch and the justification — the gap in every existing shell, what `znative` is, and why it required a stable ABI to exist at all. **Chapters 4–9** are the manager itself: the command surface, source resolution, the content-addressed store, kind detection, the manifest, and the `.zshrc` idiom the whole design converges on. **Chapters 10–15** descend into the ABI

and plugin authoring — the `zmodload -R` primitive and dispatch, the three extension models compared, how to write a native plugin, the host API, native completions, and the versioning and safety rules that keep the boundary sound. **Chapters 16–18** are the porting method and its evidence: the method worked on `forgit`, the self-reentrant `git-fuzzy`, and the full set of shipped worked ports. **Chapters 19–21** are the author's half of the book: designing a plugin's command surface, concurrency inside a plugin that cannot fork, and the rules that govern changing a published ABI once strangers are compiling against it. **Chapters 22–29** are the operational realities the native path brings — the install pipeline traced stage by stage, the build step, cross-architecture plugins, the failure modes and what state each one leaves behind, how the index and the store reconcile when they disagree, living with the store over months, where the cost of a load actually goes, and the trust and integrity model stated honestly. **Chapters 30–32** are the position: migrating from an existing plugin manager, the landscape of prior plugin managers, and the stryke dependency manager `znative` was ported from. **Chapters 33–38** are the practices the earlier chapters imply but do not spell out — many shells starting at once against one index, the parts of a load that removal cannot undo, testing a native plugin at four layers, debugging across the ABI boundary, where a plugin's configuration and state belong, and the startup file read as the manifest the tool deliberately does not have. **Chapters 39–43** are the lifecycle the tool implies over years rather than over one install — cutting a release and the four version numbers that have to agree, what happens to an installed plugin set the day the ABI moves under it, what re-sourcing a startup file does to a block of `load` lines, the completion ordering rule that points the opposite way from the one everybody knows, and the consequences of a plugin having exactly one kind. **Chapters 44–45** are the two callers the rest of the book does not assume — the store's address as a knob rather than a path, with the one deployment where the per-machine guarantee stops holding, and `znative` driven by a provisioning script rather than by a human. **Chapter 46** closes with the deliberate omissions.

Chapters 47–50 are the audit: the SDK crate read line by line, the host side of the same table read against it, the four rules that govern who owns a byte that crosses the boundary, and the fact that nothing on either side catches a panic. **Chapters 51–53** measure what is actually shipped — the four version numbers that do not agree and when that stops being harmless, the six of twelve callbacks no shipped plugin has ever called, and the eight-repository fleet counted rather than described. **Chapters 54–59** read the fleet itself: the plugin that finds its own dylib and re-loads it, the two that mutate shell state through `eval` and why both install their hooks lazily, the two that use residency and threads the way only a non-forking shell allows, the three thin ones that mark the boundaries of what a plugin needs to be, the eleven writes that bypass `host.print`, and everywhere the fleet's own documentation disagrees with its source. **Chapters 60–63** follow the parts that cross the most components: a completion traced from the macro that declares it to the `compadd` that emits it, the one plugin anywhere that uses ABI version 4 and the two of its three callbacks it declines, what a version 5 would have to add and which of the four candidates has real evidence behind it, and the twenty-five command names the fleet puts into one shared namespace. **Chapters 64–65** close the audit with the eleven-directory example tree — eight of whose entries are mirrors, every one of them behind the repository it mirrors — and the build as a plugin author performs it, including the four places a first attempt gets stuck.

Chapters 66–72 are the manager read the same way the SDK was: the error census, the resolver taken literally, what the integrity hash covers and how often it is checked, the locking that exists in the host and not in the manager, the store against the bytecode archives, the two suffix rules one store applies, and what `update` actually does. **Chapters 73–80** are the second pass over the same ground, run against a live shell rather than read: the three index fields that are consumed as paths, the ABI as a compatibility promise priced over decades, what a genuinely shared cross-architecture store would cost, the caches that share the store's directory and never meet it, uninstall and `gc` measured rather than described, what a registry would have to provide if the omission were reversed, the failure taxonomy organised by what each failure leaves behind, and the sibling manager ported out of this one into `zmax` — which inherited the design's correctness and its defects with equal fidelity.

The reference appendices at the back cover the command table, the source-spec grammar, the store layout, the manifest schema, the host API, the `zmodload -R` primitive, a porting cheat-sheet, the shell doctrine the tool obeys, the complete `HostApi` field reference with ownership rules, the fleet at a glance, and the exact commands that reproduce every count in the book.

A reader who only wants to use `znative` can stop after Chapter 9 and pick up again at Chapters 22, 25, 30,

38 and 41; a reader who wants to *write or port* a plugin should read Chapters 10–18, then 19–21, then 35–37, then 39 and 43 before publishing, and 47–50 before shipping anything that could panic; a reader running *znative* on real machines wants Chapters 22–29, 33–34, 40–45, and 77–79; a reader who wants the ABI's actual state rather than its design wants 51–53 and 74; a reader auditing the tool rather than using it wants 66–73 and 80; the appendices are for looking one fact up fast.

Every claim in the chapters is grounded in the zshrs implementation and in the invention ledger that first recorded *znative* as a “world’s first.” Where the tool deliberately *omits* something other package managers have — a lockfile, a per-project manifest, a global registry — the book says so and says why, because the omissions are as considered as the features.

A short unnumbered primer sits before Chapter 1 for readers who have used a package manager without ever having had one explained; if you already know what resolution, an ABI and a cellar are, skip it.

A Package-Manager Primer

Everything after this section assumes you already know what a package manager is and roughly how one works. If you do, skip to Chapter 1 — nothing here is new to you. If your experience of them is that you type `brew install` and a program appears, this is the twenty minutes that makes the rest of the book legible: the seven things every one of these tools does, the one of the seven that is genuinely hard, and the handful of words — *source*, *prefix*, *cellar*, *ABI*, *lockfile* — that later chapters use without stopping to define. It is deliberately short and deliberately not a reference; Appendix A is the command table, Appendix B the source-spec grammar, Appendix C the store layout.

What was run, and what was not. `znative` is not a standalone executable; it is a builtin inside `zshrs`, so every transcript below was produced by driving the shell binary directly — `/opt/homebrew/bin/zshrs`, a symlink to `../Cellar/zshrs/0.12.33/bin/zshrs`, reporting `zshrs 0.12.33`. That is a **release** build: it is what the package-manager install put in the Cellar, it is 17.6 MB, and the debug binary the source tree produces is 97 MB. Every invocation used `-f` (`--no-rcs`), so no startup file was read and nothing in a `.zshrc` could install anything as a side effect. **Only read-only capabilities were executed:** `znative --help`, `list`, `info`, `gc --dry-run`, `whence -w`, a `zmodload -R` of a path that does not exist, and — for contrast — `brew --version`, `--prefix`, `--cellar`, `deps --tree`. **Everything that changes state is described in prose and was not executed:** `znative add`, `load` in its installing form, `remove`, `update`, `gc` without `--dry-run`, and `clean`. Nothing was installed, removed, upgraded, cloned or fetched to produce this section. The machine it was written on has no plugins installed, which is why the `list` transcript below is the empty one — real output, not a redaction.

Seven things, in order

Strip away the differences between `apt`, `brew`, `cargo`, `npm` and `pacman` and the same seven acts remain, always in this order. **Resolve:** you name a thing, and the manager works out exactly which bytes you meant — a specific version from a specific place, plus, for most managers, everything *else* that has to be there for it to run. **Fetch:** get those bytes onto disk. **Verify:** check they are the bytes expected, by checksum at minimum and by signature if the ecosystem has one. **Unpack:** turn an archive or a repository into a directory tree. **Link:** make that tree reachable from wherever the system looks — `$PATH`, `$fpath`, a library search path, a shell's function table. **Record:** write down what was done, so a later run can answer “what do I have” and “how do I undo this” without re-deriving it from the filesystem. **Remove:** undo the install from the record rather than by guessing. Every chapter of this book that describes machinery is describing one of those seven — Chapter 22 walks all of them end to end, Chapter 5 is resolve, Chapter 6 is record, Chapter 7 is the step between unpack and link, Chapter 77 is remove.

The dependency graph, and why resolve is the hard one

Fetching is a download and unpacking is `tar`. Resolution is the part that has consumed decades of engineering, and one picture shows why. Ask Homebrew what it takes to install `wget`:

```

$ brew deps --tree wget
wget
├── libidn2
│   ├── libunistring
│   └── gettext
│       ├── json-c
│       └── libunistring
├── libpsl
├── openssl@3
│   └── ca-certificates
├── gettext
│   ├── json-c
│   └── libunistring
└── libunistring

```

You asked for one thing and named eight. That structure is a **dependency graph** — nodes are packages, edges are “needs” — and note that `libunistring` appears four times and `gettext` twice: the tree is a drawing of a graph, and the graph has shared nodes. Installing `libunistring` four times would be wrong; installing it once and letting four dependents share it is the whole job. Add versions and it stops being bookkeeping. If `libidn2` needs `libunistring >= 1.0` and something else needs `libunistring < 1.0`, no installation satisfies both. Deciding whether a satisfying assignment exists, and finding it, is a constraint-satisfaction problem that is NP-complete in the general case — which is why real managers ship SAT solvers, or `pubgrub`, or a hand-rolled backtracking search, and why “resolving dependencies...” is the part of an install that hangs.

Hold on to that, because Chapter 4’s line that there is no ordering solver and therefore no dependency declaration is not a gap in the tool. It is the consequence of an empty graph.

Sources: where a package comes from

A manager needs somewhere to look, and the vocabulary differs while the idea does not. A **repository** or **registry** is a server holding an index of packages — Debian’s archive, `crates.io`, `npm` — which decides what names mean. A **tap** (Homebrew’s word), **channel** or **PPA** is an additional such source you opt into, so a name the default index does not know can still resolve. A **formula**, **recipe**, **ebuild** or **PKGBUILD** is the file describing one package: where to fetch it, what it needs, how to build it. There is a second model, older and simpler: a source is a **git URL** and the package is whatever is in the repository. Every `zsh` plugin manager works this way, because there has never been a registry of shell plugins to work from — not a deficiency they share, but what a plugin *is*. Chapter 5 is `znative`’s version, Chapter 78 is what a registry would have to provide, Appendix B is the grammar.

Binary packages and building from source

The bytes you fetch are one of two kinds, and the distinction runs through the whole book. A **binary package** is already compiled: someone else ran the compiler on a build machine for your platform, and an install is a download and a copy. Homebrew calls its prebuilt binaries **bottles**, and a formula that has one is *bottled*. Building **from source** means fetching the code and compiling locally — slow, requires a toolchain, can fail for reasons unrelated to the package, and in exchange works on platforms nobody built for. This matters more here than in most package managers because a compiled artifact is only usable if it agrees with the thing that will load it, and agreement is not something you can check by looking at the file. Chapter 23 is `znative`’s build step; Chapter 24 is when the building and the loading machine differ.

ABI, and why it is the thing that breaks

An **API** is the interface you compile *against*: names, types, signatures — the source-level contract. An **ABI** is the interface you link *against* at runtime: the exact byte layout of every structure that crosses the boundary, which register holds which argument, the order of the fields, the size of the padding. Two libraries can have an identical API and incompatible ABIs, and that is the dangerous failure, because nothing detects it. An API mismatch is a compile error — your build fails, you read the message, you fix it. An ABI mismatch produces a binary that loads cleanly and then reads a struct field from the wrong offset; the symptom is not an error but a wrong answer, a corrupted heap, or a crash thousands of instructions later in unrelated code. This is why system package managers care about *sonames*, and why a distribution upgrade that bumps a shared library's major version rebuilds everything linking it.

It is also exactly why no shell before `zshrs` had a third-party native plugin ecosystem, which is Chapter 1's argument: a `zsh` C module compiles against the shell's private headers, so it has an ABI contract with one specific build of one specific shell and no way to state or check it. Chapter 2 is that contract published, versioned, and checked at load — so a bad load fails as a refusal rather than as corruption:

```
$ zshrs -f -c 'zmodload -R /nonexistent/libfoo.dylib'
zshrs:zmodload:1: cannot load `/nonexistent/libfoo.dylib':
  ↳ dlopen(/nonexistent/libfoo.dylib, 0x0005): tried: '/nonexistent/libfoo.dylib' (no such
  ↳ file), '/System/Volumes/Preboot/Cryptexes/OS/nonexistent/libfoo.dylib' (no such file),
  ↳ '/nonexistent/libfoo.dylib' (no such file)
```

That is the missing-file case rather than the version case, but the shape is the one Chapter 15 describes for a version mismatch: declined at the boundary, before any pointer from the library has been trusted.

Lockfiles and reproducibility

A **manifest** says what you want — “the 1.x series of this library”. A **lockfile** says what you got — “1.4.2, this exact hash, from this exact URL”. The manifest is written by a human and is loose on purpose; the lockfile is written by the resolver and is exact on purpose. Commit both and a colleague's install reproduces your tree, because the resolver is skipped and the recorded answers are replayed. Lockfiles exist because resolution is expensive *and* non-deterministic over time: the same manifest resolved a month later can produce different versions, because the registry moved. That is a real problem when a project's build must be reproducible on a build server, and a much smaller one when the “project” is a shell — one user, one machine, no build server. `znative` records the exact source it installed from, pin included, and re-resolving is what `update` deliberately does rather than something you opt out of. Chapter 46 is the case for the omission; Chapter 72 is what `update` actually does.

Prefixes, cellars, and the symlink farm

The last vocabulary is about where files land, and three commands on this machine draw the whole pattern:

```
$ brew --prefix
/opt/homebrew
$ brew --cellar
/opt/homebrew/Cellar
$ readlink /opt/homebrew/bin/zshrs
../Cellar/zshrs/0.12.33/bin/zshrs
```

The **prefix** is the root the whole installation lives under — `/usr/local`, `/opt/homebrew`, `/usr` for a system manager — with the conventional `bin/`, `lib/`, `share/` beneath it. Two managers sharing a prefix fight over the same filenames, which is why a user-installed manager picks one the system manager does not own. The **cellar** is the second idea and it is what makes the first safe: rather than installing a package's files directly into `prefix/bin` and `prefix/lib`, each package is unpacked into a directory of its own keyed by name and

version, then *symlinked* into the prefix — which is what the third command shows, Homebrew’s install of this book’s shell describing the mechanism itself.

The `zshrs` on `$PATH` is not a file; it is a link into a versioned directory. That indirection buys three things at once: two versions can sit on disk without colliding, the files belonging to a package are exactly the contents of one directory rather than a list you must track, and uninstalling is “remove the links, remove the directory” instead of “remember every path we wrote.” Nix and Guix take the idea to its conclusion by keying the directory on a hash of the package’s entire build inputs. `znative`’s store is the same pattern, keyed `store/<name>@<version>/` under `~/.zshrs/pkg/`, with one difference: a shell plugin need not appear on `$PATH`, so there is no symlink farm — linking means adding a directory to `$fpath` and sourcing a file, or handing a `cdylib` to `zmodload -R`. Chapter 6 is the store, Appendix C its layout.

Removal, rollback, and what neither can do

Removal uses the record — that is the whole reason step six exists. A manager that knows which files it wrote deletes exactly those and nothing else, where one that guesses either leaves debris or takes a file something else owns. **Rollback** is removal’s more ambitious relative: return to the state before an operation. It is easy when installs are directories keyed by version, because the old version is still sitting there, and hard when installs overwrite in place — which is why the cellar pattern and rollback tend to arrive together, and why `znative`’s `update` leaves the previous `store/<name>@<version>/` directory alone rather than deleting it, with a separate garbage-collection verb to reclaim the ones nothing points at. Neither operation undoes everything: a package’s *installation* is reversible, its *effects* often are not — files it wrote elsewhere, state it created, and, for a shell plugin, anything it did to the running shell that outlives unloading it. Chapter 34 is that subject in full, and it is the chapter to read before assuming `remove` is a perfect inverse of `add`.

What is different about znative specifically

Four things matter before you start, and the first is structural. **It is a builtin, not a program.** There is no `znative` on `$PATH` to find, because the package manager is compiled into the shell:

```
$ zshrs -f -c 'whence -w znative zmodload'
znative: builtin
zmodload: builtin
```

That is not packaging trivia. A manager that is a separate process cannot load a plugin into your shell — it can only put files somewhere and let a startup file source them. `znative` installs *and* loads in one act because the thing doing the installing is the thing being extended, which collapses the workflow to the single `.zshrc` line Chapter 9 is about.

The surface is eight verbs and no configuration language. Not “eight so far” — the tool’s own help is the complete list:

```

$ zshrs -f -c 'znative --help'
usage: znative <command> [args]

  load [SOURCE...]  load plugin(s); a source not yet in the store is
                    installed first, then loaded (for .zshrc startup).
                    No args loads everything installed. Zero-network once
                    stored. SOURCE: owner/repo, github:o/r, git+URL, path:DIR
  add <SOURCE>      install + load a plugin (load self-installs, so add is
                    mainly for installing without a .zshrc line)
  remove <NAME>      unload + delete an installed plugin
  list              list installed plugins
  info <NAME>        show details for one plugin
  update [NAME]      re-resolve + reinstall from the recorded source
  gc [--dry-run]     remove orphan store entries + the git clone cache
  clean             clear scratch caches (git/, cache/, bin/)
  help              this message

aliases: add=install=i  remove=rm=uninstall  list=ls  info=show
         load=source  update=up=upgrade

```

Compare that to the seven acts above: the mapping is one-to-one with nothing left over. There are no per-verb flags to discover either — that is the only help there is, and a `--help` handed to a subcommand is read as an ordinary argument, which is to say as a plugin name:

```

$ zshrs -f -c 'znative info --help'
znative: --help is not installed
$ zshrs -f -c 'znative info forgot'
znative: forgot is not installed

```

That is also the entire error contract: `znative: <reason>` on stderr, non-zero exit, no suggestion and no invitation to run anything else. Chapter 4 is the tour of the eight.

There is no dependency graph, and therefore no solver. The `brew deps --tree` output above has no counterpart here, and not because the feature is pending. A plugin declares no dependencies — the manifest schema in Appendix D has no field for one — so the graph is a set of isolated nodes and resolution is a per-package question with no interaction between packages. What is left of “resolve” is Chapter 5’s job of classifying one source string, which is why that chapter is about string shapes rather than about search.

It sits beside your other package managers, not underneath them. Homebrew, `apt` and their peers install *programs*, into a shared prefix, for everyone on the machine. `znative` installs *plugins*, into one store under one user’s home, for one shell. They neither overlap nor compete: on this machine Homebrew installed `zshrs` itself —

```

$ brew --version
Homebrew 6.0.17-97-gfba9b30-dirty
Homebrew/homebrew-core (git revision 4cd95f3653; last commit 2026-08-16)

```

— and `znative` is what extends the shell Homebrew put in the Cellar. The correct mental model is `cargo` or `npm` rather than `apt`: a language-level manager for one runtime’s extensions, global rather than per-project because a shell’s plugins belong to the user, not to a directory. Chapter 31 places it against the other plugin managers; Chapter 46 lists what it declines to be. Being global-only and per-user is what makes the read-only verbs cheap — the store is one directory, so “what do I have” is one index read rather than a filesystem scan, and `gc` has a preview mode that says what a real collection would reclaim without reclaiming it:

```

$ zshrs -f -c 'znative list'
znative: no plugins installed
$ zshrs -f -c 'znative gc --dry-run'
znative gc: nothing to collect

```

Both are the empty case on the machine this was written on, which is the honest state to show them in: nothing was installed to produce a fuller transcript.

Where to go from here

You now know enough vocabulary to read the rest of the book without stopping. Chapter 1 is why the gap this tool fills existed at all — the argument the ABI section above was preparing you for; Chapter 2 is the published ABI itself; Chapter 3 is what `znative` is in one page; Chapter 4 is the eight verbs properly; and Chapter 5 is resolution, which, having seen what resolution costs everywhere else, will read as surprisingly small. To *use* the tool rather than understand it, go to Chapter 9 — the one `.zshrc` line the design converges on. To know what it does to your disk, Chapter 6 and Appendix C are the store.

Chapter 1 – The Gap

Every shell has an extension story, and until now they came in exactly two shapes. Neither of them installs native code, and the reason why is the whole starting point for this book.

The first shape is the **script plugin**. This is the oh-my-zsh / zinit / antigen / sheldon / zplug world, and it is where essentially all real shell customization lives. A script plugin is a git repository of `.zsh` files. A plugin manager clones the repository and `sources` those files into the running shell, adding functions, aliases, completions, and key bindings. It is text the shell reads and re-parses, and every one of these managers, underneath its own configuration DSL, does the same single mechanical act: clone a repo, `source` its files. They differ in ergonomics — turbo/deferred loading, lockfiles, declarative blocks — but not in kind. The unit of installation is shell text, interpreted on load and, for anything that runs per prompt or per keystroke, re-interpreted forever after.

The second shape is the **native module**. zsh has `zmodload`, which loads a compiled C module (`zsh/zle`, `zsh/complint`, `zsh/mathfunc`, and the rest); bash has `enable -f`, which loads a compiled “loadable builtin.” These run as machine code — no interpreter cost — and that is their entire appeal. But they compile against the shell’s *internal, private* headers, with no stable ABI and no version gate. A zsh C module is built against `Src/` — the shell’s own build tree — so it is welded to one exact shell build. Load it into a shell compiled from even a slightly different source and it can crash the process, because the struct layouts, function signatures, and global symbols it was compiled against no longer match the host it was loaded into.

That single fact explains something you can verify by looking: **neither ecosystem has a meaningful third-party native-module scene**. Almost every native module that exists for zsh is one the shell itself ships in its own source tree, because tracking a moving internal API from *outside* the source tree is not viable. A stranger cannot reliably build a zsh C module against your zsh and hand it to you; the moment either of you rebuilds the shell, the module may stop loading or start corrupting memory. The native path was fast but unshippable by third parties. The script path was shippable by anyone but interpreted forever.

So a shell package manager, historically, had only one thing it could safely install: shell text. That is not a design choice any of those managers made — it is the only option the substrate gave them. There was no third shape to install, because there was no safe way to *distribute* native shell code at all. The gap this book is about is the gap where that third shape should be: a compiled plugin a stranger can build and you can install, running at native speed, that cannot crash your shell because it was built somewhere else.

Closing that gap takes two pieces that do not exist independently. You need a way to *build* a native plugin against a shell without cloning the shell’s source and without welding to one build — a stable, published interface. And you need a way to *install* the result — a package manager that understands a compiled artifact as a unit, not just a bag of `.zsh` files. `znative` is the second piece. The next chapter is the first.

Chapter 2 – The Published ABI

`znative` installs native plugins, and it can only do that because there is something safe to install. That something is zshrs's native-plugin ABI, published as an independently-versioned package, and it is the invention the package manager sits on top of. This chapter is the foundation; the manager is the rest of the book.

Recall the trade zshrs makes with zsh's own module system. zsh loads C modules at runtime with `zmodload`; zshrs instead statically links its fourteen-plus built-in modules and decides the whole module surface at link time, so a `zmodload` of a name the binary knows can never fail for reasons outside the binary itself. That trade gives up the ability to load a foreign `.so` — and with it every failure mode that came bundled with that ability: version skew, missing symbols discovered only at load time, dependence on the host's dynamic linker behaving identically across macOS and both Linux architectures, and the security surface of loading arbitrary shared objects out of `$module_path`. For the built-in modules, giving up dynamic loading is the conservative, correct default.

But there is a version of dynamic loading that carries none of those failure modes, and zshrs adds it as a **separate, additive path**. Native plugins are written in a compiled language — Rust — and loaded at runtime with `zmodload -R <path>`, with no recompile of the shell. The `-R` flavour sits beside the ordinary `zmodload`, not in place of it: the built-in modules stay statically linked and compile-time-proven; the plugin path is the one place a foreign compiled artifact enters the process, and it enters through a gate the built-in modules never had.

The gate is the invention: **a stable, versioned, independently-published ABI**. zshrs exposes its plugin interface not as its own build-tree internals but as the `znative` crate — the same crate this book is about, published from the zshrs repo to crates.io. `znative` is one crate wearing two hats: the package manager the user drives, and the plugin ABI a plugin author builds against. A plugin author runs `cargo add znative`, writes a handler against that crate's public types, and ships a `cdylib`. The author never clones zshrs. Everything that crosses the boundary between host and plugin is `#[repr(C)]` — a stable, compiler-independent memory layout, not Rust's unspecified default layout — and the host checks the plugin's `ABI_VERSION` against its own before it trusts a single pointer the plugin hands back. A version mismatch is *refused at load*, not left to corrupt the process later. That one check is the whole difference between “load a foreign `.so`” as the liability the built-in module system rejects and “load a foreign `.so`” as a safe, first-class extension path.

Mechanically the load is small. `zmodload -R libfoo.dylib dlopens` the library (through the `libloading` crate), resolves a single exported symbol — `znative_init`, the one the crate's `declare_plugin!` macro emits as `#[no_mangle] extern "C" fn znative_init` — and calls it with a host handle. The init registers its builtins into an in-process registry and, from then on, reaches the shell only through a curated host API — `print`, `eval`, `getvar`, `setvar` — rather than reaching into shell internals the way a C module does. The plugin never touches a shell global directly; it asks the host to do things on its behalf through four functions, all of them `#[repr(C)]` across the boundary. This is the second half of why the ABI is stable: not only is the layout fixed and version-checked, the *surface* the plugin can touch is a deliberately small, curated set of calls, not the shell's entire internal state.

Command resolution then slots the plugin registry into the shell's dispatch at a specific, principled point. When zshrs resolves a command, the plugin registry is consulted in `execute_external_bg ahead of PATH lookup` — the same place zsh resolves `zmodload -ab` autoloaded builtins. The effect is exact: a plugin command shadows an external program of the same name on `$PATH`, but it never shadows a real builtin or a shell function. A plugin can give you a faster `git` front-end; it cannot silently replace `cd` or a function you defined. Unloading is the mirror image: `zmodload -uR` purges the plugin's entries from the registry *before* the library's `dlclose`, so no dangling function pointer into a just-unmapped library can survive to be called.

The distinction that matters here is not “loads native code” — bash and zsh have loaded native code for decades. It is that **zshrs is the first shell to publish that interface as a stable, versioned package a stranger can build against without ever cloning the shell.** That is what makes native shell plugins *distributable*. And the moment a thing is distributable, it wants a distributor. That distributor is **znative**.

Chapter 3 — What znative Is

znative is the zshrs package manager. In one sentence: it resolves a plugin source, installs it into a content-addressed store, records it in a global index, and loads it — for script plugins and compiled native plugins alike, from one small command surface. It is the first shell package manager whose unit of installation can be native code rather than shell text, and that “can be” is the whole point: it installs the entire existing zsh script-plugin ecosystem unchanged *and* installs the new thing the published ABI made possible.

Hold the two halves of the invention side by side, because **znative** only makes sense as the second one. A published ABI (Chapter 2) makes a native plugin **buildable** by a stranger. A package manager makes it **installable** by one. Without the ABI, **znative**’s native path would have nothing safe to install and would collapse back into “clone a repo and source it” — which is all any prior manager ever did. Without **znative**, the ABI would let a stranger build a compiled plugin that every user then had to place, build, and **zmodload -R** by hand. The two ship as one invention because each is inert without the other.

What **znative** deliberately is *not* is as defining as what it is:

- **It is not a fork of oh-my-zsh, zinit, antigen, sheldon, or zplug.** It shares their script-plugin behaviour — clone a repo, add its files to `$fpath`, source them — because that behaviour is correct and universal, but it is a fresh implementation with a different core: a content-addressed store and a native install path those managers have no concept of.
- **It is not global-and-project like a language package manager.** There is no per-project manifest and no lockfile. **znative** is global only, because a shell’s plugins are part of the *user’s environment*, not a *project’s* dependency set. Chapter 31 makes the case for that omission in full.
- **It is not a new configuration DSL.** The primary interface is one line in a `.zshrc` — **znative load owner/repo** — not a declarative block with its own syntax to learn. The tool’s surface is small on purpose.

The creed for the whole thing is the self-installing line from the front matter: **znative load MenkeTechnologies/zshrs-forgit**. That single line, in a `.zshrc`, is a complete plugin lifecycle. First start: resolve, install, load. Every start after: load from the store, no network. It is the script-plugin workflow every zsh user already knows — extended, without a second tool or a second mental model, to compiled native plugins. The rest of this book is what happens underneath that line.

One crate, two audiences

Chapter 2 makes a claim in passing that is worth unpacking before the rest of the book leans on it: **znative** is one crate wearing two hats — the package manager the user drives, and the ABI a plugin author compiles against. That is unusual enough to be worth defending, because the obvious alternative is two crates: an **znative-abi** a plugin depends on, and an **znative** the shell builds in.

The reason to keep them together is that they are the same contract seen from two sides. The manager knows how to locate a `cdylib`, hand it to **zmodload -R**, and record what it found; the ABI defines what that `cdylib` must look like for the handoff to work. Split into two crates, those two definitions become two things that have to be kept in agreement by a human, and the failure mode of disagreement is a manager that installs artifacts the host will refuse. Kept together, the agreement is structural — the same crate that tells a plugin author what to build is the crate the manager was compiled from.

The visible consequence is the one the front matter mentions and does not explain: the edition string on this book’s cover tracks the **znative** crate version, because the crate is the thing whose behaviour the book

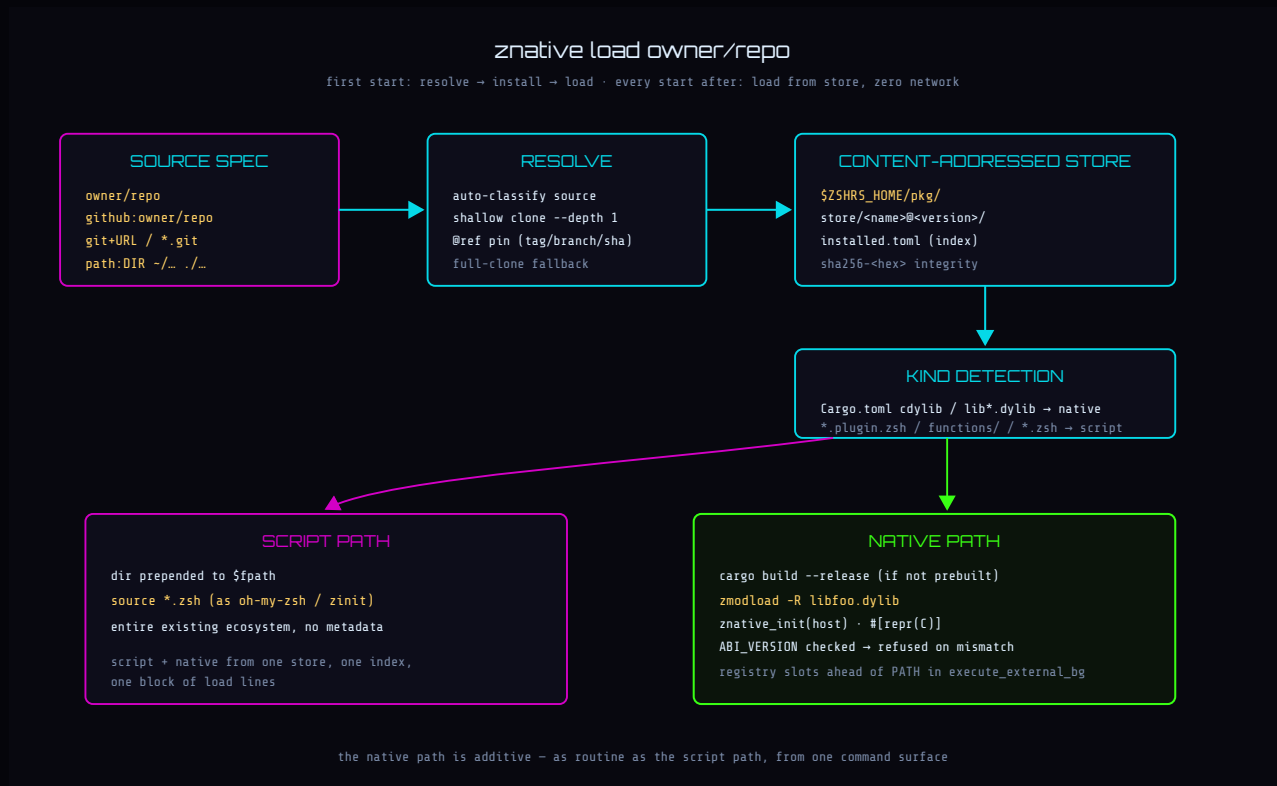


Figure 1: **znative** installs from a source spec into a content-addressed store, then loads a script plugin onto **\$fpath** or a native **cdylib** through the **zmodload -R** ABI host. The first start installs; every start after loads from the store with zero network.

describes on both sides. A reader who is a user cares about the manager's verbs; a reader who is an author cares about the ABI's types; both are reading about the same versioned artifact.

Two version numbers that are not the same number

This is the place to separate two things that are easy to conflate for exactly the reason above. The **znative crate version** is what a plugin author writes in `Cargo.toml` and what the cover tracks. `ABI_VERSION` is a `u32` constant *inside* that crate, and Chapter 15 gives its value at the time of writing. They move independently and they answer different questions.

A crate release that adds a manager verb, fixes a resolution bug, or changes how the store is walked does not touch a single type that crosses the FFI boundary, so `ABI_VERSION` does not move and every existing plugin still loads. A crate release that appends a field to `HostApi` does move it, and Chapter 21 works through why even a purely additive change has to. So the crate version advances more often than the ABI version does, and the practical rule of Chapter 15 — keep the crate's major/minor aligned with the target zshrs release — is a rule about the *crate* version precisely because the ABI version is the coarse gate underneath it.

The reason this distinction earns a section rather than a footnote is that the two numbers fail differently and a plugin author debugging a failure needs to know which one they are looking at. A crate-version mismatch surfaces at build time, as a compile error against types that changed shape in safe Rust. An ABI-version mismatch surfaces at load time, as a refusal. Chapter 25 lists both and Chapter 36 walks the diagnosis; the thing to carry from here is that they are two gates, not one, and they sit on opposite sides of `cargo build`.

Chapter 4 – The Command Surface

The command surface is deliberately small — six core verbs plus two store- maintenance verbs (`gc` and `clean`), each with a single job, all obeying the same terse error contract as the rest of the shell. Most carry short aliases so the tool answers to the muscle memory a `zsh` user already has. This chapter is the tour; the appendix at the back is the table.

`znative load` (alias `source`) is the verb a `.zshrc` calls, and it is the one that makes the whole thing collapse to a single line. Given a name, or a source already present in the store, it loads that plugin with zero network. Given a source *not* yet in the store, it installs it first and then loads it. That conditional is the entire reason there is “no separate install step” for the common case: the first shell start after you add a `load` line does the install as a side effect of the load; every start after finds the plugin already in the store and just loads it. A bare `znative load` with no argument is a special, useful case — it loads everything already present in the store, for the user who prefers to install interactively with `znative add` and then keep a single unqualified `load` line in the startup file.

Underneath, `load` resolves its argument in three steps, and the order is what lets one verb be both “load fast” and “install if needed.” First, if the argument matches an installed plugin *name* in the index, it loads from the store and stops — for a native plugin that is a `dlopen` of an already-mmap’d `cdylib`, no reinstall, no network. Second, if the argument is a *source* spec rather than a name, `znative` computes that source’s provenance label and looks for an installed plugin whose recorded source matches — this indirection matters because a repo’s basename is usually not the plugin’s name (`zshrs-forgit` the repo installs `forgit` the plugin), so `load owner/repo` has to match on the source, not the name. Third, only if no installed plugin matches does it fall through to a full install-then-load. That third branch is exactly what makes `znative load owner/repo` in a `.zshrc` self-install on the first startup and load fast on every one after.

`znative add` (aliases `install`, `i`) is the interactive install. It resolves the source, installs it into the store, records it in the global index, and loads it — all without needing a line in `.zshrc`. This is what you run at a prompt when you want to try a plugin now; if you then want it every session, you add a `load` line for it, and because it is already in the store that line will never hit the network again. On success it prints one line — `znative: added name@version (kind)`, with the description if the plugin declared one — which is the only chatter the install path emits.

`znative remove` (aliases `rm`, `uninstall`) is the clean uninstall. For a native plugin it unloads the library first (the `zmodload -uR` purge-then- `dlclose` from Chapter 2), then it deletes the plugin’s copy from the store and drops its row from the index. The order matters: unload before delete, so nothing is executing out of a directory that is about to disappear.

`znative list` (alias `ls`) prints one line per installed plugin — name, version, kind, and source. It is the fast “what do I have” read, and it comes straight off the global index rather than by scanning the filesystem, so it reflects exactly what `znative` believes is installed.

`znative info` (alias `show`) is the deep read for one plugin. It shows the full record: the store path, the integrity hash, and the specific lib (for a native plugin) or the source files and `fpath` dirs (for a script plugin) that the plugin resolves to. When you want to know precisely what a `load` line will pull in, this is the verb.

`znative update` (aliases `upgrade`, `up`) re-resolves and reinstalls from the *recorded source* — the source spec `znative` saved when it first installed the plugin — for one named plugin or, with no argument, for all of them. Because the recorded source can carry a pin (Chapter 5), `update` respects it: updating a plugin pinned to `@v1.2.0` re-fetches that tag, not a moving branch. An update to a newer version leaves the previous `store/<name>@<old>/` directory in place — it is not deleted out from under anything that might still reference it — so old versions accumulate until `gc` reclaims them.

`znative gc` (`--dry-run / -n` to preview) is the garbage collector. It removes `store/<name>@<version>/` directories that the index no longer pins — the orphans an `update` leaves behind — and clears the `git/` clone cache. The index is the authority on what is live; `gc` deletes only what the index does not name, so it can never remove a plugin you still have installed.

`znative clean` clears the scratch directories — `git/`, `cache/`, `bin/` — and leaves the store and the index untouched. Where `gc` reclaims orphaned *installed* versions, `clean` only empties `znative`'s working scratch; nothing a plugin needs to load is in those directories.

The error contract is uniform and terse. Anything that fails prints `znative: <reason>` on `stderr` and returns non-zero. There is no friendly explanation, no “did you mean,” no “run `-help` for more.” It is the same contract the rest of `zshrs` uses — state the error, exit — because the tool is built for a user who has typed these commands a million times and wants the failure, not a lecture about it.

Why the surface stops at eight verbs

The incumbent managers grew configuration languages. `zinit` has ice modifiers — a chain of adjectives attached to a plugin declaration, each one altering when, whether, and how the plugin loads. `sheldon` has a TOML file with templates. `antigen` and `zplug` have their own vocabularies of bundles and tags. Each of those grammars exists for a reason, and the reason is almost always the same one: the manager is trying to *schedule* work that is expensive, and scheduling needs knobs. Turbo loading needs a wait condition. Deferred loading needs a trigger. Conditional loading needs a predicate. Once you accept that loading a plugin is costly enough to want deferred, you have signed up for a language to describe the deferral.

`znative` does not have that language because it does not have that problem in the same shape, and the absence propagates down through the whole surface. There is no `when` to express, so there is no modifier to express it in; there is no ordering solver, so there is no dependency declaration; there is no per-project scope, so there is no scope selector. What is left after you subtract all of that is a small set of things a user actually wants to *do* to a plugin — put it here, take it away, tell me what I have, tell me about this one, bring it up to date — plus two housekeeping verbs for the directory those actions write into. That is the eight.

The verbs are also deliberately *orthogonal* rather than layered. `add` is not `load` with a flag; `update` is not `add` with a flag; `gc` is not `clean` with a flag. Each verb has exactly one job and no mode switches, which is what makes the aliases safe: `i`, `rm`, `ls`, `up` can be one or two characters precisely because there is no second meaning hiding behind an option. A manager whose primary verb takes fifteen flags cannot afford a one-letter alias for it, because the alias would be the least informative part of the line. Here the verb *is* the whole intent, and the source spec carries everything else.

There is one place the design consciously accepts overlap, and it is worth naming because it looks like a violation. `load` will install if it must, which makes it partially redundant with `add`. That overlap is the product: it is what turns the `.zshrc` line into a complete lifecycle. The two verbs differ in which case they optimise for — `load` is written once in a file and run thousands of times, `add` is typed once at a prompt — and they differ in what they assume about where you are. Collapsing them into one verb with a flag would have saved a row in the table and cost the single-line idiom the whole tool is built around.

Chapter 5 – Sources and Resolution

A `znative` source is a single string, and its form is **auto-classified** rather than flagged. You do not tell `znative` “this is a GitHub repo” or “this is a local path” with an option; it reads the shape of the string and decides. That keeps the common case — a GitHub `owner/repo` — down to the shortest thing that could possibly identify a plugin.

The classifications, in the order a reader would recognise them:

- `owner/repo` — a bare two-part slug with no scheme is a GitHub repository. `znative load ohmyzsh/ohmyzsh` clones from GitHub. This is the form nearly every `.zshrc` line uses.
- `github:owner/repo` — the same thing said explicitly, for when you want the host named in the source rather than inferred.
- a `git+URL`, or any URL ending in `.git` — clone that URL directly. This is the escape hatch to any git host: GitLab, a self-hosted forge, an internal mirror. `git+https://git.example.com/team/plugin.git` clones from there.
- `path:DIR`, or an absolute `/` or `~/` rooted path — install a local directory with **no network at all**. This is how you develop a plugin: point `znative` at the working tree, and it installs from disk. It is also how a fully offline environment installs plugins it already has on the filesystem.

Any remote form may carry an `@ref` suffix to pin what gets fetched: `owner/repo@v1.2.0`, `github:owner/repo@some-branch`, `git+https://host/x.git@main`, or a suffix naming a specific commit. The pin is a tag, a branch, or a commit — whatever git can check out — and it is recorded with the plugin so that a later `znative update` re-fetches exactly that ref rather than drifting to the head of a default branch.

The clone itself is optimised for the common case and correct in the uncommon one. `znative` clones **shallow** — `--depth 1 --branch REF` — because a plugin install needs the tree, not the history. It only falls back to a full clone when a shallow branch clone cannot reach a requested commit: pinning to an arbitrary older commit sometimes requires history a `--depth 1` fetch will not have, and in that case `znative` does the full clone rather than fail. The default path is fast (one shallow fetch); the fallback exists so that pinning to any commit always works.

Resolution produces a small, three-field result the rest of the install builds on: a *staged directory* holding the plugin tree, an *inferred name* (the repo or directory basename, used only if no manifest names the plugin), and a *provenance label* — the canonical form of the source that gets recorded in the index. The label normalises the spec you typed into one of three stable shapes: `github:owner/repo` for GitHub sources, `git+URL` for arbitrary git URLs, and `path+file://DIR` for local directories. Recording the label rather than your raw input is what lets `znative update` reconstruct a valid `add` spec later — a `path+file://` label converts back to `path:DIR`, and the `github:git+` labels are already valid specs — and it is what `znative load owner/repo` matches against when it checks whether a source is already installed.

Remote clones land in the store’s `git/` scratch directory; local `path:` sources are used in place. Either way, once the tree is staged the git clone is disposable: after the loadable subset is copied into the store, `znative` deletes the `git/` clone, because the store copy is the authoritative one from then on.

Why classification is inferred and not flagged

Auto-classification is the kind of decision that looks like laziness and is actually a constraint on the whole command surface. Consider the alternative: a `--github` / `--git` / `--path` triple, or a `--from` option taking a scheme name. Every one of those pushes information into the *verb* that already exists in the *argument*.

`owner/repo` is unambiguously not a filesystem path, because a bare two-segment relative path with no leading `./` is not how anyone writes a directory they mean; `/home/user/plugin` is unambiguously not a GitHub slug; a string containing `://` is unambiguously a URL. The shapes do not collide, so a flag telling `znative` which shape you meant would only ever restate what the string already said, and it would be one more thing to get wrong in a `.zshrc` line that is supposed to be the shortest true statement of intent.

The deeper reason is that the source spec has to survive a round trip. It is not just an argument to one invocation; it is *recorded*, and `znative update` later reads it back and re-resolves from it. A scheme carried in an out-of-band flag would have to be persisted separately and re-applied on every update, which means the index would store a spec that is not, by itself, a valid spec. Keeping the classification inside the string keeps the recorded source self-describing: one field in `installed.toml` is a complete instruction for how to fetch this plugin again.

The provenance label, round-tripped

The label is where that round trip is made concrete, and it is worth walking one all the way through. Suppose a `.zshrc` line reads:

```
znative load MenkeTechnologies/zshrs-revolver@v0.2.0
```

Classification splits that into a GitHub source and a pin. Resolution clones shallow at the tag, stages the tree, infers `zshrs-revolver` as a fallback name — which the plugin's own manifest then overrides with `revolver`, so the store key becomes `store/revolver@0.2.0/` and not `store/zshrs-revolver@.../`. That divergence between repository basename and plugin name is not an edge case; it is the norm for ported plugins, where the repo is named after the port and the plugin is named after the command a user types. It is exactly why `load` matches on the *source* rather than the name in its second resolution step (Chapter 4): the string in your `.zshrc` and the string in your `installed.toml name` field are frequently different strings referring to the same thing, and only the label bridges them.

The label recorded for that install is `github:MenkeTechnologies/zshrs-revolver` with its pin. Note what happened to the input: the bare-slug form was normalised into the explicit `github:` form. Both spellings mean the same thing, and by canonicalising to one of them the index gains a property that matters more than it looks — two users who typed the source differently produce the same recorded label for the same plugin, so “is this source already installed?” is a string comparison rather than a parse-and-compare. A `path:` source normalises the same way, to `path+file://DIR`, and converts back to `path:DIR` when `update` needs a spec again. The `github:` and `git+` labels need no conversion because they are already valid specs; that is the point of choosing those two as the canonical shapes.

The pin survives the round trip with the label, which is what makes `update` honest. Updating a plugin pinned to `@v0.2.0` re-fetches `v0.2.0`, which in the normal case is a no-op — the same tree, the same hash, the same store directory — and that is the correct outcome. A pinned plugin that silently drifted forward on `update` would make the pin decorative. A user who wants the drift removes the pin by re-adding the source without one, which rewrites the recorded label.

Whatever the source, once the tree is on disk the next question is *what kind of plugin is this* — and that is Chapter 7. But first, where the tree goes.

Chapter 6 – The Content-Addressed Store

Everything `znative` installs lives under **one store**, at `$ZSHRS_HOME/pkg/` (default `~/.zshrs/pkg/`). There is exactly one store per user; there is no per-project `.znative/` directory anywhere, because `znative` is global only. This chapter is the map of that directory and the index that governs it.

The store's layout:

- `store/<name>@<version>/` — the installed plugin itself, keyed by name and version. The tree here is the plugin *stripped for use*: `.git/` and `target/` are removed, so the store holds the source and any built library but not the version-control history or the Rust build's intermediate artifacts. That keeps the store small and makes each `store/<name>@<version>/` a clean, self-contained copy of exactly what gets loaded.
- `installed.toml` — the **global index and the source of truth**. Every installed plugin has a record here: its name, version, kind, recorded source spec, store path, and integrity hash. `znative list` and `znative info` read this file; `znative add` writes to it; `znative remove` drops a row from it. When `znative` and the filesystem disagree, this file is what `znative` believes.
- `git/`, `cache/`, `bin/` — internal scratch. Clone working directories, fetch caches, and helper binaries live here. These are `znative`'s workspace, not part of any plugin's installed footprint.

Each install is **content-addressed**: the record carries a SHA-256 hash written as `sha256-<hex>`, computed over the store copy at install time, so a plugin's index row is its own integrity check. The store is not a bag of directories you have to trust by their names — every entry knows its own hash, and that hash is what `znative info` surfaces when you ask what a plugin actually is. Content addressing is why the store can be shared, inspected, and trusted: the identity of an installed plugin is the hash of its content, not just the label on its folder.

There is a deliberate ergonomic choice in *how* the store is keyed. A pure content-addressed store (Nix is the reference point) names each entry by its hash — reproducible, but opaque: you cannot tell `/nix/store/abc123...-foo` apart from its neighbours by eye. `znative` keeps the reproducibility — the `sha256-` hash is in the index — but keys the directories by the human-readable `name@version` instead, so `store/forgit@0.1.0/` says what it is at a glance. You get the integrity guarantee of content addressing with the browsability of version paths, rather than trading one for the other.

The install itself is a copy, not a move: `znative` copies the *loadable subset* of the staged tree into `store/<name>@<version>/` — dropping `.git/` and `target/` — then computes the integrity hash over that copy, then writes the index record. For a native plugin the record names the `cdylib` it found in the store copy (located by the platform's library prefix/suffix — `lib*.dylib`, `lib*.so`); for a script plugin it records the files to `source` and the dirs to prepend to `$fpath`. Nothing is recorded until the copy and the hash succeed, so a half-finished install does not leave a live index row pointing at a partial tree.

And then the deliberate omissions, which are the design as much as the structure: **there is no per-project manifest and no lockfile**. A language package manager (cargo, npm, bundler) has both, because a project's dependency set must be reproducible across machines and pinned in version control. A shell's plugins are not that. They are part of *your environment* — the set of tools you, the human, have chosen to have at your prompt — not part of a project you hand to a collaborator. So `znative` keeps one global store and one global index, and it does not pretend a shell's plugin set is a project's lockfile. Chapter 31 returns to why “global only” is the right model rather than a missing feature.

Why the index, and not the filesystem, is the authority

A package manager has to pick one thing to believe when its two records of reality disagree, and the choice determines how every operation behaves at the edges. `znative` believes `installed.toml`. A `store/foo@1.0.0/` directory with no corresponding record is not an installed plugin; it is garbage that `gc` will reclaim. This is not an arbitrary tiebreak — it falls out of the ordering the install uses, and it is the reason that ordering is what it is.

Consider the alternative. If the filesystem were authoritative — if `list` worked by scanning `store/` and reporting what it found — then every directory that appeared for any reason would become an installed plugin. A `cargo build` that died partway through, a copy interrupted by a full disk, a directory a user created by hand to poke at something, all of them would show up in `list` and all of them would be candidates for `load`. Scanning is also strictly more expensive than reading one file: it costs a directory walk proportional to how much you have installed, on every `list`, forever.

More importantly, a directory cannot answer the questions the index answers. The store copy does not know what source it came from, because the `.git/` directory was stripped out of it deliberately. It does not know its integrity hash, because a hash stored inside the thing it hashes is circular. It does not know whether it is the version currently in use or an orphan left behind by an `update`. Every one of those facts is *about* the tree rather than *in* it, and a manager that threw them away in favour of filesystem inspection would have to reconstruct them by guessing. `znative info` can tell you a plugin's recorded source and its `sha256-` hash precisely because it is reading a record, not a directory.

The asymmetry this creates is deliberate and it only runs one way. A store directory with no index row is inert: nothing loads it, nothing lists it, and `gc` removes it. An index row with no store directory would be the dangerous case — a `load` that resolves to a path that is not there — and the install ordering is designed to make it not happen, because the record is written only after the copy and the hash both succeed. There is a failure window in which the copy exists and the record does not; there is no window in which the record exists and the copy does not. Chapter 22 traces that ordering stage by stage.

This is also what makes `gc` safe enough to be a plain, unguarded verb. Because the index names every live version, `gc` never has to reason about whether something *might* be needed; it deletes what the index does not name, which is by construction the set of things nothing can reach. A manager whose authority was the filesystem would need heuristics to decide what was garbage, and heuristics that delete are the kind of feature you have to add a confirmation prompt to — which the doctrine forbids.



Figure 2: The store at `$ZSHRS_HOME/pkg/`: `store/<name>@<version>/` holds each stripped plugin tree, `installed.toml` is the global index and source of truth (name · version · kind · source · sha256- hash), and `git/`, `cache/`, `bin/` are internal scratch.

Chapter 7 — Kind Detection

`znative` installs two kinds of plugin — **native** (a compiled Rust `cdylib`) and **script** (zsh text on `$fpath`) — and when the repository does not declare which it is, `znative` detects the kind from the shape of the tree. Detection is what lets the entire existing zsh ecosystem install with no metadata at all, and it is what lets a Rust plugin install with nothing more than a `Cargo.toml`.

A plugin is detected as **native** when either of two signals is present:

- a prebuilt `lib*.{dylib,so}` at the repository root — a library already compiled, ready to load; or
- a `Cargo.toml` whose `[lib] crate-type` includes `cdylib` — a Rust plugin meant to be built into a loadable library.

A native plugin loads through the `zmodload -R` ABI host of Chapter 2, but there is a build step to reach first, and its logic is worth stating exactly. If a prebuilt `lib*.{dylib,so}` already sits at the tree root, `znative` uses it as-is and does not build. Otherwise it consults the manifest's `build` flag, which defaults to `true` whenever a `Cargo.toml` is present, and runs `cargo build --release`. Two failure paths are explicit rather than silent: a native plugin with no prebuilt library *and* `build` disabled is an error (“no prebuilt cdylib and build is disabled”), and so is one with neither a library nor a `Cargo.toml` to build from. When the build succeeds, `znative` copies the freshly built `cdylib` out of `target/release/` up to the tree root — because the store copy deliberately skips `target/`, the library has to be at the root to survive into the store. Only then does the store copy happen. Either way, what ends up loaded is machine code crossing the `#[repr(C)]`, version-checked boundary, not interpreted shell text.

Loading a script plugin is the mirror image and equally concrete: `znative` synthesises a small shell fragment — `fpath=(<dir>/<fpathdir> $fpath)` for each recorded `fpath` entry, then `source <dir>/<file>` for each recorded source file — and evaluates it on the live shell. That is the same `$fpath`-then-`source` dance oh-my-zsh and zinit perform; `znative` just drives it from the index record rather than from a hand-written line in your `.zshrc`.

A plugin is detected as **script** when the native signals are absent and any of these are present:

- a `*.plugin.zsh` file (the oh-my-zsh convention);
- a `functions/` directory (the completion/autoload convention); or
- loose `*.zsh` files at the root.

A script plugin is handled exactly as oh-my-zsh or zinit would handle it: its directory is added to `$fpath` and its files are `sourced` as-is. This is the point of the whole detection scheme — **the entire existing script-plugin ecosystem installs with no metadata whatsoever**. You do not write a manifest to install `zsh-autosuggestions`; `znative` sees the `.zsh` files, classifies it as a script plugin, and sources it, because that repository already looks exactly like what a script plugin is.

The order is fixed: native signals are checked first, script signals second. If a repository carries neither — no `cdylib Cargo.toml`, no prebuilt library, no `*.plugin.zsh`, no `functions/`, no loose `*.zsh` — `znative` cannot classify it and says so rather than guessing, and the repository has to ship a manifest to declare its kind.

Detection as a compatibility contract

It is worth being precise about what detection is *for*, because it looks like a convenience and is really a compatibility strategy. There are thousands of zsh plugin repositories in existence, and not one of them was

written with `znative` in mind. Every one of them predates it. A manager that required metadata to install a plugin would therefore start life able to install nothing, and would spend its existence waiting for an ecosystem to come to it — which, for an ecosystem that already works fine with five other managers, would be a long wait.

So detection is aimed at repositories that cannot be changed. Each signal in the list is chosen because it is a convention that already exists and is already load-bearing somewhere else. `*.plugin.zsh` is oh-my-zsh's entry-point convention, and a repository that has one has it because oh-my-zsh users needed it there. `functions/` is the autoload convention, and a directory by that name is on somebody's `$fpath` today. Loose `*.zsh` at the root is what a plugin looks like when its author never adopted a framework's convention at all. `znative` is not proposing a new layout and hoping people adopt it; it is reading the layouts that are already true and deriving the same conclusion the incumbent managers derive from them.

The native signals work the same way one level up. `Cargo.toml` with `crate-type = ["cdylib"]` is not a `znative` convention either — it is what any Rust crate that intends to be `dlopened` says about itself, and it says so because cargo requires it, not because a shell asked. A prebuilt `lib*.{dylib,so}` at the root is what a release artifact looks like on both supported platforms. In both cases the signal already had to be there for the artifact to be what it is, which is what makes it a reliable classifier rather than a hint.

That is also why the order is native-first rather than script-first, and the reasoning is not aesthetic. A native plugin repository can plausibly contain `.zsh` files — a shim, an example, a completion fragment kept for reference — but a script plugin repository essentially never contains a `cdylib Cargo.toml`, because there would be nothing for it to build. The signals are asymmetric in how often they appear spuriously, so the rarer and more specific signal is checked first. Checking script signals first would misclassify exactly the repositories the native path exists for.

The failure case is the honest one. A tree with no signal at all is not guessed at; `znative` says it cannot classify it. Guessing here would be worse than failing, because a wrong classification does not produce a clean error later — it produces a native build against a tree with nothing to build, or a `source` of files that were never meant to be sourced into an interactive shell. Detection is confident where the conventions are unambiguous and refuses where they are absent, and the manifest is the way a repository resolves that refusal on its own terms.

Detection covers the common cases. When a repository wants to be *explicit* — to override detection, or to declare details detection cannot infer — it ships a manifest. That is the next chapter.

Chapter 8 — The Manifest: `znative.toml`

Auto-detection (Chapter 7) is enough for the common cases, but a repository that wants to be precise ships a `znative.toml` at its root. The manifest **overrides** detection; where it is present, `znative` reads it rather than guessing. Where it is absent, detection covers the rest. Nothing about the manifest is required — it is the explicit path for plugins that need it, not a barrier every plugin must clear.

The manifest has a `[plugin]` table and one of two kind tables.

`[plugin]` carries the metadata that identifies the plugin regardless of kind:

- **name** — the plugin's name, used as the `<name>` in the store path and the index.
- **version** — the version, used as the `<version>` in `store/<name>@<version>/`.
- **description** — a human-readable one-liner.

Then exactly one of the kind tables:

`[native]` declares a compiled plugin. Its central field names the **lib stem** — the base name of the `cdylib` to load (the `foo` in `libfoo.dylib` / `libfoo.so`). It carries a **build** flag whose default is `true` whenever a `Cargo.toml` is present, meaning “build this with cargo before loading”; a plugin that ships a prebuilt library can set it off. This table is how a native plugin that does not follow the “one `cdylib` at the root” convention tells `znative` exactly which library is the plugin.

`[script]` declares a script plugin explicitly. It lists the files to **source** and the directories to prepend to `$fpath`. This is how a script plugin with a non-standard layout — files that are not named `*.plugin.zsh`, or that live in subdirectories detection would not scan — tells `znative` precisely what to load and in what order.

The relationship between the manifest and detection is the thing to hold onto: the manifest is an override, not a requirement. A plugin author who follows the conventions ships nothing. A plugin author whose layout is unusual, or who wants the name/version/description pinned rather than inferred, ships a `znative.toml` and `znative` obeys it exactly. Both paths land in the same store with the same index record; the manifest only changes how `znative` decided what to install, not what installation means.

When a manifest earns its keep

There are four situations where shipping one is the difference between a plugin that installs correctly and one that installs wrong or not at all, and they are worth recognising by shape.

The repository name is not the plugin name. This is the most common reason and the least dramatic. A port lives at `zshrs-revolver` because the repository has to say what it is a port of; the plugin is `revolver` because that is what the user runs. Without a manifest the store key would be `store/zshrs-revolver@.../`, which is merely ugly, and `znative list` would report a name nobody types. A three-line `[plugin]` table fixes the identity at the source, and every downstream artifact — the store path, the index row, the `remove` argument — inherits it.

The load order within the plugin matters. Detection finds `*.zsh` files; it does not know that `lib.zsh` must be sourced before `widgets.zsh`, or that a `config.zsh` full of defaults has to run before the file that reads them. A `[script]` table's **source** list is ordered, and the order is honoured. A plugin that happens to work

today because the filesystem returned its files in a lucky order is a plugin that will break on a different machine; declaring the order is how you stop depending on luck.

Something in the tree looks like a signal and is not. A repository with a `functions/` directory that holds internal helpers rather than autoloadable functions would have that directory prepended to `$fpath`, where it does not belong. A repository with example `.zsh` files at the root would have the examples sourced into the user's shell. Detection reads conventions, and a tree that uses a conventional name for an unconventional purpose is exactly the case detection cannot get right. The `[script]` table's explicit `source` and `fpath` lists are the override: what is listed is loaded, what is not listed is left alone, and the examples stay examples.

The library is not where the convention says. A workspace with several crates, or a crate whose `[lib]` `name` differs from the package name, produces a `cdylib` whose stem is not derivable from the repository. The `[native]` table's `lib` stem names it directly. The same table's `build` flag covers the other half of that problem: a plugin shipping a prebuilt library for users who do not have a Rust toolchain turns the build off, and `znative` loads the artifact as it found it rather than trying to rebuild it.

The through-line in all four is that a manifest is how a repository states a fact about itself that its *shape* cannot state. Where the shape is sufficient, saying it again in TOML adds nothing and is one more file to keep in sync with reality — which is why the manifest is optional rather than required, and why the overwhelming majority of installable plugins will never have one.

Chapter 9 – The `.zshrc` Idiom

Everything in the previous chapters collapses, in practice, to a few lines in a `.zshrc` — one `znative load owner/repo` per plugin, in load order. This chapter is that idiom, because the idiom is the product: the machinery exists so that the startup file can be this plain.

A representative block:

```
znative load zdharma-continuum/history-search-multi-word
znative load zsh-users/zsh-autosuggestions
znative load MenkeTechnologies/zshrs-forgit           # a native Rust plugin
znative load zsh-users/zsh-syntax-highlighting       # kept last, as always
```

Three things about that block are the whole point.

Load order is source order. The lines run top to bottom, so plugins load in the order they are written — which matters, because `zsh-syntax-highlighting` must be sourced last (it wraps the line editor and must see the final state of every widget the earlier plugins defined). `znative` does not reorder or defer; the order in the file is the order of the load. This is the same discipline every zsh user already applies to their plugin block, and `znative` preserves it exactly.

The native line is indistinguishable from the script lines. The third line loads a compiled Rust plugin through the ABI host, and it is written the same way, in the same place, in the same order as the script plugins around it. There is no separate “native plugin” section, no different verb, no build step you invoke by hand. `znative load` classifies the source, builds it if it is a Rust plugin that needs building, and loads it — and from the `.zshrc`’s point of view it is one more `load` line. The native path is *as routine as the script path*, which is the sentence the whole invention is built to earn.

First start installs; every start after loads from the store with zero network. The first time this `.zshrc` runs, each `load` line finds its plugin absent from the store and installs it (Chapter 4’s conditional). Every start after that, each line finds its plugin present and loads it from the store without touching the network. There is no separate `znative install` you run once and a `znative load` you run forever; the one line does both, install-then-load on first sight and load-only thereafter. That is what “no separate install step” means at the level of the file you actually edit.

For the user who would rather install interactively than list every plugin in the startup file, there is the bare form. Run `znative add owner/repo` at a prompt for each plugin you want, and then keep a single unqualified line in `.zshrc`:

```
znative load           # load everything already in the store
```

A bare `znative load` with no argument loads every plugin present in the store, in the store’s order. Either idiom — explicit `load` lines or interactive `add` plus one bare `load` — is the script-plugin workflow zsh users already know, with compiled native plugins folded into it and nothing new to learn.

Which startup file, and why it matters more than it looks

The block goes in `.zshrc` and not in `.zshenv` or `.zprofile`, and the reason is worth being explicit about because getting it wrong is a mistake that works.

`.zshenv` is read by every zsh invocation, interactive or not — every script, every `zsh -c` in a Makefile, every subshell some tool spawned to run one command. A plugin block there would install and load a full set of

plugins for a process whose entire life is one `git rev-parse`. Nothing would break. The plugins would load correctly, define their functions correctly, and be discarded microseconds later, having contributed nothing, on every non-interactive invocation of the shell for the rest of the machine's life. Chapter 28's cost model says exactly why that is the worst place to put the cost: it is a per-invocation charge for a benefit only an interactive session can collect.

`.zshrc` is read by interactive shells, which is precisely the population that has a prompt, a line editor, and a human. Every category of thing plugins provide — widgets, completions, prompt segments, interactive git front-ends — is furniture for that population and meaningless outside it. So the placement rule is not a convention to memorise; it is a restatement of what a plugin is for.

The one case that looks like an exception is a plugin whose commands you want available to scripts. It is not an exception, because scripts should not depend on a plugin being loaded — a script that calls a plugin builtin is a script that fails on any machine where that plugin is absent or unloaded, with a “command not found” that gives no hint about which of your dotfiles was supposed to have provided it. The `zshrs -f` habit of Chapter 35 is the same discipline from the testing side: code that must work should not be relying on ambient interactive configuration.

When one line fails

A `load` line is a command in a shell program, and it behaves like one. If it fails — an unreachable source on a first start, a build failure, an unclassifiable tree — it prints `znative: <reason>` on stderr, returns non-zero, and the shell moves to the next line. The rest of the startup file runs, the rest of the plugins load, and you get a prompt.

That is the right behaviour and it has a consequence worth internalising: **a failed plugin install is a degraded shell, not a broken one, and it announces itself once and then is quiet**. The error scrolls past during startup, the shell comes up, and if the plugin was something subtle — a completion, a widget you use occasionally — you may not notice for a day. The diagnostic is `znative list`, which tells you what the index actually contains rather than what your `.zshrc` asked for, and the gap between those two lists is the set of lines that failed.

If you would rather a failure be loud, the shell already gives you the tools and `znative` does not need to duplicate them: the line's exit status is available like any other, so a `||` clause after a `load` line is how you say “and complain if this one did not work.” That is the general pattern for this whole design — the manager provides a verb with an honest exit status, and the startup file, being a program, decides what to do about it.

Chapter 10 – Loading and Unloading: `zmodload -R`

`znative load` is the high-level workflow; underneath it is one low-level primitive, and it is worth meeting directly because it is what every install ultimately calls and what a plugin developer uses by hand. That primitive is `zmodload -R`.

```
zmodload -R <path>...    # load each cdylib
zmodload -R              # list loaded plugins: name version path
zmodload -uR <name>...   # unload each plugin by name
```

`zmodload -R <path>` `dlopen`s the library, resolves the single `znative_init` symbol, calls it with the host handle, and lets the plugin register its builtins. A bare `zmodload -R` with no argument lists what is currently loaded — name, version, and path, one per line. `zmodload -uR <name>` unloads by the plugin's registered name, not its path.

The `-R` flag is a deliberate, parity-preserving extension. In C zsh, `zmodload -R` removes a *module alias* and is only meaningful alongside `-A`; zshrs keeps that exact behaviour for `zmodload -A -R <name>`, so no zsh parity is lost — `-R without -A` is the slot zshrs claimed for native-plugin loading, a spelling that could not previously mean anything. Loading a plugin whose name is already loaded is refused: you unload first, so there is never ambiguity about which of two same-named plugins a command resolves to. And unload purges the plugin's command registrations **before** the `dlclose`, so no live function pointer survives the library it points into — the single most important safety property of the unload path.

Where a plugin command resolves

The reason a plugin builtin behaves predictably is the specific point in dispatch where it is consulted. `fusevm` — the bytecode machine under zshrs — compiles any name it does not recognise as a builtin into *external* execution. A plugin command is unknown at compile time (the plugin may not even be loaded when the line is compiled), so it arrives at `execute_external_bg` in `src/vm_helper.rs`, which consults `plugin_host::dispatch` **before** it spawns a process. That is the analog of zsh's `resolvebuiltin` slot for `zmodload -ab` autoloaded builtins, and it fixes the resolution order exactly: a plugin command resolves *after* real builtins and shell functions, and *before* `PATH` lookup. A plugin can therefore shadow an external program of the same name — a faster `git` front-end, say — but it can never shadow `cd`, a keyword, or a function you defined. The precedence is not a policy the plugin can opt out of; it falls out of *where* in the pipeline the registry is consulted.

The architecture in three pieces

The whole mechanism is three files' worth of contract:

- **The host loader** — `src/extensions/plugin_host.rs`. It owns the `dlopen` (through `libloading`), the in-process command registry, and the host-callback table plugins call back through. It keeps each loaded `libloading::Library` alive for the life of the process (or until unload); the library's `Drop` is the `dlclose`.
- **The shared ABI** — the `znative` crate. Both the host and every plugin depend on it, which is the whole trick: because both sides compile against the *same* crate, both agree on the exact `#[repr(C)]` layout of every struct that crosses the boundary. `ABI_VERSION` gates loading — a plugin whose version does

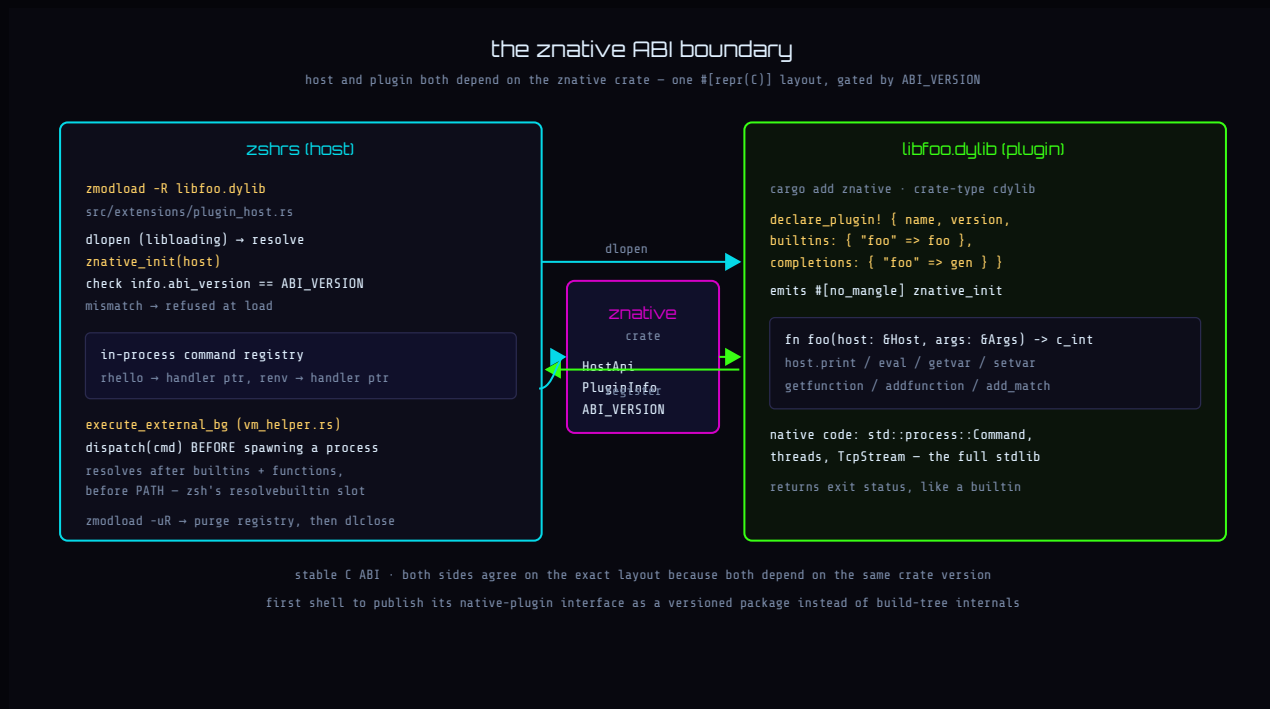


Figure 3: The ABI boundary: `zmodload -R` dlopens the plugin and calls `znative_init`; the plugin registers builtins into the host registry; `execute_external_bg` dispatches to that registry before spawning a process. Host and plugin agree on the layout because both depend on the same `znative` crate version, gated by `ABI_VERSION`.

not match the host is refused, because a mismatched layout would be undefined behaviour rather than a clean error.

- **Command resolution** — `execute_external_bg` in `src/vm_helper.rs`, the dispatch slot described above.

Everything in the rest of Part III is built on these three pieces: the plugin you write (Chapter 12), the host API you call (Chapter 13), the completions you register (Chapter 14), and the versioning and safety rules that keep the boundary sound (Chapter 15).

Chapter 11 – The Three Extension Models

zshrs runs zsh's two existing extension models unchanged and adds a third. It is worth laying all three side by side once, precisely, because the whole claim **znative** rests on lives in the differences between the columns — and because the differences are narrower and more specific than “it loads native code.”

	script plugin (.zsh)	bash enable -f / zsh zmodload C module	zshrs Rust plugin
Language	shell script (interpreted)	C	Rust (any native language via the C ABI)
Artifact	a .zsh text file	a .so built in the shell's own tree	a .dylib / .so cdylib
Build against	nothing — it is sourced	the shell's private internal headers	the published znative crate — cargo add znative
ABI stability	n/a	none — bound to one exact shell build	stable, versioned ABI_VERSION ; mismatch refused at load
Distribution	a file to source	rebuild against each shell release	one crates.io crate, independent of the shell's source
Load / unload	source (re-parse every start)	enable -f/-d ; zmodload (dlsym internal symbols)	zmodload -R/-uR (dlsym one symbol, znative_init)
Execution Registers	interpreted each call functions, aliases, options, ZLE widgets, fpath completions	native machine code builtins/params/hooks via internal API	native machine code builtins + a curated host API (print , eval , getvar , setvar)
Third-party viable	yes (oh-my-zsh, zinit)	rare in practice — needs the source tree, tracks internals	yes — one crate, no zshrs source needed
Type safety	none	C, unchecked against shell internals	Rust type system, checked against the ABI crate

Two rows carry the argument. Look at **build against**: a bash loadable builtin compiles against **builtins.h/shell.h**; a zsh C module against the **.mdd** plus **Src/Modules/**. Both are the shell's private internals, with no stable ABI and no version gate — so the artifact is welded to one shell build and can crash a mismatched one. The zshrs plugin instead compiles against a *published* crate, and **ABI_VERSION** refuses a mismatch rather than corrupting the process. Now look at **third-party viable**: because the native interfaces of bash and zsh are their internal C APIs, a stranger cannot practically ship a native module — which is why the native modules that exist for those shells are almost entirely the ones the shell bundles itself. The zshrs column is the only one where “native code” and “shippable by a stranger” are true at the same time.

That is the precise content of the “world’s first” claim, and it is narrower than it first sounds. Native runtime plugin loading is *not* new: bash has had `enable -f file.so name` since roughly 1996, and zsh has always had `zmodload`. The novelty is not loading native code; it is exposing the native-plugin interface as an **independently-published, versioned ABI package** — a crate a third party builds against without ever cloning the shell — instead of the shell’s own build-tree internals. Every other row in the table is shared with something; that one row is `znative`’s alone, and it is the row `znative` the package manager exists to make *installable*.

The fourth thing, which is not an extension model

There is a strategy the table omits and readers coming from zinit will notice its absence, because zinit has a spelling for it: install a compiled *program*, put it on `$PATH`, and let the shell call it as an external command. Chapter 30 names this as a genuine capability difference rather than a scheduling one, and it deserves to be placed against the three columns rather than left as a gap.

It belongs in a different category, and the reason is not a value judgement about binaries. A program on `$PATH` is a process. It gets an argument vector, an environment, and three file descriptors; it returns an exit status. Everything the three columns above have in common — that the extension *runs inside the shell and changes it* — is exactly what a separate process cannot do. It cannot define a function you can then call. It cannot bind a ZLE widget, because widgets are in-process callbacks into the line editor. It cannot read a shell parameter that was never exported, or set one that survives its own exit, or register a completion, or observe an option. It changes your `$PATH`; it does not change your shell.

That distinction is what makes it a packaging question rather than an extension model. A manager that fetches release binaries is doing useful work — the work of getting a file onto a machine and onto a path — and it is the same work a system package manager does, on a different schedule. `znative` does not do it, and the omission is coherent: the tool’s unit of installation is a *plugin*, meaning a thing that loads into the shell and extends it, and a standalone executable is definitionally not that no matter how it was fetched.

Worth following the consequence one step, because it sharpens the whole table. The per-call cost story of Chapter 28 also separates cleanly here. A plugin builtin resolves in-process to a function pointer; an external program costs a process creation on every invocation, which is a cost no amount of compiling the program away removes, because the cost is not in the program. Chapter 19’s rule about when *not* to write a builtin is the same observation from the author’s side: if the work is dominated by spawning a child, a compiled plugin spawns the same child and buys nothing. The three columns in the table are three ways to run code inside the shell. A binary on `$PATH` is the way to run code outside it, and the choice between those two categories precedes the choice among the three.

Chapter 12 — Writing a Native Plugin

A native plugin is an ordinary Rust `cdylib` with two required pieces: a `Cargo.toml` that says “build a `cdylib` and depend on the `znative` crate,” and a `src/lib.rs` that declares the plugin with the `declare_plugin!` macro. Nothing else is mandatory — no build script, no generated glue, no `zshrs` checkout.

The `Cargo.toml`:

```
[lib]
crate-type = ["cdylib"]

[dependencies]
znative = "0.12"
```

`crate-type = ["cdylib"]` is what makes `cargo build` emit a `dlopen`-able shared library rather than a Rust `rlib`; the `znative` dependency is the ABI crate from Chapter 2. A plugin distributed as a repo often depends on the crate by git so it builds standalone against the matching `zshrs` — `znative = { git = "https://github.com/MenkeTechnologies/zshrs" }` — but the `crates.io` form is the common one.

The `src/lib.rs`:

```
use znative::{declare_plugin, Args, Host};
use std::os::raw::c_int;

fn rhello(host: &Host, args: &Args) -> c_int {
    let who = if args.rest().is_empty() {
        "world".into()
    } else {
        args.rest().join(", ")
    };
    host.print(&format!("hello, {who}\n"));
    0
}

declare_plugin! {
    name: "hello",
    version: "0.1.0",
    builtins: {
        "rhello" => rhello,
    },
}
```

`cargo build` turns that into `libhello.dylib` on macOS or `libhello.so` on Linux, and `zmodload -R ../libhello.dylib` loads it. Three things in that snippet are the whole authoring model, and they generalise to every plugin:

A **handler is a plain function** with the signature `fn(&Host, &Args) -> c_int`. It receives the host callback table and the decoded arguments, does its work, and returns an exit status — `0` for success — exactly like a shell builtin. There is no async runtime, no trait to implement, no lifecycle beyond “called when the command runs.”

`declare_plugin!` is the **only macro**, and it is what emits the single `#[no_mangle] extern "C" fn znative_init` the host looks for (Chapter 2). You give it the plugin’s `name` and `version` — which become the store key and the `ABI_VERSION`-tagged `PluginInfo` — and a `builtins` map from command name

to handler. The macro handles the FFI: you never write an `extern "C" function` or an `#[repr(C)] struct` yourself.

Commands register under the name the user types. The map key `"rhello"` is the command a user runs, not the Rust function name. This matters most when porting: a zsh plugin's user-facing name is usually an *alias* (`ga`, not `forget::add`), and the port registers the builtin under the alias — `"ga" => ga` — so the muscle memory is preserved. Chapter 16 returns to this.

From here the two dimensions to learn are what a handler can *do* to the shell (the host API, Chapter 13) and how to offer *completions* for the commands it registers (Chapter 14).

The contract under the convenience

`declare_plugin!` and the `Host/Args` wrappers are ergonomics over a raw C-ABI contract that Appendix I states in full. You do not have to write to that contract, but two of its terms leak through the wrappers in ways that will produce bugs if you do not know them.

The arguments are borrowed for the duration of the call. At the raw level a handler receives `argc` and an `argv` of NUL-terminated C strings, and those strings are valid only while the call is executing. `Args` gives you `&[String]` over that, which is Rust's way of saying the same thing: the slice does not outlive the handler. If you spawn a thread that reads an argument, or stash one in a `static` for a later command to consult, you must copy it first. This is exactly the kind of mistake the borrow checker catches when the lifetimes are visible, and exactly the kind it cannot catch if you launder the value through a raw pointer to be clever.

The `PluginInfo` you return is `'static`, and the macro makes it so. The name and version you give `declare_plugin!` become string literals in a `static` structure the host keeps a pointer to. The host uses that name for the `zmodload -R` listing and for `zmodload -uR <name>`, which means the name in the macro is the handle a user unloads by — and it is a separate namespace from the command names in the `builtins` map. A plugin named `hello` registering a builtin `rhello` is unloaded as `hello`, not as `rhello`, and getting that backwards is the most common first confusion in the development loop of Chapter 27.

What the macro deliberately does not do

It emits the FFI and nothing else, and the omissions are worth listing because each is a thing you are responsible for.

It does not parse your arguments. `args.rest()` is the shell's word-split vector and that is all; there is no flag parser, no subcommand router, no usage generator. If your command has options, you write that, and Chapter 19's advice applies — take the vector as given, do not reconstruct a command line from it in order to split it again.

It does not catch your panics. Chapter 15's rule is that a panic across the C-ABI boundary is undefined behaviour, and the boundary in question is the very `extern "C" fn` the macro generated for you. The macro does not wrap your handler in a catch, so the handler that panics is the handler that ends the shell process. This is the single rule the compiler will not enforce and the macro will not paper over.

It does not give you a lifecycle beyond init. The only entry point the boundary defines is `znative_init`, called once right after load to register your commands and return a `PluginInfo`; the documented surface has nothing that runs on unload. That is why Chapter 20 initialises shared state lazily on first use rather than in a constructor, and why Chapter 20's rule about threads is phrased as an obligation on your handlers — if a worker has to stop before the library goes away, your code is the only thing in a position to stop it.

Chapter 13 – The Host API

Inside a handler, `Host` is the shell's callback table — the curated, `#[repr(C)]` surface a plugin is allowed to touch. It is deliberately small: a plugin does not reach into shell globals the way a C module does, it asks the host to act on its behalf through a fixed set of methods.

Method	Purpose
<code>host.print(s)</code>	Write to the shell's stdout.
<code>host.eval(code) -> i32</code>	Run shell code; return its exit status.
<code>host.getvar(name) -> Option<String></code>	Read a shell scalar parameter.
<code>host.setvar(name, value)</code>	Set a shell scalar parameter.
<code>host.getfunction(name) -> Option<String></code>	Read a shell function's deparsed body (<code>\${functions[name]}</code>) — ABI v3.
<code>host.addfunction(name, body) -> bool</code>	Define or replace a shell function (<code>functions[name]=body</code>) — ABI v3.
<code>host.register_builtin(n, f)</code>	Register a command handler dynamically, at runtime.
<code>host.add_match(word)</code>	Emit one completion candidate (Chapter 14).
<code>host.install_completion(cmd, gen)</code>	Wire a native completion into compsys (Chapter 14).

Alongside `Host`, `Args` decodes `argv`: `.name()` is `argv[0]`, `.rest()` is the arguments after it as a `&[String]`, and `.to_vec()` is the whole vector. Between `Args` for input and the table above for effects, a handler has everything it needs to be a first-class shell command.

The split in that table is the design. `print`, `eval`, `getvar`, and `setvar` are the everyday four — write output, run some shell, read and write a scalar. Then there is a deliberate boundary: `getvar` is scalars only, and `eval` returns just a status, so neither gives *structured* access to shell **functions**. That access is exactly what `getfunction` and `addfunction` add in ABI v3, and the pair does something more interesting than read-a-function/write-a-function.

Deparse as a service

Because `addfunction` then `getfunction` round-trips a body through the shell's *own* parser and pretty-printer, the pair doubles as deparsing-on-demand. Define a temporary function from arbitrary source text, read it back, and what returns is the shell's normalised form — one statement per line, tab-indented, canonical spacing — regardless of how the input was written. A plugin that wants to reformat a command line does not need to embed a shell parser; it defines a throwaway function from the line and reads back the normalised body. That is precisely how a history-formatting plugin such as `zsh-hist` auto-formats what you typed: not by parsing shell in Rust, but by handing it to the host's parser and reading the result. It is a small API with a large second use, and it exists because the host, not the plugin, is the authority on what shell code *means*.

register_builtin and dynamic registration

`declare_plugin!`'s `builtins` map is the static way to register commands; it is resolved once, at `init`. `host.register_builtin(name, f)` is the dynamic way — a handler can register *another* handler at runtime. Most plugins never need it, but it is what makes the completion machinery of the next chapter work: the macro uses it to register a completion generator as a hidden builtin without the plugin author writing the call.

Choosing between `eval` and Rust

`host.eval` is the largest lever on the table and the easiest one to overuse. It runs arbitrary shell code in the live shell and hands back a status, which means a plugin author can always fall back to it, which means the interesting question is never “can I do this with `eval`” but “should I.” There is a clean rule, and it follows from what each side of the boundary is *authoritative* about.

Use `eval` when the effect you want is a change to shell state that only the shell can make correctly. Changing the working directory is the canonical case, and it is why `zsh-z` (Chapter 18) computes its frequency ranking in Rust and then delegates the actual `cd` back to the shell. A plugin could call `chdir` directly — it is running in the shell's own process, so the syscall would succeed — and the result would be a shell whose `$PWD` is stale, whose directory stack is wrong, and whose `chpwd` hooks never fired. `cd` in a shell is not a syscall; it is a syscall plus a set of bookkeeping obligations, and the shell is the only thing that knows the full set. The same reasoning applies to anything that trips hooks, anything that touches options, and anything a user might reasonably have redefined.

Use Rust when the work is computation, I/O, or process management. A plugin does not need a host API to run `git`, read a file, or parse output, and reaching for `eval` to do those things is strictly worse in every dimension: it re-enters the parser, it re-introduces the quoting hazards the port existed to eliminate, and it turns a type-checked call into a string you have to escape correctly. The moment a plugin starts building shell source with `format!` and interpolating values into it, the port has regressed to the thing it replaced. If you find yourself constructing a command line as text, the question to ask is whether `std::process::Command` — which takes an argument vector and does no word-splitting at all — would say the same thing without a quoting layer.

The middle case is shell code you deliberately chose *not* to port, and here `eval` is the right answer rather than a compromise. A plugin that wraps an existing user-supplied hook, or that needs to run a snippet the original plugin exposed as a configuration point, should run that snippet as shell, because it *is* shell and the user wrote it expecting shell semantics. Chapter 16's rule about `zfz` preview strings is the same principle from the other end: some things are irreducibly shell, and porting them to Rust would change their meaning.

The parameter accessors sit under the same rule. `getvar` and `setvar` exist so a plugin can participate in the shell's configuration conventions — reading `$FZF_DEFAULT_OPTS`, honouring a plugin-specific `$FORGIT_PAGER` — rather than inventing its own configuration file. They are scalars only, which is a real constraint and an intentional one: it keeps the boundary's string handling to a single case with a single ownership rule, instead of a family of array and associative-array marshalling paths each with its own allocation contract. A plugin that needs a list from the shell has two honest options — a delimited scalar it splits in Rust, or `eval` of the shell code that would consume the array where it lives — and both of them keep the ABI's surface flat.

Chapter 14 — Native Completions

A native plugin can ship its command *and* a completion for that command, both in Rust. This closes a real gap: a script plugin registers `fpath` completers, but a compiled plugin that only registered builtins would leave its commands with no tab-completion at all. The `completions:` section of `declare_plugin!` fixes that.

```
const NAMES: &[&str] = &["alice", "bob", "carol", "dave", "erin"];

fn greet_complete(host: &Host, args: &Args) -> c_int {
    let a = args.rest(); // [CURRENT, word0, word1, ...]
    let current: usize = a.first().and_then(|s| s.parse().ok()).unwrap_or(0);
    let words = &a[1..]; // words[0] == "greet"
    let prefix = current.checked_sub(1).and_then(|i| words.get(i))
        .map(String::as_str).unwrap_or("");
    for &c in NAMES {
        if c.starts_with(prefix) { host.add_match(c); }
    }
    0
}

declare_plugin! {
    name: "greet",
    version: "0.1.0",
    builtins: { "greet" => greet },
    completions: { "greet" => greet_complete },
}
```

A completion *generator* is a handler like any other, with a specific argument convention: it receives `$CURRENT` — the 1-based index of the word being completed — followed by every word on the command line, and it emits candidates with `host.add_match`. In the example, `greet <TAB>` offers `alice bob carol dave erin`, filtered live by the current prefix, and every decision — which candidates exist, how they are filtered — is made in Rust.

Two rules make it work, and both are load-order rules the author has to respect.

Load the plugin after `compinit`. The macro registers the generator as a hidden builtin and records a *completion request*; the host defers the actual `compdef` wiring until the first time completion fires. That deferral is deliberate — running compsys glue during plugin-init would fire it too early, before the completion system is set up — so the wiring happens at a safe point in the completion pipeline instead. The consequence for you: `znative load` a completion plugin *after* `autoload -Uz compinit; compinit`, not before.

No leading underscore on the generator's name. `zsh` treats a command named `__*` as an autoloadable completer, so a generator named `__greet` would be picked up by the completion system as a completer rather than staying the builtin the macro registered. Name it anything without a leading underscore.

A real completion: `kubectl`

The `greet` example is a static list; the technique scales to completions that are anything but static. The shipped `kubectl` completion plugin delegates to cobra's own `kubectl __complete` machinery — it asks the installed `kubectl` binary what the candidates are — so it tracks that exact binary's version: its subcommands, its flags, and live cluster resources, with no static completion tree to maintain or let go stale. The generator is Rust;

the source of truth is the tool itself. Published as `znative load MenkeTechnologies/zshrs-kubectl-completion`, it is the proof that native completions are not a toy for hard-coded word lists but a way to wire a live, version-tracking completion in a compiled plugin.

Reading the argument convention closely

The generator's `argv` repays a second look, because two of its properties are the ones people get wrong.

`CURRENT` is the 1-based index of the word being completed, and the words that follow it start with the command itself. So for `greet ali<TAB>`, `CURRENT` is 2 and the words are `greet, ali` — the index counts the command as word one. The `greet_complete` example converts with `current.checked_sub(1)` before indexing into the words slice, and that subtraction is not a fudge; it is the translation between a convention inherited from `zsh`'s completion system, where `$CURRENT` has always been 1-based, and `Rust`'s 0-based slices. Using `checked_sub` rather than `- 1` matters because a `CURRENT` of zero is expressible in the `argv` you were handed, and a generator that panics on a malformed request is a generator that panics inside a handler, which Chapter 15 says is undefined behaviour rather than a caught error.

The whole command line is available, not just the prefix, and that is the entire reason a native generator can be context-sensitive. A completion that offers different candidates after `--namespace` than after the bare command needs to look backwards through the words, and it can, because they are all there. This is the same information a `zsh` completer gets from `$words`, delivered as a `&[String]` rather than as a shell array, and the porting mapping in Appendix G is therefore direct: a completer that read `$words[CURRENT-1]` to decide reads `words[current - 2]` here, with the same off-by-one bookkeeping made explicit.

What crosses, and what therefore cannot

The emission side is one method: `host.add_match(word)`, one candidate per call. A candidate is a string. That is the whole surface, and being precise about it saves a plugin author from designing against a capability that is not there.

What you get is a flat list of candidates, filtered however you like, computed by arbitrary `Rust` — which covers the overwhelming majority of what a completion is. What does not cross this interface is the rest of `compsys`'s vocabulary: candidate descriptions, grouping under headings, mutually exclusive option sets, the `_arguments` specification language. A generator that wants to present two labelled groups is expressing something the `add_match` surface has no field for.

That is a deliberate narrowness rather than an oversight, and it follows the same logic as `getvar` being scalars only (Chapter 13): a flat string is one case with one ownership rule across the C boundary, and each additional structure would be another marshalling path with its own allocation contract, on a boundary whose stability is the whole product. A plugin whose completion genuinely depends on that richer presentation is telling you something about which extension model it belongs in: the `fpath` completer that Chapter 11's first column describes has `compsys`'s whole vocabulary available to it because it *is* `compsys`, and a plugin can be shipped as a script plugin for that reason alone. The native generator is the right tool when the candidates are expensive to compute and cheap to describe, which is the shape `kubectl` has and the shape most command completions have.

The cost of a generator, and when it matters

A completion generator runs on a keystroke, which puts it in the one place where the per-call cost model of Chapter 28 is most visible to a human. A native generator's per-call cost is a call through a registered function pointer plus whatever the generator does, and “whatever the generator does” is the part worth thinking about.

The `greet` example filters a constant slice, which is free. The `kubectl` case runs a subprocess and, for cluster resources, waits on a network round trip — which is not free, and which is nonetheless the right design,

because the alternative is a static completion tree that is wrong. The honest framing is that the native path removed the interpretation overhead and left the inherent cost exactly where it was: if your candidates come from a slow source, the completion is slow, and compiling the generator did not change that.

The implication for a generator that does real work is the one Chapter 20 states for long-running builtins generally. It is running in the shell's process, on the thread the user is typing on, and for as long as it runs the line editor is not responding. Cache what is stable, bound what is not, and prefer returning a smaller correct set quickly over a complete set slowly — because a completion the user out-typed is a completion that cost them time and gave them nothing.

Chapter 15 — ABI Versioning and Safety

The whole native path rests on one integer and a handful of safety rules. This chapter is both.

ABI_VERSION

`ABI_VERSION` is a constant in the `znative` crate — 3 at the time of writing — and it is the version gate the whole scheme depends on. It is bumped on *any* change to the layout or semantics of the types that cross the boundary: `HostApi`, `PluginInfo`, `BuiltinFn`. Its history is legible in the crate: v2 added the `register_completion` entry that native completions ride on (Chapter 14), and v3 added `getfunction/addfunction` (Chapter 13) — each an additive field on `HostApi`, each a version bump because appending a field still changes the frozen layout a plugin compiled against.

The check runs on *both* sides, which is the belt-and-suspenders that makes it sound. The host compares the plugin's compiled-in version against its own and refuses the load on a mismatch; and the plugin's `declare_plugin!` glue *also* checks the `abi_version` field of the `HostApi` table it is handed before it trusts any other pointer in that table. Either side can reject the other. This matters because a plugin built against a different layout would not be a clean incompatibility — it would be undefined behaviour, one side reading a struct with the wrong field offsets. Refusing at load, from whichever side notices first, turns that latent memory-corruption bug into a legible “this plugin was built for a different zshrs” failure.

One consequence of the C boundary shows up in the everyday methods and is handled for you: a raw `getvar` across the ABI returns a freshly allocated C string that must be released with the table's `free_cstring`, and a bare `getvar` that returns nothing is the parameter being unset. The `Host` wrapper does that allocation dance behind `getvar(name) -> Option<String>`, so a plugin author writes safe Rust and never touches `free_cstring` directly — but it is why the host API is a curated wrapper over the raw `#[repr(C)]` table rather than the table itself.

The practical rule for a plugin author is to keep the `znative` crate's major/minor aligned with the target zshrs release, and to remember that the single exported symbol every plugin must provide — `znative_init` — is generated by `declare_plugin!`, so you never hand-write it and never have to keep its signature in sync yourself. The macro and the version constant are the two things that keep a stranger's plugin loadable without a shared source tree.

Safety at the boundary

Three properties keep the FFI boundary sound, and a plugin author has to respect the middle one:

- **The library outlives the calls into it.** The host keeps each loaded `libloading::Library` alive for the life of the process, or until an explicit unload; its `Drop` is the `dlclose`. Nothing calls into a library that has been unmapped, because unload purges the registry before it drops the library.
- **A panic across the C-ABI boundary is undefined behaviour.** Rust does not unwind across an `extern "C"` frame, so a handler that panics past the boundary is UB, not a caught error. Handlers must catch their own panics (or simply not panic) — the host does not, and cannot safely, unwind through the FFI boundary for you. This is the one rule the compiler will not enforce for you.

- **Plugin builtins run synchronously, in-process, even when backgrounded.** `zshrs` is non-forking, so `foo &` does not spawn a subprocess for an in-process builtin — there is nothing to background. A plugin that wants concurrent or background work does it with threads inside the process (Chapter 18's `revolver` and `git-repos` ports are exactly this), not by forking. This is not a limitation so much as a different, and often faster, model: shared memory instead of a forked copy and a statefile.

Chapter 16 — Porting a zsh Plugin: the Method

The worked ports ([forgit](#), [git-fuzzy](#), [zsh-z](#), [revolver](#), [git-repos](#)) were not written from scratch; they were *ported* from existing zsh plugins, and the translation follows a repeatable method. This chapter is that method, worked on [forgit](#) — a git+fzf plugin — the way the zshrs porting guide works it.

The mental model

A zsh plugin is a set of shell **functions**, usually exposed through **aliases** and **completions**. The native equivalent maps construct for construct:

zsh plugin construct	Rust plugin equivalent
a shell function <code>foo() { ... }</code>	a builtin <code>fn foo(&Host, &Args) -> c_int</code> register the builtin under the alias name (<code>"g" => foo</code>)
<code>alias g='foo'</code> (the user-facing name)	a <code>completions: { "g" => gen }</code> entry
<code>compdef _foo foo + _foo()</code> completer	
<code>\$1 \$2 "\$@"</code> positional args	<code>args.rest()</code> (<code>&[String]</code>)
<code>\$VAR / \$FZF_DEFAULT_OPTS</code>	<code>host.getvar("VAR")</code>
<code>echo / print</code> to the terminal calling <code>git</code> , <code>fzf</code> , <code>fd</code> , <code>rg</code> , ...	<code>host.print(...)</code> , or let a subprocess inherit stdout <code>std::process::Command</code> — spawn anything
shell code you do not want to port	<code>host.eval("...")</code> do it in Rust (<code>str</code> methods, iterators)
<code>sed/awk/grep</code> text munging	

The realization that makes the whole port tractable: **a Rust plugin is native code**, so it does not need a host API to run external programs, read files, or parse text — it uses the standard library directly. The host API is only for *shell integration*: registering commands and completions, reading and writing shell parameters, printing to the shell, and evaluating shell code you chose not to port. Everything else is ordinary Rust.

Why port at all

- **No per-invocation parse.** A zsh plugin function is parsed (or `.zwc`-loaded) and interpreted on every call; the builtin is machine code, resolved once at `zmodload -R`.
- **Control flow is type-checked, not re-parsed shell.** The fragile parts of shell plugins — quoting, word-splitting, `$IFS`, `[[ ]]` versus `[ ]` — move into Rust, where the compiler checks them.
- **The genuinely-shell parts stay shell.** An fzf `--preview` string runs per-item via `sh -c`; you keep it as a string. Porting the glue does not oblige you to reimplement a pager or fzf.

The recipe

1. **Inventory the commands.** List every user-facing alias and the function it maps to. `forgit.plugin.sh` aliases `ga` `grh` `glo` `gd` `gi` `gcf` `gclean` `gss` `gcp` onto nine functions; each becomes one builtin, registered under the alias name.
2. **Find the shared helpers and config.** `forgit` resolves pagers and a base `FORGIT_FZF_DEFAULT_OPTS` block at load, and every command opens with `forgit::inside_work_tree`. Port these to free functions or a small per-call context struct.
3. **Port each command's skeleton:** guard → build a list → pick with fzf → act. Keep git and fzf as subprocesses; do the list transforms in Rust.
4. **Decide what stays shell.** fzf previews and `enter:execute(...)` binds are shell strings — keep them, interpolating the values you resolved in Rust.
5. **Register with `declare_plugin!`** and build as a `cdylib`.

The patterns, forgit before and after

The guard. `forgit::inside_work_tree` in shell becomes a free function that shells out to `git rev-parse` and checks the status:

```
fn inside_work_tree() -> bool {
    Command::new("git").args(["rev-parse", "--is-inside-work-tree"])
        .stdout(Stdio::null())
        .stderr(Stdio::null())
        .status().map(|s| s.success()).unwrap_or(false)
}
```

Reading config. `${FORGIT_PAGER:-$(git config core.pager || echo cat)}` becomes an `Option` chain with `host.getvar` for the shell parameter and a small `git config` helper for the fallback — the shell's `:-` default logic expressed as Rust `.filter(...).or_else(...).unwrap_or_else(...)`.

List → fzf → selection, the heart of the plugin. In shell it is a pipeline; in Rust it is: capture the git list, spawn fzf, write the list to its stdin, read the selection back. The `fzf` helper is reusable across every command, and three details in it are load-bearing:

- **stderr is inherited; stdin and stdout are pipes.** fzf reads its candidate list from the stdin pipe and opens `/dev/tty` itself for the interactive UI, so it works even though stdin is not a terminal.
- **Write stdin on a thread.** Writing the whole list and then calling `wait_with_output()` can deadlock for large lists — fzf's stdin pipe fills before it drains while you block reading its stdout. A writer thread avoids it.
- **fzf exits non-zero on abort or no-match.** Treat that as “no selection,” not an error.

Text munging moves to Rust. `forgit`'s `ga` greps colored `git status` output and rewrites it with `sed`; the port reads stable `--porcelain` output and parses it in Rust — more robust than color-grepping, and the kind of thing the compiler checks. **What stays shell** is the fzf `--preview` and `--bind=enter:execute(...)` strings, which fzf runs per item via `sh -c`; those are irreducibly shell, and the original plugin shipped them as shell strings too. The port moves the *control flow* to Rust and leaves the per-item shell snippets where they belong.

Registering, and the gotchas

The nine builtins register under their alias names:

```
declare_plugin! {
  name: "forgit",
  version: "0.1.0",
  builtins: {
    "ga" => ga, "grh" => grh, "glo" => glo, "gd" => gd, "gcf" => gcf,
    "gclean" => gclean, "gss" => gss, "gcp" => gcp, "gi" => gi,
  },
}
```

The checklist that keeps a port honest: register under the alias name, not the internal function name; expect the same runtime dependencies as the original (`forgit` needs `git` and `fzf` on `PATH`, and so does the port — `Command::new` fails cleanly if they are missing); inherit the `tty` for interactive children (`fzf`, `less`, an editor) rather than capturing their `stderr`; map any `_foo` completers to the `completions:` block; and never give a builtin a leading-underscore name. And the meta-rule: **do not over-port**. Reimplementing `git`, a pager, or `fzf` in Rust is the wrong move — shell out to them. Port the glue, not the tools.

Chapter 17 – Self-Reentrant fzf Tools

`forgit` is “list → fzf → act”: fzf runs once, returns a selection, done. A harder class of tool is **self-reentrant** — the fzf UI calls *back* into the tool on every keystroke. `git-fuzzy` is the worked example, and porting it surfaces a problem that a compiled, non-forking shell is uniquely positioned to solve well.

In `git-fuzzy`, fzf’s `--preview` re-invokes a helper for the item under the cursor, its `--bind` actions re-invoke helpers to stage/unstage/discard and then `reload` the list, and a `--listen` port lets a background watcher push live reloads. In bash this means fzf re-execs the `git-fuzzy` script and re-sources its library **on every keystroke** — `git-fuzzy` even ships “dispatch-aware sourcing” to keep that per-keystroke re-sourcing affordable. That per-keystroke reparsing is exactly the overhead a compiled host exists to delete.

The wrinkle is that fzf runs its bind and preview commands via `sh`, and `sh` cannot call a plugin builtin — the builtin lives inside the `zshrs` process, not on `PATH`. Three parts make the port work:

1. **Helpers are builtins too.** The plugin registers `gf` and dispatches an internal `--helper <sub>` mode: `gf --helper status_preview ...`, `gf --helper status_add ...`. Same code path, same compiled plugin, no separate binary to ship or keep in sync.
2. **A shim bridges fzf back to the plugin.** Since `sh` cannot reach the builtin, the plugin generates a tiny shim once and points every fzf bind at it:

```
# ~/.cache/zshrs/gf-helper.sh
#!/bin/sh
exec <zshrs> -fc 'zmodload -R "$@" 2>/dev/null; gf --helper "$@"' <self.dylib> "$@"
```

Each fzf bind becomes `execute(<shim> status_add {+2..})`. Every invocation is a fresh `zshrs -f` that `dlopen`s the plugin’s own dylib — an mmap’d shared library, *no parsing* — and runs the helper builtin. That single `dlopen` replaces bash’s per-keystroke library sourcing. The plugin finds **its own** dylib path with `dladdr` and the `zshrs` binary with `std::env::current_exe()`, and it passes the dylib as `$0` (rather than embedding it in the `-c` script) so a path containing spaces survives intact.

3. **Live reload without curl.** `git-fuzzy`’s watcher POSTs `reload-sync(...)` to fzf’s `--listen` port; the port writes the HTTP request to a raw `TcpStream`, so there is no `curl` dependency and the watch loop itself is Rust. A failed connect means fzf is gone, which is the watcher’s signal to stop.

Verifying an interactive TUI

A self-reentrant fzf tool is mostly testable headlessly, and one part is not, and knowing which is which keeps the port honest. The *logic* — the helper subcommands, the shim round-trip, menu generation, the `--expect` interpreter, geometry, diff rendering — is unit-testable without a terminal: call `gf --helper ...` directly, drive the top-level command with a stub `fzf` that echoes a canned selection, and invoke the generated shim by hand. The **live fzf render** — the actual full-screen UI reacting to keystrokes — must be checked in a **real terminal**. A scripted PTY can drive a plain `sh | fzf`, but it does not faithfully reproduce an interactive shell handing the terminal to a full-screen child, so a blank PTY capture is not evidence of a bug. The rule the porting guide states plainly: verify the UI by running it, and do not treat the untestable part as testable.

Chapter 18 — The Worked Ports

An invention that only works in principle is a proposal. `znative` ships with a set of worked plugin ports that prove the native path end to end — real, useful plugins built against the published ABI, installed by `znative`, running as compiled code. They exist to demonstrate that a stranger *can* build a native plugin and that `znative` *can* install it, using tools people actually want at a prompt.

Two of the shipped examples are teaching cases and the rest are real tools. The teaching pair sets the floor:

- `plugin-hello` exercises the whole host API in a few lines — `rhello` echoes its argv and `$PWD`, `renv NAME [VALUE]` reads and writes a scalar — so it is the minimal end-to-end proof that a `cdylib` can register builtins, read and write shell parameters, and print.
- `plugin-complete` is the completion counterpart: a native Rust completion for a `greet` command (Chapter 14), demonstrating `add_match`, the `$CURRENT+words` convention, and the load-after-`compinit` rule with nothing else in the way.

The real tools are the point, though, because each one is a plugin people actually run, and each was chosen because the native path *changes* something about it.

`forgit` — the interactive git front-end (`ga grh glo gd gcf gclean gss gcp gi`, fuzzy-searched through `fzf`) — is the flagship port, done command-for-command from the zsh original and used as the worked example of the whole porting method (Chapter 16). Published as `znative load MenkeTechnologies/zshrs-forgit`, it is the self-installing line the front matter opens with.

`git-fuzzy` is the self-reentrant case (Chapter 17): its `status` command is a full `fzf` UI whose previews and key-binds call back into the plugin per keystroke, plus a `--listen` live-reload watcher. Where the bash original re-sources its library on every keystroke, the port answers each callback with a single `dlopen` of an already-mmap'd dylib. `znative load MenkeTechnologies/zshrs-git-fuzzy`.

`zsh-z` is the frecency directory jumper (`z <partial>`), reimplemented in Rust: the `~/.z` datafile, the frecency formula, aging, and matching, with all `z` options. The interesting seam is what it does *not* do in Rust — the actual `cd` is delegated to the shell through `host.eval`, so `$PWD`, `chpwd` hooks, and directory stack stay correct, while a `chpwd` hook does the recording. It is the model port for “compute in Rust, mutate shell state through the host.” `znative load MenkeTechnologies/zshrs-zsh-z`.

`git-repos` (from `zsh-git-repo-cache`) scans the filesystem for every git repo, caches them, and `fzf`-picks one to `cd` into, with `--clean/--dirty` filters. The shell version classifies each repo with a sequential `git status`; the native version classifies **in parallel across threads** — a real, not rhetorical, speedup, and a direct use of the non-forking model below. `znative load MenkeTechnologies/zshrs-git-repos`.

`revolver` is the progress spinner (`start/update/stop/demo`, 55 styles), ported from zsh. Its port is the sharpest illustration of the non-forking model: the upstream forks a background process and coordinates through a `$PPID`-keyed statefile; the native port keeps the animator as an in-process **thread** and the running spinner in a `static` slot, so `start/update/stop` share memory directly — no fork, no statefile. `znative load MenkeTechnologies/zshrs-revolver`.

`kubectl` is the native-completion case (Chapter 14): it delegates to cobra's `kubectl __complete`, so it tracks the installed binary's subcommands, flags, and live resources with no static tree to go stale. `znative load MenkeTechnologies/zshrs-kubectl-completion`.

These are not toy plugins chosen to be easy; they are tools with real per-call work — fuzzy matching, frecency scoring, parallel git classification, spinner animation, live cluster completion — where running as compiled machine code instead of re-interpreted shell text is measurable rather than cosmetic. And several of them lean

on the same property: **because zshrs is non-forking, a plugin's concurrent work is a thread, not a fork.** `revolver`'s animator and `git-repos`'s parallel classifier both share process memory instead of coordinating a forked child through a statefile — a model that is simpler *and* faster than the shell originals, and one only a compiled, in-process plugin can use.

And critically, they change nothing for the script ecosystem. Script plugins — oh-my-zsh, zinit bundles, any `.zsh` you `source` — keep working exactly as before, installed by the same `znative load` in the same `.zshrc`. A good stress test of that is `history-search-multi-word`, a script plugin `znative` installs unchanged: a ZLE widget with its own forked syntax-highlighter, paged `POSTDISPLAY` output, live `region_highlight`, and in-widget key reads — it installs, `znative loads`, binds `^R`, and runs under zshrs's ZLE with no port at all. The worked native ports sit alongside plugins like that as evidence of what the native path adds, not as a replacement for the script path anyone has to migrate to. You can run every native port and a hundred script plugins from one store, one index, and one block of `load` lines.

Chapter 19 – Designing a Plugin’s Command Surface

Chapter 12 covered the mechanics of registering a command and Chapter 16 covered translating an existing plugin’s commands. This chapter is the design question neither of them asks: given that you *can* register any name, which names should you register, and what should they do? The answers follow from where in dispatch a plugin command lives, and getting them wrong produces plugins that work perfectly in isolation and behave badly in a real shell.

Design around the dispatch position

The registry is consulted after real builtins and shell functions and before `PATH` (Chapter 10). Everything in this section is a consequence of that one sentence.

You can shadow an external program, and sometimes you should. A plugin that registers `git` intercepts every `git` invocation that would otherwise have found the binary on `PATH`. That is a legitimate and powerful thing to do — a compiled front-end that handles the hot subcommands itself and passes the rest through is exactly the sort of plugin the native path makes worthwhile. It is also a serious commitment: you have taken over a name the user’s entire toolchain, their scripts, and their other plugins invoke, and every behaviour of the real tool you do not reproduce is now a bug in yours. The escape hatch still exists — the real binary is still on `PATH` and still reachable by its absolute path — but you should not design assuming users will reach for it.

You cannot shadow a builtin, a keyword, or a user’s function, and you should not want to. A plugin registering `cd` does not get called; the real builtin resolves first. A plugin registering a name the user has defined as a function loses to the user’s definition. Both of those are correct precedence and both are features. The second one in particular is the user’s veto: anyone can override anything your plugin registers by defining a function with the same name, without unloading anything and without your cooperation. Design as though that veto will be used, because for any name common enough to be worth taking, it will be.

Avoid colliding with another plugin. The refusal that exists is on the *plugin* name — loading a plugin whose name is already loaded is rejected — and it is deliberately about identity rather than about the commands inside. Two plugins registering the same command name is a situation you should design out rather than resolve: prefix your commands, or use the internal-mode pattern below, or pick names specific enough that collision is implausible. The cost of a collision is a user who cannot tell which of two plugins answered, which is the same ambiguity the plugin-name refusal exists to prevent one level up.

Keep the public namespace small

The instinct when porting a plugin with many operations is to register a builtin per operation. Sometimes that is right — Chapter 16’s `forget` port registers nine, because the original exposed nine aliases and the muscle memory attaches to each of them individually. The test is whether the user types the name. If they do, it is a public name and it should be registered. If they do not, it should not be.

Chapter 17’s `git-fuzzy` port is the other pattern and it generalises well. The plugin registers one public name, `gf`, and dispatches internally on a `--helper` argument for everything `gfz` calls back into. Those helpers are numerous, they are implementation detail, and none of them is a command anyone would type. Registering

each as its own builtin would have polluted the global command namespace with a dozen names whose only caller is the plugin itself — names that would then show up in completion, shadow whatever they happened to collide with on `PATH`, and become a compatibility surface the author never intended to promise. One public name plus an internal mode keeps the promise small and the implementation as decomposed as it wants to be.

The same reasoning applies to completions. A plugin whose entire purpose is completing an existing external command registers a completion and *no* builtin at all — the `kubectl` case from Chapter 14. There is no command to add, because the command already exists and works; the plugin is adding knowledge about it. A plugin that registered a `kubectl` builtin in order to attach a completion to it would have shadowed the real binary as a side effect of a decision that had nothing to do with shadowing.

Behave like a builtin, not like a program

A registered command sits where a builtin sits, and users will treat it as one. That implies a set of conventions worth following deliberately.

Return a meaningful status. `0` for success, non-zero for failure, as the handler signature already suggests. This is not cosmetic: `$?` , `&&` , `||` , `if` , and every script that calls your command depend on it. A handler that returns `0` unconditionally has silently opted its command out of shell control flow.

Split stdout and stderr correctly. Data on stdout, diagnostics on stderr. The reason is stronger for a shell command than for a program, because shell users pipe things reflexively, and a diagnostic on stdout is a diagnostic that ends up in someone's variable. `host.print` writes to the shell's stdout, which is what you want for output; anything that is a complaint rather than a result should not go through it.

Do not narrate. No banner when the plugin loads, no progress commentary, no tips. The doctrine in Appendix H applies to plugins as much as to the manager, and the load path in particular is unforgiving: a plugin that prints on init prints on every shell start, forever, in every terminal the user opens.

Take arguments the way the shell already gave them to you. `args.rest()` is a `&[String]` that the shell has already word-split according to its own rules. Do not re-split it, do not re-quote it, and do not try to reconstruct a command line from it in order to parse it again. The most common porting bug in this area is rebuilding a string from the argument vector and handing it somewhere that splits it a second time — which works until an argument contains a space and then fails in a way that looks like a quoting mystery rather than a design error.

When not to write a builtin at all

Two cases are worth recognising before you start. If the thing you want is a transformation of shell state that the shell already exposes a good interface for — setting an option, defining a hook, binding a widget — a script plugin does it in one line and a native plugin does it through `eval` in slightly more than one. Compiling that is not wrong, but it is not buying anything, and Chapter 28's cost model says why: there is no per-call work to speed up, because it runs once.

And if the work is dominated by an external process — a command whose entire body is `exec` of something else with rearranged arguments — the native path is not the lever either. The cost is the child process, and a compiled wrapper spawns the same child. The ports in Chapter 18 were chosen because each has real per-call work of its own: fuzzy matching, frequency scoring, parallel classification, animation. That is the shape to look for when deciding whether a port is worth writing.

Your command names have no version gate

There is an asymmetry between the two contracts a native plugin participates in, and it is easy to miss because one of them is so thoroughly defended that you assume the other is too.

The ABI is version-gated on both sides: change the layout, bump `ABI_VERSION`, and a plugin built against the old shape is refused at load with a legible cause (Chapter 15). That machinery exists because a layout mismatch would otherwise be memory corruption, and it works.

The command surface your plugin registers has nothing like it. A user's `.zshrc` does not declare which commands it expects; a shell script that calls `ga` does not record which plugin it came from or which version of it. So when a plugin renames a builtin, removes one, or quietly changes what one does with its arguments, nothing anywhere refuses anything. The next `znative update` fetches the new version, the new store directory is installed, the index row is repointed, and the next shell loads a plugin whose commands are different. The first symptom is a user's script failing, or worse, a user's fingers producing a different outcome than they meant.

The manifest's `version` field does not help here, and it is important to see why rather than assuming it might. That version is the `<version>` in `store/<name>@<version>/` (Chapter 8) — it keys the directory and identifies the install. It is not compared against anything at load time, because nothing has declared an expectation for it to be compared against. It is an identifier, not a constraint.

What follows is a set of obligations that fall entirely on the plugin author, because there is no mechanism to enforce them.

Treat a registered name as permanent from the day you publish it. The names in your `builtins` map are the plugin's public API, and they are the *only* part of it a user interacts with directly. Chapter 16's rule that a port registers under the original's alias names is the same point from the other end: those names were the contract before the port, and the port's job was to keep them.

Add rather than rename. If a name was wrong, register the better name and keep the old one pointing at the same handler. The cost of an extra map entry is one line; the cost of a rename is every user who has to notice, diagnose, and fix something that used to work. This is the same reasoning Chapter 21 applies to the ABI — additive change is cheap, and the discipline is worth having even where no version constant is enforcing it.

Change behaviour more carefully than you change code. A handler that starts interpreting its arguments differently, or that changes its exit status convention, is a breaking change with no visible diff for the user. This chapter's earlier rule about returning a meaningful status is a compatibility rule as much as a correctness one: once `&&` chains and `if` statements in a user's scripts depend on your status, that status is part of your interface.

Use the pin, and tell users about it. The one mechanism that does exist is the `@ref` of Chapter 5, and it belongs to the user rather than the author. A plugin whose author documents “pin to `@v1` if you depend on the current command names” has handed the user the only lever available. Chapter 38's discipline — pin what your muscle memory depends on — is the user's side of exactly this gap.

Chapter 20 — Threads, Not Forks

Chapter 15 states the property in one line: plugin builtins run synchronously, in-process, even when backgrounded, because `zshrs` is non-forking. Chapter 18 gives two ports that exploit it. This chapter is the design consequences in full, because “you have threads instead of forks” is not merely a different spelling of the same capability — it changes what is easy, what is impossible, and what is newly your responsibility.

What the shell originals were actually doing

Start with why shell plugins fork in the first place. A shell function that wants to do something concurrently has exactly one primitive: `&`, which forks. Everything follows from that. The child gets a copy of the parent’s state at the moment of the fork and no way to share memory afterwards, so any coordination has to go through the filesystem or a signal. That is why `revolver` upstream writes a statefile keyed by `$PPID`: `start` forks an animator, `update` needs to tell that animator something, and a file is the only channel two processes with no shared memory have in common that a shell can use.

Look at what that costs. There is a fork per spinner. There is a file per shell, whose name has to encode which shell it belongs to, and which has to be cleaned up on exit or leak. There is a race between `update` writing the file and the animator reading it. There is a second race on `stop`, which has to both signal the child and remove the file. None of that complexity is about spinning; all of it is about the fact that the two halves of a spinner ended up in different address spaces.

The same shape shows up in `git-repos`. Classifying repositories in parallel from a shell means forking one `git status` per repository and collecting results — again through files or a pipe, again with the ordering and cleanup problems that implies. And in `git-fuzzy`, where a watcher process has to reach a running `fzf` and the only available channel is a network socket the watcher POSTs to.

What the native path replaces them with

In a compiled plugin, concurrency is `std::thread`, and the coordination channel is memory. `revolver`’s port keeps the animator as a thread and the running spinner in a `static` slot; `start`, `update`, and `stop` are three builtins reading and writing the same object. There is no statefile because there is nothing to serialise, no `$PPID` key because there is no second process to identify, and no cleanup on exit because the thread dies with the process that owns it. The entire category of bug the statefile design existed to manage is gone, not fixed.

`git-repos` gets the more familiar win: a worker pool over the repository list, with each thread running its own `git status` child and writing results into shared state. The children are still processes — you are still shelling out to `git`, which Chapter 16 says you should — but the *fan-out* is threads, so the collection is a join rather than a protocol.

And `git-fuzzy`’s watcher is a thread that writes an HTTP request to a raw `TcpStream`, which is what removes the `curl` dependency. Worth noticing what happened there: the socket did not go away, because `fzf`’s `--listen` port genuinely is a network interface and the plugin does not control that. What went away was the *external program* used to talk to it. The lesson generalises — the native path removes the process you were spawning to do a thing your own code can do, not the external interfaces you actually need.

What becomes your responsibility

A fork gives you isolation for free, and free isolation is what a thread takes away. Three obligations replace it.

State in `static` slots must be sound. A value that outlives a builtin call and is reachable from another one is shared mutable state in a process where multiple threads may be running, and Rust will require you to say how it is synchronised. This is a feature — the compiler is asking a question the shell version answered with a file and a hope — but it is a design decision you have to make deliberately rather than a detail you can defer. It also has a lifecycle dimension: the slot is initialised lazily, on first use, because there is no init hook that runs before the first command and no guarantee any particular command is first.

Threads must not outlive the library. This is the sharp edge and it deserves to be stated plainly. `Unload` purges the plugin's registrations and then drops the library, which is the `dlclose`; the purge-before-drop ordering guarantees that no *registered* function pointer survives its library. It says nothing about a thread you spawned, because the host does not know about your threads. A detached worker still running after `dlclose` is executing code in an unmapped region, which is not an error you will get a message about. If your plugin spawns anything long-lived, it needs a way to stop it, and stopping it is your code's job — a shutdown flag the worker checks, a channel it selects on, a join before you let go. The rule is simple: nothing of yours may still be running when your library goes away.

Panics behave differently depending on where they happen. Chapter 15's rule is that a panic across the C-ABI boundary is undefined behaviour — a handler that panics past its `extern "C"` frame is UB, not a caught error, and handlers must therefore catch their own or not panic. A panic *inside a thread you spawned* is a different situation: it unwinds that thread only and never reaches an `extern "C"` frame, so it is not UB. It is, however, a silently dead worker, which for a spinner means an animation that stops and for a worker pool means results that never arrive. The boundary rule protects the process; it does not protect your plugin's own behaviour, and those need separate attention.

The terminal is a shared resource

One more consequence of the shared address space, and the one most likely to produce a visibly broken plugin rather than a subtle one. Chapter 16's porting rule says interactive children inherit the tty — `fzf`, a pager, an editor take the terminal and repaint it. A background thread that writes to the terminal while a child owns it will interleave with the child's rendering, and the result is a corrupted screen.

This is a problem the forking design had a partial accident of a solution to: a forked animator writing to the same tty had the same collision, but shell plugins tend to structure themselves so that the spinner is stopped before anything interactive starts, because the statefile protocol made overlapping them awkward anyway. In a threaded plugin the overlap is easy, so avoiding it has to be a decision. Anything that animates, progresses, or logs to the terminal from a thread needs to know when the terminal is not its to write to, and the simplest correct answer is to stop it before handing the tty to a child and start it again after.

The related case is a long-running builtin that does no output at all. It is running in the shell's own process, on the thread the user's command line is executing on, which means for as long as it runs the shell is doing that and nothing else. A builtin that blocks for a long time is a shell that appears hung. Structure long-running work so it makes progress in observable increments, or so it can be abandoned, rather than assuming a user will wait — the non-forking model gives you the whole process, and giving it back promptly is part of the deal.

Chapter 21 – Evolving the ABI

A published ABI is a promise, and the difficulty of keeping it starts the moment strangers begin compiling against it. This chapter is the set of rules that govern change at the boundary — why an additive field still forces a version bump, why the check requires equality rather than compatibility, which parts can never change at all, and what a plugin author should conclude about how to depend on the crate.

Why appending a field is a breaking change

`HostApi` grew twice in the history visible in the crate: `register_completion` at v2, `getfunction` and `addfunction` at v3. Both were purely additive — no field moved, no signature changed, nothing was removed. The naive conclusion is that neither needed a version bump, and the naive conclusion is wrong in a way worth working through, because it is the reason the version constant exists at all.

The struct is `#[repr(C)]`, so a plugin compiled against v2's definition knows the offsets of every v2 field and knows the struct's v2 size. Hand that plugin a v3 table and nothing bad happens *as long as it only touches v2 fields*: the offsets it knows are still correct, because v3 appended rather than inserted. This direction is benign.

Now reverse it. Hand a plugin compiled against v3 a table that a v2 host allocated. The plugin believes there is a `getfunction` pointer at an offset past the end of what the host actually allocated, and reading it reads whatever memory follows the table. Then it calls through it. This is not a detectable failure — it is a call to an address derived from unrelated bytes, and there is no version of that story where you get a clean error.

So the two directions are not symmetric: growth is safe forwards and catastrophic backwards. A check that permitted “plugin version $\leq$ host version” would be sound for `HostApi` alone, but the boundary is not `HostApi` alone — `PluginInfo` and `BuiltinFn` are also part of the frozen contract, and a change to either of those has no safe direction. Rather than maintain a per-type compatibility matrix that a future maintainer must reason about correctly every time, the design uses one integer and requires equality. It is stricter than strictly necessary in one direction, and that strictness is what makes it impossible to get wrong.

Why both sides check

The host compares the plugin's compiled-in version against its own; the plugin's `declare_plugin!` glue independently checks the `abi_version` field of the table it was handed before it trusts any other pointer in it. Two checks for one mismatch looks redundant and is not, because the two checks catch the situation from different positions in time.

The host's check happens after `dlopen` and after the plugin's `znative_init` returned a `PluginInfo` — which means the plugin's init already ran. The plugin's own check happens at the very start of that init, before it has dereferenced anything else in the table. If the host is older than the plugin, the plugin's check is the one that fires, and it fires before the plugin can call through a pointer that is not there. If the plugin is older or lying about its version, the host's check is the one that fires. Neither check alone covers both, and the cheap way to cover both is to do both.

The field ordering follows from this directly. `abi_version` is the first field in `HostApi`, and its offset is the one thing in the entire design that must never change, because it is the field you have to read *in order to know* whether the rest of the struct is what you think it is. A version field at the end would be unreadable without already trusting the layout you are trying to validate.

The parts that can never change

Two more things are frozen harder than the version constant can protect, and it is worth being explicit about which.

`INIT_SYMBOL` — the NUL-terminated name `znative_init` — is resolved by `dlsym` before any version check is possible, because the version is inside the thing you have not called yet. If that name changed, an old plugin loaded by a new host would fail symbol resolution, which is at least a clean failure; but the failure would have no version information in it and no way to say *why*, because the mechanism that carries the version never ran. The name is therefore permanent in a way the struct layouts are not.

`InitFn`'s signature is frozen for the same reason and more sharply. The host resolves `znative_init` and calls it as `extern "C" fn(*const HostApi) -> *const PluginInfo`. If that signature ever changed, the host would call an old plugin's init with the wrong calling convention — and unlike a struct-layout mismatch, this one cannot be checked first, because performing the check requires making the call. The pointer types on either side of that signature can evolve behind their version gate; the shape of the call itself cannot.

What a change costs, and who pays

Every ABI bump means every plugin must be rebuilt against the new crate. That is the price of a flat, layout-frozen, equality-checked boundary, and it is a real price paid by every plugin author each time the surface grows.

The alternatives all cost something else. A self-describing table — where the host advertises which entries it has and the plugin queries before using them — removes the rebuild requirement and replaces it with a runtime negotiation on every optional call, plus the permanent obligation to keep every retired entry working, plus a class of bug where a plugin silently degrades because a capability it wanted was absent. A symbol-per-function design, where the plugin links against symbols the host exports, inverts the dependency and makes the host's symbol table the ABI — which is approximately what a zsh C module does, and Chapter 1 is about why that ends badly. A serialised boundary, where calls become encoded messages, buys version tolerance and pays for it with allocation and encoding on every call, which is the one cost a native plugin exists to avoid.

Against those, “rebuild when the version changes” is the honest trade: the cost is visible, it is paid at build time rather than per call, and its failure mode is a refusal to load rather than a subtle degradation. It also keeps the boundary readable — Appendix I fits on a few pages, and a plugin author can hold the whole contract in their head, which is itself a durability property.

What this means for depending on the crate

Chapter 15's practical rule — keep the crate's major/minor aligned with the target zshrs release — is the operational consequence of everything above, and there are two ways to satisfy it that suit different plugins.

A plugin distributed to users installing from source, built by `znative` as part of the install, is best served by a `crates.io` dependency with a version that matches the zshrs release it targets. The build happens on the user's machine against the crate the user's cargo resolves, and a version constraint is how the plugin says which host generation it is for.

A plugin developed alongside a moving zshrs — during a port, or against an unreleased change — is better served by the git dependency form Chapter 12 mentions, because it tracks the host as it moves rather than against a published snapshot. The tradeoff is the usual one for git dependencies: you get currency and you give up the reproducibility a version constraint provides.

The failure mode in both cases is legible, which is the whole point. A plugin whose crate is too far from its host either fails to compile — Chapter 25's compile-time shadow — or fails to load with a version mismatch. Neither

of those is a plugin that loads and then misbehaves, and turning an entire class of latent memory-corruption bug into a build error or a refusal is what the version constant buys for its one integer.

Chapter 22 – The Install Pipeline, Step by Step

The previous chapters described the install in pieces — resolution here, detection there, the store somewhere else. This chapter walks one install end to end, in order, and says what exists on disk after each stage. The reason to do it that way is that the *ordering* is a design artifact, not an implementation detail: nearly every property the manager offers at the edges — that `gc` is safe, that the index never dangles, that the integrity hash means what it claims — is a consequence of which step happens before which.

Take a concrete invocation:

```
znative add MenkeTechnologies/zshrs-revolver@v0.2.0
```

Stage 1 — classify. The argument is inspected and split into a GitHub source and a `v0.2.0` pin. This stage is a pure function of the string: no network, no filesystem, no state. Its errors are total and cheap — a malformed spec fails before anything has been touched, and the failure costs nothing to recover from because nothing happened. It is worth noticing that this is the only stage that *can* be pure, and it was designed to be: everything downstream depends on knowing the shape of the source, so getting that wrong late would be expensive.

Stage 2 — stage the tree. For a remote source this is a shallow clone into the store's `git/` scratch directory, `--depth 1 --branch v0.2.0`, with the full-clone fallback if the pin names a commit a shallow branch fetch cannot reach. For a `path:` source there is no clone at all; the directory on disk is used in place. This is the first stage that can fail for reasons outside the user's control — network, authentication, a tag that does not exist — and the first that leaves debris. What it leaves is a directory under `git/`, which is scratch by definition: nothing loads from it, nothing lists it, and `znative clean` empties it. The store is untouched and the index is untouched.

Stage 3 — detect the kind. The staged tree is read. If it carries a `znative.toml`, that manifest decides; otherwise the native signals are checked and then the script signals, in that order (Chapter 7). This stage also fixes the plugin's **identity** — name and version, from the manifest if present, inferred from the repository basename if not. That is the reason the pipeline cannot pre-allocate a store directory: `store/<name>@<version>/` is not knowable until the tree has been fetched and read. A manager that wanted to allocate the destination first would have to trust the source spec to predict the plugin's name, which for ported plugins is exactly the assumption that does not hold.

Stage 4 — build, for a native plugin only. If a prebuilt `lib*.{dylib,so}` is already at the tree root, this stage is skipped entirely and the artifact is used as found. Otherwise `cargo build --release` runs in the staged tree, and on success the freshly built library is copied out of `target/release/` up to the tree root. That copy looks redundant until you remember the next stage: the store copy deliberately excludes `target/`, so a library that stayed in `target/release/` would be excluded along with it and the store would hold a native plugin with no library in it. Promoting the artifact to the root is what makes it survive the strip.

This stage is also where the two explicit build errors live — a native plugin with no prebuilt library and building disabled, and a native plugin with neither a library nor a `Cargo.toml`. Both are stated rather than papered over, because both are configuration mistakes with exactly one correct fix, and a manager that guessed here would produce a plugin that installs and then does not load.

Stage 5 — strip and copy. The loadable subset of the staged tree is copied into `store/<name>@<version>/`, with `.git/` and `target/` dropped. Two things are happening at once. The copy is what makes the store *independent* of the staging area, which is why the clone can be thrown away afterwards; the strip is what

keeps the store proportional to what actually loads rather than to what a repository happens to carry. For a Rust plugin those are not close in size: a `target/` directory after a release build dwarfs the library it produced.

This is the failure window that matters, and it is the one the ordering is built around. If the process dies here, a partial `store/<name>@<version>/` exists and no index record does. That directory is inert — nothing can find it, because finding things is the index's job — and `gc` will reclaim it, because `gc` deletes exactly what the index does not name. The failure mode is a small amount of wasted disk that a housekeeping verb already knows how to clean up.

Stage 6 — hash. SHA-256 is computed over the store copy and formatted as `sha256-<hex>`. The position of this stage is load-bearing in two directions. It is after the build, so the hash covers the artifact that will actually be loaded rather than the source that produced it — which is the only version of the claim that is useful, since the loaded thing is the thing you care about the identity of. And it is after the strip, so the hash is stable against the two directories that are the least stable parts of any tree: `.git/` changes with every fetch and `target/` changes with every build. Hashing before either of those steps would produce a number that changed for reasons unrelated to the plugin.

Stage 7 — record. The `[[package]]` record is written to `installed.toml`: name, version, the recorded source label with its pin, kind, integrity hash, and either the `cdylib` to load or the source files and `fpath` directories. This is the moment the plugin becomes *installed*, in the only sense the word has here. Everything before this was preparation that left recoverable debris; from this point forward `list` reports it, `info` describes it, `load` finds it, `gc` protects it, and `remove` is the way to undo it.

Stage 8 — load. For a script plugin, `znative` synthesises the small shell fragment described in Chapter 7 — `fpath=(...)` prepends followed by `source` calls, in the recorded order — and evaluates it on the live shell. For a native plugin, it is `zmodload -R` on the recorded library: `dlopen`, resolve `znative_init`, call it, register the builtins. Then the one line of output the install path emits, and the prompt comes back.

The ordering, stated as an invariant

Everything above collapses to a single rule: **the index record is written last, after every step that can fail.** The consequences are worth stating on their own, because they are what you actually rely on day to day.

There is no window in which an index record points at a store directory that does not exist or is incomplete. That is the dangerous direction — a `load` that resolves to a broken path, in a `.zshrc`, on shell start — and the ordering eliminates it by construction rather than by checking for it. There *is* a window in which a store directory exists with no record, and that direction is harmless and self-cleaning. The pipeline trades a garbage-collection problem for a correctness problem and takes the garbage-collection problem, which is the right trade because `gc` already exists for another reason.

It also means there is nothing to roll back. A failed install does not need to undo anything, because it never committed anything: the scratch clone is scratch, the partial store directory is unnamed, and the index was never touched. This is why `znative` has no transaction machinery, no journal, and no repair verb. The absence of those things is not a gap; it is what you get for free when the only durable write happens once, at the end, after everything that could go wrong already has not.

And it explains why `load`'s fast path is as fast as it is. A `load` of a plugin already in the store runs none of stages 1 through 7. It reads the index, matches on name or recorded source, and goes straight to stage 8. The entire install pipeline is a cold-start path that a `.zshrc` line traverses exactly once in its life.

Chapter 23 — The Build Step

The native path has one step the script path does not: for a Rust plugin that ships source rather than a prebuilt library, `znative` compiles it. Chapter 7 named that step in a sentence — `cargo build --release` when a `Cargo.toml` is present and `build` is not disabled — and this chapter is the whole of it, because the build is the one place the native path asks something of the host that the script path never did, and it is worth knowing exactly what it asks and when.

When the build runs, and when it does not. The decision is the `build` flag from Chapter 7, and its default is the useful one: `true` whenever a `Cargo.toml` is present. So the ordinary Rust plugin — source in a repo, no library committed — builds on install with no configuration. The two ways out of a build are both explicit. A plugin that ships a prebuilt `lib*.{dylib,so}` at its tree root is used as-is and never compiled — `znative` finds the library and skips cargo entirely. And a plugin that sets `build = false` in its `[native]` table without shipping a prebuilt library is a declared error, not a silent no-op: `znative` says “no prebuilt cdylib and build is disabled” and stops, because the one thing it must not do is register a native plugin with nothing to load.

It builds once, at install, never at load. This is the same discipline the whole native path is built on — resolve once, run forever. The build happens inside `znative add/load`’s install branch or inside `znative update`, and its output is one `cdylib` copied up to the store. From then on every `load` is a `dlopen` of that already-built, already-mmap’d library; the compiler is not on the load path at all. A plugin you use every session compiles on the session you first install it and never again until you `update`. That is why the build is `--release` and not a debug build: you pay the optimizing compile exactly once, at install, and collect faster machine code on every load after — the opposite of the trade a script plugin makes, where the “compile” is a re-parse paid on every start.

Where the built library goes. Chapter 6’s store deliberately strips `target/`, so a `cdylib` left in `target/release/` would not survive into the store. The build step closes that gap: after a successful `cargo build --release`, `znative` copies the freshly built library out of `target/release/` up to the tree root, and *then* the store copy runs over a tree whose library is at the root where kind detection and the index both expect it. The sequence is fixed — build, hoist the library to the root, copy the loadable subset into `store/<name>@<version>/`, compute the integrity hash, write the index record — and nothing is recorded until the copy and the hash succeed, so a build that fails leaves no live index row.

The one prerequisite it adds. A native-from-source plugin needs a Rust toolchain on the machine that installs it, because that machine is the one that runs `cargo build`. This is the single thing the native build path asks that the script path does not, and it is asked of the *installing* host, not the plugin author’s — which is the point of the next chapter. When the toolchain is absent the build fails and `znative` reports it the same terse way it reports everything else: `znative: <reason>` on stderr, non-zero exit, no lecture. The build’s own chatter — cargo’s compile progress — follows zshrs’s doctrine and goes to the log under `~/.cache/zshrs/`, not to the terminal; a first install compiles quietly and the prompt is not decorated with a build transcript. A plugin that ships a prebuilt library, or a script plugin, needs no toolchain at all; the requirement lands only on the native-from-source case, exactly where it is unavoidable.

Rebuilds, and why the store carries no build cache

`znative update` re-resolves the recorded source and reinstalls, and for a native-from-source plugin that means it builds again. There is no incremental build cache carried between installs, and the reason is Chapter 6’s store: it strips `target/`, so the intermediate artifacts a warm `cargo` build would reuse are deliberately not kept. Each build therefore starts clean. That is a considered trade, not an oversight — a clean build from the

pinned source is reproducible and self-contained, and the cost is a full compile paid once per **update**, on a plugin you update rarely and load constantly. Carrying a **target/** cache into the store to shave seconds off an occasional update would bloat every installed plugin with build scratch it never loads, which is the opposite of the small, self-contained **store/<name>@<version>/** the store is designed to be.

A prebuilt-library plugin skips all of this on **update** too: it re-fetches the recorded source, and if that source carries a newer prebuilt **lib*.{dylib,so}** it adopts it with no compile, exactly as it did on first install. The build step is consistent across install and update — it is a property of *what the source contains*, source or binary, not of *which verb* triggered it.

Chapter 24 – Cross-Architecture Native Plugins

zshrs is a cross-architecture shell from day one — macOS `aarch64`, Linux `x86_64`, Linux `aarch64`, with more to come — and a compiled `cdylib` is, by its nature, none of those things portably: a shared library built for Apple Silicon does not load on an `x86_64` Linux box, and no amount of ABI stability changes that, because the ABI in question is `znative`'s struct layout, not the machine's instruction set. A native package manager has to have an answer for that, and `znative`'s answer falls directly out of the build step of the previous chapter.

Source is portable; the artifact is not — so `znative` distributes source and builds locally. This is the whole of it, and it is why `build = true` is the default rather than an option. The unit a plugin repository ships is Rust source plus a `Cargo.toml`; that source is architecture-neutral. The *artifact* — the `cdylib` — is produced on the machine that installs the plugin, by that machine's own toolchain, for that machine's own target. A `store/<name>@<version>/` on a Mac holds a `lib*.dylib` built for `aarch64`-Darwin; the same plugin installed on an `x86_64` Linux server holds a `lib*.so` built for that target; neither store ever sees the other's binary. The content-addressed store is per user *and per machine* — one store under that host's `$ZSHRS_HOME/pkg/` — so the question “will this binary load here” is answered by construction: the binary in a store was built by the host that owns the store, for the host that owns the store.

The platform library shape is already the partition. Chapter 6 located a native plugin's library by the platform's prefix and suffix — `lib*.dylib` on macOS, `lib*.so` on Linux. That is not only a naming convention; it is the seam that keeps a foreign binary from being mistaken for a loadable one. A Linux host looks for `lib*.so`; a repository that committed only a prebuilt macOS `lib*.dylib` presents *nothing* the Linux host will accept as its library, so the host does not try to `dlopen` a Mach-O object on ELF and fail deep in the dynamic linker. Instead detection falls through to the `Cargo.toml`, and the plugin builds from source for the host it is on — the correct outcome, reached without a special case.

The one hazard is a prebuilt-only, single-arch repository. A repository that ships *only* a prebuilt library for one architecture and *no* `Cargo.toml` is the single case cross-architecture cannot rescue, and the design does not pretend otherwise: on a host of the matching architecture it loads, and on any other host there is no source to build from and no library of the right shape, so `znative` reports that it cannot produce a loadable plugin and stops. The lesson for a plugin author is the one the build step already implied — **ship source, not a binary**. A plugin that ships its `Cargo.toml` installs correctly on every architecture zshrs runs on, because every one of those hosts builds it for itself. A plugin that ships a prebuilt binary trades that universality for skipping a compile, and the trade is only worth it inside a known-homogeneous fleet.

Why this is the right model and not a limitation. A package manager could instead fetch per-architecture binaries from a registry keyed by target triple — the model a language ecosystem with a central binary cache uses. `znative` does not, and the reason is the same reason it has no lockfile and no registry (Chapter 31): it has no central authority to fetch binaries *from*, and building locally needs none. Local build makes every host self-sufficient — an offline `aarch64` server with a toolchain installs a native plugin from a `path`: source with no network and no trusted binary cache anywhere in the loop. The cross-architecture story is not bolted on; it is what “distribute source, build on install” gives you for free, and it is why that was the default from the start.

When a prebuilt binary is the right call

The rule “ship source” is the default, not an absolute, and the exception is worth stating so it is chosen deliberately rather than stumbled into. Inside a *homogeneous fleet* — a set of machines of one known

architecture, built and administered together — a prebuilt binary is a legitimate optimization. If every target is Linux `x86_64` and a build server already produces the `cdylib`, committing that prebuilt `lib*.so` and setting `build = false` lets each host skip the compile and the toolchain requirement entirely: the install is a copy, not a build, and no machine in the fleet needs `cargo`. The `path:` source form pairs naturally with this — an internal mirror or a shared filesystem holds the tree with its prebuilt library, and hosts install from it offline.

What makes this safe rather than a return to the welded-C-module problem is that the trade is now *explicit and scoped*. The plugin author who commits a binary is declaring “this artifact matches these hosts,” and the `lib*.so` / `lib*.dylib` shape still refuses to load it on the wrong OS. The hazard only appears when a single-arch binary escapes its fleet — published to strangers on other architectures with no `Cargo.toml` fallback. The portable default and the fleet-local exception are the same rule read at two scales: distribute the form that every intended host can load, which is source for the open world and a matched binary inside a controlled one.

Chapter 25 – When It Fails

A tool with a terse error contract owes its user a map. `znative: <reason>` on stderr and a non-zero exit is the right output for someone who knows what went wrong; this chapter is for building the model that makes the one-liner sufficient. Each failure class below is described by what produces it, what state it leaves, and what the correct response is.

Resolution failures

A source that does not exist, or cannot be reached. A misspelled slug, a private repository without credentials, a host that is down, a network that is not there. `znative` surfaces git's failure rather than paraphrasing it, which is the correct behaviour for a class of error where the underlying tool's message is strictly more informative than any summary of it. State left behind: possibly a partial clone under `git/`. Response: fix the spec or the network and re-run; `znative clean` if the scratch is bothering you.

A pin that cannot be reached. A tag or branch that does not exist fails outright. A *commit* that a shallow branch clone cannot see is the one case that does not fail, because the full-clone fallback exists precisely for it — the install gets slower and still succeeds. This asymmetry is worth internalising: pinning to a tag or branch is always cheap, pinning to an old commit may cost a full history fetch, and neither is ever a hard failure for reachability reasons alone.

A tree that cannot be classified. No `cdylib Cargo.toml`, no prebuilt library, no `*.plugin.zsh`, no `functions/`, no loose `*.zsh`. `znative` says so rather than guessing. This one is almost always a real signal about the repository rather than about your invocation — you have pointed at something that is not a plugin, or at a monorepo whose plugin lives in a subdirectory, or at a repository whose layout genuinely needs a `znative.toml` it does not ship. The response is a manifest, or a more precise source.

Build failures

No prebuilt library and building disabled. An explicit error with an explicit message, and a configuration mistake rather than an environment problem: something declared `build = false` and then did not ship the artifact that decision assumed. Either ship the library or turn the build back on.

No library and nothing to build from. The other half of the same pair: the tree was classified native — because a manifest said so — and there is neither an artifact nor a `Cargo.toml`. The manifest and the tree disagree, and the manifest wins loudly instead of silently.

cargo missing, or the build failing. `znative` needs `cargo` on `PATH` for native plugins that ship as source; that is a stated prerequisite, and its absence is not something the manager can work around. A build that runs and fails is more interesting: the failure is the crate's, and cargo has already said everything useful about it on its own stderr. This is the same reasoning as the git case — paraphrasing another tool's diagnostics loses information and adds nothing. State left behind: a staged tree with a `target/` directory in it, under scratch. Nothing in the store, nothing in the index.

The most common real-world cause of a build failure that is not the crate's fault is ABI skew at compile time: the plugin's `znative` dependency and the running `zshrs` disagree enough that the crate no longer compiles against the version cargo resolved. That is the compile-time shadow of the load-time version check, and the fix is the same one Chapter 15 states — align the crate version with the target `zshrs` release.

Load failures

ABI version mismatch. The plugin compiled against one `ABI_VERSION` and the host has another. The load is refused, from whichever side notices first: the host checks the plugin's version, and the plugin's `declare_plugin!` glue checks the host's `abi_version` field before trusting any other pointer in the table. This is the failure the whole scheme exists to produce, and it is worth appreciating as a success rather than an error — the alternative outcome, in a world without the check, is not a message but a corrupted process. Response: rebuild the plugin against the matching crate version.

A plugin name already loaded. Refused, deliberately, rather than resolved by some precedence rule. Two plugins registered under one name would make every command that name resolves to ambiguous, and ambiguity in dispatch is worse than a refusal at load. Response: `zmodload -uR <name>` first, then load.

A script plugin that errors when sourced. This one is not `znative`'s failure at all, and recognising that saves time. Once the synthesised `fpath` and `source` fragment is evaluated, the plugin's code is running in your shell exactly as if you had sourced it by hand, and any error it produces is the plugin's error with the plugin's wording. `znative`'s job ended when the fragment was handed to the shell. The diagnostic move is to source the store copy by hand — `znative info` gives you its path — and see the same failure with `znative` out of the picture.

Symptoms that are not errors

Two common problems produce no error message at all, which makes them worth naming.

A completion plugin that silently does not complete. Almost always the load-order rule from Chapter 14: the plugin was loaded before `compinit`. The deferred `compdef` wiring has nothing to attach to, and nothing fails loudly, because nothing failed — the plugin loaded fine and registered its generator, and the completion system was simply not there yet to receive it. Move the `znative load` line after `autoload -Uz compinit; compinit`.

Syntax highlighting that stops working after adding a plugin. The other load-order rule from Chapter 9: `zsh-syntax-highlighting` wraps the line editor and has to see the final state of every widget, so it goes last. A new `load` line appended below it is the usual cause. `znative` does not reorder your lines, which is the property that makes this diagnosable — the order in the file is the order of the load, so the file is the whole explanation.

Reconciling the store

When the store and your expectations diverge, three verbs do all the work. `znative list` says what the index believes is installed. `znative info NAME` gives the record for one plugin — the store path to look at, and the `sha256-` hash to compare. `znative gc --dry-run` lists the store directories the index no longer pins, which is how you see accumulated orphans before deciding to reclaim them.

The one reconciliation `znative` does not offer is a repair verb, and its absence follows from Chapter 6's authority rule. There is no meaningful "repair," because there is no state a repair could recover: a store directory the index does not name is garbage by definition, and `gc` already deletes it; a plugin whose store copy is damaged is fixed by `znative update`, which re-resolves from the recorded source and reinstalls. Between `gc` for what should not be there and `update` for what should be there and is not right, the space of recoverable problems is already covered by verbs that exist for other reasons.

Chapter 26 – Reconciliation: When the Index and the Store Disagree

`installed.toml` is the source of truth (Chapter 6), and that sentence is a design decision with consequences worth following all the way down, because the moment you name one of two representations authoritative, you have to say what happens when they disagree — and a package manager that survives real machines, interrupted installs, and a user with `rm` in their fingers will see them disagree.

The index is the authority; the filesystem is its cache. Every read that answers “what is installed” — `list`, `info`, a bare `load` — comes off the index, not off a directory scan. So the store is, in effect, a content cache the index points into, and reconciliation always runs in one direction: the index says what should be there, and the store either has it or does not. This is why `znative list` reflects what `znative` believes rather than what happens to be sitting in `store/`, and it is the frame for every disagreement below.

A row with no directory. If the index names a plugin whose `store/<name>@<version>/` is gone — deleted by hand, lost to a botched backup — a `load` of it fails cleanly rather than half-loading: there is nothing to `source` or `dlopen`, and `znative` reports the failure the terse way. The repair is ordinary: `znative remove` drops the dangling row, or `znative update` re-resolves the recorded source and reinstalls the content the row promised. What `gc` will *not* do is help here — `gc` only ever deletes `store/` directories the index does *not* name, so it cannot touch a row at all, and a row pointing at a missing directory is exactly the kind of state `gc` is designed to be unable to worsen.

A directory with no row. The mirror case is the common one, and it is by design. An `update` to a newer version leaves the previous `store/<name>@<old>/` in place — nothing is deleted out from under something that might still reference it — so old versions accumulate as directories the index no longer pins. Those orphans are precisely what `gc` reclaims: it walks the store, removes every `store/<name>@<version>/` the index does not name, and clears the `git/` clone cache, and because the index is the authority on what is live, it can never remove a version you still have installed. A store directory with no index row is not corruption; it is garbage in the specific, reclaimable sense, and `gc --dry-run` shows you the list before it acts.

Why a half-finished install cannot leave a live lie. The install order from Chapter 6 is what makes all of this hold: copy the loadable subset into the store, compute the integrity hash over that copy, and only *then* write the index record. An install killed partway — a lost connection, a `^C`, a full disk — dies before the record is written, so it leaves at most an unreferenced directory (which `gc` reclaims) and never a live index row pointing at a partial tree. The index never promises content that was not fully staged, because the promise is the last thing written, not the first. Content addressing closes the loop: if a store copy is ever silently altered after that record is written, its bytes no longer match the recorded `sha256-<hex>`, and the mismatch is detectable rather than invisible.

The reconciliation rule, stated once. Trust the index; treat the store as its cache; repair a missing directory with `update` or clear the stale row with `remove`; reclaim an unpinned directory with `gc`; and rely on the write-record-last discipline and the integrity hash to keep an interruption or a tamper from ever becoming a silent inconsistency. There is no separate `fsck` verb and no repair database, because the index already *is* the database and the four existing verbs already *are* the repair tools — which is the whole reason the tool keeps its surface as small as Chapter 4 describes.

A worked disagreement

Make it concrete. Suppose you are clearing disk and `rm -rf ~/.zshrs/pkg/store/forgit@0.1.0` by hand — deleting a plugin's content without telling `znative`. The index still carries the `forgit` row, so `znative list` still shows `forgit` as installed: `list` reads the index, and the index has not changed. The disagreement surfaces the next time the plugin is actually needed. A `znative load forgit` finds the row, resolves it to a `store/` directory that is no longer there, and fails cleanly — `znative: <reason>` on stderr, non-zero — rather than half-loading a plugin whose files are gone. Nothing is corrupted; the shell simply reports that the content the index promised is missing.

The repair is one verb, and which one depends on intent. If you want `forgit` back, `znative update forgit` re-resolves its recorded source and reinstalls the content the row describes — the directory returns, its integrity hash is recomputed, and `load` works again. If you meant to be rid of it, `znative remove forgit` drops the dangling row and the plugin is gone from the index too, leaving nothing behind. Either way `znative gc` is irrelevant to this case: `gc` only ever deletes store directories the index does *not* name, so a missing directory for a named row is outside what it can touch. The whole episode is a single application of the reconciliation rule — the index is the truth, the store is its cache, and a lost cache entry is either refilled or its record retired.

Chapter 27 – Living With the Store

The store is a directory that accumulates. This chapter is about what happens to it over months of use — how versions pile up, why nothing is deleted automatically, how pins behave across updates, and the two workflows that involve the store most directly: developing a plugin, and running with no network at all.

Why `update` leaves the old version behind

`znative update` re-resolves from the recorded source and installs the result. If the result is a new version, it lands at a new `store/<name>@<version>/` and the index row is repointed. The previous directory stays.

That looks like a leak and is a safety property. Consider a native plugin loaded in the shell you are typing in: its `cdylib` is `dlopened` and mapped into the running process. Deleting the file out from under a live mapping is precisely the hazard the whole unload path is built to avoid — Chapter 10's `purge-before-dlclose` ordering exists so that no live pointer survives its library, and an `update` that deleted the old store directory would reintroduce the problem from the other direction. Worse, the shell that has the old version mapped may not be the shell that ran the update. In a workflow with many concurrent shells, an update in one is an unannounced deletion under all the others.

So `update` adds and repoints, and reclamation is a separate, explicit act. This is the same shape of decision as writing the index record last: prefer a garbage-collection problem to a correctness problem, and give the garbage collector a verb.

`gc`, and why it can be unguarded

`znative gc` removes `store/<name>@<version>/` directories the index no longer pins and clears the `git/` clone cache. It is safe to run at any time without thinking about it, and the safety is structural rather than heuristic: the index names every live version, so “not named by the index” is exactly the set of directories nothing can reach. There is no judgment call, no age threshold, no “probably unused” inference. A manager that had to guess would need a confirmation prompt, and the doctrine forbids confirmation prompts — which means the design had to make the guess unnecessary rather than make the prompt tolerable.

`--dry-run` exists for the human, not for safety. What it answers is “how much has accumulated and from which plugins,” which is a question about your own update habits rather than about whether the deletion is correct.

`znative clean` is the smaller hammer: it empties `git/`, `cache/`, and `bin/` and leaves store and index alone. The distinction is that `clean` touches only things that were never part of any plugin's installed footprint, so it cannot affect anything's ability to load. `gc` touches store directories, which requires the index argument above; `clean` requires no argument at all because scratch is scratch.

Pins over time

An unpinned source tracks whatever the default branch resolves to at fetch time. `znative update` on it fetches again and may get something new — that is what unpinned means, and it is the right default for the many plugins whose authors release by pushing to their main branch.

A pinned source does not move. `update` on `owner/repo@v1.2.0` re-fetches `v1.2.0`, which normally reproduces the same tree, the same hash, and the same store directory. The update is a no-op, and that is the correct outcome for a pin — a pin whose whole purpose is stability would be worthless if the verb named “update” quietly broke it. The pin lives in the recorded source label (Chapter 5), so it survives independently of whatever you typed most recently.

The practical pattern that falls out: pin the plugins whose behaviour you depend on and leave the rest unpinned. A ZLE widget that your muscle memory is built around is a pin candidate; a completion you would happily take the newest version of is not. Changing your mind in either direction is a re-add of the source with or without the `@ref`, which rewrites the recorded label.

Developing a plugin against the store

The `path:` source exists for this, and it has one property that surprises people exactly once: **the install copies**. `znative add path:~/src/my-plugin` stages the directory in place, then copies the loadable subset into `store/<name>@<version>/`, and from that moment the store copy is what loads. Edits to your working tree change nothing about the installed plugin until you install again. That is not an oversight — a store whose contents changed under it whenever someone edited a source directory would have an integrity hash that meant nothing and a `list` that described a moving target.

The consequence is that the store is the wrong tool for a tight edit-compile-test loop, and the right tool is the one Appendix F documents. During development, build your `cdylib` with cargo and load it directly:

```
zmodload -R ./target/release/libmyplugin.dylib
# ... test ...
zmodload -uR myplugin
```

That is the loop: build, load, exercise, unload, edit, repeat. No store, no index, no copy. The package manager enters the picture when you are ready to install the thing you built, and the `path:` source is how you install it without pushing it anywhere first. A `git+` or `github:` source is for after the repository is somewhere a stranger can fetch it.

Note the unload half of that loop is not optional. Loading a plugin whose name is already loaded is refused, so a second `zmodload -R` after a rebuild fails unless the first one was unloaded — which is the behaviour you want, because the alternative is a shell in which some commands come from the library you just built and others from the one you built ten minutes ago.

Running offline

Everything about the store is designed to make the offline case ordinary rather than special. Two facts do most of the work: `path:` sources touch the network not at all, and a plugin already in the store loads with no network on every subsequent start. An environment with no outbound connectivity can therefore install from directories that are already on the filesystem — a vendored bundle, an internal mirror synced by some other mechanism, a checkout copied in — and everything after that install behaves identically to the connected case.

For native plugins there is a second lever. A plugin that ships a prebuilt `lib*.{dylib,so}` and turns `build` off in its `[native]` table installs without cargo, without a toolchain, and without fetching a single crate. That is the shape of a plugin intended for machines that are not build machines, and the manifest flag exists precisely so the choice is the plugin’s to make rather than the installer’s to work around.

What actually accumulates, and how fast

It is worth having a rough model of where the disk goes, because “the store accumulates” is true and unhelpfully vague, and the answer differs sharply between the two kinds of plugin.

A script plugin's store directory is its repository minus `.git/`. For most plugins that is a handful of `.zsh` files and possibly a `functions/` directory — small enough that the count of them stops mattering long before the bytes do. Stripping `.git/` is doing real work here, since for an old repository the history is frequently larger than the tree, but the residual is small either way.

A native plugin's store directory is its source tree minus `.git/` and `target/`, plus the built `cdylib` hoisted to the root (Chapter 23). The strip is doing much more work in this case: a Rust release build's `target/` is dominated by intermediate artifacts and dependency rlibs, none of which the store keeps. What remains is source plus one shared library, which is a different order of magnitude from what the build directory held a moment earlier.

The thing that grows without bound is neither of those; it is *versions*. Every `update` that lands a new version adds a store directory and orphans the old one (Chapter 26), and orphans are only reclaimed when you ask. A plugin updated on a regular cadence for a year is a directory per landed version, all but the current one unreachable by anything. That is the accumulation `gc` exists for, and `gc --dry-run` is how you see the shape of it before deciding whether it is worth acting on.

The scratch directories are the third category and they behave differently again. Remote clones land in `git/`, and `gc` clears that cache as part of its work, so a `gc` addresses both orphaned versions and stale clone scratch in one pass. `znative clean` is the option when you want the scratch reclaimed and the store left exactly as it is — which is the right verb after a run of failed installs, since those leave debris under `git/` and nothing anywhere else.

The practical shape of a maintenance habit falls out of that: `gc` after a round of updates, `clean` after a round of failures, and neither on any schedule, because nothing here grows fast enough to need one.

Inspecting and comparing

The store is a plain directory tree, stripped of `.git/` and `target/`, and `znative info` gives you the path to any of it. That combination makes a number of otherwise-awkward questions trivial. What exactly does this plugin source into my shell? Read the files. Did this update change anything I care about? Diff the two `store/<name>@<version>/` directories, which both still exist until you `gc`. Is the plugin on this machine the same one as on that machine? Compare the `sha256-<hex>` in the two index records — that is what content addressing buys, and it answers the question without transferring or reading a byte of the trees themselves.

That last one is the everyday value of the integrity hash, and it is worth separating from the security framing Chapter 29 gives it. Most of the time the hash is not defending you from an attacker; it is telling you whether two things you believe are the same actually are, which in a world of moving branches and multiple machines is a question that comes up constantly and has no other cheap answer.

Chapter 28 — The Cost of a Load

Every design decision in this book has a cost model behind it, and the model is rarely stated explicitly. This chapter states it: where time goes when a plugin loads and when a plugin's command runs, how the script path and the native path differ in shape, and why the manager offers no deferred-loading machinery to hide any of it.

The claims here are about *shape* — which costs exist, which are paid once, which are paid forever — not about magnitudes. Magnitudes depend on the plugin, the machine, and the workload, and a book that quoted them would be quoting a number that was true on one laptop on one afternoon.

The costs a script plugin pays

A script plugin's load is text becoming shell state. The file is read, lexed, parsed, and executed, and the execution's job is mostly to install definitions: functions defined, aliases created, options set, widgets bound, `$fpath` extended. The parse is the interesting part of the cost because of *when* it recurs — every shell start, for every plugin, for the life of the configuration. It does not amortise, because each new shell begins with an empty parser state and no memory of the last one. Ten plugins in a `.zshrc` is ten parses per shell, and a user who opens a lot of shells pays it a lot of times.

That recurrence is the entire reason the incumbent managers grew scheduling features. Turbo mode, deferred loading, and asynchronous sourcing do not make the parse cheaper; they move it off the path between pressing enter and seeing a prompt. The parse still happens, just later or on another thread, and the plugin is correspondingly not available until it does. This is a reasonable engineering response to a cost you cannot remove, and it is worth being fair about: those features exist because they solve a real problem for real users.

Then there is the per-call cost. A shell function is interpreted when it runs, and a plugin whose functions run per prompt or per keystroke pays that cost at exactly the frequency where it is most visible. A `precmd` hook runs on every prompt. A ZLE widget runs on every keypress it is bound to. Chapter 18's ports were chosen partly on this axis — fuzzy matching, frequency scoring, git classification, spinner animation are all *per-call work*, which is where the difference between interpreted and compiled shows up as something a user notices rather than something a benchmark reports.

The costs a native plugin pays

A native plugin's load is a different sequence with a different profile: `dlopen` the library, `dlsym` one symbol, call it once, and let it insert its registrations into an in-process registry. There is no parse anywhere in that list, because there is no text — the library is already machine code, and `dlopen` is a mapping operation plus relocation rather than a translation. The registrations are inserts into a data structure, proportional to the number of commands the plugin declares, which for real plugins is a small number.

The per-call cost is a call through a function pointer resolved at registration time. The dispatch lookup happens at the point in the pipeline Chapter 10 describes — `execute_external_bg` consults the registry before spawning a process — so a plugin command that would otherwise have been a process spawn becomes an in-process call instead. That substitution is the second-order effect and often the larger one: the native path does not only replace interpretation with machine code, it replaces some process creation with none at all.

The costs the store removes

There is a third cost that is easy to forget because it is invisible after the first day: the network. `znative load` hits the network exactly once per plugin, on the first shell start after the line appears in your `.zshrc`. Every start after that resolves against the index, finds the plugin present, and loads from the store copy. There is no update check on startup, no revalidation, no “is this still the head of the branch” round trip — `update` is an explicit verb you run when you want it, which is a decision about *when* network cost is paid rather than an omission.

The store copy is also warm. After the first load of a session the plugin's files are in the page cache; for a native plugin the library is mapped, which is the property Chapter 17's shim exploits — each `fzf` callback is a fresh `zshrs -f` that `dlopen`s a library the kernel already has resident, which is why replacing bash's per-keystroke library re-sourcing with a `dlopen` is a change in kind rather than a tuning improvement.

Why there is no deferred loading

Given all of the above, the obvious feature request is a turbo mode for `znative` — a way to defer a slow script plugin off the startup path the way `zinit` does. The tool does not have one, and the reason is the doctrine in Appendix H: first paint equals full functionality.

The argument is not that deferral is badly implemented elsewhere. It is that deferral is a *workaround for a cost*, and workarounds accumulate. A shell that needs turbo loading needs it because sourcing is expensive; add an instant-prompt trick on top because the prompt is expensive; add compiled-file caching because parsing is expensive; add a lazy autoload layer because function definition is expensive. Each layer is evidence that the underlying cost was never removed, and each layer buys its improvement by making the shell's state *at any given moment* harder to reason about — a plugin that is loaded now and was not a moment ago is a plugin whose absence is a race for anything that touches it.

`znative`'s position is to attack the cost instead. The native path removes the parse rather than rescheduling it; the store removes the network rather than backgrounding it; the index removes the filesystem scan rather than caching it. For script plugins, which still pay a parse, the manager does what every manager does and does not pretend otherwise — but it declines to add a deferral layer, because the layer would have to be there forever and would make “is this plugin loaded” a question with a time-dependent answer.

The practical consequence for a user is a small one and a real one: when your prompt appears, everything in your `.zshrc` has already happened. There is no window in which a keystroke lands before the widget that was supposed to handle it exists. That property is worth more than the milliseconds a deferral would have saved, and it is only affordable because the costs underneath were reduced rather than hidden.

Chapter 29 – Trust and Integrity

Installing software means running code someone else wrote, and loading a *native* plugin means running compiled machine code inside the shell process. That fact deserves a chapter that says plainly what `znative` does and does not promise about it — because the honest account is more reassuring than a vague one, and because a manager that did not state clearly what trust it asks for and what it does not defend against would be lying by omission. This is not a security-features chapter: several of the things a reader may expect are absent, and the absences are stated as absences. Two words that sound alike, *safety* and *trust*, are different guarantees, and `znative` provides exactly one of them.

What you are trusting

You are trusting the source you named. `znative` fetches what you asked it for. There is no curation layer, no review process, no global registry with a namespace someone polices, no reputation signal. `znative load owner/repo` means “fetch and run whatever is at `owner/repo`,” and the manager’s contribution is that it does so accurately rather than that it vouches for the result.

This is the same trust every plugin manager has ever asked for, and it is worth saying so rather than implying `znative` is different. `antigen bundle owner/repo`, `zinit light owner/repo`, and a hand-written `source ~/plugins/foo.zsh` all extend identical trust to identical parties. The delivery mechanism is not where the risk lives.

The native path does not change the threat as much as it appears to

The trust surface is the same one the script path always had. It is tempting to read “loads native code” as “more dangerous than sourcing a script,” and it is worth being precise that it is not. Sourcing a script plugin’s `.zsh` file runs arbitrary shell code with your full privileges the instant it loads; that has always been true of every plugin manager, and nobody signs their oh-my-zsh plugins. A native plugin runs arbitrary machine code with the same privileges. In both cases the thing you are trusting is *the source you named* — `owner/repo`, the git URL, the local path — and in both cases the act of trust is choosing to install it. `znative` does not widen that surface; a native plugin is neither more nor less trusted than the script plugin loaded on the line above it. The native path changed how a plugin *runs*, not who you have to trust to run one.

The intuition is that a compiled plugin is more dangerous than a script plugin, because a `.so` is opaque and a `.zsh` file is readable. Half of that is right and the half that is right is not the half people usually mean.

What is *not* different is capability. A sourced shell script runs as you, in your shell, with your environment, your credentials, and your filesystem access. It can read anything you can read, exfiltrate anything it reads, redefine any command you subsequently type, and persist itself by editing the same startup files it was loaded from. There is no sandbox around a `sourced` plugin and there never was. A native plugin runs in the same process with the same access. Neither is contained; the ceiling is identical and the ceiling is “everything the user can do.”

What *is* different is auditability. Shell text can be read, diffed, and grepped; a compiled artifact cannot, practically, by most people. If your defence is reading what you install, that defence works against the script path and does not work against a prebuilt binary. The mitigation available in the design is to install from source rather

than from a prebuilt artifact — the tree in the store is the source, readable, with `.git/` and `target/` stripped but the code intact — and to build it yourself, which is what `build = true` already does by default whenever a `Cargo.toml` is present.

What the curated surface does and does not contain

It is worth being exact about the curated host API of Chapter 13 here, because it is easy to over-read as a sandbox. The small surface — `print`, `eval`, `getvar`, `setvar`, and the function pair — genuinely bounds how a plugin touches *the shell*: it cannot reach into shell internals the way a C module does, so the blast radius of an ordinary bug against shell state is the narrow set of effects those methods allow. That is real containment of the *shell-integration* surface, and it is one reason a native plugin is easier to reason about than a C module that pokes globals directly. But it contains only that. A native plugin is ordinary Rust with the full standard library (Chapter 16): it can open files, spawn processes, and open sockets with your privileges, entirely outside the host API, because that is exactly what makes porting a real tool tractable. So the curated surface is a containment boundary for *shell state*, not a security boundary for the *process* — it limits what a plugin does to your shell, not what it can do as code you chose to run. Naming that honestly is the point of this chapter: the guarantees are real and they are specific, and the one guarantee `znative` never pretends to is that the code you installed is code you did not need to trust.

Installation runs code before loading does

This one deserves its own statement because it surprises people. `cargo build --release` executes build scripts and procedural macros. That means installing a native plugin from source runs code from that plugin — at your privilege level, on your machine — during the install, before any of the ABI machinery in this book has had anything to check. The `ABI_VERSION` gate, the `#[repr(C)]` layout discipline, the curated host API: all of that governs the *loaded plugin*, and all of it happens after the build already ran.

There is nothing pathological about this; it is true of `cargo build` everywhere and of `make`, `npm install`, and every other build-from-source workflow. It is worth knowing where the boundary sits: the install pipeline's stage 4 is the point at which you have already extended trust, and everything downstream of it is about correctness rather than about containment.

What the integrity hash is, and what it is not

What the integrity hash is, and what it is not. Chapter 6's `sha256-<hex>` is often mistaken for a security signature; it is not one, and the distinction matters. The hash is computed over the store copy at install time and recorded in the index, so it answers exactly one question: *is the content in this store directory still the content `znative` installed?* It says nothing about *provenance*: it does not prove who wrote the plugin, that the source was authentic, or that a repository was not compromised before you installed it. A signature scheme answers that second question and `znative` does not have one, because a signature needs a trust authority — a keyring, a registry, a web of trust — and `znative` is deliberately a manager without a central authority (Chapter 31).

The question it answers, it answers *on request and never on its own*. This is the correction the second pass forced, and it is stated here rather than buried in Chapter 68 because it changes what the field is worth. `store_integrity` has one call site in the entire shell — the install, at `commands.rs:43` — and no code path ever recomputes it. Measured: appending one comment line to a file in a store directory moves its true digest from `sha256-a2abf736...` to `sha256-fb47f2a1...`, after which `znative load` loads the plugin without complaint, `znative info` still prints the install-time value, and `znative verify` is not a command (`znative: unknown command 'verify'`). So the hash detects tampering only in the sense that a person who recomputes it by hand can compare two strings — and the manager exposes no way to recompute it.

Read the field as an audit record and a cross-machine comparison key, which is what Chapter 68 concludes; do not read it as a tripwire, because nothing is watching it.

What the ABI check is and is not

ABI_VERSION is a safety gate, not a trust gate. The version check of Chapter 15 is easy to file under “security,” and it is not that either. It guarantees a plugin will not *crash* the shell by disagreeing about struct layout — a memory-safety property — and it does so for a plugin you already chose to trust. It does nothing to keep a malicious plugin out; a hostile plugin can compile against the correct **ABI_VERSION** as easily as an honest one. Naming the two gates separately is the point: **ABI_VERSION** says *this plugin will not corrupt the process by accident*; the hash says *this plugin's bytes are the ones you installed*; and neither says *this plugin is benign* — that judgement is yours, made when you named the source, exactly as it has always been for every plugin you have ever sourced.

ABI_VERSION is a safety mechanism, not a security mechanism, and conflating the two would be the easiest mistake this book could invite.

What it defends against is *skew*. A plugin built against a different layout, loaded into a mismatched host, would read structs at wrong field offsets — undefined behaviour, memory corruption, a crash at an unrelated moment. The two-sided check converts that into a refusal at load with a legible cause. That is a real and valuable guarantee and it is entirely about accidents.

What it does not defend against is a plugin that means you harm. A malicious plugin compiles against the current ABI, reports the correct version, passes both checks, loads successfully, and then does whatever it wants — because a loaded plugin runs in-process with no sandbox, which is the same position a sourced script is in. The version gate makes honest plugins fail cleanly; it has no opinion about dishonest ones.

What the design does offer

Four properties are genuine and worth using deliberately.

Content addressing gives you a comparison, not a check. Every install carries a **sha256-<hex>** in its index record, computed over the store copy. Two machines whose rows agree for the same **name@version** hold the same bytes, and that comparison is genuinely useful across a fleet. What the field does not do is fire: no verb recomputes it, so “what is installed now is what was installed then” is a question you can ask offline with your own tooling and not one the manager will answer. And it does not tell you whether the bytes were good.

Pins stop things from moving under you. An unpinned source tracks a branch, which means the code you audited on Tuesday is not necessarily the code you install on Friday. **@v1.2.0** or a commit pin is recorded and honoured by **update**, so a plugin you have reviewed stays the plugin you reviewed until you decide otherwise.

There is no automatic update. Nothing fetches on startup, nothing revalidates in the background, nothing upgrades itself. New code enters your shell when you run **znative update** or add a new source, and at no other time. For a threat model built around “when could this change without my noticing,” that is the property that matters most, and it comes for free from the same decision that removed the network from the startup path.

Local installs decouple review from fetch. **path:DIR** installs from disk with no network at all, which means the sequence “clone it, read it, then install what I read” is expressible. The alternative — fetch and install in one step — does not leave a point where a human could look.

What it does not offer

Stated plainly, so nothing is inferred: there is no signature verification, no sandbox or capability restriction on loaded plugins, no vulnerability database, no registry-level namespace protection, and no privilege separation between a plugin and the shell. A plugin gets the shell's process and the user's authority, which is what makes it fast and is also exactly what it sounds like.

Two further absences belong in this list rather than in a chapter of their own, because both are security-relevant and both are easy to assume away.

There is no integrity verification, only an integrity record. The `sha256-` field is written once at install and read by nothing. There is no `verify` verb, no check on `load`, and no check on `gc` — an altered store loads exactly like an unaltered one, and `info` keeps printing the install-time digest. Whatever the recorded hash is worth, it is worth it only to someone who recomputes it themselves; nothing in the tool will notice a store that changed under it. Chapter 68 traces the single call site; Chapter 73 shows the failure mode it leaves open.

The index's `lib`, `name`, and `version` are paths, and they are not constrained. `load_entry` computes the library to `dlopen` as `store.package_dir(name, version).join(lib)` (`commands.rs:339–343`), and Rust's `Path::join` replaces the base when its argument is absolute — so an index row whose `lib` is an absolute path loads a library from anywhere on the machine while the store copy sits untouched. Measured, `znative load hello` against such a row loaded a different plugin entirely and `zmodload -R` listed it under a name that appears nowhere in `installed.toml`; the subsequent `znative remove hello` reported success and left it loaded, because the unload is attempted under the `index` name while the host's registry is keyed on the name the library reported. The same unchecked interpolation applies to `name` and `version`, which come from the plugin's own `znative.toml`: a manifest declaring `version = "../../../escaped"` installs, with status 0, to a directory outside `store/` — and a later `znative remove` recursively deletes that directory, whatever else is in it. Neither behaviour requires the plugin's code to run; both are performed by the manager, from a TOML string, during operations a cautious user performs *because* they have decided not to trust something. Chapter 73 reads all three joins and prices the three string checks that would close them.

The one structural mitigation the dispatch design happens to provide is narrow and real: a plugin cannot shadow a builtin, a keyword, or a function you defined (Chapter 10). It cannot transparently intercept `cd`, and it cannot silently take over a function you wrote. It *can* shadow an external program on `PATH`, which is a meaningful capability and one you should be aware a plugin has. That is a consequence of the resolution order rather than a security feature designed as one, and it is stated here at the size it actually is.

Why there is no signing, no sandbox, no allow-list. This is doctrine as much as design. `zshrs` does not gatekeep its user — no “are you sure,” no safety prompts, no confirmation on a destructive op — and `znative` inherits that posture whole. It will not interpose a trust ceremony between you and a plugin you asked for, because the tool is built for a user who decides what to run and does not want to be asked twice. What it *does* provide is the two honest guarantees above — a safety gate that prevents accidental corruption and an integrity hash that detects after-the-fact tampering — and it declines to imply the third guarantee, authentic provenance, that it cannot actually deliver without becoming the registry it deliberately is not.

The local-source case, where trust is total

The one install path where trust is not a question is the one a plugin developer lives in: `path:DIR`, or an absolute `/ ./ / ~-`rooted source (Chapter 5). When you install from a local working tree, the source is code you wrote or already have on disk and already trust with your session — there is no remote author, no repository that could have been compromised between commit and clone, and no network in the loop at all. This is the maximal-trust end of the same spectrum: the farther a source is from your own disk, the more the act of naming it is an act of trust, and a local path is the near end where that act is trivial. It is also the path an air-gapped or offline host uses for everything — a machine with no network installs native plugins from a local mirror by `path:`, builds them with its own toolchain, and never needs a trust authority it could not reach anyway. The trust model and the offline story are the same fact seen twice: `znative` trusts the source you named, and a

local source is one you named by pointing at your own filesystem.

Chapter 30 – Coming From oh-my-zsh and zinit

Most readers of this book already have a plugin manager, a `.zshrc` that works, and no appetite for a migration project. This chapter is about what actually changes and what does not, which is less than the preceding chapters might suggest.

Your script plugins are not affected

The single most important fact, and the one Chapter 46 calls the most important non-feature: script plugins install and load exactly as they always have. Nothing about the native path is mandatory, nothing about your existing plugins needs rewriting, and there is no compatibility tier where old plugins run in a degraded mode. A repository that oh-my-zsh sources is a repository `znative` sources, by the same mechanism — `$fpath` prepends and `source` calls, in order.

Concretely, the translation of a plugin line is a translation of a verb and nothing else:

```
# before, with various managers
antigen bundle zsh-users/zsh-autosuggestions
zinit light zsh-users/zsh-autosuggestions

# after
znative load zsh-users/zsh-autosuggestions
```

The source spec is the same `owner/repo` those managers already accept, because that shape is the ecosystem's lingua franca rather than any one manager's invention.

What has no translation

Three things do not carry over, and being clear about them upfront is more useful than discovering them one at a time.

Ice modifiers and their equivalents have no counterpart. `wait`, `lucid`, `atload`, `atinit`, `as`, `from`, and the rest of zinit's vocabulary describe scheduling and packaging decisions `znative` does not make. Chapter 28 argues the scheduling half; the packaging half — `as "program"`, `from "gh-r"` — is a genuine capability difference and worth naming as one. `znative` installs plugins, and a plugin is a script tree or a `cdylib`. A manager you were using to fetch release binaries and put them on `$PATH` is doing something `znative` does not do.

Deferred and asynchronous loading has no counterpart, deliberately. If your current configuration depends on turbo mode to be tolerable, moving those lines to `znative` will make startup do all of that work up front. That is the doctrine working as intended — first paint equals full functionality — but it is an honest tradeoff to state rather than a strict improvement. The compensating move is the one the rest of the book is about: for the plugins where the cost actually is, a native port removes the cost rather than rescheduling it.

A lockfile has no counterpart. Sheldon users in particular should read Chapter 46's argument rather than looking for the feature. The nearest thing is the `@ref` pin, which is per-plugin and recorded in the index, and

which answers “does this plugin move” without pretending to answer “can I reproduce this set of resolutions on another machine.”

Frameworks and monorepos

oh-my-zsh is not a plugin; it is a repository containing a framework and several hundred plugins. That makes it the case where the mapping between “a source” and “a plugin” is least direct, and the honest description is that `znative` installs repositories and detection reads whatever tree it gets. A repository built as one plugin — which is what nearly every standalone plugin on GitHub is — installs cleanly with no metadata. A repository containing many plugins installs as one tree, and what gets sourced is what detection finds at its root.

The practical consequence is that the natural unit for `znative` is the standalone plugin repository, and most of what people actually load from a framework exists as one. Where it does not — where the thing you want lives inside a larger tree with a layout detection cannot read — the manifest is the mechanism, and it is the plugin repository’s to ship rather than yours to configure. This is a real boundary and Chapter 46’s framing applies: `znative` is not a framework, and it is not trying to absorb one.

Migrating incrementally

There is no requirement to move everything at once, and nothing prevents two managers coexisting in one `.zshrc` during a transition. Both are, at the level the shell sees, code that runs at startup and installs definitions; neither has a lock on the process. The one thing you own is not loading the same plugin twice — a plugin sourced by both managers gets its definitions installed twice, and for anything that appends to an array, binds a widget, or registers a hook, twice is not idempotent.

So the workable order is: move one line, start a new shell, confirm the plugin behaves, delete the old line. Repeat. The first shell after each move pays a network cost for that plugin and no shell after it does, which makes the migration’s cost visible exactly once per plugin and then gone.

Two ordering rules survive the move unchanged and are worth re-checking as you go, because a migration is exactly when ordering gets scrambled. `zsh-syntax-highlighting` stays last, for the reason Chapter 9 gives — it wraps the line editor and must see the final state of every widget. Completion plugins, including native ones, go after `compinit`, for the reason Chapter 14 gives. Neither rule is new; both are rules your current manager already made you follow, and `znative` preserves them by not reordering anything.

What you gain on the first day

Not much, and that is the point. On day one you have the same plugins, in the same order, doing the same things, installed by a different verb into a store you can read. The network stops being touched after the first start of each plugin. `list` and `info` tell you what you have without a scan. `gc` reclaims what updates leave behind.

What you gain later is the option that did not previously exist: when one of your plugins is slow enough to matter, and its slowness is per-call work rather than a process spawn, you can replace it with a compiled port that loads through the same line in the same file. Chapter 18’s ports exist as evidence that this is a real option rather than a promise, and Chapter 16 is the method for taking it. The migration is not the interesting part; the migration is the prerequisite for the interesting part being available at all.

Chapter 31 — The Landscape

`znative` earns “first shell package manager whose unit of installation can be native code” only against the field of managers that came before it, so this chapter is that field and where `znative` sits in it. The claim is narrow and specific on purpose: not “first plugin manager,” not “fastest,” but *first whose unit of installation can be native compiled code rather than shell text*.

The incumbents all share one mechanism:

- **oh-my-zsh** — the framework: a large repository of plugins and themes you enable by name, each a directory of `.zsh` files it `sources`.
- **zinit** — the ergonomics leader: turbo mode, deferred/async loading, its own ice-modifier DSL. Underneath the DSL, it clones repos and sources their files.
- **antigen** — the bundle-declaration manager: `antigen bundle owner/repo`, clone and source.
- **sheldon** — the TOML-configured, lockfile-carrying manager: a declarative plugins file, and again clone-and-source underneath.
- **zplug** — the feature-broad manager (tags, dependencies, parallel install): clone and source.

Every one of them, underneath its configuration model, does the same single act: **clone a git repository and source its `.zsh` files**. They differ — and some differ a great deal — in *ergonomics*: how you declare a plugin, whether loading is deferred, whether there is a lockfile, how parallel the installs are. They do not differ in *kind*. The unit of installation, for all of them, is shell text.

None of them installs a compiled plugin, and the reason is not that they never thought of it — it is that **until the stable ABI of Chapter 2, there was nothing safe to install**. A native shell module before `zshrs` was welded to one shell build; a manager that “installed” one would be shipping a crash waiting for the next shell rebuild. The incumbents install shell text because shell text was the only thing safe to distribute. `znative` installs compiled plugins because `zshrs` first made compiled plugins safe to distribute. The manager did not invent the native path; the ABI did. The manager is what made the native path *installable*, and that is the leg of the claim it owns.

Two omissions distinguish `znative` from some of the incumbents, and both are deliberate. Sheldon carries a lockfile; `znative` does not. Language package managers carry per-project manifests; `znative` does not. `znative` is **global-only** — one store, one index, under `$ZSHRS_HOME/pkg/` — because a shell’s plugins are part of the user’s environment rather than a project’s dependency set, and pretending otherwise would import a reproducibility model that does not fit what shell plugins are. A missing lockfile in a project package manager is a bug; in a shell plugin manager it is a correct read of what the tool is for. `znative` is not “sheldon minus a lockfile” — it is a different category, built on a substrate none of the incumbents had.

What the incumbents got right, and what `znative` kept

A claim of novelty is only credible from someone who can say what the prior art solved. The incumbents solved a great deal, and three of their conclusions are load-bearing in `znative` rather than merely tolerated.

owner/repo as the universal source spec. None of these managers invented the idea that a plugin is a GitHub repository, but collectively they made the two-part slug the canonical way to name one, to the point that it is now the shape a plugin author assumes users will type. `znative` adopts it as the primary form specifically because it is not `znative`’s to redefine — a source syntax that disagreed with the ecosystem’s would make every existing plugin’s README wrong. Chapter 5’s classification rules are, read one way, an argument that the ecosystem already settled this and the correct move is to conform.

Clone-and-source as the script-plugin semantics. The mechanism every incumbent converges on — put the directory on `$fpath`, source the entry-point files, in order — is not a lowest common denominator. It is the right answer, and it is right because it is what a plugin *is*: a set of definitions the shell installs by reading them. `znative` reimplements it rather than inheriting an implementation, but it reimplements the same semantics deliberately, and Chapter 7's detection signals are all conventions those managers established or popularised. A manager that changed the semantics here would break plugins for no gain.

Order is the user's to declare. Every one of these tools loads plugins in the order they appear, and none of them tries to infer a dependency graph and sort it. That looks like a missing feature and is a correct one: shell plugins interact through global mutable state — widgets, hooks, arrays, options — and the interactions are not declarable in any vocabulary a manager could check. The `zsh-syntax-highlighting`-last rule is the standing proof; it is not a dependency in any formal sense, it is a fact about what wrapping the line editor means, and no manifest field would have captured it. `znative` inherits the discipline exactly: source order is load order, and the file is the whole explanation.

Where `znative` diverges is narrower than the shared ground. It declines the scheduling layer, for the reasons Chapter 28 gives. It declines the configuration DSL, because without scheduling there is nothing left for a DSL to say. It replaces the ad-hoc install directory with a content-addressed store and an index, which is a real improvement in kind — `list` and `info` answer from a record rather than a scan, integrity is checkable, and `gc` has a sound definition of garbage. And it adds the native path, which is the one thing no substrate before `zshrs` could support.

Reading the divergences together, a pattern shows up that is worth naming explicitly. The incumbents' most elaborate machinery — turbo mode, async loading, compiled-file caching, lazy autoloading — is all aimed at the same target: the cost of turning shell text into shell state, paid on every startup. Each is a well-built response to a cost that could not be removed from inside `zsh`, because `zsh`'s unit of extension is text and text must be parsed. That accumulation is not a criticism of the tools; it is the strongest available evidence that the cost is structural. A community does not independently build four different workarounds for a problem that has a fix.

`znative`'s position is that the fix has to come from underneath — a shell in which a plugin can be compiled code, and a published ABI that makes compiled code safe to distribute. Given that substrate, most of the scheduling machinery becomes unnecessary rather than unimplemented, which is why its absence here is a different claim than the same absence would be in a manager built on `zsh`. This is also precisely the boundary of the novelty claim: `znative` is not better-engineered than `zinit` at the thing `zinit` does. It is a manager for a substrate `zinit` did not have.

Chapter 32 – Lineage: the Stryke Dependency Manager

`znative` was not designed from nothing. It is **ported from stryke's own dependency manager** — the package system for strykelang, the parallel Perl-5 superset that shares the fusevm bytecode machine with zshrs — and understanding that lineage explains why `znative`'s native path is as considered as it is.

Stryke's dependency manager does something more aggressive than any shell plugin manager: it AOT-compile a whole dependency graph to native code. A stryke program that pulls in packages does not carry an interpreter that reads those packages at runtime; the closed-world compiler lowers the whole graph — program plus dependencies — to native registers and a standalone binary. The dependency manager, in that world, is not “fetch some text to interpret later.” It is “fetch source that will be compiled into the artifact.” Native code as the *unit of a dependency* is stryke's normal case, not an exotic one.

`znative` carries that instinct into the shell. It takes the stryke dependency manager and teaches it to install a compiled plugin the way every other shell manager installs a script — same command surface, same “add this and it is at your prompt” ergonomics, but with a `cdylib` as a legitimate unit of installation rather than only a bag of `.zsh` files. The reason the shell path could accept that idea at all is the reason this book keeps returning to: the published ABI (Chapter 2) gave the shell a safe way to load a foreign compiled artifact, which is the same shape of thing stryke's manager already dealt in. The lineage is not cosmetic; it is why `znative` treats native code as a first-class package instead of an afterthought bolted onto a script manager.

There is a consistency to it worth naming. zshrs and stryke share fusevm — one bytecode machine, one JIT, one AOT compiler underneath both. It is fitting that they also share a dependency-management instinct: that native compiled code is a normal thing to fetch, build, and install, not a special case. `znative` is that instinct applied to shell plugins, made routine by the ABI that made it safe.

What did not port, and why

A lineage is only informative if you can say what was dropped, and the drops here are as instructive as the inheritance. Three capabilities that a dependency manager for a programming language must have are absent from `znative`, and in each case the absence traces to a structural difference between what a program's dependencies are and what a shell's plugins are.

There is no dependency graph, so there is no resolver. A stryke program pulls in packages that pull in packages, and the manager's central job is to turn that transitive closure into a consistent set — the version-solving problem every language package manager has. Shell plugins have no such closure. A plugin does not declare that it needs another plugin; the manifest of Chapter 8 has a `[plugin]` identity table and a kind table and nothing that names another package. What a user has is a flat set they curated by hand, in an order they chose, which the closing chapter states directly as the reason there is no lockfile: there is no graph to pin. A resolver with nothing to resolve is not a missing feature.

There is no closed-world compile, because loading happens at runtime. Stryke's manager exists in service of an AOT compiler that lowers the whole graph — program plus dependencies — into one artifact, which is what makes native code the natural unit there: the dependency is not a thing you load, it is a thing you *link*. `znative`'s native plugin is the opposite arrangement. It is compiled separately, it is loaded into an already-running process, and the interface between the two halves therefore has to survive the two halves being built at different times by different people. That is precisely the burden the published ABI carries and the

reason Chapter 21's rules are as strict as they are. A closed-world compiler does not need an `ABI_VERSION`, because there is only ever one world and the compiler saw all of it.

There is no project scope, because a shell is not a project. Stryke's manager resolves dependencies for a program, and a program is a thing with a directory, a manifest, and a collaborator who needs the same resolution. A shell's plugin set has none of those, which is Chapter 46's argument for global-only, and it is worth noticing that the argument only becomes visible when you hold the two managers next to each other. Per-project scoping is not something `znative` failed to inherit; it is something the shell context makes meaningless.

What survived the port is the instinct rather than the machinery: that a compiled artifact is an ordinary thing to name, fetch, build, and install, with the same ergonomics as fetching text. Everything downstream of that instinct — the content addressed store, the kind detection, the index as authority, the build-on-install default — was designed for the shell, against a substrate the shell only recently acquired. Reading the lineage accurately means reading it as a transfer of assumptions, not a transfer of code.

Chapter 33 – Concurrent Shells and the Index

Every chapter up to here has described `znative` as though one invocation runs at a time. That is the easy case and it is not the interesting one. A user who opens terminals in bulk — split panes, a restored session, a window manager reopening everything at login, a build script that starts a dozen non-interactive shells — runs many `.zshrc` files at once, and every one of them contains the same block of `load` lines. This chapter is what that means, what the design's invariants give you regardless of how the invocations interleave, and where the remaining sharp edge actually is.

The stampede

Consider a fresh machine, a `.zshrc` with a block of `load` lines, and several shells starting simultaneously. Each `load` line runs the Chapter 4 conditional: is this source already installed? On a fresh machine the answer is no, in every one of those processes, at more or less the same instant. Nothing in the conditional is a claim on the plugin; it is a read of the index followed by a decision. So all of them decide to install, and all of them enter the Chapter 22 pipeline for the same source.

Follow what each of them does. Stage 1 is pure and cannot conflict. Stage 2 clones into the store's `git/` scratch directory — one clone per shell of the same repository, which is wasteful and, because `git/` is scratch that nothing loads from and `znative clean` empties, is not dangerous. Stage 3 reads the staged tree and arrives at the same name and version everywhere, because it is reading the same tree. Stage 4 builds, once per shell, which on a native-from-source plugin means that many `cargo build --release` invocations competing for the machine. Stage 5 copies into `store/<name>@<version>/` — and here they converge, because the store key is a function of the plugin's identity and every one of them computed the same identity.

That convergence is the property that makes the stampede survivable rather than corrupting. Many installs of one source are many installs *of the same content into the same destination*. Stage 6's hash is computed over that destination and is the same number in every process, because a hash of identical bytes is identical. Stage 7 writes a `[[package]]` record whose every field — name, version, recorded source label, kind, integrity hash, and the lib or the source-files list — is the same in every process. Whichever write lands last, the row it leaves is the row all of them intended. Last-writer-wins is harmless when all the writers agree.

The cost is real and it is entirely in the wasted work: many clones, many builds, many copies, for one plugin. On a script plugin that is a few network round trips you did not need. On a native-from-source plugin it is a set of simultaneous optimizing compiles, which is the kind of thing that makes a login feel like the machine is broken.

Where agreement breaks

The interesting failure is not two shells installing the same plugin; it is two shells installing *different* plugins. The index is one file with one record per plugin, and a manager that adds a record to it has to read the file, add its row, and write the file back. Two such sequences overlapping is the classic read-modify-write race: process A reads an index with rows for X and Y, process B reads the same index, A writes back X, Y, and its new row for P, B writes back X, Y, and its new row for Q. Whichever wrote second wins, and the other's row is gone.

This is worth stating precisely because the consequence is much milder than the mechanism suggests. A lost row does not lose the plugin's content — the store copy was written before the record and is still there, complete and hashed. What the lost row produces is exactly the Chapter 26 state of a *store directory with no index row*, and Chapter 26 already says what that is: an orphan. It is inert, `list` does not report it, and `gc` will reclaim it.

So the failure mode of a concurrent first start is not corruption and not a broken shell. It is a plugin that quietly did not end up installed, in a configuration where the next shell start will notice — because the next `load` line for that source runs the same conditional, finds no matching row, and installs it again. The self-healing property here is not a repair mechanism anyone wrote; it is the load-or-install conditional doing on start number two exactly what it did on start number one. The `.zshrc` idiom is idempotent in the sense that matters: running it again converges.

That is a real answer, and it is not an argument that the race is fine. A user whose plugin silently failed to install and then silently installed on the next shell has been given a confusing hour if they happened to be looking. The honest characterisation is that the ordering discipline of Chapter 22 turns a concurrency hazard into a garbage-collection outcome, which is the same trade that chapter makes everywhere else, and the same trade is available here because the durable write is one write at the end.

What does not contend

Three things are worth separating out, because they are the ones people assume are the problem and they are not.

Loading is per-process. The refusal of Chapter 10 — a plugin whose name is already loaded cannot be loaded again — is a statement about one shell's in-process registry. Each shell `dlopens` its own copy of the library into its own address space and registers into its own registry. There is no shared state to contend for, and a plugin loaded in one terminal is not loaded in the next one.

Reads do not conflict with each other. `list` and `info` and the fast path of `load` are reads of `installed.toml`. Any number of them can run at once, and the worst thing a reader can observe is a slightly stale view: an index that does not yet contain a row another process is about to add. A stale read on the `load` fast path produces an install rather than an error, which is the same conditional resolving the same way it does on any first start.

The store copies do not overlap. Two different plugins install into two different `store/<name>@<version>/` directories, and the same plugin at two versions installs into two more. The directory key is derived from the identity that stage 3 fixed, so the only way two concurrent installs write the same directory is if they are writing the same content. The index is the shared resource; the store is not.

Living with it

The practical advice follows from the shape rather than from any flag, because there is no flag. The first start of a configuration is the expensive, contended one, and it happens exactly once per plugin per machine. So the cheap move on a new machine is to make that first start a single start: open one shell, let the block install, and only then restore the session that opens all the others. The same effect is available deliberately with `znative add` at a prompt for each new plugin — the interactive install of Chapter 4 — after which every `load` line in every shell takes the fast path and touches neither the network nor the index's write path.

For a configuration that is already established, none of this applies. The steady state of the `.zshrc` idiom is that every `load` line finds its plugin in the index and goes straight to Chapter 22's stage 8. Many shells doing that are many readers of a file nobody is writing, which is not a concurrency problem in any sense. The stampede is a property of *change* — a new line, a new machine, an `update` — and change is not what most shell starts are.

And when the two do coincide, the reconciliation tools of Chapter 26 are the same ones you would use for any other disagreement. `znative list` says what the index believes. `znative gc --dry-run` shows the store directories no row names, which after a contended first start is where a lost row's content is sitting. Reclaim them or leave them; the next shell start will reinstall what it needs either way.

Chapter 34 — What `remove` Cannot Undo

`znative remove` is described in Chapter 4 in one sentence: unload the library, delete the store copy, drop the index row. That is accurate and it is not the whole story, because *undoing an install* and *undoing a load* are different operations with different feasibility, and the difference is not symmetric between the two kinds of plugin. A user who removes a plugin and finds its commands still working has not hit a bug; they have hit the part of loading that was never reversible in the first place.

The native case is genuinely reversible

For a native plugin, removal really does remove. The whole load was: `dlopen` a library, resolve one symbol, call it, and let it insert entries into an in-process registry the host owns. Every effect of that sequence lives in structures `zshrs` controls. Unload walks them back — purge the registrations, then drop the library, in that order, which is the Chapter 10 property that keeps a function pointer from outliving its code.

The reason this works is worth naming, because it is the one structural advantage the native path has over the script path in this specific dimension. A native plugin's entire contribution to the shell is *mediated*. It did not define functions; it asked the host to record handlers. It did not modify `$fpath`; it recorded a completion request. The host knows the full set of things the plugin added because the host is what added them. Reversal is therefore a matter of consulting a list, not of inferring what changed.

The Chapter 20 caveat is the exception that proves it. The host can purge what it registered; it cannot stop a thread the plugin spawned, because that thread was never mediated — it is ordinary Rust the plugin ran on its own account. So a plugin's obligation to shut down its own workers is precisely the obligation to be reversible in the one dimension the host cannot reverse for it.

The script case is not reversible at all

Loading a script plugin is Chapter 7's synthesised fragment: prepend directories to `$fpath`, then `source` files, evaluated on the live shell. What that fragment does is run the plugin's code, and the plugin's code does whatever shell code does — defines functions, sets aliases, sets options, binds widgets, appends to hook arrays, exports variables, possibly writes files.

None of that is recorded anywhere. There is no ledger of “the definitions this `source` produced,” because `source` does not produce a ledger; it executes a program whose effect is to mutate the shell's global state. A manager that wanted to undo it would have to have snapshotted the entire shell state before the `source` and diffed it after — which would catch the direct effects and still misattribute anything that happened for a second reason in between.

So `znative remove` on a script plugin does exactly what it says and no more: the store copy goes, the index row goes, and the *running shell you typed the command in* still has every function, alias, widget, and hook the plugin installed. They keep working until that shell exits. This is not a shortcoming relative to the incumbents — no plugin manager unsources a plugin, because none of them can — but it is a difference from the native path that lives in the same tool, and the asymmetry is easier to accept once you can see it comes from what loading meant in each case.

There is one wrinkle sharp enough to name. The `$fpath` entry the fragment prepended is a path into `store/<name>@<version>/`, and `remove` just deleted that directory. Your live shell now has a directory on its `$fpath` that does not exist. Completion lookups walk `$fpath`, and a missing entry is an entry that contributes nothing — the practical effect is that autoloadable functions the plugin provided stop being findable, while the functions it already defined by sourcing keep working. That mixed state is the clearest possible signal that the shell you are in is no longer the shell the index describes.

The correct response is the simple one: start a new shell. A fresh shell reads a `.zshrc` whose `load` line for that plugin you have presumably also deleted, finds nothing to load, and comes up clean. Removal is complete for every shell started after it, and complete for no shell started before it, which is the honest rule and the one to hold.

What removal never touched

A plugin's state outside the store is not `znative`'s to delete, and it does not try. Chapter 18's `zsh-z` keeps its frequency database at `~/.z` — a file the plugin created, in a location the plugin chose, that `znative` has no knowledge of. Removing the plugin removes the code that reads and writes that file. The file stays.

That is the right default in both directions. Deleting it would mean the manager guessing which files in a user's home directory belong to which plugin, from no information, and getting it wrong destructively. Keeping it means an uninstall-then-reinstall preserves the history you had built up, which for anything that accumulates value over time — a frequency database, a cache, a learned ranking — is what you wanted. Chapter 37 goes through where a plugin should put such state and why the store is the wrong place for it.

update on a plugin that is currently loaded

The same reversibility question shows up in a milder form on `update`, and Chapter 27 gives half the answer: the old `store/<name>@<version>/` stays, because deleting a file out from under a live mapping is the hazard the unload path exists to prevent.

The other half is what the shell you are typing in actually has. It has the library it `dlopened` at load time, mapped, with its registrations in the registry. An `update` in that shell installs a new version into a new store directory and repoints the index row. It does not swap the mapping — nothing reaches into a running process and exchanges its code. So the shell that ran the update is running the old plugin, and the index describes the new one, and both of those are correct statements about different things.

Every shell started after the update loads the new version, because `load` resolves through the index and the index now names the new store directory. The one that ran the update catches up when it exits. For a native plugin you can force the issue by hand with the Chapter 27 development loop — `zmodload -uR` the name and `zmodload -R` the new library — but the ordinary answer is that a new shell is how a shell picks up a new version, which is the same answer as for every other change to a startup file.

Chapter 35 – Testing a Native Plugin

A `cdylib` that loads into a shell is an awkward thing to test, and the awkwardness is mostly avoidable. Most of what a plugin does is not shell integration at all — Chapter 16’s central realization is that a Rust plugin runs external programs, reads files, and parses text with the standard library, and only reaches for the host API to touch the shell. That split is also the testing strategy: the part that is ordinary Rust is testable as ordinary Rust, and the small remainder needs a real shell.

Layer one: the handler is an adapter, not the logic

The signature `fn(&Host, &Args) -> c_int` is a boundary, and treating it as one is the single decision that makes a plugin testable. A handler that opens with argument validation, continues with the actual work, and ends by printing through the host has fused three separable things to a type you can only construct inside a running shell. Split them and the middle one becomes a plain function.

Take Chapter 16’s `forget` port. The guard is `inside_work_tree()`, already a free function returning `bool`. The list-building step captures `git` output and transforms it; the transformation from the raw `--porcelain` text to the list of candidate lines is a function from `&str` to `Vec<String>` with no host and no subprocess in it, and that function is where the parsing bugs live. Test it with recorded `--porcelain` output as a string literal. The handler that calls it needs no test of its own, because after the split it contains nothing but sequencing.

The same applies to the shape Chapter 13 recommends for configuration: resolving `${FORGIT_PAGER:-$(git config core.pager || echo cat)}` becomes an `Option` chain, and the chain is testable if the two lookups it composes are parameters rather than direct calls to `host.getvar` and a `git` subprocess. A function taking two closures — one for the shell parameter, one for the git config — is testable with stubs and is called with the real ones from the handler.

The rule that falls out is worth stating flatly: **nothing that needs a `Host` should contain a decision**. Reads, writes, prints, and `evals` are effects; put them at the edges and keep the middle pure. This is not a testing-specific discipline — it is the same factoring that makes the `git-fuzzy` port’s helper subcommands directly invocable in Chapter 17 — but testability is where it pays most obviously.

Layer two: the real shell, scripted

The remainder needs a shell, and the shell is available non-interactively. Chapter 27’s development loop is `zmodload -R` on a freshly built library, exercise, `zmodload -uR`. That loop scripts:

```
zshrs -f -c '
  zmodload -R ./target/release/libmyplugin.dylib
  mycmd --some-flag
  print "status=$?"
'
```

Two details in that invocation matter. The `-f` is not optional in a test: it skips the user’s startup files, so the test exercises the plugin rather than the plugin plus whatever else is in your `.zshrc`. A test that passes only on the author’s machine because a personal alias was in scope is worse than no test. And the library path is the build output directly, not a store path, because the store is the wrong tool for an edit-test loop for exactly the reason Chapter 27 gives: the install copies, so a store-based test tests the copy you made last, not the code you just compiled.

What this layer is for is the things layer one cannot reach. Does the command actually register under the name you declared? Does it return the status you think it returns, as `$?` in a real shell? Does `host.setvar` produce a parameter the shell can read on the next line? Does the plugin load at all — which is to say, does the `ABI_VERSION` in the crate you compiled against match the shell you are testing with? That last one is a build-configuration check masquerading as a test, and it is the one that fails most often on someone else's machine.

The unload half deserves its own test rather than being a cleanup step. Load, exercise, unload, load again, exercise again: a plugin with static state that survives the first unload incorrectly will misbehave on the second load, and that sequence is the cheapest way to find it. Chapter 20's rule — nothing of yours may still be running when your library goes away — is directly checkable this way, because a worker thread that outlived its `dlopen` will not fail quietly on the second cycle.

Layer three: the completion generator is just a builtin

Chapter 14's generator is registered as a hidden builtin, and its argument convention is fully specified: `$CURRENT` followed by the words on the command line. That means testing it needs neither a terminal nor the completion system — it needs the ability to call a builtin with a chosen argv, which a script has.

Load the plugin, call the generator by the name you gave it, pass a synthetic `CURRENT` and word list, and compare the candidates it emits against what you expect for that prefix. Every interesting case is expressible: completing the first word after the command, completing after a flag, completing with an empty prefix, completing a prefix that matches nothing. None of it requires pressing tab.

What that does *not* verify is the wiring — whether the deferred `compdef` actually attached your generator to your command. That is the load-order rule of Chapter 14, it fails silently when broken, and it needs an interactive shell to observe. Which is the general shape of the boundary: the generator's *behaviour* is scriptable, the completion system's *arrival* at your generator is not.

Layer four: testing the install, not the plugin

There is one more thing worth testing and it is the manager's side rather than the plugin's: does this repository install correctly? The tools for that are already in the book. A `path:` source installs from a directory with no network at all (Chapter 5), and the store root is `$ZSHRS_HOME/pkg/` (Chapter 6) — so pointing `$ZSHRS_HOME` at a scratch directory and running `znative add path:.` exercises the whole Chapter 22 pipeline against your working tree, in a store you can delete afterwards.

That is the test that catches the class of problem no unit test can: a manifest that names a lib stem the build does not produce, a `[script] source` list with a typo in a filename, a tree whose detection outcome is not the one you assumed, a `build = false` on a repository that does not ship the artifact. All four of those compile fine and fail at install, which is to say they fail on your users' machines and not on yours. `znative info` on the result is the assertion: it prints the store path, the integrity hash, and the specific lib or source-files-and-`fpath` the plugin resolved to, which is exactly the set of facts you are trying to confirm.

What is not testable, restated

Chapter 17 draws the line for interactive TUIs and the line generalises. A full-screen child process taking the terminal from an interactive shell is not faithfully reproducible under a scripted PTY, so a blank capture is not evidence of a bug and a passing capture is not evidence of correctness. Verify that part by running it, in a real terminal, with your eyes. The discipline is to keep the untestable region as small as possible — helper subcommands, geometry, rendering of a single item, the selection protocol all pushed down into layer one — so that what is left to check by hand is the part where a human genuinely is the only available oracle.

Chapter 36 – Debugging Across the Boundary

When a native plugin misbehaves, the ordinary debugging instincts point in the wrong direction, because the plugin is not a program you can run. It is a library inside a shell, its output is the shell's output, its crashes are the shell's crashes, and its errors are terse by doctrine. This chapter is where the information actually lives and how to narrow a problem down to the side of the boundary it is on.

Establish what is loaded before anything else

The first question is never “why is this plugin wrong” but “is this plugin the thing that ran.” Two commands answer it and they answer different halves.

A bare `zmodload -R` lists what is loaded in *this process*: name, version, and path, one line each. That is ground truth for the running shell — not what the index says should be loaded, not what your `.zshrc` asked for, but what is mapped and registered right now. A plugin missing from that list did not load, and the reason is upstream of anything in its code.

`znative info NAME` answers the other half: what the index says a `load` of that plugin resolves to — the store path, the hash, and the specific library or source-files list. Between the two, a load failure localises immediately. Present in `info`, absent from the `zmodload -R` listing means the load itself failed. Absent from `info` means the plugin is not installed and the `.zshrc` line either is not there or did not run. Present in both, with the wrong version string, means you are running a shell older than your last `update` — Chapter 34's case.

Where the words are

zshrs's doctrine puts almost nothing on the terminal, which is correct and is also why a first-time debugger thinks nothing happened. Three destinations exist and each carries a different class of information.

stderr carries failures, terse. `znative: <reason>`, non-zero exit. That is the whole of what the manager says about a failure it detected, and for the classes in Chapter 25 it is sufficient because the reason names the class.

The log under `~/.cache/zshrs/` carries the chatter. Build output in particular: Chapter 23 states that cargo's compile progress goes to the log rather than the terminal. So a native install that failed to build reports the failure on stderr and the *compiler's actual diagnostics* are in the log. Looking at the terse line and concluding there is no more information available is the most common wasted hour in this whole area.

The plugin's own output goes wherever it sent it. `host.print` writes to the shell's stdout. A subprocess the plugin spawned writes wherever its stdio was wired — inherited, piped, or discarded, per the plugin's own choice. This is worth checking early when a plugin appears silent: Chapter 16's `fzf` helper deliberately inherits stderr and pipes stdin and stdout, and a plugin that captured a child's stderr instead has swallowed exactly the messages you are looking for.

Deciding which side is wrong

The dispatch order of Chapter 10 is a decision procedure, and using it as one resolves the most confusing class of symptom: the command ran and did something you did not expect.

Plugin commands resolve after real builtins and shell functions and before `PATH`. So if a name you registered appears to do something else, the candidates are, in order: a real builtin of that name (your registration never gets consulted), a shell function of that name (same), or your handler running and behaving differently than you think. There is no fourth possibility in which `PATH` won, because `PATH` is consulted after the registry — unless the plugin did not load, in which case there is no registry entry and the external program is exactly what you got. Which is why the loaded-or-not question comes first.

The inverse symptom — “command not found” for something you registered — has the same short list. The plugin did not load; or it loaded and the name in the `builtins` map is not the name you are typing, which is easy when Chapter 12’s rule is that the map key is the user-facing name and the Rust function name is irrelevant; or the handler exists and the shell is being run without the plugin, which is the `zshrs -f` case and is a feature when it is deliberate.

What a crash looks like, and what it means

A panic that crosses the C-ABI boundary is undefined behaviour (Chapter 15), which in practice means a plugin bug can end the shell process with no message that identifies the plugin. That is a hard failure mode and its signature is recognisable: the shell dies while running a plugin command, and the terminal is simply gone. Nothing in the shell’s terse error contract will have told you which plugin, because the process did not survive to say.

The narrowing move is the Chapter 27 loop with one plugin at a time: `zshrs -f, zmodload -R` the suspect library, run the command. A crash there is the plugin’s, observed in isolation, with none of your configuration in the picture. This is also the point at which a debug build earns its cost — the store holds a `--release` artifact by design (Chapter 23), and a locally built debug `cdylib` loaded by path with `zmodload -R` is a strictly better thing to attach a debugger to, without disturbing the installed copy at all.

The quieter cousin is Chapter 20’s panicking worker thread. A panic inside a thread the plugin spawned unwinds that thread only, never reaches an `extern "C"` frame, and is therefore not undefined behaviour and not a crash. What it is, is a worker that stopped: an animation that froze, a parallel classification whose results never arrived, a watcher that stopped watching. The symptom is absence rather than failure, which is why it is worth knowing that the two panic situations look nothing alike and have nothing in common except the word.

Completions that do not fire

Chapter 25 names the usual cause — the plugin loaded before `compinit` — and the diagnosis is a sequence rather than a guess, because Chapter 35’s third layer makes the generator directly callable.

Call the generator by name with a synthetic `CURRENT` and word list. If it emits the right candidates, the generator is correct and the problem is wiring: check the load order against `compinit`, and check that the generator’s name has no leading underscore, which `zsh` would read as an autoloadable completer rather than the builtin the macro registered. If it emits nothing or the wrong things, the problem is the generator and the completion system was never involved. Two outcomes, two disjoint places to look, and no need to press tab to find out which.

The debugging that is really a build problem

One last class, because it presents as a runtime mystery and is not. A plugin that refuses to load with a version mismatch is Chapter 15's check doing its job, and the fix is in the plugin's `Cargo.toml`, not in its code. A plugin that fails to compile against a `znative` version cargo resolved is Chapter 25's compile-time shadow of the same skew. A plugin that loads but is missing a host method you expected is a plugin compiled against an older crate than you think — which the version check would have caught if the versions differed, and which means they do not and your expectation is what is wrong.

The unifying observation is that the boundary was designed to make skew loud, and it succeeds, so a symptom that *looks* like ABI trouble and produced no version mismatch is almost never ABI trouble. Believe the check. Look somewhere else.

Chapter 37 — Plugin State and Configuration

A plugin that does anything durable has to decide two things the book has so far only gestured at: where its settings come from, and where its data goes. Both have a right answer that follows from the design, and one of them has a wrong answer that will silently break the store's integrity guarantee.

Configuration is shell parameters, and that is not a compromise

`host.getvar` exists so a plugin can read a shell scalar, and Chapter 13's examples are the conventional shapes: `$FZF_DEFAULT_OPTS`, a plugin-specific `$FORGIT_PAGER`. The temptation, for a plugin author with a whole standard library available, is to skip that and read a config file — TOML in the user's config directory, parsed properly, with types and defaults and validation. Resist it, and the reason is not aesthetic.

The user already has a configuration language for their shell, and it is the shell. Their `.zshrc` is where every other plugin's settings live, where their conditional logic lives, and where they will look first. A plugin that reads a separate file has introduced a second place where its behaviour is determined, and the second place is invisible from the first. Worse, it is invisible to the *shell* — a setting in a file cannot be changed for one command with a prefix assignment, cannot be exported, cannot be computed, and cannot differ between two shells the user has open for two different purposes. A shell parameter can do all four, because that is what shell parameters are.

The conventions to follow are the ecosystem's rather than anything `znative` imposes. Uppercase, prefixed with something clearly attributable to the plugin, one parameter per setting. Chapter 16's translation of `${VAR:-default}` into an `Option` chain is the pattern for reading one: ask the host, treat an unset parameter the way the shell would, and fall back to a default in Rust rather than requiring the user to set anything.

Read per call, not at init. This is the part that is easy to get wrong and tedious to debug. A plugin that reads its configuration once during `znative_init` and caches it in a static has frozen the user's settings as of shell startup. If the user changes the parameter later — at a prompt, in a function, for one invocation via a prefix assignment — the plugin will not see it, and there is no error and no clue. Reading through `host.getvar` on each call is a lookup into the shell's parameter table, which is the same work the shell does when it expands `$VAR` itself, and it makes the plugin behave the way a shell function would have. Cache only when you have measured that you need to, and cache in a way that a change can invalidate.

Never write into your own store directory

Here is the rule that has a wrong answer with teeth. `store/<name>@<version>/` is the installed plugin tree, and Chapter 6 computes a SHA-256 over that copy at install time and records it as the plugin's `integrity` field. Chapter 26 makes the consequence explicit: if a store copy is altered after that record is written, its bytes no longer match the recorded `sha256-<hex>`, and the mismatch is detectable.

That mechanism does not distinguish between an attacker, a corrupted disk, and a plugin writing its own cache file next to its own library. All three produce the same outcome — content that no longer hashes to what the index says — and the whole value of the integrity hash is that the outcome means something. A plugin that

writes into its store directory converts a meaningful signal into noise, for itself and for the user's habit of trusting the signal.

The second reason is more prosaic and just as decisive: the store directory is keyed by version. A plugin that kept its data there would lose it on every update, because `update` installs into a new `store/<name>@<version>/` and leaves the old one for `gc` to reclaim. State that vanishes when the code is upgraded is not state; it is a cache with a very surprising invalidation policy.

Where state actually goes

Chapter 18's `zsh-z` is the model: the frequency database is `~/.z`, a file in the user's home directory, created and owned by the plugin, entirely outside anything `znative` manages. Chapter 34 covers what that means for removal — the file survives, which is the right default because it is the accumulated value the user would not want an uninstall to destroy.

The properties that make this the correct location are worth being explicit about, because they are the criteria for any plugin's choice. State outside the store survives `update`, which is what you want, since a version bump should not reset a history. It survives `gc`, because `gc` only touches store directories the index does not name. It is not covered by the integrity hash, which is correct, because mutable data has no business being covered by a hash of installed content. And it is inspectable and deletable by the user with ordinary tools, which for anything that accumulates is a property they will eventually want.

A plugin should let the user choose the path — through a shell parameter, per the section above, with a sensible default — for the same reason `zsh-z` does. A machine with a shared home directory, a user who keeps caches on a different filesystem, a test run that wants an isolated database: all of them are one parameter away if the plugin exposes one and impossible if it hardcodes.

State and the many-shells problem

Chapter 33's concurrency discussion applies to a plugin's own state with more force than it applies to the index, because a plugin's state is written far more often. A frequency database updated on every directory change, across a screenful of open shells, has that many writers to one file.

The design choices available are the ordinary ones and the tradeoffs are the ordinary ones, but two observations are specific to this setting. First, the non-forking model of Chapter 20 means concurrent access *within* one shell is a threading problem you can solve with a lock in your own address space, while concurrent access *across* shells is a genuine multi-process problem and no amount of Rust type safety helps — those are separate concerns and it is easy to solve only the first and believe you are done. Second, a read-modify-write of a whole state file is the same hazard as Chapter 33's index race, and it has the same mitigation shape: either make the update append-only, so that concurrent writers do not clobber each other's contributions, or make the write atomic in the filesystem's terms rather than in your program's.

The failure mode differs from the index's in one important way, though: there is no self-healing conditional here. A lost index row is reinstalled by the next `load`. A lost frequency entry is simply gone, and the user's only signal is that the directory they visited yesterday ranks lower than it should. Silent, gradual data loss is worse than a loud failure, which argues for the more conservative write strategy in exactly the case where it is tempting to skip it.

Chapter 38 — The `.zshrc` as the Manifest

The omissions list — no lockfile, no per-project manifest, no declarative plugins file — reads as three things `znative` does without. This chapter reads it the other way: those three things are absent because the artifact that would have contained them already exists, is already under version control on most of the machines this matters on, and can do things none of them could.

The startup file is already declarative enough

A block of `load` lines names a set of plugins and an order. That is what a declarative plugins file contains, expressed as commands rather than as a table, and the expression difference is smaller than it looks: `znative load owner/repo` has one verb and one argument, and the argument is the ecosystem’s canonical source spec. Nobody has to learn a schema to read it or to write it.

What it carries that a table would carry: the set of sources, with their pins, in their load order. What it carries that a table could not: everything else in the startup file, in position. The `compinit` call that Chapter 14’s completion plugins must come after is *in the same file, above them*, which makes the ordering rule visible rather than a constraint the manager has to encode. `zsh-syntax-highlighting` at the bottom is in the same file below everything it must observe. A separate plugins file would have to either duplicate that ordering information or leave the user to keep two files consistent, and Chapter 31 already argues that the ordering is not derivable — it is a fact about what wrapping the line editor means, and no manifest field captures it.

And it is executable, which a table is not

This is the capability difference and it is not small. A `.zshrc` is a shell program, so a `load` line can be conditional on anything the shell can test.

```
if [[ $OSTYPE == darwin* ]]; then
  znative load owner/mac-only-plugin
fi

[[ -n $SSH_CONNECTION ]] || znative load owner/local-only-plugin
```

Neither of those needs a feature from `znative`, and that is the argument. A declarative plugins file that wanted to express “this plugin only on macOS” would need a conditional syntax, which is a predicate language, which is the first step down the road Chapter 4 declines to walk. Here the predicate language is the shell — the thing the user already knows better than any DSL a plugin manager could offer, with the full vocabulary of parameter tests, command availability checks, hostname comparisons, and anything else they can write.

The same mechanism covers the case a per-project manifest is usually proposed for. A user who genuinely wants a different plugin set for different work can express it with a conditional on a directory, a parameter, or a marker file, in the file where they already express everything else. What they cannot express is a plugin set that belongs to a *repository* they hand to a collaborator — and the closing chapter’s argument is that this is correct, because a collaborator’s shell is their environment and not the project’s to configure.

What it reproduces on a new machine, and what it does not

Copy the `.zshrc` to a fresh machine and start one shell. Every `load` line finds its source absent, installs it, and loads it — the Chapter 4 conditional, on a new machine, doing the same thing it did on the first shell of the old one. There is no separate install step, no import verb, and no second artifact that could have drifted out of sync with the first. That is the bootstrap property a lockfile-based manager buys with a second file and a second verb, and it falls out here from the load-or-install conditional alone.

Being precise about what converges: **the set and the order converge; the resolved content converges only where you pinned it.** Two machines provisioned a week apart from the same `.zshrc` will have the same plugins in the same order. For every plugin whose source carries an `@ref`, they will have the same tree, because install and `update` both honour the pin (Chapter 5). For every unpinned source, they will have whatever the default branch held at their respective fetch times, which may be two different trees.

That is a real gap relative to a lockfile and it should be named as one rather than explained away. A lockfile's whole job is to record the resolution so that a second machine reproduces the first exactly, including for dependencies nobody pinned. `znative` does not do that, and a user who needs it has one mechanism — pin the sources whose content must match — which is per-plugin, explicit, and recorded in the index where `update` will respect it.

The compensating property is that the divergence is *detectable*. The `sha256-<hex>` in each index record is over the store copy, so comparing two machines' records for the same plugin answers whether their content is identical, without transferring or reading the trees. Chapter 27 makes the point in the context of a single user with several machines; it is the same point here. A lockfile prevents divergence; the integrity hash detects it. Those are different guarantees and only the second one is on offer, which is the same honest split Chapter 29 draws between prevention and detection everywhere else.

The practical discipline

What falls out is a small set of habits rather than a feature.

Pin what must not move, and leave the rest unpinned — Chapter 27's rule, restated here because the multi-machine case is where it earns most. The plugins whose behaviour your muscle memory depends on, and any plugin you have actually audited, are pin candidates. The rest are not, and pinning everything converts a zero-maintenance configuration into a manual upgrade queue.

Keep the `load` block contiguous and ordered, with `compinit` above the completion plugins and the syntax highlighter at the bottom. The ordering is information, the file is where it lives, and a block broken up across a long startup file is a block whose order is hard to verify.

Treat `znative update` as a deliberate act with a date, not a thing that happens. Nothing in the design updates on its own (Chapter 29), which means the moment new code enters your shell is a moment you chose. On several machines, choosing it at different times is exactly what makes unpinned sources diverge — so if you care that two machines match, update them together and compare hashes afterwards, which is a two-command check rather than a workflow.

Chapter 39 – Publishing a Plugin: Versions, Tags, and the Release

The book has covered writing a plugin, porting one, designing its command surface, testing it, and debugging it. It has not covered the act that turns all of that into something a stranger can type one line to get: cutting a release. That act is where four separate version numbers, each owned by a different party and read by a different mechanism, have to be brought into agreement — and nothing in the tool checks whether you did.

Four numbers, four jobs

Chapter 3 separates two of them: the `znative` crate version a plugin author writes in `Cargo.toml`, and `ABI_VERSION`, the constant inside that crate. A publishing author has two more to think about, and the four are not variants of one idea.

Number	Written by	Read by	Fails at
the <code>@ref</code> pin	the user , in a source spec	git, at fetch	resolution, if the ref is absent
<code>[plugin] version</code>	the author , in <code>znative.toml</code>	the store key and the index row	never — nothing compares it
the <code>znative</code> dependency	the author , in <code>Cargo.toml</code>	cargo, at build	compile, on skew
<code>ABI_VERSION</code>	the crate	host and plugin, at load	load, on mismatch

The bottom two are Chapter 3's pair and Chapter 21's subject. The top two are this chapter's, and they interact in a way the mechanics make possible and nothing in the mechanics prevents.

Notice what the second row says. Chapter 19 already establishes that the manifest's `version` is an identifier rather than a constraint — it is the `<version>` in `store/<name>@<version>/` and nothing compares it against anything at load. Chapter 19 draws the author-facing conclusion that command names have no version gate. The release-facing conclusion is different and sharper: because nothing compares that number, nothing notices when it stops describing the content it keys.

What a moved tag does to a store key that did not move

Follow the mechanics. A user has a plugin installed at `store/foo@1.0.0/`. The recorded source is unpinned, or pinned to a branch. The author pushes new work. The user runs `znative update foo`.

Stage 2 re-fetches and gets the new tree. Stage 3 reads the manifest and gets `version = "1.0.0"`, because the author changed code and not the manifest. So stage 5's destination is `store/foo@1.0.0/` — the key the previous install already occupies. Stage 6 hashes the content that is there afterwards, and stage 7 rewrites the row with that hash.

The outcome is a plugin whose recorded version did not change and whose recorded integrity hash did. Nothing is broken; every invariant Chapter 22 establishes still holds, because they are all invariants about ordering rather than about identity. But three things a user relies on quietly stopped working.

There is no orphan, so `gc` has nothing to reclaim and `gc --dry-run` shows nothing — which is what a user checks after an update to see what happened. There is no second directory, so Chapter 27’s suggestion to diff the two `store/<name>@<version>/` trees has one tree to diff. And `znative list` prints the same line it printed yesterday, because `list` prints name, version, kind, and source, and three of those four are unchanged while the fourth was never a version indicator.

The only surviving signal is the `sha256-<hex>` in the record, which `info` prints. That signal still works, and Chapter 38 is right that comparing hashes across two machines detects divergence. What changed is its *interpretation*. Before, a hash mismatch between two machines for a plugin at the same version was a strong signal of something odd. After, it is the expected result of two machines having updated an actively developed plugin at different times, and the version column tells you nothing about which case you are in.

The rule that falls out

Bump [plugin] version whenever the content changes. The number is not decoration and it is not for humans; it is the key on the directory that holds your code, and Chapter 6’s whole browsability argument — `store/forgit@0.1.0/` says what it is at a glance — is only true while the number is a claim about content rather than a string that happens to sit in a TOML file.

Chapter 8 gives four situations where a manifest earns its keep, and all four are about the *shape of the tree*: the name is not the repository name, the load order matters, something looks like a signal and is not, the library is not where the convention says. Publishing is a fifth reason and it is about the *release* rather than the tree. A plugin nobody but you installs can rely on inference. A plugin you have told strangers to install cannot, because inference gives you a name derived from the repository basename and gives the version question no answer you chose. If you publish, ship a `[plugin]` table, and treat its `version` field as the thing you edit in the same commit as the code.

Tag what you told people to pin

Chapter 19’s advice to the author is to use the pin and tell users about it. That advice has a prerequisite the author owns: the ref has to exist, and it has to mean what the README says it means.

Here is the part worth being exact about, because Chapter 27 states it with a hedge that is load-bearing. A pinned source, it says, re-fetches the pinned ref on `update`, “which normally reproduces the same tree.” *Normally* is doing real work in that sentence. `znative` re-fetches the ref; git resolves the ref to whatever it currently names; and a git tag is a mutable pointer. An author who moves `v1` to a newer commit has moved every user’s pin, and no mechanism anywhere in this book will tell them. The manager honoured the pin exactly as documented. The pin moved.

So the three pin forms are not three spellings of one guarantee:

- **A commit pin is immutable.** Nobody can move it, including you. Chapter 5 notes the cost — a shallow branch clone may not reach an arbitrary older commit, so the install may fall back to a full history fetch — and that cost is the price of the only pin the author cannot break.
- **A tag pin is as stable as the author’s discipline.** For an author who cuts `v1.2.0` once and never touches it, it is effectively immutable and much cheaper to fetch. For an author who maintains a moving `v1`, it is a channel rather than a pin, and users who read it as a pin will be surprised exactly once, at a time they did not choose.
- **A branch pin is explicitly a channel** and nobody mistakes it for anything else.

The author’s obligation follows: publish immutable tags, say so, and if you also want to offer a moving major-version channel, give it a name that reads like a channel rather than like a version. The user’s obligation is

Chapter 27's, with one addition — a user who needs a pin that genuinely cannot move under them pins a commit and accepts the fetch cost.

Prerequisites you cannot declare

Chapter 32 explains why there is no dependency graph: shell plugins are a flat set, not a transitive closure, so a resolver would have nothing to resolve. That is the right conclusion and it leaves the author with a practical question it does not answer. What do you do when your plugin genuinely requires something else to be present?

There is no field for it. The manifest of Chapter 8 has an identity table and a kind table, and neither names another package, an external binary, or a minimum host generation. So every prerequisite a plugin has is undeclared by construction, and the only honest strategies are runtime ones.

An external program is the easy case and the book already handles it. Chapter 16's rule is that a port keeps the original's runtime dependencies and that `Command::new` fails cleanly when the program is absent. Turn that failure into a status and a message on stderr rather than letting it surface as a confusing empty result, and you have done everything available.

Another script plugin is checkable, narrowly. The host API has no “is plugin X loaded” method — Appendix E's table is `print`, `eval`, `getvar`, `setvar`, `getfunction`, `addfunction`, `register_builtin`, `add_match`, `install_completion`, and none of those asks about plugins. But `getfunction` returns `Option<String>`, and a script plugin's contribution to the shell is functions. So `host.getfunction("some_other_plugin_fn")` returning `None` is a usable probe for “that plugin's functions are not defined in this shell.” It is indirect, it depends on a name the other plugin never promised you, and it is the best available.

Another native plugin is not checkable through the documented surface. A bare `zmodload -R` lists loaded plugins, but `host.eval` returns only an exit status, so a plugin cannot read that listing back. There is no method that answers the question. If your plugin requires another native plugin, the requirement is documentation and nothing else, and the failure mode when a user misses it is a command that does not exist.

The conclusion is uncomfortable and worth stating rather than working around: **prefer not having prerequisites**. A plugin that depends on another plugin has taken on a coordination problem with no mechanism, in a tool that deliberately has no mechanism, for a category of software whose ordering is the user's to declare. If the two things must go together, they are one plugin.

What a publishable repository contains

Everything below is derived from mechanics already in the book; none of it is a requirement the tool enforces.

A `znative.toml` with a `[plugin]` table whose `name` is the command users type rather than the repository slug, whose `version` you bump with the content, and whose `description` is the line `znative add` prints on success. A kind table only where the tree's shape does not already say it correctly. For a Rust plugin, a `Cargo.toml` with `crate-type = ["cdylib"]` and a `znative` dependency whose constraint names the host generation you target — and `source`, not a committed binary, unless Chapter 24's homogeneous-fleet exception genuinely applies to your audience. An immutable tag for the release, matching the manifest version, so that the `@ref` you tell users to pin resolves to the tree you tested. And prose for every prerequisite, because prose is the only place a prerequisite can live.

The install-side check is Chapter 35's fourth layer, and it is the one that catches release mistakes rather than code mistakes: point `$ZSHRS_HOME` at a scratch directory, `znative add path:.`, and read `znative info` on the result. The store path tells you the key your manifest produced. The lib or the source-files list tells you what a user's `load` line will actually pull in. Both of those are release facts, and both are wrong more often than the code is.

Chapter 40 – The Day the ABI Moves

Chapter 21 prices an ABI bump for the plugin author: every plugin must be rebuilt against the new crate, paid at build time, with a refusal rather than a degradation as the failure mode. That is the correct framing for the person making the change. This chapter is the same event from the position of a user who did not make it, did not schedule it, and discovers it at a prompt.

What the morning looks like

You upgrade zshrs. `ABI_VERSION` has advanced. Every native plugin in your store holds a `--release` library compiled against the previous constant, and Chapter 15's check is two-sided, so each of them is refused by whichever side notices first.

Chapter 9 already describes what a failing `load` line does: it prints `znative: <reason>` on stderr, returns non-zero, and the shell moves on. Apply that per line. The startup you get is a burst of refusals scrolling past, followed by a prompt, followed by a shell that is missing every compiled command you had and nothing else.

The partition is the diagnostic. Script plugins load normally, because a script plugin has no version gate — it has no ABI to be gated on. So if the things that stopped working are *exactly* the compiled ones and *all* of the compiled ones, the cause is not any plugin. It is the host underneath them, and no amount of looking at an individual plugin will show it.

Chapter 36's localisation procedure confirms it in two commands, applied at scale rather than to one plugin. A bare `zmodload -R` lists what is loaded in this process, and after an ABI move the listing is short or empty. `znative info` still describes every plugin in full, because the index was never touched — the install records are intact, the store copies are intact, the hashes still match their content. Present in `info`, absent from the listing, for every native plugin at once.

Why the store cannot narrow it for you

There is a query you would like to run here and it does not exist. Appendix C lists what an index record carries: `name`, `version`, `source`, `kind`, `integrity`, and either `lib` or `source_files` plus `fpath`. Nothing in that set records which ABI generation a library was built against.

That absence is reasonable — recording it would mean the index carried a fact about a build rather than about an install, and it would have to be maintained correctly forever for a query nobody runs on a normal day. But it has a consequence worth knowing on the abnormal day: **the recovery is unconditional rather than targeted**. You cannot ask the index which installed libraries are stale. You rebuild the native plugins because they are native, which is a fact the `kind` field does carry, not because anything told you they were out of date.

The recovery, and what it costs

`znative update` with no argument re-resolves and reinstalls every plugin. Reinstall runs the build step — Chapter 23 is explicit that the build happens in `add/load`'s install branch or in `update`, never on a plain `load` — and the build resolves the `znative` crate afresh. For any plugin whose `Cargo.toml` constraint admits the new crate generation, that produces a library against the new `ABI_VERSION`, and the next shell loads it.

That is the whole fix, and it is one command, which is the good news. The bad news is what else it does.

It reinstalls every script plugin too, which is wasted work but harmless. More importantly, it re-fetches every *unpinned* source, which means the newest content lands for all of them. Chapter 29 makes the point that new code enters your shell only when you run `update`, and treats that as the property that matters most for a threat model built around unnoticed change. A blanket update run for ABI reasons collects that change as a side effect. You went looking for a rebuild and also took delivery of everything that moved since last time.

So the sharper move is per-plugin: `znative update NAME` for each native plugin, which rebuilds exactly what needs rebuilding and leaves the script set alone. `znative list` prints the `kind` column, so the set of names to run it against is one read of the index away. It is more typing and it keeps the two changes — “this is now built for the current host” and “this is now the author’s latest work” — from arriving in the same undifferentiated batch.

Where the recovery does not reach

Three cases survive the rebuild, and each has a different owner.

A plugin whose crate constraint predates the bump. Its `Cargo.toml` names a `znative` version cargo will still resolve, and the crate at that version does not match the host you now have. What you get is not a load refusal but a build failure — Chapter 25’s compile-time shadow of the load-time check — and the fix is in the plugin’s manifest, which is the author’s file and not yours. Your levers are to move the plugin to a newer `@ref` if the author has published one, or to remove it and do without.

A plugin you pinned. `update` honours the pin, which is exactly the property Chapter 27 defends and exactly the property that hurts here: the rebuild runs against the tree that tag froze, whose `Cargo.toml` is whatever was correct when you pinned. This is a real tension between two pieces of good advice in this book and it does not resolve cleanly. Chapter 27 says pin what your muscle memory depends on; Chapter 38 repeats it for the multi-machine case; and an ABI move is the event that turns a pin from protection into an obstacle. The honest statement is that a pin is a decision to stop taking the author’s changes, and an author’s changes are how a plugin keeps up with the host.

A plugin that ships a prebuilt library with `build = false`. There is nothing to rebuild. `update` re-fetches the recorded source and adopts whatever prebuilt `lib*.dylib,so` that source now carries, exactly as it did on first install (Chapter 23) — so the plugin recovers if and only if the author has published an artifact built against the new ABI, on the schedule the author chose.

That third case deserves to be connected to where it came from. Chapter 24 presents the prebuilt binary as a legitimate optimization inside a homogeneous fleet: skip the compile, skip the toolchain requirement, install by copy. Every word of that stands. What it does not price is this morning. A fleet that committed to prebuilt artifacts has moved the ABI rebuild from “runs automatically on `update`” to “requires a build host to produce a new artifact and a fetch to distribute it,” and the trigger for that work is a host upgrade rather than a plugin change. The optimization is real and the bill arrives on a schedule the fleet does not control.

Sequencing an upgrade you control

The order is more constrained than it looks, and one plausible ordering is actively wrong.

You cannot usefully rebuild first. The `znative` crate is published independently, so cargo would happily resolve the newer generation and produce a library for it — and that library would be refused by the host you are still running, because the check is equality (Chapter 21) and equality has no tolerant direction. Pre-building against a host you have not installed yet does not shorten the outage; it starts it early.

So the sequence is forced: upgrade the host, accept exactly one degraded shell, run the updates in that shell, and open a new one. Chapter 34 supplies the last step’s justification — the shell that ran the update is running whatever it had mapped when it started, and a new shell is how a shell picks up a new version.

Do it in one shell rather than in all of them. Chapter 33's advice for a first start on a new machine applies unchanged here, for the same reason and a different trigger: many shells each discovering that the same set of plugins needs rebuilding, and each starting its own `cargo build --release`, is a contended first start wearing a different hat.

Why this is the design working

The alternative to a burst of refusals is a burst of struct reads at wrong field offsets, which Chapter 21 works through in detail and which produces no message at all. That argument is already made and does not need remaking.

The point worth adding is about *shape* rather than about safety. This cost is concentrated and scheduled. It lands once per host generation, on a day you chose by running an upgrade, on every native plugin simultaneously — which is what makes it a morning's work rather than a mystery. The failure mode the version constant replaced was not only more dangerous; it was diffuse, arriving one plugin at a time, at whatever moment a mismatched struct field happened to be read. Loud and all at once is a better failure than quiet and staggered, and it is the failure this design chose.

Chapter 41 – Re-entering the Shell

Every zsh user has the same reflex in their fingers: edit `.zshrc`, then `source ~/.zshrc` to pick up the change without losing the session. The reflex predates every plugin manager and survives all of them. What it does to a block of `znative load` lines is different for each kind of plugin, is not what the reflex assumes in either case, and is worth knowing before you conclude something is broken.

What the native lines do the second time

A `load` line for an installed plugin takes the fast path: read the index, match, go straight to Chapter 22's stage 8, which for a native plugin is `zmodload -R` on the recorded library. In a shell where that plugin is already loaded, the result is the refusal of Chapter 10 — loading a plugin whose name is already loaded is rejected.

So you get one `znative: <reason>` on stderr per native plugin, each returning non-zero, and Chapter 9's rule carries the rest: the line failed, the shell moved on, the block finished.

Nothing was damaged and nothing needs fixing. The refusal is the correct outcome, because the alternative is two registrations under one name and a shell where “which of these answered” has no answer — which is precisely the ambiguity the refusal exists to prevent. The only cost is noise, proportional to how many native plugins you have, arriving in the middle of a re-source you expected to be quiet.

What the script lines do the second time

There is no equivalent refusal for a script plugin, and there could not be. The load is Chapter 7's synthesised fragment — `fpath=` prepends followed by `source` calls — and both halves succeed unconditionally, because neither is a registration into anything that could detect a duplicate.

So the plugin's code runs again, and whether that matters is a property of the plugin rather than of `znative`. Idempotence here is per-construct, not per-plugin:

- **Function definitions, aliases, and option settings are idempotent** by overwrite. The second run replaces what the first run installed with the same thing.
- `fpath=(<dir> $fpath)` is **not**. The same store directory is now on `$fpath` twice. Lookups still resolve, so this is harmless in effect, and it is a real, observable difference between the shell you have and the shell a fresh start would have given you.
- **Appends to hook arrays are not**. A plugin that adds a function to a per-prompt hook array and does not guard against being loaded twice now has it there twice, and it runs twice per prompt, for the life of that session.
- **Widget wrapping is not, and this is the sharp one**. A plugin that wraps an existing widget wraps whatever is currently bound — which, the second time, is its own wrapper. Chapter 9's syntax-highlighting-last rule is about wrapping *order*; this is about wrapping *depth*, and the two produce symptoms that look alike and have nothing in common.

The honest summary is that re-sourcing a startup file is a shell operation with shell semantics, and `znative` neither improves nor worsens it. What the design does contribute is a false reassurance: the `load` line itself is perfectly idempotent-looking, which invites the assumption that the plugin behind it is too.

exec is the right primitive, and this design makes it cheap

Replacing the shell with a fresh one is the only reliable way to arrive at the state your `.zshrc` describes, because it is the only way that starts from an empty shell rather than from the one you already have.

The interesting part is why that is affordable here, since “just restart your shell” is advice whose cost is exactly the thing plugin managers spent a decade engineering around. Chapter 28 supplies both halves. Every `load` line takes the fast path, which means the index and no network — the network was paid once per plugin, on the first start after the line appeared, and never again. And for a native plugin the library is already resident, so the load is a `dlopen` of pages the kernel has, which is the same property Chapter 17’s shim exploits for a per-keystroke callback.

The incentive that produces is worth naming precisely, because it points the opposite way from the incumbent design. In a manager whose per-plugin startup cost is a parse, restarting is the expensive operation and re-sourcing is the cheap one — so users are pushed toward the operation with the worse semantics. Here restarting is cheap and correct, and re-sourcing is cheap and approximate. Chapter 28’s doctrine adds the last piece: because nothing is deferred, the new shell is fully configured when it paints, so there is no window in which the restarted shell is a worse shell than the one you replaced.

Changing one thing without restarting

The block-level reflex is the wrong tool for a one-plugin change, and there is a better one for most cases.

Adding a plugin is `znative add` at the prompt (Chapter 4). It installs and loads in the running shell, which is the effect you wanted from re-sourcing, with none of the double-loading, because it touches one plugin instead of replaying the block. Add the `load` line to the file afterwards for future shells; that line will take the fast path forever, since the plugin is already in the store.

Removing one is Chapter 34’s subject and the answer differs by kind: genuinely reversible for a native plugin, not reversible at all for a script plugin, with the `$fpath`-into-a-deleted-directory wrinkle in between.

Reloading a native plugin you just rebuilt is Chapter 27’s development loop — `zmodload -uR` the name, `zmodload -R` the new library — and the unload half is mandatory for the same reason the re-source refusal above happens.

Reloading a script plugin whose file you edited has no answer, and that is worth stating rather than leaving as an exercise. There is no unsource. And the file you edited is not the file that loads anyway: Chapter 27’s copy property means the store holds a copy taken at install time, so editing your working tree changes nothing until you install again. The loop for a script plugin under development is edit, install from the `path:` source, and start a new shell — three steps, none of which can be collapsed.

The non-interactive case

Chapter 9 argues that the plugin block belongs in `.zshrc` and not `.zshenv`, and that scripts should not depend on ambient interactive configuration. Both stand. There is a construction they leave available that is worth naming, because it is the load fast path used for something other than a prompt.

A `zshrs -f` shell reads no startup files, so it has no plugins — and a `znative load` inside it is an index read plus a `dlopen`, with no network and no compile. That makes it possible to write a script that runs with *exactly* one plugin present and nothing else, deterministically, in a process with none of your configuration in it. Chapter 35 uses that shape for testing a plugin by path; the same shape with `znative load` names an installed plugin instead, which is the difference between exercising a build artifact and exercising an install.

The caveat from Chapter 9 does not go away and should be attached to it: such a script fails on any machine where that plugin is not installed, with a “command not found” that names nothing useful. This is a construction for scripts whose machines you control, not a general technique.

Chapter 42 – Completion Ordering in Both Directions

The book states one completion ordering rule and states it several times: load a native completion plugin *after* `compinit`. Chapter 14 gives it with the reason, Chapter 25 gives it as the diagnosis for a silent failure, and Chapters 30 and 38 repeat it as migration and startup-file discipline. What none of them states is the mirror rule, which governs the far larger population of script plugins that ship completers — and which points the opposite direction.

What `compinit` does with `$fpath`, and when

Be precise about the two things that are easy to fuse. A completer for a command is a file named `_command` sitting in a directory on `$fpath`, and two separate mechanisms have to find it.

The *association* — the knowledge that `_command` is the thing that completes `command` — is built when `compinit` runs, from the tag at the top of each `_*` file it finds by walking `$fpath` at that moment. The *body* is autoloaded later, on demand, by ordinary `$fpath` lookup when the completer is actually invoked. Chapter 34's wrinkle is about the second mechanism: a `$fpath` entry whose directory has been deleted contributes nothing to lookups, so the plugin's autoloadable functions stop being findable while functions it already defined keep working.

The consequence for load order is about the first mechanism. A directory that joins `$fpath` *after* `compinit` has run holds completer files that are perfectly findable and completely unassociated. Nothing scanned them, so nothing knows what they complete. Pressing tab on the command does whatever it did before the plugin existed, the file is sitting right there, and nothing is wrong with it.

Therefore script completion plugins load before `compinit`

A script plugin whose value is a `functions/` directory of `_*` completers — which is exactly what Chapter 7's `functions/` detection signal is a signal of — has to be on `$fpath` when `compinit` runs. So its `load` line goes above the `compinit` line, not below it.

That is the opposite of Chapter 14's rule, and the two are not in conflict; they are the same fact approached from two sides. Completion setup happens once, at a point in your startup file. A script plugin needs to be **present before** that point, because the scan is what discovers it. A native plugin needs the completion system to **already exist**, because the macro records a completion request and the host defers the compsys wiring until completion first fires — which is Chapter 14's reason for the deferral in the first place.

So the canonical block has three regions rather than one:

```
# 1. script plugins that ship completers – before the scan
znative load zsh-users/zsh-completions

autoload -Uz compinit; compinit

# 2. native completion plugins – after the system exists
znative load MenkeTechnologies/zshrs-kubectl-completion

# 3. everything else, in whatever order it needs
znative load MenkeTechnologies/zshrs-forgit
znative load zsh-users/zsh-syntax-highlighting # last, always
```

Chapter 38's discipline — keep the block contiguous and ordered — is what makes that structure legible. The ordering information is in the file, and the file is the whole explanation, which is the property Chapter 31 identifies as the incumbents' correct conclusion and this one inherits.

The plugin that is both

Most real script plugins are not purely completers. A plugin that ships `_*` files *and* functions, widgets, or key bindings has one `load` line and two ordering constraints on it: above `compinit` for its completers, and wherever its widgets need to be relative to other plugins for everything else.

Usually those are compatible, because `compinit` sits early and most widget ordering constraints are about being late. The case where they are not is specific: a plugin that must wrap a widget defined by a plugin that itself has to load after `compinit`. Then one line cannot satisfy both.

There is no lever for that, and it is worth saying so rather than implying one exists. The `[script]` table lets an author declare `source` files and `fpath` directories separately, but Chapter 7's fragment applies them together — `fpath=` prepends followed by `source` calls, synthesised from one index record and evaluated at one moment. A user cannot ask for half of a plugin now and half later, because the record is the unit. The available responses are to install the completer half from a separate source if the author publishes one, or to accept whichever half loses.

A plugin added mid-session

The same symptom arrives by a second route with no `.zshrc` line to blame. `znative add` at a prompt installs and loads immediately, in a shell where `compinit` ran at startup, minutes or hours ago. The plugin's `functions/` directory joins `$fpath` now. Nothing scans it.

So a plugin you just added does not complete, in the session you added it in, even though its `load` line is in exactly the right place in a file that has not been read yet.

The distinguishing question is one shell start: does a **fresh** shell complete correctly? If yes, the session is the problem, the file is fine, and the fix is either a new shell or re-running the completion setup in this one. If no, the ordering in the file is the problem and the fresh shell reproduced it faithfully. Two branches, one experiment, no guessing — which is the shape Chapter 36 uses for native generators and which that chapter does not cover for this case, because its completion section is about a generator's behaviour rather than about a completer's discovery.

Why `znative` does not sort this out for you

Chapter 31 gives the general principle: order is the user's to declare, because shell plugins interact through global mutable state and the interactions are not declarable in any vocabulary a manager could check. That argument covers this case and is not worth restating.

The specific reason is narrower and sharper. Detection *does* tell the manager that a plugin contributes completers — the `functions/` signal and the `[script]` table's `fpath` list are exactly that information. What the manager cannot know is where `compinit` is. `compinit` is a line in a program that is calling `znative`, not a thing `znative` owns, schedules, or can observe. A manager that wanted to place completion plugins correctly would have to reach up into its caller and reorder it, which is a strictly larger request than “install this plugin” and one the tool declines on the same grounds it declines every other scheduling feature.

The failure signatures, side by side

Symptom	Kind	Cause	Fix
tab does nothing; plugin loaded, completer file present	script	joined <code>\$fpath</code> after the scan	move the line above <code>compinit</code>
tab does nothing; generator emits correct candidates when called directly	native	loaded before the completion system existed	move the line below <code>compinit</code>
tab does nothing; generator emits nothing when called directly	native	the generator is wrong	fix the generator
tab does nothing only in the session where you ran <code>add</code>	script	the scan ran at startup, before the plugin existed	start a new shell
completer never fires despite correct ordering	native	the generator's name has a leading underscore	rename it

The last row is Chapter 14's naming rule and Chapter 36's second check, included here because this table is where someone will look for it.

Chapter 43 – One Kind Per Plugin

Kind detection classifies a tree as native or script. The manifest carries exactly one of two kind tables. The index record holds either a `lib` or a `source_files` plus `fpath` pair. The exclusivity runs cleanly through every layer of the design, and a plugin author who needs both halves does not discover it as a warning — they discover it as a wall, usually after writing the compiled half.

What each kind can install into a shell

Chapter 11's table has a row for this and it is compressed to the point of being easy to skim past. Expanded:

A **script plugin** installs whatever shell code can install, because it *is* shell code being executed. Functions, aliases, options, ZLE widgets, key bindings, hook-array entries, `$fpath` contributions, exported parameters. There is no list of permitted effects, because the mechanism is `source` and the effect of `source` is “run this program.”

A **native plugin** installs builtins and completion requests. That is the list. Not a curated subset of a larger set — the whole documented surface by which a compiled plugin changes the shell at load is the `builtins` and `completions` maps of `declare_plugin!`.

So the absences are specific and total: a native plugin does not bind a ZLE widget, define an alias, set a shell option, add a per-prompt hook, or put anything on `$fpath`. Not because those are forbidden by a policy, but because there is no method for them in the host table and — as the next section works out — no moment at which to call one if there were.

The missing code slot

Chapter 12 states that the macro gives you no lifecycle beyond init, and that the init in question is the one `declare_plugin!` generated rather than one you wrote. Chapter 20 states the same thing from the other side, as the reason shared state is initialised lazily: there is no init hook that runs before the first command, and no guarantee which command is first.

Put those together and follow them one step further than either chapter does. **A native plugin's first opportunity to run code you wrote is the first invocation of one of its builtins.** Loading is registration and nothing else. The library is mapped, the symbol is resolved, the macro's init checks the ABI version and installs your handler pointers into the registry, and control returns. Not one line of your logic has executed.

That is why `host.eval` — the one method on the table that could install a widget or an alias, by running the shell code that installs it — has no moment at load in which to be called. The escape hatch is real and there is nowhere to stand while using it. A plugin that wants shell-level constructs present *when the plugin loads* is asking for something the load path does not have a slot for.

Three ways to ship a plugin that needs both

Each of these composes documented parts. None is a feature; all are consequences.

Two plugins, two lines. Publish the compiled half and the shell half as two repositories, and tell users to load both. This is the cleanest and the most honest, and it costs exactly what it looks like: two `load` lines, two store

entries, two versions to keep in step, and — per Chapter 39 — no mechanism whatsoever for keeping them in step, since a plugin cannot declare a prerequisite on another plugin and a native plugin cannot even detect one at runtime. One repository cannot be the answer here, because identity comes from one manifest and the store key is one `name@version`: one repository is one plugin.

A script plugin that loads its own library. Declare `[script]`, ship a prebuilt `cdylib` in the tree, and have one of your sourced files run `zmodload -R` on it. Chapter 10 documents that primitive and Chapter 27 shows it driven by hand, so this is composition rather than invention; a sourced file can derive the directory it was sourced from, which is where the library sits in the store copy.

What it costs is severe and worth spelling out. The tree is classified script, so the native build step does not run at all — `build` is a field on the `[native]` table, and there is no `[native]` table here. The library must therefore be committed, prebuilt, which is exactly Chapter 24’s unrescuable case: the plugin installs on hosts of the matching architecture and fails everywhere else, with no `Cargo.toml` path to rescue it, because the manifest already told `znative` this is a script plugin. And Chapter 40’s morning is worse for this shape than for any other, since a prebuilt artifact is the one thing `update` cannot rebuild for you.

A native plugin with a setup builtin. Register a builtin whose entire job is to run the shell-side installation through `host.eval`, and document one extra line after the `load`:

```
znative load owner/my-plugin
my-plugin-setup
```

This is the option that keeps every property the rest of the book spends chapters establishing. Build-on-install still runs, so the plugin ships source and installs on every architecture. There is one store entry, one integrity hash, one version. An ABI move is fixed by `znative update` like any other native plugin. And the cost is one visible line in a startup file — a line the user can see, place correctly relative to `compinit` and everything else, and make conditional if they want to.

Approach	build on install	cross-arch	store entries	extra lines	survives an ABI move
two plugins	yes	yes	2	1	yes
script + prebuilt lib	no	no	1	0	only if the author ships a new binary
native + setup builtin	yes	yes	1	1	yes

Read the table as a ranking and the middle row is the one that trades the most for the least: it saves a line and gives up architecture portability and automatic ABI recovery. It remains defensible inside the homogeneous fleet Chapter 24 describes, and for the same reasons, which is the only place it is defensible.

Why there is no [hybrid] kind

The obvious feature request writes itself: let a manifest declare both tables and let the manager build the library, source the files, and be done. It is worth pricing rather than dismissing.

The index record’s shape would change from either/or to both, which touches Appendix C’s schema and everything that reads it — `info`’s output, the load path’s branch on kind, and reconciliation’s assumptions about what a record describes. That part is mechanical.

Detection is where it gets structural. Chapter 7’s native-first ordering rests on a specific asymmetry: a native repository plausibly contains stray `.zsh` files, while a script repository essentially never contains a `cdylib` `Cargo.toml`, because there would be nothing for it to build. That asymmetry is an argument for how to choose between the two kinds. A hybrid kind does not resolve the question; it replaces it with a harder one. A

tree carrying both a `cdylib Cargo.toml` and a `*.plugin.zsh` would now have three possible readings — native, script, or hybrid — and nothing in the tree's shape distinguishes “a native plugin that keeps a shim for reference” from “a plugin that means both.” Detection would have to stop inferring and start requiring a manifest for exactly the repositories the native path exists for, which is the outcome Chapter 7 is built to avoid.

And the payoff, measured honestly against the table above, is one line in a `.zshrc`. The setup-builtin option already delivers the capability, with every other property intact, at that price. A schema change plus a detection ambiguity plus a permanent widening of what a record can mean, to save one line the user can see and control, is not a trade this design would take.

The rule to carry

Decide which half of your plugin is load-bearing. If the value is compiled work — per-call computation, parallelism, a fast front-end — the plugin is native and the shell-level furniture is delivered by a documented setup step. If the value is shell integration — widgets, key bindings, hooks, completers — the plugin is a script plugin, and any compiled work it wants belongs in a separate plugin the user loads alongside it. The kinds are exclusive at every layer because they are answers to the question *what is this thing*, and a plugin that cannot answer that question is usually two plugins.

Chapter 44 – The Store’s Address, and the Per-Machine Assumption

The store root is `$ZSHRS_HOME/pkg/`, default `~/.zshrs/pkg/`. That sentence appears in Chapter 6 as a location and in Chapter 35 as a testing lever, and in both places it reads like a path rather than like a design decision. It is a design decision, and one of the book’s guarantees rests on a property of that path which is true on almost every machine and false on a specific and not-rare class of them.

What the variable actually selects

`$ZSHRS_HOME` names the root of everything `znative` owns: the `store/` directory of installed trees, `installed.toml`, and the `git/`, `cache/`, and `bin/` scratch. Point it somewhere else and you have selected a different store and a different index — not an additional one, a different one. Every verb follows it. `list` reads that index. `add` writes into that store. `gc` reclaims against that index.

Chapter 35’s fourth testing layer is the honest use of that: point `$ZSHRS_HOME` at a scratch directory, run `znative add path:.`, exercise the whole Chapter 22 pipeline, and delete the directory afterwards without having touched anything real. The isolation is complete because there is no shared state anywhere else — no per-project directory, no cache outside the root, no registry. The tool is global-only, and “global” turns out to mean exactly “under this variable.”

That also means the variable is the only mechanism the design offers for having more than one plugin set. There is no profile concept, no `--store` flag, no environment selector inside the tool. If you want two independent sets of installed plugins, you run with two values of `$ZSHRS_HOME`, and the two know nothing about each other. Whether that is a feature or a workaround depends entirely on what you are doing with it, and the next section is the case where it stops being optional.

The assumption underneath the cross-architecture guarantee

Chapter 24 makes a strong claim and Appendix K restates it as a table row: the store is per user **and per machine**, so the question “will this binary load here” is answered by construction — the binary in a store was built by the host that owns the store, for the host that owns the store.

That claim is true, and it is true because of something outside `znative`. The default root is under `$HOME`, and on an ordinary machine `$HOME` is local storage belonging to one machine. “Per machine” is not enforced by the manager; it is inherited from where the path points.

Now put `$HOME` on shared storage — a network-mounted home directory, the arrangement that has been standard in university and enterprise environments for decades and that a person with a laptop and a server can arrive at by accident. Log in on a macOS `aarch64` workstation and on an `x86_64` Linux server, both reading the same home. Both resolve `$ZSHRS_HOME` to the same directory. There is now one store, one `installed.toml`, and two machines of different architectures using it as though it were theirs.

Chapter 24’s guarantee is false in that deployment, and nothing anywhere reports it.

Exactly how it goes wrong

Trace it, because the failure is more specific than “binaries do not match.”

The store key is `store/<name>@<version>/`. Nothing in that key names an architecture, an operating system, or a target triple — Chapter 6 chose human-readable `name@version` over an opaque hash precisely so the directory would say what it is at a glance, and what it says does not include where it can run. So a native plugin installed from the Mac and the same plugin at the same version installed from the Linux box are the same key.

The index has one record per plugin. Appendix C gives its fields, and the native record carries `lib` — the name of the `cdylib` to load. Install from the Mac and that field names a `lib*.dylib`. Install from Linux and it names a `lib*.so`. One field, one record, two mutually exclusive correct values.

So the sequence is: install on the Mac, and the Linux box’s next `load` resolves a record naming a Mach-O object and fails. Run `znative update` on the Linux box to fix it — which is the reasonable response, and which does the reasonable thing: it re-resolves, rebuilds from source for *this* host, writes into the same store key, recomputes the hash, and rewrites the row to name a `lib*.so`. The Linux box now works. The Mac is now broken, by exactly the same mechanism, in the opposite direction.

Two machines, one record, and a repair on either one is a break on the other. Nothing is corrupt; every write was correct and every invariant held. The two hosts simply disagree about a fact the record has one slot for.

Why the manager cannot detect it

It is worth being clear that this is not an oversight the tool could cheaply close, because the reason it cannot is the same reason several other things in this book are the way they are.

Recording the target in the index would help this case and would mean the index carried a fact about a *build* rather than about an install — the same objection Chapter 40 raises against recording an ABI generation, for the same reason. Keying the store directory by target would restore correctness and give up Chapter 6’s browsability, which was a deliberate trade against a pure content-addressed store. And detecting the situation at runtime would require the manager to know something about the filesystem `$ZSHRS_HOME` sits on, which is not a question a package manager is in a position to answer and not one it should be asking.

The honest framing is that “per machine” was never a property `znative` implemented. It was a property of `$HOME` that the design read as an environment guarantee, and it holds everywhere `$HOME` is local — which is nearly everywhere, and not everywhere.

The fix, and it is the variable

Give each machine its own root. `$ZSHRS_HOME` is the whole mechanism, it is set in the shell before any `znative` line runs, and the shell is a program, so the value can be computed:

```
export ZSHRS_HOME=$HOME/.zshrs-$(uname -s)-$(uname -m)
```

That is Chapter 38’s argument in its most concrete form. A declarative plugins file could not express this, because it is not a fact about plugins — it is a fact about where this machine’s plugins live, computed from the machine. The startup file is executable, so it can compute it, and no feature had to be added to the manager for the case to be expressible.

Two consequences follow and both are the correct ones. Each machine gets its own store, its own index, and its own builds, so Chapter 24’s guarantee is restored by construction rather than by exception. And each machine pays its own first start — Chapter 33’s stampede, once per machine rather than once per home directory — which is the cost that was being silently avoided by sharing a store that could not actually be shared.

The residual case worth naming: a fleet of machines that genuinely are identical, sharing a home directory, all of one architecture and one OS. There the sharing is sound, for the same reason Chapter 24's prebuilt-binary exception is sound — homogeneity makes the assumption true again. The hazard is not sharing; it is sharing across a boundary the store's key does not encode.

What else the variable is good for

Two uses fall out that are worth knowing, since the mechanism is already there.

Reviewing before committing. Chapter 29 argues that the local-source path decouples review from fetch — clone it, read it, then install what you read. The store-root variable extends the same idea one step: install into a scratch root first, read what landed with `znative info`, and only then install into your real one. The install is cheap the second time for a `path:` source, since there is no network in either pass.

Keeping an experiment out of your working set. A plugin you want to try but do not want in `znative list` tomorrow can be installed into a scratch root by a shell that exports the variable before it runs. When you are done you delete a directory, and there is no uninstall to get right, no orphan to collect, and no index row anywhere that ever knew about it.

Both are the same observation: because the tool keeps all of its state under one root and nothing anywhere else, moving the root is a complete and reversible isolation. That is a property worth having and it is a direct consequence of the global-only design that Chapter 39 — and every incumbent comparison in Chapter 31 — treats as a limitation.

Chapter 45 – Driving `znative` From a Script

Everything in this book assumes a human at a prompt or a `.zshrc` running at startup. Neither assumption holds for the third caller: a provisioning script, a dotfiles bootstrap, a health check, a machine-comparison tool. The doctrine that makes the tool pleasant for the first two callers — terse errors, no chatter, no flags to shape output — is exactly what a script has to work with, and the good news is that the design gives a script something better than a formatting flag.

The output that is for humans, and the file that is not

`znative list` prints one line per plugin: name, version, kind, source. `znative info` prints a record. Both are human output, and there is no `--json`, no `--format`, no machine mode — Appendix H's doctrine has no room for one, and adding one would be a second surface to keep in agreement with the first.

A script that parses `list` output is coupling to a presentation format. There is a better target, and it is the same one the tool itself uses: `installed.toml`. It is TOML, it is at a known path under `$ZSHRS_HOME`, and Appendix C documents its record shape field by field — `name`, `version`, `source`, `kind`, `integrity`, and either `lib` or `source_files` plus `fpath`. Chapter 6 makes it the source of truth, which means anything you could learn by parsing `list` is there in structured form, plus the two fields `list` does not print: the integrity hash and the resolved load target.

Be honest about what that coupling is. `installed.toml` is the manager's own state file, not a published interface, and reading it is a decision to depend on an internal format that the tool's own compatibility rules do not obviously cover. What makes it the better choice anyway is that it is *structured* and *documented*, while the alternative is column-splitting text whose stability was never claimed either. Between two undeclared contracts, take the one a parser can read without guessing at whitespace.

The verbs that are safe to script, and the one that is not

Sort the surface by what it does to state.

Reads are unconditionally safe. `list`, `info`, and `gc --dry-run` change nothing. `gc --dry-run` in particular is a reporting verb rather than a safety feature — Chapter 27 makes that point about the human case, and for a script it is simply the query “which store directories does the index not name,” which is a question a disk-usage report wants to ask.

Installs are safe and expensive. `add` and `load` are idempotent in the sense that matters — Chapter 33's whole argument is that running the `.zshrc` idiom again converges — so a provisioning script that runs `znative add` for a set of sources can be re-run without special-casing “already installed.” What it cannot be is cheap on the first run: a native-from-source plugin compiles, and a script that provisions ten machines compiles ten times, on ten machines, by design (Chapter 24).

update is the one to think about. It is the only verb that changes what code is installed without you having named a new source, and Chapter 29's strongest property — nothing enters your shell except when you run `update` — is a property of *when you run it*, not of the verb. A cron job that runs `znative update` has converted a deliberate act into an automatic one and given up that property on your behalf. If a script must

update, have it update named plugins rather than everything, for the same reason Chapter 40 recommends the narrow form.

The exit status is the interface

Chapter 4's error contract is `znative: <reason>` on stderr and a non-zero return, and Chapter 9 shows the startup-file consequence: a failed `load` line does not stop the file. For a script the same contract reads differently, because a script *can* stop.

This is the part where the terse doctrine is an advantage rather than something to work around. Every verb returns an honest status, so ordinary shell control flow is the whole interface:

```
znative add "$src" || {
  print -u2 "provision: $src failed"
  exit 1
}
```

There is nothing to parse to find out whether that worked. Chapter 9 makes the same point for the startup file and reaches the opposite default — there, you usually want the shell to come up anyway, so a failure is logged and the block continues. A provisioning script usually wants the reverse. Both are expressible because the manager reports and the caller decides, which is the division the whole design keeps.

The corollary is about stderr. A script that captures `znative`'s stderr in order to keep its own output clean has captured the only description of the failure that exists. Chapter 36's most useful observation applies here more than anywhere: the terse line names the failure class, and for a build failure the compiler's actual diagnostics are in the log under `~/.cache/zshrs/`. A provisioning script that reports "install failed" and swallows both has left the person debugging it with nothing.

Two things scripts want that are worth building

Neither is a feature request; both are compositions of what exists.

Comparing two machines. Chapter 38 states the property — the `sha256-<hex>` in each index record is over the store copy, so comparing records answers whether two machines have identical content without transferring the trees. As a script that is: read both `installed.toml` files, join on `name`, and report the rows where `integrity` differs or where one side has no row at all. The output is the list of plugins where two machines have diverged, computed from two small files.

Chapter 39's warning attaches directly to that report and is the reason it needs interpretation rather than just a diff: a hash difference at the *same* version string is the expected result of an actively developed plugin updated at different times, not a signal of anything wrong. The report tells you where the divergence is. It does not tell you whether the divergence matters.

Auditing what is native. The `kind` field is in every record, so "which installed plugins are compiled" is a filter rather than an investigation. That is the query Chapter 40 needs on an ABI-upgrade morning, the query a security review wants when it asks what runs as machine code in your shell, and the query Chapter 43's advice generates when you want to know which plugins might have a setup step you have forgotten to call.

What a script cannot do

Two limits, stated so they are not discovered late.

A script cannot load a plugin into a shell that is not itself. Loading is per-process (Chapter 33) — each shell `dlopens` into its own address space and registers into its own registry. A provisioning script that installs plugins has installed them; the shells that will use them are the ones started afterwards. There is no way to

reach into a running shell and load something into it, which is the same fact Chapter 34 states about `update` not swapping a live mapping.

A script cannot ask whether a plugin is loaded anywhere but here. A bare `zmodload -R` lists this process's plugins, and that is the only question with an answer. "Is `forget` loaded in the terminal on the other desktop" is not a question the design has a place to store the answer to, because there is no shared runtime state — only a shared index describing what is *installed*, which is a different word.

Both limits have the same root and it is worth ending on it. `znative` manages installation, which is durable and shared; the shell manages loading, which is per-process and ephemeral. A script can drive the first completely and cannot touch the second, and every apparent gap in what scripting can reach is that line showing through.

Chapter 46 — What znative Is Not

A tool is defined by its boundaries as much as its features, and `znative`'s boundaries are chosen, not accidental. This chapter collects the deliberate omissions the book has argued for in pieces, so the shape of the tool is visible whole.

It is not per-project. There is no `.znative/` in a repository, no project manifest, no lockfile committed to version control. `znative` keeps one global store and one global index under `$ZSHRS_HOME/pkg/`. A language package manager is right to be per-project — a project's dependency set must be reproducible on someone else's machine — but a shell's plugins are the *user's* environment, not a *project's* contract. Making `znative` global-only is a read of what shell plugins are, not a feature it lacks.

It is not a lockfile manager. The store *is* content-addressed — every install carries a `sha256-<hex>` integrity hash in the index — so a record can verify its own content. But there is no separate lockfile pinning a resolved graph across machines, because there is no graph to pin: shell plugins are a flat set the user curated, not a transitive dependency tree a resolver produced. The integrity hash answers “is this the content I installed”; a lockfile would answer “can I reproduce this resolution elsewhere,” and that is a question shell plugins do not ask.

It is not a new DSL. There is no ice-modifier syntax, no declarative block with its own grammar, no configuration language to learn. The interface is `znative load owner/repo` in a `.zshrc`, and the optional `znative.toml` a *plugin* ships is TOML, not a bespoke format. The tool's surface is eight verbs (six for plugins, `gc` and `clean` for the store) and one optional manifest.

It is not chatty. Errors print `znative: <reason>` on stderr and return non-zero — no friendly explanation, no “did you mean,” no “run `-help`.” It obeys the same doctrine as the rest of `zshrs`: no startup banner, no progress chatter to the terminal, no deprecation nags, no tips. It does what you typed and nothing more.

It is not a registry, and therefore not a discovery tool. There is no namespace `znative` owns, no index of available plugins, no search verb, and no notion of a package that exists independently of the source you name. `znative load owner/repo` resolves a location, not a listing. The absence is load-bearing rather than incidental: Chapter 29's account of trust turns on there being no central authority, because an authority is what a signature scheme would have to be anchored to, and Chapter 24's cross-architecture model works precisely because there is no binary cache to fetch per-target artifacts from. A manager with a registry is a manager with an operator, and the tool declines to be one. Discovery happens where it already happens — a repository someone linked, a plugin someone recommended — and what `znative` contributes is that naming it is enough.

It is not a state manager for plugins. The store holds what was installed; it does not hold what a plugin has since accumulated. A plugin's datafiles, caches, and histories live where the plugin put them, `znative` neither creates nor deletes them, and Chapter 37 argues that this is the only defensible division — the manager owns the content it installed and hashed, and the plugin owns everything it wrote afterwards. An uninstall that swept a user's home directory looking for files that might belong to a plugin would be guessing destructively, which is the opposite of the posture the rest of the tool takes.

It does not replace the script path. The single most important non-feature. `znative` does not deprecate script plugins, does not ask you to rewrite oh-my-zsh plugins in Rust, does not make the native path mandatory for anything. Script plugins install and load exactly as they always have; the native path is purely additive, for the cases where a plugin wants compiled speed and real type checking instead of re-interpreted text. The invention is not “native instead of script.” It is “native *as well as* script, from one manager, made as routine as the script path always was.”

Chapter 47 – The Crate, Line by Line

Appendix I sketches the `znative` crate's types. This chapter reads the whole thing, because the crate is small enough that “the whole thing” is a reasonable unit of study and because everything the previous forty-six chapters asserted about the ABI is a consequence of 548 lines of Rust that contain no logic.

The crate is three files:

File	Lines
<code>znative/Cargo.toml</code>	21
<code>znative/README.md</code>	134
<code>znative/src/lib.rs</code>	548
total	703

There is no `tests/` directory, no `build.rs`, no `examples/`, and no second module. `src/lib.rs` is the crate.

Twenty-one lines of manifest, and the empty table at the bottom

```
[package]
name = "znative"
version.workspace = true
edition = "2021"
description = "Stable C-ABI SDK for writing native (Rust) zshrs plugins loaded at runtime
  ~ via `zmodload -R`"
license = "MIT"

[lib]
name = "znative"
path = "src/lib.rs"

# No dependencies – the whole point of a stable ABI crate is that it is
# tiny, has zero transitive deps, and "still builds in 2035" (CLAUDE.md
# durability bar). Everything crosses the boundary as `#[repr(C)]`.
[dependencies]
```

Three things in that file are worth stopping on.

`[dependencies]` is empty, and the comment above it is the argument. A plugin that depends on `znative` acquires no transitive dependency at all. That is not a minimalism aesthetic; it is the only way the durability claim survives contact with time. Every crate in a dependency tree is a future build failure — a yanked version, an edition bump, a `proc-macro` that stops compiling on a newer toolchain. A crate with zero dependencies has zero of those failure modes, and a plugin whose only dependency is that crate inherits the property. The 2035 in the comment is a specific bar, and an empty `[dependencies]` table is what clearing it looks like.

The bar is met at the plugin's *own* dependencies only when the plugin author chooses to meet it. Of the eight plugins in the shipped fleet (Chapter 53), six declare `znative = "0.12"` and nothing else. `zshrs-revolver`

adds `libc = "0.2"`, for one `ioctl` call. `zshrs-fasd` adds `regex = "1"`, and is therefore the only plugin in the fleet with a transitive dependency tree at all — `libc` has none of its own, `regex` has several.

No `crate-type` is set. The `[lib]` table names the crate and its path and stops. So `znative` builds as an ordinary `rlib`, statically linked into whatever depends on it — the shell on one side and each plugin on the other. It is the `plugin` that declares `crate-type = ["cdylib"]`, in its own `Cargo.toml`, and the crate's own doc comment shows exactly that (`lib.rs:33-39`). This is the correct split and worth naming because getting it backwards is a natural mistake: if `znative` were itself a `cdylib`, both sides would be dynamically linking against a shared SDK object, and the ABI would have to cover the SDK's own internals rather than just the two `#[repr(C)]` structs.

`version.workspace = true`. The crate has no version literal of its own; it inherits from the workspace. Chapter 51 is about what that produces downstream, and Appendix P notes the one build consequence.

One constant, one symbol

Two `pub const` values are the entire fixed surface of the protocol.

```
pub const ABI_VERSION: u32 = 4; // lib.rs:73
pub const INIT_SYMBOL: &[u8] = b"znative_init\0"; // lib.rs:87
```

`INIT_SYMBOL` is a byte string with an explicit trailing NUL, because it is handed to `dlsym` through `libloading`'s `Symbol` lookup and that call wants a C string. The host builds its error message by slicing the NUL back off (`plugin_host.rs:386`), so a dylib missing the symbol reports ``<path>`: not a zshrs plugin (no znative_init)` with the name spelled out.

The name was not always that. Commit `fc38701af7` (2026-07-17) renamed the SDK crate from `zshrs-plugin` to `znative` “matches the `znative` package manager”, and the exported symbol moved with it. That is the only rename in the ABI's history and it happened on the same day as the ABI's introduction, so no plugin ever shipped against the old symbol.

`ABI_VERSION` carries its own changelog in the doc comment above it (`lib.rs:61-72`), which is the right place for it — a reader who hovers the constant gets the history:

Version	Added	Commit	Date
1	the ABI itself	<code>4a181954a8</code>	2026-07-17
2	<code>register_completion</code>	<code>b4388c5e08</code>	2026-07-17
3	<code>getfunction</code> , <code>addfunction</code>	<code>f9804b895c</code>	2026-07-17
4	<code>register_compfm</code> , <code>comp_dispatch</code> , <code>empty_command_hash</code>	<code>01516458ec</code>	2026-07-18

Four versions in two days, and none since. Chapter 21 argues that appending a field to `HostApi` is a breaking change and must bump the version; the table is that argument's receipts — every one of those three bumps added fields and nothing else.

HostApi: fourteen fields, twelve of them functions

```
#[repr(C)]
pub struct HostApi {
    pub abi_version: u32,
    pub ctx: *mut c_void,
    // ...twelve extern "C" fn fields...
}
```

`abi_version` is first so that a plugin can read it before trusting anything else in the table — and the generated `znative_init` does exactly that, reading `(*host).abi_version` through a raw pointer before constructing a `Host` (`lib.rs:477-480`). Field order is load-bearing for that check: a plugin built against a version where `abi_version` was *not* first would read whatever happened to be at offset zero.

`ctx` is documented as “Reserved for the host; opaque to plugins. Currently null” (`lib.rs:118-119`), and the host sets it to `std::ptr::null_mut()` (`plugin_host.rs:344`). It is a slot for a future per-instance context that no version has needed yet, and it costs one pointer of table size to keep open. Since appending to the struct is a breaking change (Chapter 21), a reserved field in the middle of the layout is cheap insurance — filling it later is not an ABI change, because the offset does not move.

The twelve callbacks, in declaration order, with the version that introduced each:

Field	v	Signature summary
<code>register_builtin</code>	1	<code>(host, name, BuiltinFn) -> c_int</code>
<code>print</code>	1	<code>(host, text)</code>
<code>eval</code>	1	<code>(host, code) -> c_int</code>
<code>getvar</code>	1	<code>(host, name) -> *mut c_char</code>
<code>setvar</code>	1	<code>(host, name, value) -> c_int</code>
<code>free_cstring</code>	1	<code>(host, *mut c_char)</code>
<code>register_completion</code>	2	<code>(host, cmd, generator) -> c_int</code>
<code>getfunction</code>	3	<code>(host, name) -> *mut c_char</code>
<code>addfunction</code>	3	<code>(host, name, body) -> c_int</code>
<code>register_compfn</code>	4	<code>(host, name, CompFn) -> c_int</code>
<code>comp_dispatch</code>	4	<code>(host, name, argc, argv) -> c_int</code>
<code>empty_command_hash</code>	4	<code>(host)</code>

Every one takes `host: *const HostApi` as its first argument even though the host’s implementations all ignore it — every `extern "C" fn host_*` in `plugin_host.rs` names that parameter `_host`. The parameter is there because it is the C convention for a vtable-style table and because a future host might need it; carrying it costs one register per call and preserves the option.

Three return types recur and mean three different things. `c_int` is an exit status or a success flag, with 0 meaning success in both readings. `*mut c_char` is an ownership transfer to the plugin (Chapter 49). No return at all — `print`, `free_cstring`, `empty_command_hash` — means the call cannot fail in a way the plugin could act on.

PluginInfo, and the one unsafe impl

```
#[repr(C)]
pub struct PluginInfo {
    pub abi_version: u32,
    pub name: *const c_char,
    pub version: *const c_char,
}

unsafe impl Sync for PluginInfo {}
```

Three fields, and the plugin returns a pointer to one of these from `znative_init`. `abi_version` appears a second time here, and the doc comment is candid about why ([lib.rs:196-197](#)): “Redundant with the host-side check, but lets the host reject a plugin that lied about its ABI.” Chapter 15 covers the double gate; the structural point is that the two checks read *different* numbers — the plugin checks the host’s `HostApi.abi_version`, and the host checks the plugin’s `PluginInfo.abi_version` — so neither side is trusting the other’s arithmetic.

The `unsafe impl Sync` exists so the `declare_plugin!` macro can place the struct in a `static` ([lib.rs:463](#)). The safety argument is in the comment two lines above it: the struct “is only ever pointed at `'static` data; it carries no interior mutability.” That is true by construction in the macro, which builds both pointers from `concat!($name, "\0")` string literals. It would stop being true if someone hand-wrote a `PluginInfo` pointing at heap data — which is legal, since the type is `pub` — and the crate has no way to prevent it. The comment is the whole enforcement.

Host and Args: the half that does not cross

Everything from [lib.rs:211](#) down is marked, in a banner comment, as convenience: “None of this crosses the ABI; it is convenience over the raw pointers above.” That line divides the file in two, and the division is worth taking seriously when reasoning about compatibility — a change to `Host`’s method set is a source-level change to plugins that recompile, not an ABI change to plugins already built.

`Host` is one pointer wide ([lib.rs:218-220](#)) and every method is a thin translation: build a `CString`, call through the table, translate the result. Two patterns recur.

Every string-taking method can fail before it calls anything. `CString::new` returns `Err` for a string containing an interior NUL, and each method takes the early exit — `register_builtin` returns `false` ([lib.rs:243-245](#)), `print` simply does nothing ([lib.rs:251](#)), `eval` returns 1 ([lib.rs:260](#)), `comp_dispatch` returns 1 ([lib.rs:361, :370](#)). So a plugin that passes a string with an embedded NUL gets a silent no-op or a generic failure, never a diagnostic. It is the correct behaviour at the boundary — there is no channel to report on — and it is a silent failure mode a plugin author should know exists.

Every returned-string method frees before it returns. `getvar` ([lib.rs:265-277](#)) and `getfunction` ([lib.rs:323-334](#)) are the same nine lines: call the table, null-check, `CStr::from_ptr(...).to_string_lossy().into_owned()`, then `free_cstring`, then `Some(s)`. A plugin using the wrapper cannot leak the host’s allocation, because the wrapper does not give it the chance to.

`add_match` ([lib.rs:293-296](#)) is the one method with no field behind it:

```
pub fn add_match(&self, word: &str) {
    self.print(word);
    self.print("\n");
}
```

Two `print` calls. It exists so completion generators read as “emit a candidate” rather than “print with a newline,” and it makes `Host` a thirteen-method surface over a twelve-slot table. The asymmetry is worth remembering when counting the ABI against the API: they are not the same list.

`Args` ([lib.rs:386-428](#)) decodes `(argc, argv)` into `Vec<String>` eagerly and defensively — it null-checks `argv` itself, then breaks out of the loop on the first null element ([lib.rs:398-405](#)) even though the host's contract says `argc` is authoritative. Chapter 49 covers why the defence is there.

The macro, and its three trampolines

`declare_plugin!` ([lib.rs:454-541](#)) accepts a name, a version, and three optional sections — `builtins:`, `completions:`, `compfns:` — and expands to a `static __ZSHRS_PLUGIN_INFO` plus the one `#[no_mangle] extern "C" fn znative_init`.

The generated init does five things in order:

1. Null-check the host pointer, return null if so ([lib.rs:473-475](#)).
2. Read `(*host).abi_version` and return null on mismatch ([lib.rs:477-480](#)).
3. Wrap the pointer in a `Host`.
4. For each section, emit one `extern "C" fn` trampoline per entry and register it.
5. Return `&__ZSHRS_PLUGIN_INFO`.

The trampolines are the interesting part, because they are how an ergonomic `fn(&Host, &Args) -> c_int` becomes a C-ABI function pointer. Each is a nested `extern "C" fn` defined inside a block, which gives it a distinct type per expansion:

```
extern "C" fn __trampoline(  
    host: *const $crate::HostApi,  
    argc: usize,  
    argv: *const *const c_char,  
) -> c_int {  
    let h = unsafe { $crate::Host::from_raw(host) };  
    let a = unsafe { $crate::Args::from_raw(argc, argv) };  
    $handler(&h, &a)  
}
```

Three of these exist, one per section, and they are near-identical ([lib.rs:486-494](#), [:502-510](#), [:526-534](#)). The duplication is structural rather than accidental: the three sections register through three different host functions, and a single shared trampoline would need the handler type to be uniform across `BuiltinFn` and `CompFn` — which it is, today, since both type aliases have the same signature ([lib.rs:82-83](#), [:99-100](#)). They are kept distinct so the two can diverge without touching the macro's shape.

The `completions:` arm has the comment that explains the one naming rule in the whole ABI ([lib.rs:511-515](#)):

```
// NB: no leading underscore - zsh/compsys treats  
// `*_` command names as autoloadable completion  
// functions, which would shadow this builtin before  
// plugin dispatch. Keep the prefix alphanumeric.  
let gen_name = concat!("zshrs_complete_", $cmd);
```

So a `completions: { "kubectl" => ... }` block silently registers a builtin named `zshrs_complete_kubectl`. It is a hidden command in the user's namespace — typing it at a prompt runs the generator — and nothing in the tool tells the user it is there. Chapter 19's advice about keeping a plugin's public namespace small is partly out of the plugin author's hands for exactly this reason.

The last twelve lines of the file are a workaround with its reasoning attached ([lib.rs:543-548](#)): the macro cannot name `ABI_VERSION` inside a downstream crate's `const` initializer without the user importing it, so the crate re-exports the same value under `ABIVERSION_FOR_MACRO`, marked `#[doc(hidden)]`, and the macro hard-codes that path. It is a small ugliness in service of the plugin author never having to write an import they did not ask for.

Chapter 48 – The Host Side of the Table

The crate defines the shape; `zshrs/src/extensions/plugin_host.rs` fills it in. 699 lines, twelve `extern "C" fn host_*` implementations, nine global registries, and the three public entry points `zmodload -R` drives. Its module doc opens with the sentence that positions the whole subsystem (`plugin_host.rs:1`): “Native (Rust) plugin host — extension; no zsh C counterpart.”

That is a claim the rest of the file has to earn, and it does so by pointing at what it generalises. zsh’s `Src/module.c` `dlopens` C modules that call `addbuiltin` against the shell’s own symbols — a plugin is compiled against zsh’s internals and breaks when they change. zshrs replaces the shell’s internals with a frozen table, and that is the entire difference.

Nine registries, all the same shape

Every piece of state the host keeps about plugins lives in a `static OnceLock<Mutex<...>>` behind an accessor function (`plugin_host.rs:60-125`):

Accessor	Holds	Purpose
<code>plugins()</code>	<code>Vec<LoadedPlugin></code>	the loaded dylibs, each keeping its <code>Library</code> alive live command name → handler
<code>registry()</code>	<code>HashMap<String, BuiltinEntry></code>	
<code>staging()</code>	<code>Vec<(String, BuiltinFn)></code>	builtins registered during the current <code>init</code> serialises load and unload
<code>load_lock()</code>	<code>()</code>	
<code>ownership()</code>	<code>HashMap<String, String></code>	command name → owning plugin
<code>compfn_registry()</code>	<code>HashMap<String, (CompFn, String)></code>	compsys <code>_fn</code> override → handler + owner
<code>compfn_staging()</code>	<code>Vec<(String, CompFn)></code>	the same, during <code>init</code>
<code>pending_completions()</code>	<code>Vec<(String, String, String)></code>	completions requested but not yet wired
<code>installed_completions()</code>	<code>HashMap<String, String></code>	completions already wired, by owner

Four of the nine come in staged/live pairs, and that pairing is the load-time transaction. A plugin’s `znative_init` calls back into the host while the host is still deciding whether to accept the plugin at all, so the callbacks cannot write to the live registries. They write to `staging()` and `compfn_staging()` instead, and `load` promotes the staged contents only after every gate has passed. `load_lock()` is what makes that safe — it is held across the whole of `load` and the whole of `unload` (`plugin_host.rs:371, :513`), so the staging buffers have exactly one writer at a time.

load, and its four rejection points

`load(path)` (`plugin_host.rs:370-466`) runs six steps and can refuse at four of them.

1. `dlopen`. `libloading::Library::new(&expanded)`, where `expanded` is the path with `~` expanded — the comment notes why (“shells hand raw tokens here”, `:375`). Failure is `cannot load `<path>`: <dl error>`.
2. `dlsym`. `lib.get(INIT_SYMBOL)`. Failure is ``<path>`: not a zshrs plugin (no znative_init)`.
3. **Call `init`**. Before calling, the host clears both staging buffers and snapshots `pending_completions().len()` into `pc_start` — because the plugin’s name is not known until `init` returns, so entries staged by *this* `init` must be identifiable by position to be tagged with their owner afterwards (`:391-396`). A null return is ``<path>`: plugin init failed (ABI mismatch or error)`. Note what that message conflates: the plugin returns null both when it refused the host’s ABI version and when its own logic failed, and the host cannot tell which. The parenthetical is the honest form of that ambiguity.
4. **Check `info.abi_version`**. ``<path>`: ABI version N != host M`. This is the second half of the double gate, and it is the one that produces a number the user can act on.
5. **Refuse a duplicate name**. `plugin `<name>` already loaded`. The comment gives the reason (`:420-421`): the second load’s builtins “would shadow the first with no clean unload story.”

Every one of the last three rejections runs the same three-line rollback before returning:

```
staging().lock().unwrap().clear();
compfn_staging().lock().unwrap().clear();
pending_completions().lock().unwrap().truncate(pc_start);
```

Three times, verbatim (`:399-401`, `:409-411`, `:423-425`). The `truncate(pc_start)` is the interesting one — the two staging buffers are cleared wholesale because they were cleared before `init` too, but `pending_completions` accumulates across plugins, so the rollback has to cut back to the snapshot rather than to empty. It is the only place in the file where a partial undo is required, and the `pc_start` variable exists solely for it.

What the rollback does *not* undo is the `dlopen`. The `Library` is a local binding that goes out of scope on the error return, so its `Drop` runs `dlclose` and the dylib is unmapped. That is correct and it is invisible in the code — the cleanup happens because nothing kept the value, which is the ownership model doing the work.

6. **Commit**. Staged builtins move into `registry()` tagged with the owner in `ownership()` (`:429-438`); staged compfns move into `compfn_registry()` with the owner in the tuple (`:441-447`); the pending completions from `pc_start` onward are tagged (`:449-455`); and the `LoadedPlugin` — name, version, path, and the `Library` itself — is pushed (`:457-462`). The success line goes to `tracing::info!` (`:464`), not to the terminal, which is the doctrine the whole shell follows.

unload, and why the order is the whole point

`unload(name)` (`plugin_host.rs:512-565`) is short and every statement in it is ordered against one hazard: the `Library`’s `Drop` is a `dlclose`, and a `dlclose` with a live function pointer still in a registry leaves a dangling pointer into unmapped memory. The module doc says so at the top (`:27-29`) and the code says so three more times.

The sequence:

1. Confirm the plugin is loaded, else `plugin `<name>` not loaded`.
2. Purge `registry()` and `ownership()` of every command this plugin owns (`:520-533`).
3. Purge `compfn_registry()` of its overrides — with the comment “BEFORE `dlclose`, so no dangling `CompFn` pointer survives” (`:535-536`).

4. Drop its rows from `pending_completions()` and `installed_completions()`.
5. Remove the `LoadedPlugin` from `plugins()` and `drop(p)` explicitly, with the comment `// explicit: dlclose here, after registry purge. (:562)`.

Step 4 carries the one thing `unload` deliberately does *not* clean up, and the comment is unusually complete about it (:542-547): the host does not eval to tear down the `_zshrs_plug_<cmd>` compsys function it created, “because evaling deep in the `zmodload` call stack hangs the VM.” The orphaned shell function stays defined, calls a builtin that no longer exists, gets an empty command substitution, and produces no matches. A leak that degrades to a no-op is the right trade against a hang, and the reasoning being in the source is why this book can state it as fact rather than inference.

dispatch, and the lock that is not held

Command resolution is the hook that makes a plugin builtin behave like a builtin. The module doc places it precisely (`plugin_host.rs:11-19`): `fusevm` compiles an unrecognised command name into *external* execution, so a freshly-loaded plugin command arrives at `vm_helper`’s `execute_external_bg`, which consults `dispatch` before spawning a process — “the same slot `zsh` uses for `zmodload -ab` autoloaded builtins (`resolvebuiltin`).”

The resolution order that produces is: real builtins, then shell functions, then plugin commands, then `PATH`. A plugin cannot shadow `cd`; it can shadow `ls`.

The implementation detail that matters most is not in `dispatch`’s body but in the type it reads. `BuiltinEntry` derives `Clone`, `Copy` (`plugin_host.rs:52-58`), so the dispatcher can copy the function pointer out of the `HashMap` and release the registry lock *before* calling into the plugin. Everything that makes a plugin useful depends on that release: `host.eval` re-enters the shell, which can run a command, which can be another plugin builtin, which needs the registry lock. Held across the call, that is a deadlock on the first re-entrant `eval`. The `Copy` derive on a struct whose only real field is a function pointer is what prevents it, and the `_pad: ()` field next to it — with its comment explaining that ownership is tracked elsewhere — is the residue of a design that once carried more.

The leaked table

```
fn host_api() -> *const HostApi {
    static API: OnceLock<usize> = OnceLock::new();
    let addr = API.get_or_init(|| {
        let boxed = Box::new(HostApi { /* ...thirteen fields... */ });
        Box::into_raw(boxed) as usize
    });
    *addr as *const HostApi
}
```

(`plugin_host.rs:339-361`.) Two deliberate oddities in nine lines.

The table is leaked. `Box::into_raw` with no matching `from_raw` anywhere. The comment says why (:336-338): “Leaked so its address is `static` — plugins may retain the `*const HostApi` they are given and call through it from any builtin at any time.” The ABI’s own doc makes the same promise from the other side (`lib.rs:111-112`): “A single instance lives for the whole process; plugins may store the `*const HostApi` they are given.” A deliberate leak of one struct, once, for the process lifetime, is the cheapest possible implementation of that promise.

It is stored as a `usize`, not a pointer. `OnceLock<T>` requires `T: Send + Sync` to be shared across threads, and `*const HostApi` is neither. Casting to `usize` for storage and back for use sidesteps that without an `unsafe impl`. It is a workaround for a type-system constraint that does not correspond to a real hazard here

— the pointer is to a leaked, immutable, function-pointer-only struct — and it is the kind of thing worth writing down precisely so a future reader does not “fix” it into an `unsafe impl Send for` a type that has other uses.

The completion flush, and the hazard it exists for

`register_completion` does not wire anything. It appends to `pending_completions()` and returns. The actual work happens in `flush_pending_completions()` (`plugin_host.rs:478-508`), called from the top of the completion pipeline, and the doc comment states the constraint in one clause (`:470-472`): the flush point is “a safe point where compsys itself evals, unlike plugin-init (deep in the `zmodload` call stack, where evaling hangs the VM).”

That hazard — evaluating shell code during plugin init hangs the VM — is stated three separate times in the codebase and is the single most consequential thing a plugin author has to know. It is why `register_completion` stages. It is why `unload` leaks a shell function. And it is why both hook-installing plugins in the fleet (Chapter 55) install their hooks lazily on first invocation behind an `AtomicBool` rather than in `znative_init`.

What the flush generates is worth reading, because it is the only shell code the host writes on a plugin’s behalf:

```
_zshrs_plug_<cmd>() {
  local -a _zp_m
  _zp_m=("${@f}${<generator> $CURRENT $words}")
  compadd -- $_zp_m
}
(( ${+functions[compdef]} )) && compdef _zshrs_plug_<cmd> <cmd> 2>/dev/null
:
```

`${@f}...` splits the generator’s stdout on newlines into an array — which is precisely why `Host::add_match` is “print the word, print a newline” and nothing more. The `(( ${+functions[compdef]} ))` guard makes a pre-`compinit` flush a no-op rather than an error. And the trailing `:` keeps the whole fragment’s exit status zero regardless of what `compdef` did.

One more comment in that function is worth quoting because it records a bug that was found the hard way (`:499-504`): the glue is evaluated through the free function `crate::ported::exec::execute_script` rather than `try_with_executor`, because “during `do_completion` the thread-local `CURRENT_EXECUTOR` is unset — completion runs on `SESSION_EXECUTOR`. `try_with_executor` would no-op here and the `compdef` glue would never eval.” A silent no-op that manifests as “completions do not fire” is exactly the class of failure Chapter 36 is about, and this one is pinned in a comment instead of a test.

Chapter 49 – Who Owns the Bytes

Every string that crosses the boundary is a bare `*const c_char` or `*mut c_char`. There is no length, no capacity, no tagged union, and no ownership annotation in the type — the types are `c_char` pointers, and the whole ownership protocol is prose in doc comments. This chapter collects that prose into rules, because getting one of them wrong is a leak or a double free rather than a compile error.

There are four rules and they are short.

Rule 1 – `argv` is borrowed, and `argc` is the length

`BuiltinFn`'s doc comment states it (`lib.rs:93-95`): the strings are “Valid only for the duration of the call; copy anything you need to keep.”

That is a hard borrow. A plugin that stores an `argv` pointer past the return is holding a dangling pointer, and there is no lifetime in the signature to stop it. The `Args` wrapper exists so that the natural way to use `argv` is also the correct one: `Args::from_raw` copies every element into an owned `String` eagerly (`lib.rs:396-408`), so a handler written against `GArgs` cannot retain anything the host owns.

The array is **not NULL-terminated**. `argc` is authoritative, and a plugin that walks `argv` looking for a null sentinel is reading past the end. `Args::from_raw` nevertheless breaks on a null element:

```
for i in 0..argc {
    let p = *argv.add(i);
    if p.is_null() {
        break;
    }
    items.push(CStr::from_ptr(p).to_string_lossy().into_owned());
}
```

That `break` is defence against a host that violated its own contract, not a documented feature of the protocol. It also means a plugin receiving a malformed `argv` gets a *short* `Args` rather than a crash — it will see fewer arguments than `argc` claimed, which is a recoverable wrongness rather than an unrecoverable one. The `argv.is_null()` check one line above it does the same for the array pointer itself.

Rule 2 – a returned `*mut c_char` is yours, and only `free_cstring` may release it

Two callbacks transfer ownership to the plugin: `getvar` and `getfunction`. Both doc comments say the same thing (`lib.rs:130-132`, `:148-152`): “Returns a freshly allocated C string the caller MUST release with `free_cstring`, or null if ...”.

The reason it must be `free_cstring` and not the plugin's own `free` is that the allocator never crosses the boundary. The host allocates with `CString::into_raw` and releases with `CString::from_raw`, so both ends of the allocation are inside the host's Rust runtime. A plugin calling `libc::free` on that pointer is calling a different allocator than the one that produced it, which is undefined behaviour on every platform where the Rust global allocator is not the system `malloc` — and the plugin cannot know which platform it is on.

This is the single most important consequence of the “nothing about Rust’s allocator crosses the boundary” claim in the crate’s opening doc comment ([lib.rs:11-13](#)). It is why the table has a `free_cstring` slot at all. A protocol that let the plugin free host memory would have to standardise the allocator, which would make the ABI depend on a Rust implementation detail — the exact thing the `#[repr(C)]` discipline is there to avoid.

The wrapper methods make the rule unmissable by never exposing the raw pointer. `Host::getvar` ([lib.rs:265-277](#)) calls, null-checks, copies to an owned `String`, frees, and returns `Option<String>`. A plugin that only uses `Host` cannot get the rule wrong; a plugin that calls through the raw table can.

Rule 3 – everything you pass in is borrowed by the host for the call only

The three registration callbacks each say `name` is copied by the host: `register_builtin` ([lib.rs:122](#)), `register_compfn` ([lib.rs:168](#)). And it is — `host_register_builtin` builds an owned `String` and pushes it into the staging `Vec`. So a plugin may pass a stack `CString` that drops immediately after.

The wrappers rely on that. `Host::register_builtin` builds a `CString`, passes `cname.as_ptr()`, and lets the `CString` drop at the end of the method ([lib.rs:242-247](#)). If the host retained the pointer instead of copying, every registration in the fleet would be a use-after-free. The doc comment is not documentation of an implementation detail — it is the contract that makes the wrapper sound.

The same holds for `print`, `eval`, `setvar`, and `addfunction`: the host reads the string during the call and does not keep the pointer.

Rule 4 – an interior NUL is not representable, and fails silently

`CString::new` rejects a string containing a NUL byte, and every wrapper method takes an early exit when it does (Chapter 47). The failure is silent: `print` does nothing, `eval` returns 1, the setters return `false`.

This is a real constraint on what a plugin can pass to the shell. Shell strings can contain NUL bytes; C strings cannot; the ABI uses C strings. Anything a plugin wants to hand the shell must be NUL-free, and the boundary silently drops what is not. A plugin doing binary work — reading a file, hashing bytes — has to encode before it prints.

The reverse direction is more forgiving. Decoding uses `to_string_lossy()` ([lib.rs:272, :330, :404](#)), so invalid UTF-8 coming *from* the host becomes U+FFFD replacement characters rather than an error. A shell parameter holding a Latin-1 filename reaches the plugin as a lossy string, and the plugin cannot recover the original bytes through this ABI. That is a genuine limitation for a plugin that wants to manipulate arbitrary filenames, and there is no byte-preserving alternative in the table.

The rules as a table

Direction	Who allocates	Who frees	Valid for
<code>argv</code> into a <code>BuiltinFn</code>	host	host	the call only
<code>name/text/code/body</code> into a callback	plugin	plugin	the call only
<code>getvar</code> / <code>getfunction</code> return	host	plugin, via <code>free_cstring</code>	until freed

Direction	Who allocates	Who frees	Valid for
<code>PluginInfo</code> and its strings	plugin (' <code>static</code>)	never	process lifetime
the <code>HostApi</code> table	host (leaked)	never	process lifetime

Five rows, and the third is the only one that requires a plugin to do anything. Everything else is either borrowed for a call or immortal.

Chapter 50 – There Is No Panic Handler

A Rust panic that unwinds across a C-ABI boundary is undefined behaviour. The `znative` boundary has twelve host callbacks and an unbounded number of plugin handlers crossing it in both directions, and nothing anywhere catches an unwind on either side.

That is a verifiable claim, so verify it:

```
$ grep -rn 'catch_unwind' zshrs/src/extensions/ zshrs/znative/
zshrs/src/extensions/worker.rs:23://! 3. catch_unwind wraps every task – panics logged,
  ~ worker stays alive
zshrs/src/extensions/worker.rs:128: // catch_unwind keeps the worker alive if a task
  ~ panics
zshrs/src/extensions/worker.rs:129: if let Err(e) =
  ~ std::panic::catch_unwind(std::panic::AssertUnwindSafe(task))
```

Three hits, all in `worker.rs`, none on the plugin path. `plugin_host.rs` has zero. `znative/src/lib.rs` has zero.

The documentation is equally direct. [zshrs/docs/PLUGINS.md:408-410](#):

A plugin panicking across the C-ABI boundary is undefined behaviour. ... the host does not unwind through the FFI boundary for you.

Why the worker has one and the plugin path does not

The contrast is instructive because it is a deliberate difference in posture, not an oversight.

`worker.rs` runs *tasks* — closures the shell itself constructed, submitted to a persistent pool. A panicking task is a bug in the shell, and the right response is to log it and keep the worker alive, because killing a pool thread on a bug in one task degrades the shell for every subsequent task. `catch_unwind` there is a resilience measure over code the shell owns.

The plugin path is different in the one way that matters: the frames being unwound through are not Rust frames. A plugin handler is called through an `extern "C" fn` pointer, from Rust, into a separately compiled dylib. Wrapping the call in `catch_unwind` on the host side would catch a panic that had *already* unwound through the `extern "C"` frame — which is the undefined behaviour, not a protection against it. There is nowhere on the host side to put a guard that would help.

The place a guard *can* go is inside the plugin, before the unwind reaches the boundary — in the trampoline the macro generates, or at the top of each handler. The macro does not do it. `declare_plugin!`'s trampolines ([lib.rs:486-494](#) and its two siblings) call the handler directly with no wrapper.

What that means for a plugin author

The rule is: **a panic anywhere in your handler is a shell crash**. Not an error message, not a non-zero exit status, not a killed subshell — the panic unwinds into a frame that has no unwind tables, and what happens next is platform-defined.

So the ordinary Rust habits are not safe here:

- `unwrap()` and `expect()` on anything a user can influence.
- Indexing a slice or `Vec` with a computed index.
- Integer arithmetic that can overflow in debug builds.
- `parse()` chains ending in `unwrap`.
- Any `assert!` outside `#[cfg(test)]`.
- Anything in a library you called that panics on bad input.

The fleet's eight plugins are written accordingly — they are dense with `unwrap_or`, `unwrap_or_default`, `ok()?`, and `let ... else` — and that style is a requirement rather than a preference. A handler's return type is `c_int`, which is a shell exit status, and the correct response to any failure is to return a non-zero one.

A plugin author who wants a hard guarantee can add it in one place. Wrapping the handler body in `std::panic::catch_unwind(AssertUnwindSafe(|| ...))` inside the plugin, and returning a non-zero status on `Err`, keeps the unwind entirely within the plugin's own Rust frames and never lets it reach the `extern "C"` boundary. None of the eight fleet plugins do this. It is four lines each, and it is the difference between a bug in a plugin producing a non-zero exit status and a bug in a plugin taking down an interactive shell with the user's session in it.

Why the SDK does not do it for you

Putting the `catch_unwind` in `declare_plugin!`'s trampolines would fix it for every plugin that uses the macro, in one place, with no plugin author having to know the rule exists. The crate does not do this, and the honest reading is that the crate is silent on the question — there is no comment in `lib.rs` explaining the omission, and `PLUGINS.md` states the hazard without proposing the fix.

Two arguments against doing it are available and neither is written down anywhere, so both are inference and are marked as such. Catching an unwind requires the closure to be `UnwindSafe` or to be wrapped in `AssertUnwindSafe`, and asserting unwind safety over an arbitrary handler that holds `GHost` is an assertion the SDK cannot actually justify — a panic mid-handler can leave shell state the handler was mutating in a torn condition, and continuing past it is not obviously better than the undefined behaviour it replaced. And the crate is built to a zero-cost, zero-dependency, minimal-surface standard where every line is a line that has to still be correct in 2035; a panic hook is state.

Neither argument is conclusive, and the tradeoff is a real one: today the default is undefined behaviour, and the safe alternative requires each plugin author to know a rule that is documented in one paragraph of one file. This is the sharpest open question in the ABI as it stands, and a book that listed it under “safety at the boundary” without saying so would be underselling it.

Chapter 51 – Four Version Numbers That Do Not Agree

The `znative` crate has one version. Four different files state it, and no two of them state the same thing.

Where	What it says	Citation
the workspace's <code>[workspace.package]</code>	<code>0.12.28</code>	<code>zshrs/Cargo.toml:48</code>
the workspace's path-dependency pin	<code>0.12.26</code>	<code>zshrs/Cargo.toml, [workspace.dependencies]</code>
both READMEs' instruction to plugin authors	<code>"0.12"</code>	<code>znative/README.md, lib.rs:38</code>
every one of the eight fleet lockfiles	<code>0.12.22</code>	eight <code>Cargo.lock</code> files

Reproduce the last row directly:

```
$ for i in zshrs-*/; do grep -A1 'name = "znative"' "$i/Cargo.lock" | grep version; done
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
version = "0.12.22"
```

Eight identical answers, six patch releases behind the SDK the shell was built from.

Why none of this is currently a bug

Every one of the four numbers is doing a different job, and at present all four jobs are being done correctly.

`0.12.28` is the workspace version, which `znative/Cargo.toml` inherits with `version.workspace = true`. It moves whenever anything in the `zshrs` workspace is released, whether or not the `ABI` crate changed a line — and the last hundred commits show that the `ABI` crate mostly does not change. Two commits touched `znative/src/lib.rs` in that window and neither was semantic: `ed68380782` was a `rustfmt` pass and `7f0cb7969f` was a clippy/doc fix. The version advanced anyway, because it is the workspace's number.

`0.12.26` is the version constraint attached to the `path` dependency. In a workspace, a `{ path = ..., version = ... }` dependency uses the `path` for local builds and the `version` for publication. Cargo requires the two to be compatible when publishing, and `0.12.26` is compatible with `0.12.28` under semver's caret-by-default rule for `0.x` releases — a `0.12.z` dependency accepts any `0.12.z'` with `z' >= z`. So the pin being stale by two patch releases is legal and resolves to the local path anyway.

"0.12" is what the plugin author writes, and it is the right thing to write. A caret-free 0.12 requirement means "any 0.12.z", so a plugin's `Cargo.toml` does not need editing when the workspace bumps. Both the crate's doc comment (`lib.rs:38`) and its README tell authors exactly this, and all eight fleet plugins follow it.

0.12.22 is what each plugin's committed `Cargo.lock` resolved to at the time it was last regenerated. A lockfile pins; a lockfile that is not regenerated stays pinned; and CI runs `--locked`, so the lockfile is the version that is actually built and tested.

Where it stops being harmless

The four numbers agree in the only sense that matters — `ABI_VERSION` is 4 in all of them — and that is the number the runtime gate reads. A plugin built against `znative 0.12.22` and loaded into a shell built against 0.12.28 passes both halves of the double gate, because both crates define `ABI_VERSION: u32 = 4`.

It stops being harmless at the next ABI bump, and the failure has a specific shape worth walking.

Suppose `ABI_VERSION` moves to 5 in workspace version 0.13.0. A plugin whose `Cargo.toml` says `znative = "0.12"` will not pick it up — 0.13 does not satisfy 0.12 — so `cargo build` succeeds, produces a dylib compiled against ABI 4, and the plugin author sees no error at all. The failure arrives at the user's shell, as ``<path>`: plugin init failed (ABI mismatch or error)`, because the plugin's generated `znative_init` read the host's `abi_version`, saw 5 against its own 4, and returned null.

That is the design working — a refusal rather than a corrupt struct read — and it is also the worst possible place for the author to learn about it. The gate is at runtime because it has to be; the *warning* could be at build time and is not, because a version requirement that spans the change would be needed for the compile to notice, and "0.12" is precisely a requirement that does not span it.

A plugin that wanted the build-time signal would write `znative = "0.13"` the day the bump lands. Nothing tells it that day has come. Chapter 40 walks the recovery; this chapter's contribution is naming the mechanism by which the news arrives late.

The stale lockfiles make it later still. A plugin whose `Cargo.lock` pins 0.12.22 and whose CI runs `--locked` will keep building against 0.12.22 forever, including after the author edits `Cargo.toml` to "0.13" — because `--locked` fails rather than updating, so CI would at least go red. That is the one place the lockfile discipline helps: it converts silence into a failed build.

The one that is a real inconsistency

Of the four, the path-dependency pin at 0.12.26 is the only number with no job it is doing correctly. It should track the workspace version — the whole point of `[workspace.package] version` plus `version.workspace = true` is that one edit moves everything — and it is two patch releases behind. The comment immediately above `[workspace.dependencies]` in `zshrs/Cargo.toml` describes a release script whose purpose is exactly this, so a single `bp.sh <new-version>` run "touches one file." The pin sitting at 0.12.26 while `[workspace.package]` sits at 0.12.28 is evidence that the pin is not in the set that script updates.

It is inert today and it will stay inert as long as the crate is never published to a registry from a state where the difference matters. It is written down here because a book that reports four numbers and then explains away all four is not reporting.

Chapter 52 – Six Slots Nobody Calls

`HostApi` has twelve callbacks. Across the entire shipped plugin fleet — eight repositories, 3,905 lines of Rust — six of them have zero call sites.

The measurement, run over all eight `src/` trees at once:

Callback	ABI v	Fleet repos using it
<code>print</code>	1	7
<code>getvar</code>	1	6
<code>eval</code>	1	3
<code>add_match</code> (over <code>print</code>)	2	3
<code>register_builtin</code>	1	0 direct — used via <code>declare_plugin!</code>
<code>install_completion</code>	2	0 direct — used via <code>declare_plugin!</code>
<code>setvar</code>	1	0
<code>getfunction</code>	3	0
<code>addfunction</code>	3	0
<code>register_compfm</code>	4	0
<code>comp_dispatch</code>	4	0
<code>empty_command_hash</code>	4	0

Two of the zeros are not really zeros: `register_builtin` and `install_completion` are called by the macro's generated `znative_init`, once per entry in a `builtins:` or `completions:` block, so every plugin uses both without naming either. The remaining six are genuinely unexercised outside the shell's own example plugins.

What that means, stated carefully

It does **not** mean the six are broken. They have host-side implementations in `plugin_host.rs`, and the two v4 completion callbacks have an example plugin (`examples/plugin-command-names/`) built specifically to drive them. The fleet is a sample of eight real tools, not a conformance suite, and a real tool uses what it needs.

It does mean something narrower and worth saying: **ABI versions 3 and 4 have no production consumers.** Every plugin in the fleet would work identically against ABI version 2. The three v4 callbacks were added on 2026-07-18 by commit `01516458ec`; nothing in the fleet has reached for them since.

And it means `setvar` — a version 1 callback, present since the first day of the ABI — has never been used by a shipped plugin, while its counterpart `getvar` is used by six of the eight. That asymmetry is the interesting one, because it is not about recency.

Why `setvar` has no users

Six plugins read shell parameters and none writes one. The reason is visible in what each plugin is doing with the read.

`getvar` is how a plugin reads its own configuration. `$PWD`, `$HOME`, and a plugin-specific `$FORGIT_...` or `$ZSHZ_DATA` are all inputs — the user's environment, set in a `.zshrc`, read at the moment a builtin runs. Chapter 37 makes the argument that shell parameters are a plugin's configuration mechanism, and six plugins reading them is that argument confirmed.

Writing back is a different act, and there are two better ways to do it that the fleet reaches for instead. A plugin that wants to change shell state — `cd` somewhere, install a hook, define a function — uses `eval`, because the thing it wants is a shell *action* and not a variable assignment. `zshrs-zsh-z` is the model: it computes the destination in Rust and then evals a `cd`, so `$PWD`, `chpwd` hooks, and the directory stack all update through the shell's own machinery rather than through one parameter write that would leave the rest inconsistent. And a plugin that wants to hand a value *back* to the caller prints it, because a builtin's stdout is capturable with `$( )` and a parameter write is not composable.

So `setvar`'s emptiness is not a gap in the fleet; it is the fleet demonstrating that the shell already has two better channels for the same intent. The slot is still correct to have — a plugin that genuinely wants to set `$REPLY` should not have to eval a quoted assignment — and it costs one pointer in the table.

Why the v3 pair has no users

`getfunction` and `addfunction` are the only structured access to shell functions, and the README makes an argument for them beyond the obvious one (`znative/README.md:102`): because `addfunction` then `getfunction` round-trips a body through the shell's own parser and pretty-printer, the pair “doubles as **deparse-as-a-service** — define a temp function from arbitrary source, read back its normalized form.”

That is a genuinely novel capability and nothing in the fleet needs it. The plugins that install shell hooks — `zshrs-fasd` and `zshrs-zsh-z` — could use `addfunction` to define their hook function and would then still need `eval` to run `add-zsh-hook`, so the pair buys them one call and costs them a second mechanism. Both use `eval` for the whole thing instead (Chapter 55).

The honest summary is that v3 solved a problem the fleet does not have. The capability is real; the demand for it, so far, is zero.

What the emptiness costs

Three concrete costs, in ascending order of seriousness.

Table size. Six unused function pointers, forty-eight bytes on a 64-bit target, in a struct that exists once per process. Not a cost.

Untested paths. Six host implementations with no fleet consumer and no unit test — `plugin_host.rs` has zero `#[cfg(test)]` blocks (Chapter 50's `grep` applies here too). The examples exercise some of them; nothing exercises them continuously. A regression in `comp_dispatch` would be found by a user, not by CI.

Frozen-in evidence. This is the one that matters for the ABI's future. Chapter 21 argues that a field cannot be removed from `HostApi` without breaking every compiled plugin, so the six slots are permanent regardless of whether anyone ever calls them. The ABI can only grow. Six of its fourteen fields are, as of today, growth that no shipped plugin needed — which is the strongest available argument for the chapter's central discipline: **an ABI addition is a decision you cannot revisit, so the bar for making one should be a plugin that cannot be written without it, not a capability that would be nice to have.**

By that bar, `register_completion` (v2) earns its slot: `zshrs-kubectl-completion` and `zshrs-zsh-z` both ship completions and neither could without it. The v3 and v4 additions do not yet have that receipt.

Chapter 53 – The Fleet, Measured

The book has referred to “the worked ports” since Chapter 18 and described them one paragraph each. This chapter measures them. Eight repositories, all siblings, all published, all installed the same way — `znative load MenkeTechnologies/<repo>`.

Repository	LOC	Tests	Version	Extra deps	Builtins	Completions
<code>zshrs-git-fuzzy</code>	868	6	0.1.1	—	2	—
<code>zshrs-fasd</code>	783	7	0.1.0	<code>regex</code>	10	—
<code>zshrs-forgit</code>	599	2	0.1.1	—	9	—
<code>zshrs-zsh-z</code>	491	7	0.1.1	—	1	1
<code>zshrs-revolver</code>	403	3	0.2.1	<code>libc</code>	1	—
<code>zshrs-git-repos</code>	315	2	0.1.1	—	1	—
<code>zshrs-reveal</code>	305	7	0.1.0	—	1	—
<code>zshrs-kubectl-completion</code>	141	5	0.1.1	—	0	1
total	3,905	39			25	2

Reproduce the first two columns with:

```
$ find <repo>/src -name '*.rs' | xargs wc -l | tail -1
$ grep -c '#\[test\]' <repo>/src/lib.rs
```

Three numbers in that table are worth pulling out before the per-plugin chapters.

Twenty-five builtins across eight plugins, and the distribution is extremely uneven: `zshrs-fasd` and `zshrs-forgit` account for nineteen of them, and four plugins register exactly one. Chapter 19’s advice to keep the public namespace small is followed by half the fleet and deliberately violated by the two that are porting a tool whose *interface* is a set of short command names (`ga`, `grh`, `glo` ... for `forgit`; `a`, `s`, `d`, `f`, `v` ... for `fasd`). Porting faithfully means inheriting the original’s namespace, and both chose fidelity.

One plugin registers zero builtins. `zshrs-kubectl-completion`’s `declare_plugin!` has a `completions:` block and no `builtins:` block at all. It is the proof that the completion path is a first-class install target rather than a feature bolted onto a command plugin. It does still put a name in the user’s namespace — the macro registers `zshrs_complete_kubectl` behind its back (Chapter 47) — which is the one caveat on “zero builtins.”

Thirty-nine tests, and no ABI test among them. Every one of the 39 is a unit test of the plugin's own logic — argument parsing, scoring, formatting, path handling. None of them loads a dylib, none exercises the host table, and none could: the host is the shell, and a `cargo test` binary is not the shell. Chapter 35's "layer one" is where the entire fleet's testing lives, and layers two through four are exercised by hand.

The CI that every one of them shares

All eight repositories carry a `.github/workflows/ci.yml`, and the eight files are **byte-identical**:

```
$ md5 -q zshrs-*/.github/workflows/ci.yml | sort | uniq -c
8 fc36e8b0ac1ed33275a5a4fbd2d3c256
```

One hash, eight files. The workflow runs formatting, clippy, doc, build, and test gates, and it runs `cargo` with `--locked`.

The uniformity is a real asset — a plugin author cloning the pattern gets a correct pipeline for free, and a fix to the pipeline is a copy to eight places rather than a redesign in eight places. It is also the reason for a gap that is structural rather than accidental, and that is worth stating precisely because it has already caused a real class of defect.

Nothing in the pipeline compares the `declare_plugin!` version literal to the crate version. The macro takes `version: "0.1.1"` as a string literal (`lib.rs:457-458`), and that literal is what `PluginInfo.version` points at, which is what `znative list` and `zmodload -R` listings display. `Cargo.toml`'s `[package] version` is a separate string in a separate file. A release that bumps one and not the other produces a plugin that reports a stale version to the user with every gate green.

That is not hypothetical. Six of the eight repositories carry a most-recent commit that fixes exactly this, and the commit messages say so:

```
zshrs-kubectrl-completion  declare_plugin: bump version to 0.1.1 to match
  ↳ Cargo.toml/znative.toml
zshrs-revolver             revolver --version: derive from CARGO_PKG_VERSION instead of a
  ↳ stale literal
zshrs-git-repos            README: document -l, bare 'regen', and -h/--help; bump plugin
  ↳ version
zshrs-zsh-z               README: document -h, --add, --complete, and the three env vars;
  ↳ bump plugin version
zshrs-forgit              README: correct the diff-renderer claim, document the 8 env
  ↳ vars; bump plugin version
zshrs-git-fuzzy           README: document the git-fuzzy alias, gf menu, and all 14 env
  ↳ vars; bump plugin version
```

Every literal now matches its crate version — verified across all eight — so the current state is clean. The mechanism that let them drift is unchanged.

`zshrs-revolver` is the one that fixed the *cause* rather than the instance. Its `--version` output derives from `env!("CARGO_PKG_VERSION")` (`lib.rs:309`) rather than from a literal, so the number the user sees cannot disagree with the crate. Its `declare_plugin!` literal is still a literal, so `PluginInfo.version` can still drift — but the user-facing surface is now pinned to the build.

The generalisable fix is smaller than it looks. `declare_plugin!` accepts `version: $version:literal`, and `env!("CARGO_PKG_VERSION")` is not a literal in the macro-matcher sense, so a plugin cannot pass it today. Relaxing the matcher to `$version:expr` would let every plugin write `version: env!("CARGO_PKG_VERSION")` and make the drift structurally impossible. That is a change to the SDK's macro rather than to eight CI files, it is source-compatible with every existing plugin, and it does not touch `ABI_VERSION` — which makes it the rare improvement to this system that costs nothing on the compatibility ledger.

Chapter 54 — zshrs-git-fuzzy: The Plugin That Loads Itself

868 lines, six tests, two builtins, twelve helper subcommands, and one technique that no other plugin in the fleet needs: it finds its own dylib on disk and writes a shell script that re-loads it.

Chapter 17 described the pattern in the abstract. This chapter reads the implementation, because the four mechanisms it stacks are each interesting on their own and because the plugin is the fleet's demonstration that a znative plugin is not confined to the process that loaded it.

Finding itself with `dladdr`

```
/// Absolute path of THIS dylib, via `dladdr` on one of our own functions.
```

(`zshrs-git-fuzzy/src/lib.rs:223`.) The implementation declares `dladdr` and its `DlInfo` struct by hand in an `extern "C"` block (`:233`) — the plugin has no `libc` dependency, so it writes the two declarations itself — takes the address of one of its own functions as the lookup key, and reads `info.dli_fname` back out.

The failure path is guarded: `if unsafe { dladdr(marker, &mut info) } == 0 || info.dli_fname.is_null()` (`:237`) returns `None`, and every caller has a fallback string (`:298, :345`) so a `dladdr` failure produces a shim path of `gf-helper-unavailable` rather than a panic. Chapter 50's rule, observed.

The shell binary is found separately with `std::env::current_exe()` (`:252`), which is the standard library's answer to the same question about the *process*. Two different mechanisms, because the plugin needs two different paths: the shell to re-exec, and its own dylib to re-load.

The shim

```
#!/bin/sh
exec <zshrs> -fc 'zmodload -R "$0" 2>/dev/null; gf --helper "$@"' <self.dylib> "$@"
```

That is the format string at `:276`, written once to `$HOME/.cache/zshrs/gf-helper.sh` with mode 0755. Two details in it are deliberate and easy to get wrong.

The dylib path is passed as `$0` — the argument immediately after the `-c` script — rather than interpolated into the script text. So a dylib living under a path with a space in it survives, because `"$0"` is quoted at use rather than at construction. This is the kind of care that a shell-script original would have had to get right too, and the port did not lose it in translation.

`zmodload -R "$0" 2>/dev/null` discards the load's stderr. A fresh `zshrs -f` has never loaded the plugin, so the load should succeed; the redirect is there so that if it fails, `zsf`'s preview pane shows the helper's real output rather than an error banner.

Every `zsf --bind` and `--preview` in the plugin points at that shim. `zsf` runs its bind commands through `sh`, and `sh` cannot call a builtin that lives inside the `zshrs` process — so each keystroke becomes a fresh `zshrs -f` that `dlopen`s an already-page-cached dylib and runs one function. The bash original's equivalent step is re-sourcing a shell library, which is a parse.

Twelve helpers behind one flag

`gf --helper <sub> ...` routes to `helper_dispatch` (:314), whose match has twelve arms:

```
status_menu_content  status_preview_content  status_diff
status_add           status_reset             status_discard
status_amend         status_add_patch         status_commit
status_edit          status_watch             preview_resize
```

None of the twelve appears in the plugin's README. The README explains the *mechanism* in two places (README.md:38, :98) — the self-reentrant pattern, the shim, the one-`dlopen`-per-action claim — and documents the user-facing surface (`gf`, its menu, its environment variables) without listing the helper vocabulary.

That is defensible: the twelve are an internal protocol between the plugin and `fzf`, not a surface a user is meant to type. It is also incomplete in a way that matters to exactly one reader — someone debugging why a keybinding does nothing, who will find the shim in `~/.cache/zshrs/` and want to run a helper by hand to see what it prints. For that reader the list is in the source and nowhere else, which is why it is reproduced above.

HTTP without an HTTP client

`git-fuzzy`'s live-reload watcher pushes `reload-sync(...)` into `fzf`'s `--listen` port. The original uses `curl`. The port opens a `TcpStream` to `127.0.0.1:$FZF_PORT` and writes the request bytes directly (:793-794).

The reasoning is the same one Chapter 1 of the `arb-registry` book makes about `git` as a transport, arrived at from the opposite direction: there, a program with no TLS delegates to a binary that has it; here, a program that needs one unencrypted loopback POST declines to shell out for it. `std::net::TcpStream` is in the standard library, the request is a handful of bytes to localhost, and the alternative is a runtime dependency on `curl` being installed. The plugin's `Cargo.toml` remains `znative = "0.12"` and nothing else.

The failure handling is where the design shows: a failed `connect` means `fzf` has exited, which is the watcher's signal to stop. An error is not an error; it is the termination condition.

The polling loop

The watcher compares an FNV-1a hash of `git status --porcelain` output every 500 ms and pushes a reload when it changes. Hashing the output rather than diffing it means the watcher holds 8 bytes of state instead of a snapshot, and FNV-1a is a dozen lines the plugin implements itself rather than a dependency.

This is a genuine thread doing genuine periodic work inside a shell process, which is only possible because `zshrs` does not fork per command. Chapter 20 makes the general argument; `git-fuzzy`'s watcher is the version of it that survives across multiple `gf` invocations and outlives none of them — the loop's exit condition is `fzf` going away, not the builtin returning.

The seven writes that skip the host

`zshrs-git-fuzzy` contains seven direct `print!/println!` calls (:488, :510, :560, :568, :579, :623, :747) that write to the process's stdout rather than through `host.print`. Chapter 58 covers the implications for the fleet as a whole; the reason they are concentrated here is specific: six of the seven are inside helper subcommands, which run in a *fresh* `zshrs -f` process whose stdout is a pipe `fzf` is reading. In that process the plugin is effectively the whole program, and `host.print` and `println!` reach the same file descriptor.

That reasoning is correct for the helper path and it is not written down anywhere in the plugin. The seventh write (:747, a `change-preview-window:` directive) is on the same path. Nothing enforces that the calls stay

on that path, and a future refactor that moved one of them into the top-level `gf` builtin would write straight past whatever redirection the shell had established.

Chapter 55 — Two Ways to Mutate a Shell: `zshrs-fasd` and `zshrs-zsh-z`

Two plugins in the fleet do the same class of thing — track directories or files the user visits, then jump to one — and both have to solve the same problem that no other plugin has: **a builtin cannot change the shell’s state by returning**. A `cd` has to happen in the shell, and a recording hook has to be installed in the shell, and both of those are shell actions that a Rust function cannot perform by computing a value.

They solve it the same way, with the same three-part pattern, and reading them side by side is the clearest available statement of what `host.eval` is for.

The pattern: compute in Rust, mutate through `eval`

`zshrs-zsh-z` states it in its module doc (`zshrs-zsh-z/src/lib.rs:14`): recording “is driven by a `chpwd` hook (`z --add \"$PWD\`) that the plugin installs.” The frequency store, the aging rule, the ranking, and the matching are all Rust. The two things that are not Rust are the hook installation and the `cd` itself, and both go through `eval`.

`zshrs-fasd` does the same with a `preexec` hook (`zshrs-fasd/src/lib.rs:9`), which fires before each command rather than after each directory change — the right hook for a tool that records *files mentioned on command lines* rather than directories visited.

Three of the eight fleet plugins call `host.eval` at all, and these are two of them.

The lazy install, and the hazard it dodges

Neither plugin installs its hook in `znative_init`. Both defer it to the first invocation of a builtin, behind an `AtomicBool`:

```
static INSTALLED: AtomicBool = AtomicBool::new(false);
```

(`zshrs-fasd/src/lib.rs:373`, `zshrs-zsh-z/src/lib.rs:285`.) `zshrs-zsh-z`’s comment names the reason directly: “Done lazily at the ...” first call.

The hazard is the one from Chapter 48: evaluating shell code during plugin init hangs the VM, because init runs deep inside the `zmodload` call stack. The host works around it for completions by staging them and flushing at a safe point. There is no equivalent staging mechanism for hooks — **the ABI has no hook-registration entry point at all** — so each plugin implements its own deferral, with the same `AtomicBool` idiom, independently.

That is a duplicated workaround for a documented host constraint, in two plugins, in identical form. It is the strongest available case for what a v5 addition might look like: a `register_hook(kind, body)` callback that stages exactly the way `register_completion` does. Chapter 52’s bar applies — the capability would need to be something a plugin *cannot* write without, and clearly these two could and did. What it would remove is a rule every hook-installing plugin author has to learn from a source comment.

The `eval` both plugins run is small enough to quote. `zshrs-zsh-z`’s (`lib.rs:291-292`):

```
_zshrs_z_chpwd() { z --add "${PWD:A}" 2>/dev/null }
add-zsh-hook chpwd _zshrs_z_chpwd 2>/dev/null
:
```

`${PWD:A}` is the absolute, symlink-resolved form — a zsh parameter expansion flag, running in the shell, doing work the plugin would otherwise have to reimplement against the filesystem. The trailing `:` normalises the exit status, exactly as the host’s own completion glue does (Chapter 48). The two `2>/dev/null` redirects make the fragment safe to eval in a shell where `add-zsh-hook` has not been autoloaded.

Where they diverge: the datafile

`zshrs-zsh-z` writes its store atomically — build the whole file in memory, write a temp, `std::fs::rename` over the target (`lib.rs:100-109`) — and the comment notes it mirrors upstream’s `_zshz_add_path`. A shell reading the file mid-write sees either the old file or the new one, never a torn one, which matters because the plugin is running in an interactive shell that may have siblings.

The formatting detail underneath it is the more interesting one. `fmt_rank` (`lib.rs:92`) prints a whole-number rank as an integer rather than as `3.0`, and there is a test named `fmt_rank_integer_vs_float` pinning it (`lib.rs:443-444`). The reason is compatibility rather than aesthetics: zsh-z’s own shell implementation computes ranks with `${( ... )}`, which renders whole numbers without a decimal point, so a datafile written by the Rust port stays byte-compatible with one written by the original. A user can switch between the two and neither will reformat the other’s file.

That is a compatibility floor decision made at the level of a single number’s text representation, and it is the kind of fidelity that separates a port from a rewrite.

Where they diverge: the dependency, and the shell it depends on

`zshrs-fasd` is the only plugin in the fleet with a transitive dependency tree, and it is there for one job: `build_regex` (`lib.rs:254-274`) constructs the pattern that `fasd`’s matching is defined in terms of, escaping literal terms, turning spaces into `[^]*`, and honouring a trailing `$`. The line comments cite the original’s C source by line (`c:404-406`, `c:421-425`), which is the porting discipline Chapter 16 describes applied at the granularity of individual expressions.

`zshrs-fasd` also has a dependency that does not appear in any manifest: it relies on zsh’s `{{(z)...}` word-splitting behaving correctly inside `zshrs`, because the `preexec` hook it installs has to split a command line into words the way the shell would. No other plugin in the fleet leans on a specific parameter-expansion flag this way. It is a compat-floor dependency — the plugin is correct only to the extent that `zshrs`’s implementation of one zsh expansion flag is correct — and it is the sort of coupling that only becomes visible when you go looking for it.

It is also the only plugin that evals *user-supplied* text: `fasd -e <cmd>` runs what the user asked for. Every other `eval` in the fleet is a fixed string the plugin authored.

The documented flag that does not do what it says

`zshrs-fasd`’s README documents `-i` as “interactive select” (`README.md:84`), alongside the other flags it inherits from `fasd`. The source does something else (`lib.rs:556-566`):

```

if o.show || o.interactive {
  // c:591-593 / c:581-587 - scores + paths (interactive selection needs
  // a tty read the plugin ABI can't drive; degrade to the scored list).
  for (s, p) in &results {
    host.print(&format!("{:<10.2} {}\\n", s, p));
  }
  if o.interactive {
    // Still record/return the best so `-e`/cd paths stay usable.
    let best = results.last().map(|r| r.1.clone());
    return finish(host, best, o.exec.as_deref());
  }
  return 0;
}

```

`-i` prints the scored list and then silently picks the best match. There is no prompt and no selection.

The comment gives the reason and it is a genuine ABI limitation rather than an unfinished feature: “interactive selection needs a tty read the plugin ABI can’t drive.” `HostApi` has twelve callbacks and none of them reads from the terminal. There is no `readline`, no `read_key`, no way to get a line of input from the user — the table is write-only with respect to the terminal, plus `getvar` for the environment. A plugin that wants a prompt has exactly two options: shell out to something that owns a tty (`fzf`, which is what four other plugins do), or `eval` a `read` and pick the answer back up through `getvar`.

The degradation is a defensible choice — printing the list and taking the best is more useful than refusing — and the drift is that the README promises the flag’s original behaviour. It is the clearest single example in the fleet of documentation describing the tool being ported rather than the port.

Chapter 56 – Resident State and Real Threads: `zshrs-revolver` and `zshrs-git-repos`

Chapter 20 argues that a znative plugin's concurrency is threads rather than forks, because the shell it lives in does not fork per command. Two plugins in the fleet are the argument's evidence, and they use the property in opposite directions: `zshrs-revolver` keeps one thread alive *across* builtin calls, and `zshrs-git-repos` spawns sixteen threads *inside* one.

`zshrs-revolver`: the plugin as a resident object

403 lines, three tests, one builtin, and one static:

```
fn active() -> &'static Mutex<Option<Spinner>>> {
    static A: OnceLock<Mutex<Option<Spinner>>> = OnceLock::new();
    ...
}
```

(`zshrs-revolver/src/lib.rs:138-139`.) That slot holds the currently running spinner. `revolver start` fills it, `revolver update` mutates it, `revolver stop` empties it — three separate invocations of a builtin, communicating through process memory.

The upstream zsh `revolver` cannot do that. It forks a background process to run the animation and coordinates through a statefile keyed on `$PPID`, because in a forking shell the three commands are three processes and shared memory is not available to them. Every design decision downstream of that — the statefile format, its location, its cleanup, the PPID keying to avoid collisions between shells — exists to reconstruct in a file what the native port gets from a `static`.

This is the sharpest available demonstration of what “a plugin is a resident object” means. The dylib is `dlopened` once, its `statics` are initialised once, and they persist for the shell's lifetime across arbitrarily many builtin calls. It is not a re-exec'd script that reconstructs its world each time.

Three more details in the plugin are downstream of the same property.

Zero subprocesses. The plugin spawns nothing. Upstream calls `tput cols` to get the terminal width; the port issues a `TIOCGWINSZ ioctl` directly (`lib.rs:148`), which is the one reason it carries `libc = "0.2"`. Upstream forks a process to animate; the port runs a thread (`lib.rs:157`). A spinner that updates several times a second is exactly the workload where per-frame process creation is the dominant cost, and the port removes all of it.

It writes to `/dev/tty`, not to `stdout`. The animator opens `/dev/tty` for writing and falls back to `stderr` if that fails (`lib.rs:162`). The reason is composability: a spinner is decoration around a command whose `stdout` the user may be capturing, and writing frames into that stream would corrupt it. `cmd | revolver-wrapped-thing > out.txt` gets a clean `out.txt` and a spinner on the terminal, which is only true because the frames never touch `fd 1`.

That is also why `revolver` is one of the two plugins whose direct writes bypass `host.print` (Chapter 58) — its whole output discipline is about *not* going where `host.print` goes.

A 55-entry style table. `static SPINNERS: &[Style]` (`lib.rs:62`), where `Style` is `(&'static str, f64, &'static [&'static str])` — name, frame interval, frames. Fifty-five entries, all `'static`, all in the binary's read-only data. The shell-script original keeps the same table as shell arrays that are re-parsed on every load; here it is compiled data with no initialisation cost at all.

zshrs-git-repos: a bounded worker pool inside a builtin

315 lines, two tests, one builtin. The interesting fifty lines are the classifier (`zshrs-git-repos/src/lib.rs:148-171`):

```
    .min(16);
let queue = Arc::new(Mutex::new(repos.to_vec()));
let out = Arc::new(Mutex::new(Vec::<String>::new()));
for _ in 0..workers {
    let queue = Arc::clone(&queue);
    let out = Arc::clone(&out);
    handles.push(std::thread::spawn(move || loop {
        ...
        match q.pop() { ... }
    }));
}
...
let mut v = Arc::try_unwrap(out).unwrap().into_inner().unwrap();
```

A shared work queue, up to sixteen workers popping from it, a shared output vector, and a join at the end. It is the textbook shape, and the notable thing about it is where it is: inside a shell builtin, in the shell's own process, with no fork and no IPC.

The zsh original classifies each repository with a sequential `git status` — one subprocess per repo, run in order. On a machine with a few hundred repositories that is a few hundred serialised process spawns. The port runs the same `git` invocations, but sixteen at a time, and collects into memory rather than into a pipe.

Two implementation choices are worth marking. The pool size is ... `.min(16)` rather than “one thread per repository” — an unbounded spawn over a few hundred repos would create a few hundred threads all contending for the same disk, which is slower than a bounded pool and rougher on the system. And the exit from the loop is `q.pop()` returning `None`, so workers drain the queue and terminate rather than being signalled — the simplest correct shutdown for a finite batch.

The plugin is also the fleet's third `host.eval` user, and it uses it for one thing: the `cd` into the selected repository. Same reason as `zshrs-zsh-z` (Chapter 55) — the destination is computed in Rust, and the state change is performed by the shell so that `$PWD`, `chpwd` hooks, and the directory stack all stay consistent.

What the two have in common

Neither plugin could be written this way against any other shell's plugin system, and the reason is not the ABI. It is that zshrs does not fork per command, so a plugin's threads and statics outlive the call that created them and share memory with the call that follows.

An oh-my-zsh plugin is shell text re-interpreted in a process that may or may not be the same one next time. A zsh C module (`Src/module.c`) is closer — it is `dlopened` into the shell and could hold statics — but it is compiled against zsh's internal symbols, so writing one means shipping against a moving target with no versioned contract. `znative` is the combination: the residency of a C module with the stability of a published ABI.

Chapter 57 — The Thin Ones: `forgit`, `reveal`, `kubectl-completion`

Three plugins in the fleet do less, and each does less in an instructive way. Read together they mark the boundaries of what a native plugin needs to be.

`zshrs-forgit` — nine builtins, zero evals

599 lines, two tests, nine registered commands (`ga`, `grh`, `glo`, `gd`, `gcf`, `gclean`, `gss`, `gcp`, `gi`), and not a single call to `host.eval`. It is the only multi-command plugin in the fleet that never asks the shell to run anything.

That is possible because every one of `forgit`'s commands is subprocess orchestration: build a `git` command line, pipe it into `fzf`, read the selection, build a second `git` command line, run it. None of that needs the shell — it needs `git`, `fzf`, and `std::process::Command`. The plugin reads its configuration with `host.getvar` (the `FORGIT_*` parameters) and writes its output with `host.print`, and those two callbacks plus subprocess spawning are its entire interaction with the world.

The comparison with `zshrs-zsh-z` is the useful one. Both plugins wrap an interactive selection; one evals and one does not. The difference is what the selection is *for*. `zsh-z`'s answer is a directory the shell must move to, which is shell state. `forgit`'s answer is a set of files `git` must act on, which is not. When the result of a plugin's work is a subprocess invocation, `eval` buys nothing and costs a shell parse.

Two gaps are worth recording. The plugin has ten `--bind=` arguments across its `fzf` invocations — `alt-k/alt-p`, `alt-j/alt-n`, `ctrl-r`, `ctrl-s`, `?`, `alt-w`, `enter`, and `ctrl-y` (`src/lib.rs:149-154`, `:344-345`, `:397`, `:435`) — and none of them appears in any documentation — the README was recently expanded to cover the eight environment variables and to correct a claim about the diff renderer, and the keybindings were not part of that pass. And none of the nine commands handles `-h` or `--help`:

```
$ grep -c '"-h"|--help' zshrs-forgit/src/lib.rs
0
```

Zero. Nine commands with no usage output between them. The shell's own doctrine (Chapter 46) is against unsolicited help text, and this is not that — the doctrine is about not *volunteering* help, and `-h` is a user asking. For a plugin whose commands are two-letter names inherited from an upstream tool, the absence is felt most by the reader who has forgotten which of `gd` and `gss` is which.

`zshrs-reveal` — the smallest complete plugin

305 lines, seven tests, one builtin. It opens a path in the platform's file manager. That is the whole feature.

Two things make it worth a section.

Runtime OS dispatch, including WSL. The platform decision is not a `#[cfg(target_os)]` compile-time branch but a runtime check (`zshrs-reveal/src/lib.rs:27-31`):

```
let is_wsl = std::fs::read_to_string("/proc/version")
    .map(|s| s.to_lowercase()).contains("microsoft"))

...
v(6["cmd.exe", "/c", "start", ""])
```

A Linux binary running under WSL must open a Windows file manager, and no compile target distinguishes the two — a WSL build *is* a Linux build. The only way to know is to look at `/proc/version` at runtime. Chapter 24’s cross-architecture argument is about building the same source for different targets; this is the case where one target has two runtime environments, and the plugin handles it in five lines.

Fire-and-forget spawning. The launch is `.spawn()` (`lib.rs:52`) with no `.wait()`. A file manager is a long-lived GUI application; waiting for it would block the shell until the user closed the window. The builtin returns as soon as the process exists.

Seven tests over 305 lines is the fleet’s highest test-to-line ratio, and the reason is that the plugin’s logic is almost entirely pure: given an OS and a path, produce an argv. That is a function, and functions are testable. The parts that are not — did the file manager actually open — are not testable anywhere, and the plugin does not pretend otherwise. It is a small, honest demonstration of Chapter 35’s layer-one discipline.

zshrs-kubectl-completion — a plugin with no commands

141 lines, five tests, zero builtins. Its `declare_plugin!` is a `completions:` block and nothing else:

```
completions: {
    "kubectl" => kubectl_complete,
}
```

Installing it adds no command to the user’s namespace by name. What it adds is a completion for a command the user already has.

The generator’s implementation is the reason the plugin is 141 lines rather than several thousand. `kubectl` is a cobra-based CLI, and every cobra CLI implements a hidden `__complete` subcommand that takes the current argv and prints candidates. The plugin builds that argv from `$CURRENT` and the words it was handed (`lib.rs:24-40`), runs `kubectl __complete ...`, and parses the pre-tab column out of the output (`lib.rs:43`).

The consequence is the one its module doc claims (`lib.rs:12`): it “tracks whatever” the installed binary knows. Subcommands added in a new `kubectl` release, flags, and — because `__complete` queries the cluster — live resource names, all appear with no change to the plugin. A hand-written completion tree for `kubectl` is thousands of lines and is stale the day it ships; this is 141 lines and cannot go stale, because it does not encode any knowledge about `kubectl` at all.

The failure handling is a single line worth quoting (`lib.rs:77`):

```
return 0; // kubectl not on PATH — no completions, no error noise.
```

A completion generator that printed an error when its tool was missing would put that error on the user’s command line, mid-completion. Returning zero with no output produces “no matches,” which is the correct and quiet answer.

The one thing the plugin does put in the namespace is the generator builtin the macro registers behind it — `zshrs_complete_kubectl` (Chapter 47). The SDK’s own comment explains the missing leading underscore: `zsh/compsys` treats `_*` command names as autoloadable completion functions, “which would shadow this builtin before plugin dispatch.” So a plugin with an empty `builtins:` block still occupies one name, and that name is derived rather than chosen.

Chapter 58 — Eleven Writes That Bypass `host.print`

`HostApi` has a `print` callback and the fleet contains eleven direct writes to the process's own stdout and stderr that do not use it.

```
$ grep -rnoE 'println!|eprintln!|print!|eprint!' zshrs-git-fuzzy/src/lib.rs | wc -l
7
$ grep -rnoE 'println!|eprintln!|print!|eprint!' zshrs-revolver/src/lib.rs | wc -l
4
```

Seven and four. The other six plugins have zero — every byte they emit goes through `host.print`.

Why the callback exists at all

`host.print` is not a wrapper around `println!`. It is a call into the shell, and the shell is what knows where “stdout” currently points. A builtin runs inside a command that may be part of a pipeline, may have had its output redirected, may be inside a command substitution whose output is being captured. All of that is state the shell tracks and the plugin does not.

A `println!` inside a plugin writes to file descriptor 1 as the *process* has it. When the shell's notion of the builtin's stdout and the process's fd 1 are the same thing, the two are indistinguishable. When they are not, the direct write goes somewhere the user did not ask for — past a redirection, out of a captured substitution, into the terminal in the middle of a pipeline's output.

The two plugins, and their two different reasons

`zshrs-revolver`'s four writes are all `eprintln!` error messages (`lib.rs:255, :264, :330, :352`), and three of the four carry raw ANSI:

```
eprintln!("{}", "\x1b[0;31mRevolver process could not be found\x1b[0;m");
eprintln!("{}", "\x1b[0;31mSpinner '{style}' is not recognised\x1b[0;m");
```

These are on stderr, and the plugin's whole output discipline is about not touching the shell's stdout (Chapter 56) — its animation goes to `/dev/tty` deliberately. Errors going to the process's stderr rather than through the host is consistent with that posture and is very unlikely to surprise anyone: a shell rarely redirects a builtin's stderr in a way that the direct write would defeat.

What is worth flagging is not the stream but the escapes. Hardcoded `\x1b[0;31m` is unconditional colour: it is emitted whether or not stderr is a terminal, whether or not the user's theme wants red, and whether or not the shell is running with colour disabled. A message captured into a log file arrives with the escape bytes in it. The shell has a notion of whether output is a terminal; the plugin, writing directly, does not consult it.

`zshrs-git-fuzzy`'s seven are `print!/println!` on stdout (`lib.rs:488, :510, :560, :568, :579, :623, :747`), and Chapter 54 gives the reason they are defensible: six of the seven are inside helper subcommands, which run in a `fresh zshrs -f` process whose stdout is a pipe `fzf` is reading. In that process, the shell's stdout and the process's fd 1 are the same descriptor, so the two calls are equivalent, and `print!` avoids a `CString` allocation per write in a path that runs on every keystroke.

That argument is correct and it is nowhere in the plugin's source. There is no comment at any of the seven sites explaining that they are helper-path-only, and nothing structural prevents a future refactor from calling one of those functions from the top-level `gf` builtin — at which point the write happens in the user's interactive shell and bypasses whatever redirection was in effect.

The rule, and where it actually bites

The rule is one line: **inside a builtin running in the user's shell, print through the host.** Outside it — in a helper process where the plugin is effectively the whole program — the distinction has no content.

Where it bites hardest is command substitution. A user who writes

```
files=$(gf --helper status_menu_content)
```

is asking the shell to capture the builtin's stdout, and the shell implements that by redirecting the builtin's output stream. A `host.print` call lands in the capture; a `println!` may not. That is the failure mode: not a crash, not an error, just output appearing on the terminal and an empty variable. It is exactly the class of bug Chapter 36 describes as hardest to attribute, because the plugin “works” when run interactively and fails only under capture.

Eleven writes across 3,905 lines is a low rate, and both concentrations have a reason. Recording them precisely is worth more than either overstating the hazard or leaving it out.

Chapter 59 – What the Fleet’s Documentation Says, and Where the Source Disagrees

Every plugin in the fleet ships a README. So does the SDK. Chapter 16 of the companion registry book makes the case that a book which cites source is only trustworthy if it also reports where the *other* documents disagree with that source, and the same obligation applies here.

The fleet’s documentation is in unusually good shape, because six of the eight repositories’ most recent commits were documentation passes — environment variables, aliases, hidden flags, a corrected claim about a diff renderer. What follows is what remains after that pass.

The SDK README omits three of the twelve callbacks

`znative/README.md` has a Host API table (:88-99) listing nine methods:

```
print  eval  getvar  setvar  getfunction  addfunction
register_builtin  add_match  install_completion
```

`HostApi` has twelve fields. The three v4 additions — `register_compfn`, `comp_dispatch`, `empty_command_hash` — appear nowhere in the README, and neither do their `Host` wrappers, which do exist (`lib.rs:349`, `:359`, `:379`).

The list is also not quite the ABI’s list in the other direction: `add_match` is a `Host` convenience with no ABI field behind it (Chapter 47), and `install_completion` is the wrapper over the `register_completion` field. So the README’s nine rows are nine *methods*, mixing eight table fields with one pure convenience, and omitting four table fields. Neither list is wrong for what it is; they are simply different lists, and a reader counting one against the other will not get the same number twice.

The crate’s own doc comment (`lib.rs:49-56`) has the same omission — its table covers v1 through v3 and stops. So a plugin author browsing the rendered docs finds `register_compfn` only by reading the `Host` impl or the `HostApi` struct directly. Given Chapter 52’s finding that the v4 callbacks have zero fleet users, an undocumented capability with no consumers is a coherent state; it is also self-reinforcing.

PLUGINS.md undercounts the examples

`zshrs/docs/PLUGINS.md:313` says “Two runnable examples ship in `examples/`.”

```
$ ls -d zshrs/examples/plugin-*/ | wc -l
11
$ find zshrs/examples/plugin-*/ -name '*.rs' | xargs wc -l | tail -1
3871 total
```

Eleven directories, 3,871 lines. The two the sentence refers to are almost certainly `plugin-hello` and `plugin-complete`, the teaching pair Chapter 18 describes, and the other nine — mirrors of the eight fleet

plugins plus `plugin-command-names`, which is the only thing anywhere that exercises the v4 completion callbacks — arrived afterwards without the sentence being updated.

The undercount matters more than a stale number usually would, because `plugin-command-names` is the sole worked example of ABI v4 and a reader who believes there are two examples will not go looking for it.

zshrs-fasd's `-i` flag

Covered in full in Chapter 55: the README documents `-i` as “interactive select” (`README.md:84`) and the implementation prints the scored list and silently picks the best match, with the reason inline — “interactive selection needs a tty read the plugin ABI can't drive” (`src/lib.rs:557-558`).

This is the fleet's one genuine documentation-versus-behaviour disagreement, as opposed to an omission. It is also the most defensible one, because the sentence is true of `fasd` and the port inherited both the flag and its description.

zshrs-git-fuzzy's twelve helper subcommands

The README explains the self-reentrant mechanism twice (`:38`, `:98`) and never lists the twelve `--helper` subcommands or documents `--helper` as a flag. They are an internal protocol, which is a fair reason not to put them in a user-facing README; they are also the first thing a user debugging a dead keybinding will want. Chapter 54 lists them.

zshrs-forgit's keybindings

Ten `--bind=` arguments across its `zfz` invocations, documented nowhere:

```
$ grep -ci 'ctrl-\\|keybind' zshrs-forgit/README.md
0
```

Zero mentions. Combined with the absence of any `-h` handling across all nine commands (Chapter 57), a user who forgets which key stages a hunk has no in-tool and no in-README answer.

What is not wrong

Worth stating, because the list above reads as a catalogue of faults and the proportions matter.

All eight `declare_plugin!` version literals currently match their crate versions — verified across the fleet, after the recent fixing pass (Chapter 53). All eight `Cargo.toml` files declare `znative = "0.12"`, which is the requirement the SDK's own documentation asks for. Every plugin's README documents its environment variables, after the recent pass. And the SDK's doc comments on `HostApi` are unusually complete — every field carries a description including which ABI version introduced it, which is what made Chapter 47's table possible to write without inference.

The pattern across all six findings is the same one: documentation drifts where nothing checks it, and the things nothing checks are exactly the things a CI file cannot check — a prose sentence's count of examples, a flag's semantics, a list of keybindings. The one drift that was mechanically checkable, the version literal, is the one that had drifted in six of eight repositories and has now been fixed by hand in all six, with no gate added to keep it fixed. Chapter 53 proposes the gate, and it is a macro change rather than a CI change.

Chapter 60 — The Completion Path, End to End

Chapter 14 covers native completions from the plugin author's side and Chapter 42 covers the ordering rule. This chapter traces one completion from the macro invocation that declares it to the `compadd` that emits its candidates, because the path crosses four components and no single earlier chapter follows it all the way.

There are actually two paths — the v2 *generator* path and the v4 *override* path — and they meet the completion system at different points. Take them in order.

The v2 path: a generator wired in by the host

Step 1 — the declaration. A plugin writes a `completions:` block:

```
declare_plugin! {
  name: "kubectl-completion",
  version: "0.1.1",
  completions: { "kubectl" => kubectl_complete },
}
```

Step 2 — the macro expands it into two calls. The `completions:` arm ([znative/src/lib.rs:498-519](#)) emits a trampoline, derives a name, registers the trampoline as a *builtin* under that name, and then asks the host to wire it:

```
let gen_name = concat!("zshrs_complete_", $ccmd);
h.register_builtin(gen_name, __cgen);
h.install_completion($ccmd, gen_name);
```

So a native completion is, mechanically, a hidden builtin plus a note to the host that the builtin should be consulted for a given command. There is no separate “completion object” anywhere in the ABI — the generator is a command, and the only thing distinguishing it from a user-facing one is a naming convention.

The name has no leading underscore, and the comment above it explains why in terms of dispatch order ([lib.rs:511-514](#)): `zsh/compsys` treats `_*` command names as autoloadable completion functions, “which would shadow this builtin before plugin dispatch.” Plugin commands resolve after real builtins and shell functions (Chapter 48), and `compsys`'s autoload machinery would present an `_`-prefixed name as a function — so the generator would never be reached.

Step 3 — the host stages it. `host_register_completion` ([plugin_host.rs:217](#)) does not touch `compsys`. It appends a `(cmd, generator, owner)` triple to `pending_completions()` and returns. The owner field is filled in later, by `load`, once the plugin's name is known ([plugin_host.rs:449-455](#)).

This staging is not an optimisation. It is the workaround for the constraint stated three times in the codebase: evaluating shell code during plugin init hangs the VM, because init runs deep inside the `zmodload` call stack. `compdef` is shell code. Therefore `register_completion` cannot run `compdef`, and the wiring has to happen somewhere else.

Step 4 — the flush, at a safe point. `flush_pending_completions()` ([plugin_host.rs:478-508](#)) is called from the top of the completion pipeline. Its doc comment names the safety property ([:470-472](#)): this is “a safe point where `compsys` itself evals, unlike `plugin-init`.”

For each pending triple it evaluates:

```
_zshrs_plug_<cmd>() {  
    local -a _zp_m  
    _zp_m=("${@f}")<generator> $CURRENT $words}"")  
    compadd -- $_zp_m  
}  
(( ${+functions[compdef]} )) && compdef _zshrs_plug_<cmd> <cmd> 2>/dev/null  
:
```

Four decisions are visible in that fragment. `${@f}...` splits on newlines, which is why `Host::add_match` is exactly “print the word, print a newline” (`lib.rs:293-296`) and nothing more. `$CURRENT $words` is the generator’s argument convention — the 1-based index of the word being completed, then every word on the line — documented in `install_completion`’s doc comment (`lib.rs:308-310`). The `(( ${+functions[compdef]} ))` guard makes a flush before `compinit` a silent no-op instead of an error. And the trailing `:` pins the fragment’s exit status to zero.

The entry then moves from `pending_completions()` to `installed_completions()`, so the flush is idempotent — a second completion does not re-eval the glue.

Step 5 — the generator runs. Completion fires, `compsys` calls `_zshrs_plug_kubectl`, the shell function runs the generator builtin inside a command substitution, plugin dispatch resolves the name to the trampoline, the trampoline decodes argv into an `Args` and calls the handler, the handler prints candidates through `host.print`, and the shell splits the captured stdout into an array and `compadds` it.

Count the boundary crossings: shell → plugin (the generator call), plugin → shell (each `host.print`), shell → `compsys` (the `compadd`). The candidates cross as one newline-delimited blob of text, which is why Chapter 14 notes that structured completion metadata — descriptions, groups, per-match display strings — cannot cross. `compadd` supports all of them; the pipe between the generator and `_zshrs_plug_<cmd>` is a stream of words.

The v4 path: replacing a completion function outright

ABI version 4 added a second, deeper hook. Instead of supplying candidates for a command, a plugin can replace a `compsys` function — one of the `_NAME` functions `compsys` itself calls.

Registration is `compfn`: in the macro (`lib.rs:520-537`), which emits a trampoline and calls `register_compfn`. The host stages it in `compfn_staging()` and promotes it into `compfn_registry()` on a successful load, tagged with the owner.

Dispatch is `compfn_override(name)` (`plugin_host.rs:572`), consulted by `compsys::router::try_rust_dispatch`. Its doc comment states the precedence (`:567-571`): a plugin’s override is consulted **before** the built-in Rust port, “so a plugin’s `_command_names` (etc.) supersedes both the port and the shell autoloader.” That is a three-layer stack — plugin override, then native port, then autoloader shell function — with the plugin on top.

Unload has to purge `compfn_registry()` before the `dlclose`, and the code says so at the site (`:535-536`): “Purge compfn overrides owned by this plugin BEFORE dlclose, so no dangling CompFn pointer survives.”

Two supporting callbacks came with it. `comp_dispatch` lets an override delegate back into the completer chain — call `_alternative` or `_path_commands` and have the host resolve it “exactly as the completer chain would ... in the *current* completion scope so `local/tag` state nests correctly” (`lib.rs:171-176`). And `empty_command_hash` reproduces the observable effect of `local -A +h commands` inside a completion function.

Which of the two paths is actually used

The v2 generator path has two users in the fleet: `zshrs-kubectl-completion` and `zshrs-zsh-z` (Appendix O).

The v4 override path has exactly one user anywhere in the repository, and it is an example rather than a shipped plugin. Chapter 61 reads it. What matters here is the count: of the three callbacks version 4 added, one has a single user and two have none at all — including in the example built specifically to demonstrate v4.

Chapter 61 – The Only Plugin That Uses ABI v4

`zshrs/examples/plugin-command-names/` is 131 lines of Rust and a 22-line manifest. It is the sole consumer of ABI version 4 anywhere — not in the fleet, not in the other ten example plugins, nowhere else in the repository. It is worth a chapter because it is simultaneously the strongest demonstration of what the override path is for and the sharpest evidence that two of the three callbacks version 4 added were not needed.

What it does

Its module doc states the problem precisely (`src/lib.rs:1-14`). A user's zsh configuration replaces the stock `_command_names` with a version that *unconditionally* shadows the command-name hash, so completing a bare command offers builtins, reserved words, functions, and aliases — never the raw external command hash. zshrs's built-in `_command_names` port mirrors stock zsh, where the shadow is gated behind a style. The two therefore produce different candidate sets for the same keystroke.

The plugin is the faithful port of the customised body, registered with `register_compfn` so it supersedes the built-in port. The stated success criterion is byte-identity: `l<TAB>` should offer exactly what the user's real zsh offers.

The doc comment then does something unusual and worth naming as a practice: it includes the **entire original shell function**, deparsed from the user's `.zcompdump`, as a 25-line fenced block (`:16-43`). The port and its spec are in the same file, and a reader checking fidelity does not have to find the original. That is the porting discipline of Chapter 16 taken one step further than any of the fleet plugins take it.

How the port is structured

The Rust half computes; the shell half executes. That division is the same one Chapter 55 describes for `zshrs-zsh-z` and it is applied here at a much finer granularity, because the function being ported is itself a piece of shell plumbing.

Rust does the branching. Whether to add the `executables:` alternative depends on whether `$PATH` contains `.` or `$PREFIX` contains a slash — the plugin reads `PATH` and `PREFIX` with `getvar` and decides in Rust (`:76-84`). Whether to apply the `ffilt` function filter depends on a `zstyle -t` test and a prefix check; the plugin runs the `zstyle` through `eval` and reads its exit status, then combines it with a Rust string test (`:64-72`). The nine `defs` entries are pushed into a `Vec<String>` (`:87-99`).

The shell does the scoping. The final step builds one script and evals it once (`:104-124`), and the comment above it explains why it must be one script rather than several calls:

The remaining statements MUST run in one dynamic scope so the `local -A +h commands` shadow is live while `_alternative` (and the `_path_commands` it calls) run — a scoped EMPTY `commands` param is what makes `compadd -k commands` add nothing (NOT emptying the global command hash, which would just refill).

That is the technical heart of the plugin and it decides the ABI question below.

The argument quoting is done properly: `sq()` (:50-52) single-quotes each word and escapes embedded single quotes the POSIX way (`'\''`), so a completion argument containing a quote cannot break out of the generated script. In a function that builds shell text from words the user typed, that is the difference between a port and a vulnerability.

The comment also records what was *elided* and why: `_saved_path, path=($_saved_path)`, and the `command-path` reassignment “leave `$path` unchanged with the styles unset, so they are elided.” A port that drops three statements and says which three and on what condition is auditable; one that silently drops them is not.

What it does not use

The plugin’s callback usage, counted:

Callback	Uses
<code>eval</code>	3
<code>getvar</code>	2
<code>register_compfn</code>	via <code>compfn</code> : in the macro
<code>comp_dispatch</code>	0
<code>empty_command_hash</code>	0
<code>print / add_match</code>	0

It uses one of version 4’s three additions and declines the other two — and for `empty_command_hash` the declining is explicit and reasoned. The callback’s own doc comment describes it as reproducing “the observable effect of `local -a +h path; local -A +h commands` inside a completion function” (`lib.rs:183-187`). The plugin’s comment says that is the wrong mechanism for this job: what makes `compadd -k commands` add nothing is a *scoped empty commands* parameter, not an emptied global hash, “which would just refill.”

Those two comments are in tension, and the plugin’s is the one written by the code that had to work. `empty_command_hash` empties `cmdnamtab` (`plugin_host.rs:333`); the shell refills the hash on demand; so a completion that emptied it and then ran `_path_commands` would see it repopulate mid-completion. The scoped-shadow approach does not have that problem because it never touches the global at all.

`comp_dispatch` is unused for a simpler reason: the plugin’s final act is `_alternative -0 args "$defs[@]"`, and it reaches `_alternative` through the same `eval` that establishes the scope. Splitting that into `eval` for the scope plus `comp_dispatch` for the call would break the property the comment insists on — the shadow has to be live *while* `_alternative` runs, and two calls are two dynamic scopes.

So the one worked example of the override path uses `eval` where `comp_dispatch` was designed to be used, and declines `empty_command_hash` on correctness grounds.

What that means for the ABI

Chapter 52 argues that an ABI addition is a decision that cannot be revisited, so the bar for making one should be a plugin that cannot be written without it. Apply that bar to version 4’s three additions with this example as the evidence:

- **`register_compfn` clears it.** There is no other way to supersede a compsys function from a compiled plugin. Without it this plugin does not exist.
- **`comp_dispatch` does not clear it, on this evidence.** Its intended user needed a single dynamic scope and used `eval` instead. It may well clear the bar for a different override — one that delegates without needing to establish scope first — and no such override has been written.
- **`empty_command_hash` does not clear it, on this evidence.** Its intended user examined it and concluded it does the wrong thing for the case it was added for.

Two of three, on the sole available worked example. That is not a claim that the two are useless; it is the observation that the receipts for them do not exist yet and the one place they should have appeared, they did not.

The constructive reading is the one Chapter 21 already argues: the cost of adding a field is permanent and the cost of *not* adding one is a version bump later. When in doubt, the cheaper mistake is to wait for the plugin that cannot be written without it. Version 4 is the case study for what happens when the doubt is resolved the other way.

Chapter 62 – What Version 5 Would Have to Add

The ABI has not moved since 2026-07-18. This chapter collects the gaps the previous fifteen chapters found, states what each one costs today, and applies Chapter 52's bar to each: is there a plugin that cannot be written without it?

Four candidates, in descending order of the evidence behind them.

1. Hook registration – two plugins already work around its absence

The gap. `HostApi` has no entry point for installing a shell hook. A plugin that wants a `chpwd`, `preexec`, or `precmd` hook has to `eval` a fragment that defines a function and calls `add-zsh-hook`.

The cost today. Evaluating shell code during plugin init hangs the VM (Chapter 48), so the `eval` cannot happen where it naturally belongs. Both hook-installing plugins in the fleet therefore defer it to the first invocation of a builtin, behind an `AtomicBool`:

```
static INSTALLED: AtomicBool = AtomicBool::new(false);
```

(`zshrs-fasd/src/lib.rs:373`, `zshrs-zsh-z/src/lib.rs:285`.) Two plugins, independently, with the same idiom, working around the same documented host constraint. And the deferral has a real behavioural consequence: the hook is not installed until the user runs one of the plugin's commands, so a `zshrs-zsh-z` installed today records nothing until the first `z` invocation. A user who loads the plugin and then spends an hour navigating directories has recorded none of it.

Against the bar. Both plugins *can* be written without it, and are. So the bar as literally stated is not cleared.

Why it is still the strongest candidate. The bar exists to prevent additions that are conveniences. This one removes a class of bug rather than a class of typing: the current workaround is correct only if every plugin author independently discovers a rule documented in one comment, and the failure mode of missing it — a hung shell at load time — is the worst in the system. A `register_hook(kind, body)` that staged exactly the way `register_completion` does would move the deferral from eight plugin authors into one host function, and the host already has the staging machinery built.

2. A `tty read` – one documented flag already lies because of its absence

The gap. The table is write-only with respect to the terminal, plus `getvar` for the environment. There is no `read_line`, no `read_key`, no way to get input from the user.

The cost today. `zshrs-fasd`'s `-i` flag documents “interactive select” and does not select interactively. The source says why, at the site (`zshrs-fasd/src/lib.rs:557-558`): “interactive selection needs a `tty read` the plugin ABI can't drive; degrade to the scored list.” Chapter 55 covers it; Chapter 59 lists it as the fleet's one genuine documentation-versus-behaviour disagreement.

The workarounds that exist. Four fleet plugins shell out to `fzf`, which owns the tty for the duration and hands back a selection on stdout. That works, it is what the shell originals did too, and it means the plugin depends on `fzf` being installed. A plugin that wants one line of input could also `eval` a `read` into a parameter and then `getvar` it — two crossings and a shell round trip for one line.

Against the bar. `fzf` is a complete answer for selection and the `eval+read` dance is a complete answer for a line, so nothing is unwritable. What the absence buys is a smaller ABI, and what it costs is that “prompt the user” is the one ordinary builtin behaviour a native plugin cannot do natively.

3. Arrays and associative arrays — six plugins read scalars and none reads a list

The gap. `getvar` and `setvar` handle scalars. There is no way to read `$path` as a list, or `$aliases` as a map, through the table. A plugin that wants either has to `getvar` a scalar-joined form and split it, or `eval` an expansion.

The cost today. `plugin-command-names` reads `PATH` and splits on `:` (`examples/plugin-command-names/src/lib.rs:79-83`) — which works for `PATH` specifically because the scalar form is canonical, and would not work for an array whose elements can contain the separator. For anything else, the plugin runs a `zstyle` or an expansion through `eval` and reads the exit status, which is a single bit.

Against the bar. Nothing in the fleet is blocked. `PATH` has a scalar twin; most plugin configuration is scalars; the one plugin that needed real array semantics reached for `eval` and got them from the shell.

The observation that matters. Six of the eight fleet plugins call `getvar` and none calls `setvar` (Chapter 52), which suggests the read side of parameter access is what plugins actually want and the write side is already over-provisioned. Adding array reads would follow the demand; adding array writes would repeat `setvar`’s mistake.

4. A panic guard — nothing in the system has one

The gap. No `catch_unwind` anywhere on the plugin path, on either side (Chapter 50). A panic in a handler unwinds through an `extern "C"` frame, which is undefined behaviour.

The cost today. Every plugin author has to know a rule documented in one paragraph of `docs/PLUGINS.md`, and the consequence of not knowing it is a crashed interactive shell. None of the eight fleet plugins carries a guard.

Why it is not an ABI change. This is the candidate that does not need a version bump at all, which is why it is listed last and is arguably the most actionable. Wrapping the handler body in `catch_unwind(AssertUnwindSafe(...))` inside `declare_plugin!`’s three trampolines is a change to the macro — source-level, not layout-level. `ABI_VERSION` does not move, existing compiled plugins are unaffected, and every plugin that recompiles gets the guard.

Chapter 50 states the argument against: asserting unwind safety over an arbitrary handler holding `Host` is an assertion the SDK cannot justify, and a panic mid-handler can leave shell state torn. That argument is real and it is weighed against a default of undefined behaviour. Nothing in the source records which way the tradeoff was decided or whether it was considered, so the book records the question rather than an answer.

The one thing that is not on the list

ZLE widget registration. There is no way for a native plugin to register a ZLE widget through the ABI, and the omission looks like a gap until you look at what the fleet does with it: nothing. No plugin in the fleet wants one, and the script path handles widgets perfectly well — `history-search-multi-word`, a script plugin with

a ZLE widget, its own forked highlighter, paged `POSTDISPLAY`, live `region_highlight`, and in-widget key reads, installs and runs under `znative` with no port at all (Chapter 18).

That is Chapter 43's "one kind per plugin" rule doing its job. A plugin that needs a widget ships as a script plugin; a plugin that needs compiled speed for per-call work ships as a native one; the manager installs both from the same `.zshrc` line. The absence of widget registration in the ABI is not a missing feature — it is the boundary between the two kinds, and moving it would make the native path a superset that the script path no longer has a reason to exist alongside.

The pattern across all five

Four gaps and one non-gap, and the honest summary is that **none of the four clears the bar as literally stated**, because every one of them has a working shell-side workaround. That is not an accident; it is what having `eval` in the table means. A callback table that can run arbitrary shell code can, in principle, do anything the shell can do, so no capability is ever strictly unreachable and the "cannot be written without it" bar can never be cleared by a capability argument alone.

Which means the bar has to be read as Chapter 52 intended rather than literally: the question is not "is this reachable" but "does routing it through `eval` cost correctness." By that reading, hook registration is the candidate with real evidence — the workaround is duplicated, its failure mode is a hang, and the deferral it forces has a user-visible consequence — and the panic guard is the change that needs no version bump and would remove the system's only undefined behaviour.

Chapter 63 – Twenty-Five Names in a Shared Namespace

Eight plugins register twenty-five builtins between them, and every one of those names lands in the same flat namespace: the one the user types into. There is no prefix, no plugin qualifier, no `plugin::command` syntax, and no way for a user to disambiguate two plugins that want the same name. This chapter is what that implies.

The distribution

Plugin	Builtins	Names
<code>zshrs-fasd</code>	10	<code>fasd</code> , <code>fasd_cd</code> , <code>a</code> , <code>s</code> , <code>d</code> , <code>f</code> , <code>sd</code> , <code>sf</code> , <code>v</code> , <code>j</code>
<code>zshrs-forgit</code>	9	<code>ga</code> , <code>grh</code> , <code>glo</code> , <code>gd</code> , <code>gcf</code> , <code>gclean</code> , <code>gss</code> , <code>gcp</code> , <code>gi</code>
<code>zshrs-git-fuzzy</code>	2	<code>gf</code> , <code>git-fuzzy</code>
<code>zshrs-zsh-z</code>	1	<code>z</code>
<code>zshrs-revolver</code>	1	<code>revolver</code>
<code>zshrs-git-repos</code>	1	<code>gitrepos</code>
<code>zshrs-reveal</code>	1	<code>reveal</code>
<code>zshrs-kubectl-completion</code>	0	— (plus the hidden <code>zshrs_complete_kubectl</code>)

Nineteen of the twenty-five belong to two plugins, and both of those plugins are porting a tool whose *interface* is a set of very short names. Chapter 19 advises keeping a plugin's public namespace small; `fasd` and `forgit` are the two cases where following that advice would mean not porting the tool.

The short names are the interesting half. `a`, `s`, `d`, `f`, `v`, `j` — six single-letter commands from one plugin. `ga`, `gd`, `gi` — three two-letter commands from another. These are names a user is extremely likely to have already bound to something, and the resolution order decides what happens.

What a plugin name can and cannot shadow

Plugin commands resolve in one specific slot: after real builtins, after shell functions, before `PATH` lookup (`plugin_host.rs:11-19`, and Chapter 48). The consequences follow directly.

A plugin cannot shadow a builtin. `cd`, `echo`, `print`, `read`, `set`, `bind` — a plugin that registered any of these would find its handler never called, silently. There is no error at registration; `host_register_builtin` stages whatever name it is given.

A plugin cannot shadow a shell function. A user with a `gd` function in their `.zshrc` keeps it, and `zshrs-forgit`'s `gd` is unreachable. Again silently.

A plugin does shadow a `PATH` executable. A user with `/usr/local/bin/z` gets `zshrs-zsh-z`'s `z` instead once the plugin is loaded. This is the case that can surprise, because the executable was working yesterday and the plugin's arrival changed which one runs.

A plugin does not shadow an alias, because aliases are resolved before command lookup entirely. A user with `alias gd='git diff'` never reaches the plugin's `gd`.

Read as advice for a plugin author, the ordering is friendlier than it looks: the two things a plugin cannot shadow are the two whose shadowing would be most destructive, and the one thing it can shadow is the one a user can most easily inspect with `which`.

Read as advice for a user, it means **a plugin's command names are its most compatibility-relevant surface**, and there is no way to rename them at install time. Chapter 19's advice to keep the surface small is advice to the author because the user has no lever.

What happens when two plugins collide

The host refuses duplicate *plugin* names (`plugin_host.rs:420-427`) and does not refuse duplicate *command* names. Registration is a `HashMap` insert (`plugin_host.rs:435`):

```
reg.insert(cmd.clone(), BuiltinEntry { func, _pad: () });
own.insert(cmd, name.clone());
```

A second plugin registering `gd` replaces the first plugin's entry, and the ownership map is updated to say the second plugin owns it. No warning, no error, no log line at the registration site.

The unload path then behaves correctly, which is worth noting because it easily might not: `unload` purges by ownership (`plugin_host.rs:520-533`), so unloading the *first* plugin does not remove the `gd` the second plugin now owns. The ownership map being the authority rather than a name-prefix scan is what makes that work — and the `_pad: ()` field's comment in `BuiltinEntry` records that an earlier design did use a prefix scan (`plugin_host.rs:55-57`).

What it does mean is that load order decides who wins, and load order is the order of `znative load` lines in a `.zshrc`. Last one wins. A user with both a plugin that registers `d` and another that registers `d` will get the later line's, and will find out by running it.

The name a plugin does not choose

Every plugin with a `completions:` block gets one extra name it did not ask for: `zshrs_complete_<cmd>`, registered as a real builtin by the macro (Chapter 47, Chapter 60). It is typeable. Running `zshrs_complete_kubectl 1 kubectl get` at a prompt runs the generator and prints its candidates.

That is two names in the user's namespace for `zshrs-kubectl-completion`, a plugin whose `declare_plugin!` declares zero builtins. It is not a problem — the name is long, prefixed, and collision-proof by construction — but it is the one case where the count of names a plugin occupies is not the count of names it wrote down.

The practical rules

For a plugin author, three rules fall out of the above and none of them is enforced by anything:

Do not register a name that is a real builtin. It will never be reached and nothing will tell you. Check against `zshrs`'s builtin list before choosing.

Prefer one name plus subcommands over many names. Four of the eight fleet plugins do exactly this — `revolver start|update|stop|demo`, `gitrepos`, `reveal`, `gf` with twelve internal helpers behind one flag. It puts one name in the namespace instead of ten and it gives the plugin a place to put `-h`.

If you are porting a tool, inherit its names and say so. `fasd` and `forgit` took nineteen names between them because the names *are* the tool, and a port that renamed them would not be a port. That is the correct call and it makes the plugin's namespace footprint a documented property rather than an accident — which is exactly what the recent README passes across the fleet (Chapter 59) were for.

Chapter 64 – The Example Tree, and How Far Behind It Is

`zshrs/examples/` contains eleven plugin directories, 3,871 lines of Rust between them. `docs/PLUGINS.md:313` says “Two runnable examples ship in `examples/`.”

The gap between those two facts is the subject of this chapter, and it is more interesting than a stale sentence, because nine of the eleven are *mirrors* of the eight published fleet repositories — and every one of the mirrors is behind the repository it mirrors.

The inventory

Directory	Crate name	LOC	Role
<code>plugin-hello</code>	<code>hello</code>	54	teaching: the whole host API in two commands
<code>plugin-complete</code>	<code>greet</code>	87	teaching: a native completion for <code>greet</code>
<code>plugin-kubectrl</code>	<code>kubectrl_completion</code>	74	mirror of <code>zshrs-kubectrl_completion</code>
<code>plugin-command-names</code>	<code>command_names</code>	131	the sole ABI v4 worked example (Chapter 61)
<code>plugin-git-repos</code>	<code>git_repos</code>	279	mirror of <code>zshrs-git-repos</code>
<code>plugin-reveal</code>	<code>reveal</code>	304	mirror of <code>zshrs-reveal</code>
<code>plugin-revolver</code>	<code>revolver</code>	401	mirror of <code>zshrs-revolver</code>
<code>plugin-zsh-z</code>	<code>zsh_z</code>	411	mirror of <code>zshrs-zsh-z</code>
<code>plugin-forgit</code>	<code>forgit</code>	552	mirror of <code>zshrs-forgit</code>
<code>plugin-fasd</code>	<code>fasd</code>	782	mirror of <code>zshrs-fasd</code>
<code>plugin-git-fuzzy</code>	<code>git_fuzzy</code>	796	mirror of <code>zshrs-git-fuzzy</code>
total		3,871	

Two teaching cases, eight mirrors, one v4 demonstration. The two the `PLUGINS.md` sentence refers to are almost certainly `plugin-hello` and `plugin-complete`, and everything else arrived afterwards without the sentence moving.

plugin-hello is the only place setvar is used

54 lines, two builtins, and its module doc claims to demonstrate “the full host API: printing, reading/writing shell parameters, and evaluating shell code.”

`renv NAME [VALUE]` is the reason it matters. With one argument it calls `host.getvar`; with two it calls `host.setvar` (`plugin-hello/src/lib.rs:22-44`). Chapter 52 reports that `setvar` has zero users across the eight shipped plugins — this is the qualification: it has exactly one user anywhere, and that user is the teaching example whose purpose is to exercise every callback.

That is a sharper statement of the same finding rather than a softening of it. A callback whose only consumer is the example written to demonstrate it has no production consumer, and the ABI’s own worked examples are where you would look first for evidence that a slot earns its place.

The plugin also demonstrates the return-status convention properly: `0` for success, `1` when `setvar` fails, `2` for a usage error (`:38`, `:42`). Three different non-zero statuses from a 54-line plugin, which is more status discipline than several of the shipped ones show.

Every mirror is behind its repository

Line counts, side by side:

Plugin	<code>examples/</code>	published repo	delta
<code>git-fuzzy</code>	796	868	-72
<code>fasd</code>	782	783	-1
<code>forgit</code>	552	599	-47
<code>zsh-z</code>	411	491	-80
<code>revolver</code>	401	403	-2
<code>git-repos</code>	279	315	-36
<code>reveal</code>	304	305	-1
<code>kubectl-completion</code>	74	141	-67

Eight mirrors, eight negative deltas, ranging from one line to eighty. Not one is ahead of or equal to its published counterpart.

The pattern is what you would expect from the history: each plugin was written as an example first, extracted into its own repository, and then developed there. Chapter 59 records that six of the eight repositories’ most recent commits were documentation and version-literal passes, and several bumped behaviour along the way — `zshrs-kubectl-completion` nearly doubled, `zshrs-zsh-z` gained eighty lines. None of that flowed back.

This is not a defect in the mirrors, because a mirror’s job is to be a readable worked example rather than the shipping artifact. It is a hazard for exactly one reader: someone who reads `examples/plugin-zsh-z/` to understand what `znative load MenkeTechnologies/zshrs-zsh-z` installs. Those are different programs by eighty lines, and nothing in either tree says so.

The practical rule that falls out: **read `examples/` to learn the ABI; read the repository to learn the plugin.**

Every example uses a path dependency

All eleven declare:

```
znative = { path = "../../znative" }
```

while all eight published repositories declare `znative = "0.12"`. That difference is correct for both — an in-tree example should build against the in-tree SDK, and a published plugin has to resolve from a registry — and it has one consequence worth naming.

An example always builds against the *current* workspace `znative`, including an `ABI_VERSION` bump made in the same commit. A published plugin builds against whatever `0.12.z` its lockfile pinned (Chapter 51). So the examples are the only plugins in the world that cannot be out of step with the host, and they are therefore the only ones that would catch an ABI change at compile time. That makes them a useful canary and a misleading model: an author who copies an example's `Cargo.toml` verbatim gets a path dependency to a directory that does not exist on their machine.

plugin-revolver is the only one that is also an rlib

Ten examples declare `crate-type = ["cdylib"]`. One declares `crate-type = ["cdylib", "rlib"]`, with the reason at the site ([plugin-revolver/Cargo.toml](https://github.com/alexcrichton/plugin-revolver/blob/master/Cargo.toml)):

```
[lib]
# rlib alongside cdylib so `cargo test` can link the frame-table checks;
# znative still loads the cdylib artifact.
crate-type = ["cdylib", "rlib"]
```

That is the answer to a question every plugin author eventually hits: a pure `cdylib` cannot be linked by a test harness, so `cargo test` against a `crate-type = ["cdylib"]` crate has nothing to link against for integration tests. Adding `rlib` produces both artifacts from one build; the shell loads the `cdylib` and the test binary links the `rlib`.

`plugin-revolver` needs it because its tests check the 55-entry frame table (Chapter 56), which is a `static` inside the crate. The other ten examples get away with `cdylib` alone because their tests are unit tests in the same crate, compiled into the test binary directly rather than linked from outside.

Its manifest carries two more things none of the others do. A block comment at the top giving the exact build and load commands for both platforms:

```
# cargo build --manifest-path examples/plugin-revolver/Cargo.toml
# zmodload -R examples/plugin-revolver/target/debug/librevolver.dylib # macOS
# zmodload -R examples/plugin-revolver/target/debug/librevolver.so   # Linux
# revolver start 'Working...' ; sleep 3 ; revolver stop
```

and an empty `[workspace]` table, with its own comment: "Standalone: empty workspace table so this example is not pulled into the parent zshrs workspace." That is the mechanism that keeps a nested crate from being absorbed into the enclosing workspace's member set, and it is the difference between an example that builds standalone and one that only builds from the repository root.

What the tree is good for

Read in the right order, the eleven directories are a curriculum, and it is worth stating because nothing in the repository states it:

1. `plugin-hello` (54 lines) — registration, `print`, `getvar`, `setvar`, exit statuses. The whole shape of a plugin in one screen.
2. `plugin-complete` (87 lines) — the `completions:` block, `add_match`, the `$CURRENT+words` convention, the load-after-`compinit` rule.
3. `plugin-kubectrl` (74 lines) — the same completion path against a real tool, delegating to a subprocess.
4. `plugin-reveal` (304 lines) — the smallest complete real plugin: runtime OS dispatch, detached spawn, a high test ratio over pure logic.

5. `plugin-revolver` (401 lines) — resident `static` state, a thread, `/dev/tty`, and the `cdylib+rlib` build.
6. `plugin-zsh-z` (411 lines) — `eval` as shell mutation, lazy hook installation, an atomic datafile.
7. `plugin-git-repos` (279 lines) — a bounded worker pool inside a builtin.
8. `plugin-forgit` (552 lines) — many builtins, zero evals, subprocess orchestration.
9. `plugin-fasd` (782 lines) — a real dependency, a real port with C line citations, and the tty limitation stated in a comment.
10. `plugin-git-fuzzy` (796 lines) — self-reentrancy, `dladdr`, the shim, a raw `TcpStream`.
11. `plugin-command-names` (131 lines) — ABI v4, the compsys override path, and a port whose spec is quoted in its own doc comment.

Eleven steps from 54 lines to a plugin that loads itself. The `PLUGINS.md` sentence counting two of them is the single most consequential documentation drift in the project, because a reader who believes it stops at step two.

Chapter 65 — What Building a Plugin Actually Involves

Chapter 12 covers writing a plugin and Chapter 23 covers the build step as `znative` performs it. This chapter is the build as the *author* performs it — the manifest decisions, the artifact names, and the four places a first-time author gets stuck. Everything here is read off the eleven example manifests and the eight published ones.

The manifest, minimally

```
[package]
name = "myplugin"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib"]

[dependencies]
znative = "0.12"
```

Nine lines. That is the complete set of decisions, and three of them have consequences worth spelling out.

`crate-type = ["cdylib"]` is not optional and not defaulted. Without it, cargo builds an `rlib`, there is no dynamic library to `dlopen`, and `zmodload -R` fails with `cannot load ....` The crate's own doc comment shows the two lines (`znative/src/lib.rs:33-39`) precisely because forgetting them is the first thing that goes wrong.

`znative = "0.12"` is the version requirement to write, and Chapter 51 explains why: a bare `0.12` accepts any `0.12.z`, so a workspace patch release does not require an edit. All eight published plugins use exactly this string.

`[package] version` and the `declare_plugin!` literal are two different strings, and nothing checks that they agree (Chapter 53). Six of the eight published plugins had drifted; all six were fixed by hand.

The artifact, and its three names

`cargo build` writes to `target/debug/`, and the filename depends on the platform:

Platform	Artifact
macOS	<code>lib<crate_name>.dylib</code>
Linux	<code>lib<crate_name>.so</code>

`<crate_name>` is `[package] name` with hyphens replaced by underscores — which is why the examples' package names are `git_fuzzy`, `zsh_z`, `kubectrl_completion`, and `command_names` rather than the hy-

phenated directory names. A crate named `my-plugin` produces `libmy_plugin.dylib`, and an author who tries to `zmodload -R target/debug/libmy-plugin.dylib` gets a file-not-found.

`plugin-revolver`'s manifest comment spells both platforms out (Chapter 64), and it is the only one that does.

Debug or release

`cargo build` — not `cargo build --release` — is the development default across the project, and the example manifest comments load from `target/debug/`. A debug build loads and runs fine; the difference the author will notice is that debug builds have overflow checks on, so an arithmetic overflow that would wrap in release becomes a panic in debug — and a panic in a plugin is undefined behaviour (Chapter 50). Testing in debug is therefore the *stricter* configuration for exactly the class of bug that matters most here.

The four places authors get stuck

1. The plugin loads but the command is not found. The name resolves after real builtins and shell functions (Chapter 63). If the plugin registered `cd` or a name the user has as a function, the handler is unreachable and nothing reports it. `zmodload -R` succeeded, so the plugin is loaded; the command just never routes. Check with a name that cannot collide.

2. The completion never fires. Two causes, and they look identical. The plugin loaded before `compinit`, so the flush's `(( ${+functions[compdef]} ))` guard made the wiring a silent no-op (Chapter 60) — the fix is the load-after-`compinit` ordering of Chapter 42. Or the generator prints candidates through something other than `host.print`, so they never reach the command substitution that `_zshrs_plug_<cmd>` captures.

3. `cargo test` will not link. A pure `cdylib` cannot be linked by an integration test. Add `rlib` to the `crate-type` array and both artifacts come out of one build. This is settled practice rather than a workaround: one of the eleven examples does it, and **all eight** published plugins do:

```
$ grep -h 'crate-type' zshrs-*/Cargo.toml | sort | uniq -c
      8 crate-type = ["cdylib", "rlib"]
```

So the nine-line minimal manifest above is the *minimum*, and the shipping convention is `crate-type = ["cdylib", "rlib"]`. Write the pair from the start.

4. The plugin hangs the shell at load. Something in `znative_init` evaluated shell code. That includes anything a handler-adjacent helper does at init time. The rule is absolute: `znative_init` may register and nothing else. Anything that needs the shell to run code — a hook, a `compdef`, a `zstyle` — is deferred to first invocation behind an `AtomicBool`, the way both hook-installing fleet plugins do it (Chapter 55), or staged through `register_completion` and left to the host's flush.

What the eight published plugins add on top

Every one of them ships more than the nine-line minimum, and the additions are the same set across all eight, which makes them a de facto template:

- A `znative.toml` manifest, so kind detection does not have to infer — present in all eight.
- A `README.md` documenting commands and environment variables.
- A committed `Cargo.lock`, with CI running `--locked`.
- A `.github/workflows/ci.yml` — byte-identical across all eight (Chapter 53) — running `fmt`, `clippy`, `doc`, `build`, and `test`.
- Unit tests in `src/lib.rs` under `#[cfg(test)]`, between two and seven of them.

Not one of them ships an integration test that loads the dylib, and none could without a running shell (Chapter 35). The 39 tests in the fleet are all layer one: pure logic, called directly, no host in sight.

The one thing worth adding that none of them has is the `catch_unwind` of Chapter 50 — four lines in each handler, converting the system's only undefined behaviour into a non-zero exit status. It is the cheapest available improvement to a plugin's safety and it is absent from all eight.

Chapter 66 – Thirty-Eight Ways to Fail

Chapter 25 gave the map: failure classes, what produces each one, and what to do about it. This chapter gives the census. Every place the package manager can construct an error is a literal in the source, and there are thirty-eight of them across four files. Counting them is worth doing because the count is small enough to be exhaustive, and because the shape of the distribution says something about where the design believes the risk lives.

The variants, and why the user never sees them

`PkgError` has five variants (`src/extensions/pkg/mod.rs:36–49`): `Io`, `Manifest`, `Resolve`, `Unknown`, and `Other`. Each carries a `String` and nothing else — no error code, no source chain, no structured payload.

The first thing to know about those five is that they are invisible at the terminal. The `Display` implementation collapses them:

```
impl std::fmt::Display for PkgError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            PkgError::Io(s)
            | PkgError::Manifest(s)
            | PkgError::Resolve(s)
            | PkgError::Unknown(s)
            | PkgError::Other(s) => write!(f, "{}", s),
        }
    }
}
```

`src/extensions/pkg/mod.rs:51–61`. One arm, five patterns, one behaviour: print the payload. The variant is a compile-time classification for whoever is reading the code; the user gets the sentence and nothing more. That is consistent with the tool's terse-error posture, and it means the taxonomy below is a map of the source rather than a map of anything you can observe from a prompt.

The distribution:

Variant	Sites	What it labels
<code>Resolve</code>	16	Anything between “you named a source” and “there is a loadable artifact” — clone, checkout, build, <code>dlopen</code> .
<code>Io</code>	11	A filesystem call that failed, with the path interpolated.
<code>Other</code>	9	Index parse/serialize, the not-installed cases, script-load failure, missing <code>\$HOME</code> .
<code>Manifest</code>	1	A <code>znative.toml</code> that is not valid TOML.
<code>Unknown</code>	1	A tree whose kind could not be determined.

By file: `commands.rs` 16, `resolver.rs` 11, `store.rs` 8, `manifest.rs` 3. The weighting is the design restated as arithmetic. Sixteen of thirty-eight errors are `Resolve`, and eleven of the thirty-eight sites live in the resolver, because fetching and building somebody else's code is where a package manager actually touches an uncontrolled world. The store — the part that owns your disk — has eight, and every one of them is a wrapped filesystem call. The manifest parser has three, because there is almost nothing in a `znative.toml` to get wrong.

The errors that lose their path

The `From<std::io::Error> impl` is three lines:

```
impl From<std::io::Error> for PkgError {
    fn from(e: std::io::Error) -> Self {
        PkgError::Io(e.to_string())
    }
}
```

`src/extensions/pkg/mod.rs:63–67`. That conversion is what makes a bare `?` on a filesystem call compile, and it is why some failures arrive without a path. Most of the store's I/O is wrapped deliberately — `store.rs:86` produces `create <dir>: <reason>`, `store.rs:184` produces `write <path>: <reason>` — but the sites that use a bare `?` do not. `std::fs::create_dir_all(&dst)?` at `store.rs:106`, inside `install_dir`, yields exactly what the OS said and no indication of which directory it was trying to create. The same applies to the `read_dir` and `metadata` calls inside `copy_dir_filtered` (`store.rs:213–222`) and to every filesystem call inside `store_integrity` (`mod.rs:83–99`).

This is worth knowing at a prompt, because it is the difference between `znative: create /home/u/.zshrs/pkg/store: Permission denied` and `znative: Permission denied (os error 13)`. Both are true; only one tells you where to look. When you get the second form, the operation was inside the copy or the hash, and the path in question is under `store/<name>@<version>/`.

The complete census

Every site, with the exact format string as it appears in the source.

Where	Variant	Text
<code>store.rs:38</code>	Other	<code>HOME environment variable not set</code>
<code>store.rs:86</code>	Io	<code>create {}: {}</code>
<code>store.rs:104</code>	Io	<code>clear {}: {}</code>
<code>store.rs:166</code>	Io	<code>read {}: {}</code>
<code>store.rs:168</code>	Other	<code>parse {}: {}</code>
<code>store.rs:179</code>	Other	<code>serialize installed.toml: {}</code>
<code>store.rs:184</code>	Io	<code>write {}: {}</code>
<code>store.rs:233</code>	Io	<code>copy {}: {}</code>
<code>manifest.rs:99</code>	Manifest	<code>znative.toml: {}</code>

Where	Variant	Text
manifest.rs:109	Io	read {}: {}
manifest.rs:168	Unknown	could not determine plugin kind: no znative.toml, no *.plugin.zsh, no functions/, and no Rust cdylib/Cargo.toml
resolver.rs:41	Resolve	path {}: {}
resolver.rs:43	Resolve	path {} is not a directory
resolver.rs:61	Io	clear {}: {}
resolver.rs:163	Resolve	unrecognized source '{}': expected owner/repo, github:owner/repo, git+URL, or a local path
resolver.rs:180	Resolve	git clone: {} (is git installed?)
resolver.rs:187	Resolve	git clone {} failed: {}
resolver.rs:207	Resolve	git clone: {}
resolver.rs:209	Resolve	git clone {} failed: {}
resolver.rs:220	Resolve	git checkout: {}
resolver.rs:222	Resolve	git checkout {} failed: {}
commands.rs:57	Resolve	{}: no cdylib after build
commands.rs:93	Other	{} is not installed
commands.rs:101	Io	remove {}: {}
commands.rs:173	Io	remove {}: {}
commands.rs:222	Io	remove {}: {}
commands.rs:237	Other	{} is not installed
commands.rs:290	Other	{} is not installed
commands.rs:318	Other	{} is not installed
commands.rs:368	Other	{}: unknown plugin kind '{}'
commands.rs:386	Resolve	{}: native plugin has no prebuilt cdylib and build is disabled
commands.rs:392	Resolve	{}: native plugin has neither a cdylib nor a Cargo.toml to build

Where	Variant	Text
<code>commands.rs:402</code>	Resolve	<code>cargo build: {} (is cargo installed?)</code>
<code>commands.rs:404</code>	Resolve	<code>{}: cargo build failed:\n{}</code>
<code>commands.rs:413</code>	Resolve	<code>{}: cargo build produced no cdylib in {} (need crate-type=["cdylib"])</code>
<code>commands.rs:420</code>	Io	<code>stage cdylib: {}</code>

Two entries in that list carry an entire diagnostic strategy. `git clone {} failed: {}` and `{}: cargo build failed:\n{}` both interpolate the other tool's `stderr` verbatim (`resolver.rs:190`, `commands.rs:407`), which is Chapter 25's “do not paraphrase another tool” rule visible as a `String::from_utf8_lossy` call. The `cargo` one embeds a newline, so a build failure produces a multi-line `znative:` message whose second line onward is cargo's own output.

One sentence, four causes

`{}` `is not installed` appears at four distinct sites, and they are not the same condition:

- `commands.rs:93` — `remove NAME` where the index has no row named `NAME`.
- `commands.rs:237` — `info NAME`, same condition.
- `commands.rs:290` — `load SPEC` where `SPEC` matched no index row, matched no recorded source label, and was not recognised as a source form at all. This is the one that surprises people, because the argument may have been a perfectly well-formed source spec that simply could not be resolved to a label — a `path:DIR` whose `DIR` does not exist takes this branch, since `source_label` canonicalises and returns `None` on failure (`resolver.rs:90–92`).
- `commands.rs:318` — `update NAME` where the index has no such row.

So the sentence `znative: /tmp/gone is not installed` can mean “there is no plugin by that name” or it can mean “that path does not exist.” Chapter 67 works through why the second case lands here.

Where the prefix and the exit code come from

None of the thirty-eight strings contains the word `znative`. The prefix is added once, at the outer edge:

```
match result {
    Ok(()) => 0,
    Err(e) => {
        eprintln!("znative: {}", e);
        1
    }
}
```

`src/extensions/pkg/builtin.rs:92–98`. Every failure is one line on `stderr` and exit status 1. There is no status 2 for usage errors, no distinct code per variant, and no `--verbose` that would restore the classification. The `usage_err` helper is the only other error writer, and it prints the message *and* the whole `USAGE` block to `stderr` before returning 1 (`builtin.rs:102–106`); it is reached from four places — `add` with no argument, `remove` with no argument, `info` with no argument, and an unrecognised subcommand (`builtin.rs:46, :58, :63, :89`).

The help paths are the only ones that write to stdout and succeed: `help`, `-h`, `--help`, and the *empty* subcommand all print `USAGE` and return 0 (`builtin.rs:85–88`). A bare `znative` with no arguments is therefore not an error.

The multi-argument loops keep only the last failure

Three subcommands accept several arguments, and all three are written the same way. `add`:

```
let mut last = Ok(());
for spec in rest {
    if let Err(e) = commands::add(spec) {
        last = Err(e);
    }
}
```

`builtin.rs:37–42`; `remove` is `builtin.rs:50–55` and `load` is `builtin.rs:73–78`. The variable is only ever assigned inside the failure arm, so a success after a failure does not clear it — the exit status is 1 if *any* argument failed, which is the right contract. What is lost is the intermediate diagnostics: with `znative add a b c` where both `a` and `c` fail, the only message printed is `c`'s, because `a`'s `PkgError` was overwritten before anything formatted it. The failure of `a` is real, is reflected in the exit status, and is silent.

The practical consequence is one line of advice: when a multi-argument invocation exits non-zero with a message that names only one of the arguments, the others are not thereby known to have succeeded. Re-run them individually to find out.

The six errors that come from outside the manager

Loading is not the package manager's code. `load_entry`'s native arm calls into the plugin host and re-labels whatever comes back:

```
crate::plugin_host::load(&lib.to_string_lossy())
    .map(|_| ())
    .map_err(PkgError::Resolve)
```

`commands.rs:343–345`. So six further strings can appear behind the `znative:` prefix without existing anywhere in `src/extensions/pkg/`:

Where	Text
<code>plugin_host.rs:378</code>	<code>cannot load `{}`: {}</code>
<code>plugin_host.rs:383</code>	<code>`{}`: not a zshrs plugin (no znative_init)</code>
<code>plugin_host.rs:402</code>	<code>`{}`: plugin init failed (ABI mismatch or error)</code>
<code>plugin_host.rs:412</code>	<code>`{}`: ABI version {} != host {}</code>
<code>plugin_host.rs:426</code>	<code>plugin `{}` already loaded</code>
<code>plugin_host.rs:517</code>	<code>plugin `{}` not loaded</code>

The last two are the ones you meet without doing anything unusual, and Chapter 72 shows how an ordinary `znative update` reaches the “already loaded” refusal. The `not a zshrs plugin` message reconstructs

the symbol name from `INIT_SYMBOL` by trimming its trailing NUL (`plugin_host.rs:385–387`) rather than hard-coding the string a second time — a small thing, and the reason the message can never drift from the symbol the host actually looks for.

The script arm re-labels the other way, into `Other` (`commands.rs:364–366`), so a plugin whose `.plugin.zsh` fails to execute produces whatever the shell executor returned, prefixed with `znative:`. Chapter 25's advice for that case — source the store copy by hand and take `znative` out of the picture — is the right one precisely because the manager is contributing nothing to the message but the prefix.

Chapter 67 – The Resolver, Read Literally

Chapter 5 described the source forms `znative` `add` accepts and what each one means. This chapter reads the code that implements them, because the resolver is 280 lines of string handling and process spawning, and string handling has edges. Every rule below is a line in `src/extensions/pkg/resolver.rs`, and the three unit tests that pin the trickiest of them pass:

```
test pkg::resolver::tests::local_path_forms ... ok
test pkg::resolver::tests::split_ref_only_after_last_slash ... ok
test pkg::resolver::tests::git_url_forms ... ok
```

The @ rule

A source spec may carry a trailing `@REF`, and the resolver has to split it off without breaking an SSH URL, which contains an `@` of its own. The rule:

```
fn split_ref(spec: &str) -> (&str, Option<&str>) {
    if let Some(at) = spec.rfind('@') {
        let after_slash = spec.rfind('/').map(|s| at > s).unwrap_or(true);
        if after_slash && at + 1 < spec.len() {
            return (&spec[..at], Some(&spec[at + 1..]));
        }
    }
    (spec, None)
}
```

`resolver.rs:101–111`. Three conditions, each doing work. It splits on the *last* `@`, so a ref containing an `@` cannot confuse it. It splits only when that `@` falls after the last `/`, which is what saves `git@github.com:o/r.git` — the `@` there precedes the final `/`, so nothing is split. And it requires at least one character after the `@`, so a trailing `owner/repo@` is a name, not an empty pin.

The consequences are exact and occasionally surprising. `owner/repo@v1` splits as expected. `git@github.com:o/r.git` does not split — pinned by the test at `resolver.rs:257–260`. But `git@host:repo` — an SSH URL with no `/` at all — *does* split, because `rfind('/')` returns `None` and the `unwrap_or(true)` treats the absent slash as “the `@` is after it.” Everything after the `@` is taken as the ref, and what remains, `git`, matches no source form and produces `unrecognized source 'git'`. The SSH shapes that survive the split are the ones with a path separator in them, which is the common form.

The split happens before anything else, including the local-path check (`resolver.rs:35`), so it applies to paths too: `path:/opt/plugins/tool@2` is split into a directory `path:/opt/plugins/tool` and a ref `2`, and the ref is then discarded because the local-path branch never looks at it (`resolver.rs:38–51`). A directory whose name ends in `@something` is not addressable by `path:.`

What counts as a local path

```
fn local_path(spec: &str) -> Option<PathBuf> {
    if let Some(rest) = spec.strip_prefix("path:") {
        return Some(PathBuf::from(rest));
    }
    if spec.starts_with('/')
        || spec.starts_with("./")
        || spec.starts_with("../")
        || spec.starts_with('~')
    { ... }
    None
}
```

`resolver.rs:114–135`. Five recognised shapes: an explicit `path:` prefix, an absolute path, `./`, `../`, and anything beginning with `~`.

The tilde branch is the one to read closely. The test at the top of the function accepts *any* leading `~`, but the expansion inside only handles `~/`:

```
let expanded = if let Some(rest) = spec.strip_prefix("~/") {
    if let Some(home) = std::env::var_os("HOME") {
        PathBuf::from(home).join(rest)
    } else {
        PathBuf::from(spec)
    }
} else {
    PathBuf::from(spec)
};
```

`resolver.rs:123–131`. So `~/plugins/foo` becomes an absolute path under `$HOME`, and `~otheruser/plugins/foo` — zsh’s named-directory syntax — becomes the literal relative path `~otheruser/plugins/foo`, which will not canonicalise. This is not a bug in the sense of a mistake; the shell expands `~user` before the builtin ever sees the word, in every context where filename expansion applies. It matters only when the tilde reaches the resolver unexpanded, which happens when it is quoted. `znative add '~user/tree'` fails with `path ~user/tree: No such file or directory` (`resolver.rs:41`); `znative add ~user/tree` unquoted does not reach that code at all, because zsh substituted the home directory first.

Note also what is *not* a local path: a bare relative name. `plugins/foo` has no recognised prefix, falls through to `git_url`, and — having exactly two slash-separated segments — is treated as the GitHub shorthand `plugins/foo`. A relative directory must be written `./plugins/foo` or `path:plugins/foo` to be seen as one.

The four git forms, and the two-segment rule

```
fn git_url(spec: &str) -> PkgResult<(String, String, String)> {
```

`resolver.rs:138`. Four branches, tried in order:

Test	Clone URL	Recorded label
starts with <code>git+</code>	the remainder	<code>git+<remainder></code>
starts with <code>github:</code>	<code>https://github.com/<rest></code>	<code>github:<rest></code>

Test	Clone URL	Recorded label
ends with <code>.git</code> , or contains <code>://</code>	the spec verbatim	<code>git+<spec></code>
exactly two <code>/</code> -separated segments, no space	<code>https://github.com/<spec></code>	<code>github:<spec></code>

The `github:` and shorthand branches both strip a trailing `.git` from the label as well as the URL (`resolver.rs:144, :148, :158`), so `github:o/r.git` and `o/r` record the identical label `github:o/r` and are the same installed plugin.

The fourth branch is the strict one:

```
if spec.split('/').count() == 2 && !spec.contains(' ') {
```

`resolver.rs:157`. Exactly two segments. `owner/repo/subdir` has three and does not match, so it falls through to the `unrecognized source` error at `resolver.rs:163` rather than being interpreted as a monorepo path. That is the mechanism behind Chapter 25’s observation that a plugin living in a subdirectory of a repository needs a different approach: there is no source form that names a subtree, and the resolver’s error message lists the four forms that exist rather than guessing.

A trailing slash also breaks the count: `owner/repo/` splits into three segments, the last empty. It does not match, and it never reaches `repo_basename`’s trailing-slash trimming (`resolver.rs:241`), because that function is only called after a branch has already matched.

Where the clone lands, and what it overwrites

```
let dir = store.git_dir().join(&name);
if dir.exists() {
    std::fs::remove_dir_all(&dir)
        .map_err(|e| PkgError::Io(format!("clear {}: {}", dir.display(), e)))?;
}
git_clone(&url, &dir, git_ref)?;
```

`resolver.rs:58–63`. The clone directory is `pkg/git/<name>`, where `<name>` is `repo_basename(url)` — the last `/`- or `:`-separated segment with any `.git` suffix trimmed (`resolver.rs:239–246`). The owner is not part of it.

That is a flat namespace over repository basenames, and it collides. `alice/tools` and `bob/tools` both resolve to `pkg/git/tools`, and the second install unconditionally deletes the first’s clone before starting its own. In the ordinary sequential case this is harmless — the clone is scratch, the store copy is what gets installed, and `add` deletes the clone afterwards anyway (`commands.rs:71–73`). It is worth knowing about for the concurrent case that Chapter 33 examines: two shells installing *different* repositories that happen to share a basename are writing the same scratch directory, which is not the benign “many clones of the same repository” that chapter describes. The store keys are still distinct if the plugin names differ, so the outcome is a failed or truncated clone rather than a wrong install, but it is the one place where concurrent installs of unrelated plugins touch the same path.

Shallow first, full clone as the fallback

```
let mut cmd = Command::new("git");
cmd.arg("clone").arg("--depth").arg("1");
if let Some(r) = git_ref {
    cmd.arg("--branch").arg(r);
}
```

[resolver.rs:172–176](#). Every clone starts shallow. When a ref was given and the shallow clone fails for *any* reason, the resolver does not report the failure — it retries with a full clone and an explicit checkout:

```
if !out.status.success() {
    if git_ref.is_some() {
        return git_clone_checkout(url, dir, git_ref.unwrap());
    }
}
```

[resolver.rs:181–186](#). The intent is stated in the comment above it: a commit SHA cannot be used with `--branch` on a shallow clone. The implementation is broader than the intent, because the retry is triggered by a non-zero exit and not by inspecting why. A pinned install against an unreachable host therefore attempts the clone twice — once shallow, once full — before reporting `git clone <url> failed:` from the second attempt ([resolver.rs:209](#)). The message you see is the fallback's, not the original's.

`git_clone_checkout` removes the directory before starting ([resolver.rs:199–201](#)) so the failed shallow attempt's debris does not confuse the full clone, and it reports checkout failure separately ([resolver.rs:222](#)) so a reachable repository with an unreachable ref is distinguishable from an unreachable repository.

The label, and the read that needs no network

What gets recorded is not the spec but a normalised label, with the pin reattached:

```
fn label_with_ref(label: String, git_ref: Option<Gstr>) -> String {
    match git_ref {
        Some(r) => format!("{}", r, label, r),
        None => label,
    }
}
```

[resolver.rs:75–80](#). So `o/r@v1.2.0` records `github:o/r@v1.2.0`, and feeding that label back through `split_ref` and `git_url` reproduces the same clone — which is exactly what `update` does (Chapter 72).

The label is also computable without touching the network, which is what makes the `.zshrc` idiom cheap:

```
pub fn source_label(spec: Gstr) -> Option<String> {
    let (base, git_ref) = split_ref(spec);
    if let Some(p) = local_path(base) {
        let dir = p.canonicalize().ok()?;
        return Some(format!("path+file://{dir}", dir.display()));
    }
    git_url(base)
        .ok()
        .map(|(_url, label, _name)| label_with_ref(label, git_ref))
}
```

[resolver.rs:87–97](#). `znative load owner/repo` computes `github:owner/repo` from the string alone, looks for a row with that source, and takes the fast path if it finds one ([commands.rs:279–282](#)). The comment above the function explains why the index is searched by *source* and not by name: a repository called `zshrs-forgit` installs a plugin whose `znative.toml` names it `forgit`, so the name in the index is not the name in the `.zshrc` line.

The local branch is the asymmetric one. It calls `canonicalize`, which touches the filesystem and fails when the directory does not exist — and `.ok()?` turns that failure into `None`, meaning “not a source form.” For `load` that is the difference between two very different messages. A `path:` source that exists and is not installed gets installed on first use (`commands.rs:287`). A `path:` source that does not exist produces `znative: path:/gone is not installed` (`commands.rs:290`), which names the wrong problem: the plugin is not installed *because the directory is not there*, and the message reports only the first half. `znative add path:/gone`, by contrast, goes through `resolve` and produces `znative: path /gone: No such file or directory (os error 2)` — the accurate one. When a `load` line for a local plugin reports “is not installed” and you know you installed it, check that the directory still exists before anything else.

What survives an install

One line decides whether the resolver’s working directory outlives the install:

```
if staged.source.starts_with("github:") || staged.source.starts_with("git+") {  
    let _ = std::fs::remove_dir_all(&staged.dir);  
}
```

`commands.rs:71–73`. Clones are deleted; local sources are not. The guard is on the *label*, so it is structurally impossible for the branch to fire on a `path+file://` source — which is the property that keeps `znative add path:.` from deleting the tree you are developing in. The deletion is also `let _ =`: a clone that cannot be removed does not fail the install, it just stays until `gc` or `clean` reclaims it (`commands.rs:183–201, :215–230`).

Chapter 68 – What the Integrity Hash Covers

Chapter 29 placed the `sha256-` field correctly in the trust model: it is tamper-detection over your own store, not a signature, and it makes no claim about provenance. That is the right framing and it leaves two questions unanswered, both of which have exact answers in twenty lines of code. What bytes go into the digest? And when is it ever compared against anything?

The algorithm, stated as a grammar

```
pub fn store_integrity(root: &std::path::Path) -> PkgResult<String> {
```

`src/extensions/pkg/mod.rs:74`. The function walks the tree, sorts, and feeds a `Sha256`. The walk collects paths, not content:

```
    if meta.is_dir() && !meta.file_type().is_symlink() {
        walk(root, &path, out)?;
    } else {
        out.push(path.strip_prefix(root).unwrap_or(&path).to_path_buf());
    }
}
```

`mod.rs:87–91`. A real directory is recursed into and contributes no entry of its own. Everything else — regular files, and symlinks including symlinks to directories — is pushed as a root-relative path. Then `entries.sort()` (`mod.rs:96`), which is what makes the digest independent of `read_dir` order and therefore reproducible across filesystems.

Each entry contributes one of two records:

```
file:      <relative-path> 0x00 'F' 0x00 <decimal-length> '\n' <contents> '\n'
symlink:   <relative-path> 0x00 'L' 0x00 <link-target>      '\n'
```

`mod.rs:101–115`. The length prefix on the file record is what makes the encoding unambiguous — without it, a file named `a` containing `b` and a file named `ab` containing nothing would be indistinguishable once concatenated. The type byte distinguishes the two record kinds. The output is `format!("sha256-{:x}", ...)` (`mod.rs:117`), lowercase hex with a fixed prefix, which is what lands in the index's `integrity` field.

What is not in the digest

The list is short and each omission is load-bearing:

Not hashed	Consequence
File modes	A <code>chmod +x</code> or <code>chmod -w</code> on a stored file leaves the hash unchanged.
Ownership	Same.

Not hashed	Consequence
Timestamps	A <code>touch</code> on a stored file leaves the hash unchanged — which is what makes the hash reproducible in the first place.
Empty directories	A directory containing nothing contributes nothing to the digest; adding or deleting one is invisible.
Anything under <code>.git/</code> or <code>target/</code>	Excluded from the store copy before the hash ever runs (<code>store.rs:217–219</code>).

The mode omission is the one with a practical edge. A script plugin that ships an executable helper — the pattern Chapter 17 documents for self-reentrant fzf tools — carries that helper’s executable bit through `std::fs::copy`, which is documented to copy the source file’s permission bits, but the bit is not part of the recorded hash. If something later clears it, the plugin breaks and the integrity field still matches. The hash answers “are these the bytes” and not “is this tree still usable.”

The exclusion of `.git/` has a different consequence worth stating plainly: the recorded hash is a hash of the *store copy*, not of the upstream repository. Two machines that installed the same tagged commit will agree, because they copied the same files and the exclusion is deterministic. But there is no relationship between this number and any hash git computes, and no way to check a store copy against a remote without re-cloning.

The symlink branch cannot fire

The `0x00 'L' 0x00` record exists in `store_integrity`, and for a tree produced by the installer it is unreachable. The reason is in `copy_dir_filtered`:

```

    } else if ft.is_symlink() {
        // Copy the resolved file content (plugins rarely ship symlinks; a
        // dangling link in the store would break loads).
        if let Ok(target) = std::fs::read(&from) {
            std::fs::write(&to, target)?;
        }
    }
}

```

`store.rs:225–230`. `std::fs::read` follows the link. What lands in the store is a regular file containing the linked-to file’s bytes, so by the time `store_integrity` runs over `store/<name>@<version>/` — which is exactly what `add` does, at `commands.rs:42–43`, immediately after `install_dir` returns — there are no symlinks left for the `L` branch to encode. `store_integrity` is a general function that is also called on nothing but installer output, and the symlink half of it is dead code in that configuration.

The flattening has an observable consequence beyond the hash. A plugin that ships a symlink to a *directory* loses it silently: `std::fs::read` on a directory returns an error, the `if let Ok` swallows it, and the entry is simply absent from the store copy. No error is raised, the install succeeds, the hash is computed over a tree that is missing something, and the plugin fails later — at `source` time with a file-not-found, or at completion time with an `fpath` directory that is not there. This is the one place in the install pipeline where content is dropped without a diagnostic.

A plugin that ships symlinks to *files* fares better but not identically: the store copy has content where the source had links, so a plugin that relies on two paths being the same inode does not get that, and a self-referential or circular link becomes an error from `std::fs::read` and is likewise dropped.

Computed once, compared never

The most important fact about the integrity hash is a grep result:

```
src/extensions/pkg/commands.rs:43: let integrity =
  ~ super::store_integrity(&store_path)?;
src/extensions/pkg/mod.rs:74:pub fn store_integrity(root: &std::path::Path) ->
  ~ PkgResult<String> {
```

One definition, one call site. The hash is computed at install time, written into the index record (`commands.rs:50`), printed by `info` when non-empty (`commands.rs:247–249`), and never recomputed by any code path in the manager. `load` does not check it (`commands.rs:338–373` reads `p.lib`, `p.fpath`, and `p.source_files` and nothing else). `list` does not check it. `gc` does not check it. There is no `znative verify`.

That is a deliberate shape and it should be read as one. Verifying on `load` would mean hashing every installed plugin's entire tree at every shell start, which is the kind of startup cost this shell's whole design exists to eliminate; the field is an audit record, not a runtime gate. But it means the detection Chapter 29 describes is *available* rather than *automatic*. Nothing tells you a store copy changed. What the field gives you is the ability to find out, by hand, when you have a reason to ask — and one half of the comparison is one command:

```
# what the index recorded at install time
znative info forget | grep integrity
```

There is no built-in way to recompute the current value for comparison, because `store_integrity` is not exposed through the builtin's surface. The honest summary is that the index carries a fingerprint that a future verb could check, that no verb checks today, and that its value right now is comparison *across machines* — two stores whose `installed.toml` rows carry the same `sha256-` for the same `name@version` hold identical bytes.

Chapter 69 – Nine Locks in the Host, None in the Manager

Chapter 33 examined what happens when several shells run `znative` at once and concluded, correctly, that the shared resource is `installed.toml` and that the write is a read-modify-write with last-writer-wins semantics. This chapter asks the narrower mechanical question that chapter left implicit: what synchronisation exists, where, and what is it protecting? The answer is asymmetric enough to be worth its own chapter — the package manager has none at all, and the plugin host, one module away, has nine mutexes.

The manager holds nothing

`src/extensions/pkg/` contains no `Mutex`, no `RwLock`, no atomic, and no file lock. The only `Lock` token in the whole directory is a `LazyLock` around the builtin table (`builtin.rs:128`), which is initialisation, not exclusion. The word “lockfile” appears three times and every occurrence is a comment explaining that there is no lockfile, in the dependency-resolution sense, because the model is global (`mod.rs:6`, `mod.rs:70`, `commands.rs:146`).

The index write is a plain overwrite:

```
let body = toml::to_string_pretty(&self)
    .map_err(|e| PkgError::Other(format!("serialize {}: {}", INSTALLED_FILE,
        e)))?;
std::fs::write(
    &path,
    format!("# znative - auto-generated. Do not edit.\n{}", body),
)
```

`store.rs:178–184`. No advisory lock, no temporary file, no `fsync`, no atomic rename. `std::fs::write` truncates the target and writes the new content into it.

What the shell does one directory away

The comparison that makes this stark is `script_cache`, the rkyv bytecode shard that lives under the same `$ZSHRS_HOME`. Its documented write protocol is four steps (`src/extensions/script_cache.rs:31–34`):

```
//! - `bin_zsystem_flock(LOCK_EX)` on `scripts.rkyv.lock` so concurrent writers
    ~ serialize.
//! - Read existing shard into owned form, mutate, `rkyv::to_bytes`,
//!   write to `scripts.rkyv.tmp.<pid>.<nanos>`, fsync, atomic-rename.
//! - Drop the in-process `mmap` so the next read picks up the new shard.
```

And the code does all of it: the lock is acquired at `script_cache.rs:227`, `:304`, and `:326`, one per mutating entry point; the temporary name carries the pid and a nanosecond stamp (`script_cache.rs:379`); the file is `sync_all`'d before the rename (`script_cache.rs:390`); the rename is the durable commit (`script_cache.rs:393`).

Two files under one directory tree, written by the same process, with entirely different durability disciplines. Setting the two side by side:

	<code>scripts.rkyv</code>	<code>installed.toml</code>
Exclusion between processes	<code>flock(LOCK_EX)</code> on a sidecar lock file	none
Write	temp file, <code>fsync</code> , atomic rename	<code>fs::write</code> in place
Torn-write window	none	between truncate and completion
Recovery from a torn write	previous shard intact	TOML parse error on next read

The last row is the one that matters. A `znative` process killed mid-write leaves `installed.toml` truncated, and the next read fails at `store.rs:167–168` with `parse <path>: <toml message>`, which is a hard error out of `InstalledIndex::load_from` and therefore out of every subcommand — `list`, `info`, `load`, all of them. Chapter 26’s guarantee that an interrupted *install* cannot leave a live lie is about the ordering of the store copy relative to the record, and it holds; it does not extend to an interruption inside the record write itself, because that write is not atomic. The window is small — the file is a few hundred bytes for a realistic plugin set — and it is not zero.

The host’s nine

`src/extensions/plugin_host.rs` is the opposite. Nine `OnceLock<Mutex<...>>` statics, each with an accessor function:

Accessor	Line	Protects
<code>plugins</code>	60	the list of loaded plugins, each owning its <code>libloading::Library</code>
<code>registry</code>	66	command name → handler
<code>staging</code>	75	builtins registered during one <code>init</code> call
<code>load_lock</code>	81	nothing — it serialises <code>load/unload</code>
<code>ownership</code>	88	command name → owning plugin
<code>compfn_registry</code>	97	<code>compsys_fn</code> overrides
<code>compfn_staging</code>	104	<code>compfn</code> overrides registered during one <code>init</code>
<code>pending_completions</code>	116	completion wirings not yet installed into <code>compsys</code>
<code>installed_completions</code>	123	completion wirings already installed

The reason the host needs them and the manager does not is that the host’s state is in-process and shared between threads, while the manager’s state is on disk and shared between processes. Different problem, different tool — and the shell has a tool for the second problem, in `zsystem flock`, which the manager does not use.

Why `load_lock` exists

`load_lock` guards no data. It is taken at the top of `load` (`plugin_host.rs:371`) and at the top of `unload` (`:513`), and its job is to make the staging protocol single-writer. That protocol is a sequence, not a datum:

```
staging().lock().unwrap().clear();
compfn_staging().lock().unwrap().clear();
let pc_start = pending_completions().lock().unwrap().len();
let info_ptr: *const PluginInfo = init(host_api());
```

`plugin_host.rs:394–397`. Clear the staging buffers, remember how many pending completions exist, then call the plugin's `znative_init` — which calls back into `host_register_builtin` (`:133`), `host_register_compfn` (`:283`), and `host_register_completion` (`:217`), each of which pushes into one of those buffers. When `init` returns, everything in staging belongs to this plugin, and everything appended to `pending_completions` past `pc_start` does too (`:450–455`). Two concurrent loads would interleave their registrations into one buffer with no way to tell them apart, which is why the whole sequence is inside one critical section rather than each buffer having its own.

The buffers still have individual mutexes because they are reachable from the `extern "C"` callbacks, which the host cannot assume anything about.

The lock dispatch does not hold

Command dispatch is the hot path — every bare command name the VM does not recognise as a builtin passes through it — and it is written to hold a lock for as short a time as possible:

```
pub fn dispatch(cmd: &str, args: &[String]) -> Option<i32> {
    let entry = { registry().lock().unwrap().get(cmd).copied() }?;
    ...
    let rc = (entry.func)(host_api(), ptrs.len(), ptrs.as_ptr());
```

`plugin_host.rs:601–616`. The block on line 602 exists to bound the borrow: look the name up, copy the `BuiltinEntry` out — it is `Clone`, `Copy` for exactly this reason (`:52–53`) — and drop the guard. Everything after that, including the `CString` construction and the call into plugin code, runs with no lock held.

This is the right shape for the common case, and it has a stated consequence. `unload` is careful about ordering — it purges the registry and the compfn registry *before* dropping the `Library`, because dropping it is a `dnclose` and a registered function pointer into an unmapped library is a dangling pointer (`:510–511`, `:520–563`). That ordering makes unload safe against *future* dispatches: once the purge is done, no lookup can find the handler. It does not make it safe against a dispatch already in flight, because `dispatch` holds no lock across the call and `unload`'s `load_lock` does not exclude it. A thread executing a plugin's handler while another thread unloads that plugin is a sequence the code permits.

Whether that sequence is reachable in practice depends on the shell's threading, and this book will not assert what it has not measured. What can be stated from the source is the shape of the invariant: unload is ordered correctly with respect to *resolution*, and is not synchronised with respect to *execution*. The single-threaded interactive case — where the `zmodload -uR` that unloads a plugin is itself a command, and therefore cannot be running while one of that plugin's handlers is running — is never affected.

What this means for the manager

Two lines of practical advice fall out.

`znative remove NAME` calls `plugin_host::unload(name)` for a native plugin and discards the result — `let _ =`, at `commands.rs:95–97`, described in the function's own doc comment as “unload (best-effort)” (`commands.rs:88`). A shell that never loaded the plugin gets `plugin `NAME` not loaded` back from that call and proceeds anyway. The index row and the store directory are removed regardless of whether the library

was mapped, which is what makes `remove` behave identically in a shell that had the plugin loaded and one that did not.

And the index's lack of a lock is worth one habit: do not run `znative add` or `znative update` in several shells at the same moment on purpose. Chapter 33 established that the *outcome* is a lost row rather than corruption, which is recoverable; this chapter adds that the *write* is not atomic, so the file itself is at risk in a way the rkyv shards next to it are not.

Chapter 10 – The Store and the Byte-code Archives

`znative`'s store is not the only thing `zshrs` keeps under `$ZSHRS_HOME`. It sits beside three other persistent caches, all serving the same shell, all keyed by different things, all with different invalidation rules. Understanding where the store fits among them explains two things that surprise people: why rebuilding `zshrs` does not disturb an installed plugin, and why a script plugin's code is compiled at every shell start no matter how many caches the shell has.

Four artifacts, one root

Every one of them resolves its location the same way — `$ZSHRS_HOME` if set, otherwise `~/.zshrs`:

Artifact	Path	Format	Resolver
Script bytecode	<code>scripts.rkyv</code>	rkyv, mmap'd	<code>script_cache.rs:3</code>
Autoload bytecode	<code>autoloads.rkyv</code>	rkyv, mmap'd	<code>autoload_cache.rs:424–432</code>
Plugin/compsys metadata	<code>plugins.db</code>	SQLite	<code>plugin_cache.rs:1042–1048</code>
The <code>znative</code> store	<code>pkg/</code>	directories + TOML	<code>store.rs:33–44</code>

The `znative` resolver is the odd one out in a small way worth noting: it reads `ZSHRS_HOME` via `std::env::var_os` and falls back to `$HOME` from the `environment` (`store.rs:34–40`), erroring with `HOME environment variable not set` if that is absent, while the cache modules fall back through `dirs::home_dir()` (`autoload_cache.rs:426–430`) and never error. Under any normal shell the two agree.

Two invalidation philosophies

The rkyv shards are aggressively self-invalidating. Both carry a header, and both validate four fields before trusting a single cached byte:

```
fn header_ok(&self) -> bool {
    let h = &self.shard().header;
    let magic: u32 = h.magic.into();
    let fv: u32 = h.format_version.into();
    let pw: u32 = h.pointer_width.into();
    magic == SHARD_MAGIC
    && fv == SHARD_FORMAT_VERSION
    && pw as usize == std::mem::size_of::<usize>()
    && h.zshrs_version.as_str() == env!("CARGO_PKG_VERSION")
}
```

`autoload_cache.rs:115–124`. The magic is `ZRAL` — `0x5A52414C` (`autoload_cache.rs:48–49`); `scripts.rkyv` uses `ZR5C`, `0x5A525343` (`script_cache.rs:54–55`); both are at `SHARD_FORMAT_VERSION = 1` (`autoload_cache.rs:51`, `script_cache.rs:57`). The pointer-width check refuses a shard written by a build with a different `usize`. And the version check compares against the compiled-in `CARGO_PKG_VERSION` — `0.12.28` at the workspace root as this is written — so upgrading zshrs by a single patch release makes every cached chunk in both shards unreadable at once, silently, with no migration and no error. `script_cache` goes further and invalidates per entry on the running binary's own mtime (`script_cache.rs:29–30`), so even an unchanged version number does not let a stale chunk survive a rebuild.

The `znative` index does none of this. Its entire versioning is one field:

```
pub fn new() -> InstalledIndex {
    InstalledIndex {
        version: 1,
        packages: Vec::new(),
    }
}
```

`store.rs:150–157` — a schema version, declared at `store.rs:117–118`, that nothing currently reads back for a compatibility decision. There is no magic, no pointer width, no zshrs version, and no binary mtime. The file is TOML with a banner comment (`store.rs:182`) and it is meant to survive.

The asymmetry is not an oversight; it follows from what each artifact *is*. The rkyv shards hold compiled bytecode for a specific VM built into a specific binary, so a different binary's bytecode is not merely stale, it is wrong. The index holds provenance — names, versions, source labels, hashes, file lists — none of which changes meaning when the shell is rebuilt. Rebuilding zshrs throws away every cached chunk in the shell and touches nothing in `pkg/`.

There is a matching asymmetry in the off switch. `ZSHRS_CACHE=0` (or `false`, or `no`) disables both rkyv shards (`autoload_cache.rs:438–450`), and an in-process `CACHE_DISABLED` flag disables them without the environment variable. Neither has any effect on `znative`; there is no environment variable in `src/extensions/pkg/` that turns the store off, because the store is not an optimisation you can decline.

What a rebuild does to an installed native plugin

Put the two facts together. You rebuild zshrs. Both rkyv shards are invalidated wholesale. `installed.toml` is untouched, `store/<name>@<version>/` is untouched, and the `lib*.dylib` sitting in it — built against whatever `znative` crate version was current when you installed it — is still exactly where the index says it is.

The only thing standing between that library and the new shell is the ABI check:

```
if info.abi_version != ABI_VERSION {
    ...
    return Err(format!(
        "{}: ABI version {} != host {}",
        path, info.abi_version, ABI_VERSION
    ));
}
```

`plugin_host.rs:408–416`, against `ABI_VERSION = 4` in `znative/src/lib.rs:73`. That is the entire gate. If the rebuild did not change `ABI_VERSION`, the old library loads into the new shell and works — which is the guarantee Chapter 15 makes and the reason the ABI number is separate from the crate version. If the rebuild did change it, the load is refused with a message that names both numbers, and the fix is `znative update`, which rebuilds from source.

The index carries no record of which ABI version a stored library was built against — the `InstalledPlugin` fields are name, version, source, kind, integrity, lib, source_files, and fpath (`store.rs:126–148`), and none of them is an ABI number. So the manager cannot tell you in advance that a stored plugin will be refused; you

find out at load. Chapter 40 works through what that day looks like.

The path a script plugin's code actually takes

The other half of the relationship is less comfortable. A script plugin is loaded by synthesising shell code and running it:

```
for f in &p.source_files {
    let path = dir.join(f);
    code.push_str(&format!("{}", shquote(&path.to_string_lossy())));
}
```

`commands.rs:357–360`. So the store path ends up as the argument to the `source` builtin, and the question is whether that path engages the script bytecode cache. It does not, and the chain is short enough to walk.

`source` is `bin_dot` (`src/ported/builtin.rs:11806`), which resolves the file and delegates to the port of zsh's `source()` at `src/ported/init.rs:1563`. That function reads the file and hands the text to the pipeline:

```
let contents = match zwc_prog {
    Some(prog) => Ok(crate::ported::text::getpermtxt(Box::new(prog), None, 0)),
    None => std::fs::read_to_string(path),
};
```

`init.rs:1629–1632`, and then `execute_script_zsh_pipeline(&body)` at `init.rs:1656`. There is no `script_cache` call anywhere in that path.

The cache's read entry point, `script_cache::try_load_bytes`, has exactly one caller in the tree — `ShellExecutor::execute_script_file` at `vm_helper.rs:2390` — and `execute_script_file` itself has two: `bins/zshrs.rs:2156`, which is the `zshrs <script>` invocation, and `dap.rs:451`, the debug adapter. Sourcing is not among them. Neither, for the same reason, is a `.zshrc`: rc files are loaded through `sourcehome`, which calls the same `source()` (`init.rs:1674–1697`, called from `init.rs:1507` for `.zshrc`).

Stated plainly: **the shell's bytecode cache accelerates scripts it is asked to run, not files it is asked to source**. A script plugin installed by `znative` is lexed, parsed, and compiled at every shell start, exactly as a plugin sourced by hand from any other manager would be. The store gave that plugin a stable path, a content hash, and a version — it did not give it a cache entry.

The autoload shard is the one that does reach the plugins, and it reaches them by the front door rather than through `znative`. A script plugin that ships a `functions/` directory gets that directory prepended to `$fpath` (`commands.rs:350–356`), and from that moment its function files are ordinary autoloads, indexed by function name (`autoload_cache.rs:3–5`) like any other entry in `$fpath`. The store contributed the path; the caching that follows knows nothing about `znative` and does not need to.

What `plugins.db` is not

`plugins.db` is easy to mistake for `znative`'s database. It is not related to it at all. It is a SQLite file tracking source files by mtime for `compinit` and `compaudit` (`plugin_cache.rs:283, :662`), and its notion of a “plugin” is the directory-layout kind — its classifier recognises `zinit`'s `~/zinit` and `$XDG_DATA_HOME/zinit` layouts and oh-my-zsh's `plugins/`, `custom/plugins/`, `themes/`, and `custom/themes/` trees, among others (`plugin_cache.rs:817–853`). It is the cache for the plugin world `zshrs` inherited. `pkg/installed.toml` is the index for the plugin world `znative` introduced. They share a parent directory and nothing else.

Chapter 71 – One Store, Two Suffix Rules

Chapter 24 set out the cross-architecture model: source is portable, the artifact is not, so `znative` distributes source and builds locally, and the platform library shape keeps a foreign binary from being mistaken for a loadable one. That is the design and it is right. This chapter reads the two functions that implement the “library shape” half of it, because they do not use the same rule, and the difference is visible in one error message that names a decision nobody made.

The store key has no architecture in it

```
pub fn package_dir(&self, name: &str, version: &str) -> PathBuf {
    self.store_dir().join(format!("{}", name, version))
}
```

`store.rs:73–75`. Two components. No target triple, no OS, no pointer width. Chapter 44 already established that a store is per-machine and that the address in it is a local one; this is the mechanical statement of the same fact. A store whose `$ZSHRS_HOME` is on a filesystem shared between machines of different architectures has one directory per `name@version` for all of them, and whichever machine installed last owns its contents. The design’s answer is that you should not do that, and the design has no mechanism that would notice if you did.

Two functions, two rules

Kind detection asks whether a tree contains a shared library:

```
fn has_cdylib(dir: &Path) -> bool {
    ...
    if n.starts_with("lib") && (n.ends_with(".dylib") || n.ends_with(".so")) {
        return true;
    }
}
```

`manifest.rs:180–192`. Both suffixes, on every host. A macOS `.dylib` counts as a cdylib on Linux and a Linux `.so` counts as one on macOS.

Installation asks which file to record and load:

```
fn find_cdylib(dir: &Path) -> Option<String> {
    let suffix = std::env::consts::DLL_SUFFIX; // .dylib / .so / .dll
    let rd = std::fs::read_dir(dir).ok()?;
    for entry in rd.flatten() {
        let n = entry.file_name().to_string_lossy().into_owned();
        if n.ends_with(suffix) && n.starts_with(std::env::consts::DLL_PREFIX) {
            return Some(n);
        }
    }
    None
}
```

`commands.rs:425–435`. The host's suffix only. Those constants are per-target: on macOS, `DLL_PREFIX` is `lib` and `DLL_SUFFIX` is `.dylib`; on Linux, `lib` and `.so` (Rust's standard library, `library/std/src/sys/env_consts.rs:198–199` and `:187–188`).

So detection is permissive and selection is strict. Four combinations, and the outcomes differ:

Tree contents	Detected as	Install outcome
host-shaped library	native	copied, <code>lib</code> recorded, loads
foreign library + <code>Cargo.toml</code>	native	builds from source, host-shaped library produced and recorded
foreign library, no <code>Cargo.toml</code>	native	fails — see below
no library, <code>Cargo.toml</code> with <code>cdylib</code>	native	builds from source

Row two is Chapter 24's claim working exactly as described, by a slightly different mechanism than that chapter states: detection does not “fall through to the `Cargo.toml`” — it classifies the tree as native on the strength of the foreign library — but `prepare_native`'s first test is `find_cdylib`, which does not see the foreign file, so the build runs anyway and the outcome is the one Chapter 24 promises.

The message that names a decision nobody made

Row three is the case Chapter 24 calls “the one hazard,” and the message it produces is worth reading against the code that emits it:

```
let has_cargo = dir.join("Cargo.toml").is_file();
let want_build = spec.build.unwrap_or(has_cargo);
if !want_build {
    return Err(PkgError::Resolve(format!(
        "{}: native plugin has no prebuilt cdylib and build is disabled",
        name
    )));
}
```

`commands.rs:383–390`. `spec.build` is `Option<bool>` and is `None` unless the plugin's `znative.toml` set it (`manifest.rs:70–71`), so `unwrap_or(has_cargo)` means: build if the manifest says to, otherwise build if and only if there is something to build. A repository with a macOS `.dylib`, no `Cargo.toml`, installed on Linux, reaches this branch with `has_cargo == false` and therefore `want_build == false`, and the error says **build is disabled** — when nothing was disabled and nobody wrote `build = false`.

The next branch is the one whose message fits:

```

if !has_cargo {
    return Err(PkgError::Resolve(format!(
        "{}: native plugin has neither a cdylib nor a Cargo.toml to build",
        name
    )));
}

```

`commands.rs:391–396`. It is unreachable from the `has_cargo == false` path, because the `want_build` test above it already returned. Reaching it requires `want_build == true` with `has_cargo == false`, which means an explicit `build = true` in a `znative.toml` for a tree with nothing to build. That is the manifest-and-tree-disagree case Chapter 25 describes under “No library and nothing to build from,” and it is real; it is simply not the case most users hit.

Chapter 25’s other entry, “No prebuilt library and building disabled,” describes this message as “a configuration mistake rather than an environment problem: something declared `build = false` and then did not ship the artifact that decision assumed.” That is one of the two ways to reach the message. The other, and the more common one in a cross-architecture fleet, is an environment problem wearing a configuration problem’s words: the right binary for a different machine.

If you meet `native plugin has no prebuilt cdylib and build is disabled` and you never wrote `build = false` anywhere, check the tree for a library with the other platform’s suffix. That is the diagnosis.

What the store keeps from the losing side

In row two — foreign library plus `Cargo.toml` — the install succeeds, and the store copy contains **both** libraries. `copy_dir_filtered` excludes `.git/` and `target/` and nothing else (`store.rs:217–219`), so the committed `.dylib` is copied into a Linux store alongside the `.so` the build produced. The integrity hash covers both, since it hashes whatever is in the store directory (`commands.rs:42–43`). The index’s `lib` field records only the one `find_cdylib` returned (`commands.rs:56–57`), so only the host-shaped library is ever loaded; the other is inert weight in the store and in the hash.

That has a consequence for the cross-machine comparison Chapter 68 describes. Two stores that installed the same source on the same architecture agree. Two stores on different architectures do not, and cannot — the built library differs byte for byte. The integrity field compares meaningfully only within an architecture.

There is one further detail in `find_cdylib` worth stating because it is not deterministic: the loop returns the *first* matching entry from `read_dir`, which has no defined order. A tree with two host-shaped libraries at its root — say a build that produced both `libfoo.so` and a vendored `libbar.so` — records whichever the directory iteration yielded first, and that choice can differ between machines or between reinstalls on the same machine. A native plugin should have exactly one library at its tree root.

Installing from a path writes into your tree

The build step’s last act is a copy, and its destination is worth reading twice:

```

let rel = dir.join("target").join("release");
let built = find_cdylib(&rel).ok_or_else(|| { ... })?;
std::fs::copy(rel.join(&built), dir.join(&built))
    .map_err(|e| PkgError::Io(format!("stage cdylib: {}", e)))?;

```

`commands.rs:411–420`. `dir` is the staged tree. For a git source that is scratch under `pkg/git/` and the copy is invisible. For a `path:` source, `dir` is the canonicalised directory you named (`resolver.rs:39–50`) — your working tree — and the copy puts a `lib*.dylib` at its root. The reason is stated in the comment above the function (`commands.rs:375–378`): the store copy skips `target/`, so the library has to be somewhere the copy will find it.

That is a side effect on a directory the user owns, and it has a second-order consequence at the top of the same function:

```
fn prepare_native(dir: &Path, spec: &super::manifest::NativeSpec, name: &str) ->
    ~ PkgResult<()> {
    if find_cdylib(dir).is_some() {
        return Ok(()); // prebuilt cdylib already at the root.
    }
}
```

`commands.rs:379–382`. On the next `znative add path:.` in that same tree, the library staged by the previous install is still sitting at the root, `find_cdylib` finds it, and the build is skipped entirely. The plugin you just edited is not rebuilt; the previous binary is copied into the store and loaded. This is the plugin-development loop's sharpest edge, and the fix is one command:

```
# in the plugin's working tree, before reinstalling
rm -f lib*.dylib lib*.so # drop the staged artifact so prepare_native rebuilds
znative add path:.
```

The alternative — building by hand and letting the staged artifact be the install — is legitimate, and is exactly what `build = false` formalises for a fleet that builds elsewhere (Chapter 24). What is not legitimate is being unaware of which of the two you are doing, because both produce a successful install and a plausible `znative info`.

Chapter 72 — What update Actually Does

`znative update` is documented in one line — “re-resolve + reinstall from the recorded source” (`builtin.rs:19`) — and the implementation is fourteen lines. It is worth those fourteen lines of reading, because three of its behaviours are not obvious from the description and one of them makes a successful update report failure.

The whole of it

```
pub fn update(name: Option<String>) -> PkgResult<()> {
    let store = Store::user_default()?;
    let index = InstalledIndex::load_from(&store)?;
    let targets: Vec<String> = match name {
        Some(n) => vec![n.to_string()],
        None => index.packages.iter().map(|p| p.name.clone()).collect(),
    };
    for n in targets {
        let Some(p) = index.find(&n) else {
            return Err(PkgError::Other(format!("{n} is not installed", n)));
        };
        let spec = source_to_spec(&p.source);
        add(&spec)?;
    }
    Ok(())
}
```

`commands.rs:309–324`. Load the index once, compute a list of names, and for each one look up its recorded source and run the full `add` pipeline on it. There is no version comparison, no “is there anything newer” check, and no network probe that precedes the work. `update` is `add` in a loop, and `add` is unconditional.

Two structural notes follow directly. The index is read once, before the loop, and `add` writes the index on every iteration (`commands.rs:56–68`) — so the snapshot `update` is iterating is stale from the second iteration onward. It is used only for names and source labels, which do not change under it, so the staleness is harmless; it is worth knowing because it means `znative update` with no argument updates exactly the set of plugins that existed when it started, and a plugin that some other process installs mid-run is not picked up.

The second note is the error path: the loop uses `add(&spec)?`, so the first failure aborts the whole run. `znative update` over twelve plugins where the third fails leaves nine untouched and reports one message. This is the opposite of the builtin’s multi-argument loops (Chapter 66), which continue past failures.

The label goes back the way it came

```
fn source_to_spec(source: &str) -> String {
    if let Some(rest) = source.strip_prefix("path+file://") {
        format!("path:{}", rest)
    } else {
        source.to_string()
    }
}
```

`commands.rs:327–334`. Only local sources need translating, because `github:owner/repo` and `git+URL` are already valid `add` specs — and, per Chapter 67, so is `github:owner/repo@v1.2.0`, which is why a pinned install stays pinned across updates rather than silently drifting to the branch tip. `update` on a pinned plugin re-clones the same ref and reinstalls the same content. That is the property Chapter 29 relies on when it says a plugin you have reviewed stays the plugin you reviewed.

For a `path+file://` source the round trip is exact, and the update reinstalls from the directory as it is now. If that directory has been moved or deleted, the resolve fails with `path <dir>: No such file or directory` — the accurate message, because `update` goes through `add` and therefore through `resolve`, not through `source_label`.

A renamed plugin becomes two rows

`add` computes the plugin's name from the manifest of the tree it just fetched, not from the row it is replacing:

```
let name = manifest
    .as_ref()
    .map(|m| m.plugin.name.clone())
    .filter(|n| !n.is_empty())
    .unwrap_or_else(|| staged.name.clone());
```

`commands.rs:19–23`. And the index insert is keyed on that name:

```
pub fn upsert(&mut self, p: InstalledPlugin) {
    if let Some(slot) = self.packages.iter_mut().find(|e| e.name == p.name) {
        *slot = p;
    } else {
        self.packages.push(p);
    }
}
```

`store.rs:194–200`. So if an upstream release changes `[plugin] name` in its `znative.toml`, `update` does not rename anything. It installs under the new name, finds no row with that name, and pushes a second row. The old row survives, still pointing at the old store directory, still loadable, still listed. You now have the same upstream plugin installed twice under two names, and what a bare `znative load` does when it walks both rows depends on the *other* name — the one the Rust side reports in its `PluginInfo`. If that is unchanged, the second load is refused as a duplicate (`plugin_host.rs:422–427`) and `load` collects the failure into its error list (`commands.rs:293–303`). If it changed too, both libraries load, and any command name they share is overwritten in the registry by whichever loaded second, since registration is a `HashMap::insert` (`plugin_host.rs:435`).

The version-change case is the normal one and is benign, because Chapter 27 already describes it: the new version installs at a new `store/<name>@<newversion>/`, the row is reprinted, and the old directory becomes an orphan for `gc`. A name change is the case where the *row* is orphaned too, and `gc` cannot help with that — `gc` only removes directories the index does not name (`commands.rs:159–180`), and this directory is named. Clearing it takes an explicit `znative remove <oldname>`.

Reinstalling the same version clears first

When the version has not changed, the destination already exists, and `install_dir` empties it:

```
pub fn install_dir(&self, name: &str, version: &str, src: &Path) -> PkgResult<PathBuf>
{
    let dst = self.package_dir(name, version);
    if dst.exists() {
        std::fs::remove_dir_all(&dst)
            .map_err(|e| PkgError::Io(format!("clear {}: {}", dst.display(), e)))?;
    }
    std::fs::create_dir_all(&dst)?;
    copy_dir_filtered(src, &dst)?;
}
```

store.rs:100–108. Delete, recreate, copy. Nothing is merged, so a file that the new upstream deleted does not survive in the store, and any file you edited in the store by hand is gone. This is also the window in which a store directory is briefly absent while its index row still names it — the Chapter 26 “row with no directory” state, held for the duration of one directory copy.

The update that succeeds and reports failure

This is the behaviour worth the chapter. `add` ends by loading what it installed:

```
let mut index = InstalledIndex::load_from(&store)?;
index.upsert(entry.clone());
index.save_to(&store)?;

...
load_entry(&store, &entry)?;

...
println!(
    "znative: added {}@{} ({}){}",
    name, version, entry.kind, desc
);
```

commands.rs:66–84. The order is: write the index, then load, then announce. `load_entry` for a native plugin calls `plugin_host::load`, which refuses a name that is already present:

```
if plugins().lock().unwrap().iter().any(|p| p.name == name) {
    ...
    return Err(format!("plugin `{}` already loaded", name));
}
```

plugin_host.rs:422–427. Put those together. You are in an interactive shell. The plugin is loaded — your `.zshrc` loaded it at startup. You run `znative update forgit`. The clone succeeds, the build succeeds, the store copy is replaced, the integrity hash is recomputed, the index row is written — and then the load is refused, because a plugin by that name is already in this process’s registry. `add` returns that error, `update` propagates it, and the builtin prints:

```
znative: plugin `forgit` already loaded
```

with exit status 1, and no `znative: added ...` line. Every durable effect of the update happened. The only thing that failed is the part that could not have succeeded — replacing an already-mapped library in a running process, which Chapter 10 explains the host deliberately refuses rather than resolving by some precedence rule.

So the message is accurate and the framing is misleading: it reads like the update failed, and what it means is that the update landed on disk and this shell is still running the old code. `znative info forgit` confirms it — the store path and the integrity hash are the new ones. The new code takes effect in the next shell, or after

`zmodload -uR forgit` followed by a `znative load forgit` in this one.

Script plugins do not hit this. `load_entry`'s script arm synthesises `fpath=` and `source` lines and evaluates them (`commands.rs:347–366`); re-sourcing a plugin is not refused by anything, so an updated script plugin's new code is live in the shell that ran the update, layered over whatever the old version had already defined. The two kinds diverge here in exactly the way their loading mechanisms diverge: a `dlopen` is a one-shot registration the host protects, and a `source` is an idempotent-ish re-execution the shell has always allowed.

The one-line summary

`update` is `add` on the recorded label, once per target, aborting on the first failure. It re-resolves rather than checking for newer content, it honours a pin because the pin is part of the label, it keys the result on the manifest's current name, and in a shell that has the plugin loaded it will do all of its work and then tell you it could not do the last step. None of that is hidden — it is fourteen lines — and all of it is easier to recognise than to deduce from a message at the wrong altitude.

Chapter 73 – The Index Is a Path Interpreter

`installed.toml` is described everywhere in this book as a record: identity, provenance, and the few fields a load needs. That description is accurate about what the file *contains* and incomplete about what the file *is*. Three of its fields — `name`, `version`, and `lib` — are not consumed as labels. They are consumed as path components, joined against a store root and handed to the filesystem. A field that becomes a path is an instruction, and the manager executes it without asking whether the result is still inside the store.

This chapter reads the three joins, then shows what each one does when the value it is given is not the shape the design assumed. Everything below was produced against a scratch `ZSHRS_HOME` under `/private/tmp`, never a real one.

The three joins

The first is the store address, built from two index fields:

```
pub fn package_dir(&self, name: &str, version: &str) -> PathBuf {
    self.store_dir().join(format!("{}", name, version))
}
```

`store.rs:73–75`. The second is the library address, built from the store address and a third field:

```
"native" => {
    let lib = dir.join(&p.lib);
    crate::plugin_host::load(&lib.to_string_lossy())
}
```

`commands.rs:341–343`. The third is the script address, the same join with a different field, once per entry in `source_files` and `fpath` (`commands.rs:353` and `commands.rs:358`).

None of the three validates. There is no check that the result is a descendant of `store/`, no rejection of `..`, no rejection of a leading `/`, and no comparison between the identity in the index and the identity the loaded artifact reports.

`Path::join` with an absolute argument replaces, it does not append

This is Rust standard-library behaviour rather than a `znative` decision, and it is the mechanism the `lib` field inherits: joining an absolute path onto a base discards the base. So an index row whose `lib` is absolute does not name a file inside the store copy — it names a file anywhere on the machine, and the store directory computed one line above becomes dead.

The demonstration used a second plugin, `outside`, built in a directory with no relationship to the store, exporting one command that identifies itself. The index row for the installed plugin `hello` was left intact except for its `lib` field:

```
lib = "/private/tmp/znative-book2/outside/target/debug/liboutside.dylib"
```

Then, in a fresh shell with `ZSHRS_HOME` pointed at the scratch tree:

```
$ zshrs -c 'znative load hello; routside; zmodload -R'
LOADED-FROM-OUTSIDE-THE-STORE
outside 9.9.9 /private/tmp/znative-book2/outside/target/debug/liboutside.dylib
```

`znative load hello` loaded a library that is not in the store, registered a command the store copy does not contain, and the host's own listing reports the plugin under the name `outside` at version `9.9.9` — neither of which appears anywhere in `installed.toml`. The store directory was untouched throughout; it still holds `libhello.dylib`, which was never opened.

Two facts follow, and they are separable. The first is that the index's `lib` is a capability rather than a filename. The second is subtler and worse: after that load, the manager's idea of what is loaded and the host's idea of what is loaded disagree completely, and nothing reconciles them. Chapter 26's reconciliation matrix covers index-versus-store disagreements; this is index-versus-*process*, which the matrix does not have a row for because the design did not anticipate the two identities being able to differ.

The consequence shows up immediately at `remove`:

```
$ zshrs -c 'znative load hello; znative remove hello; routside; zmodload -R'
znative: removed hello
LOADED-FROM-OUTSIDE-THE-STORE
outside 9.9.9 /private/tmp/znative-book2/outside/target/debug/liboutside.dylib
```

`remove` reported success, deleted the store directory, dropped the index row — and the plugin is still loaded, because the unload it attempted was:

```
if entry.kind == "native" {
    let _ = crate::plugin_host::unload(name);
}
```

`commands.rs:95–97`. The name it passes is the *index* name, `hello`. The host's registry is keyed on the name the plugin's `PluginInfo` reported, `outside`. The unload found nothing, returned `Err`, and the `let _ =` discarded it. Chapter 34's account of the native path — “removal really does remove,” because every effect lives in structures the host owns — holds only while the two names agree, and nothing enforces that they do.

name and version escape the store root

The `lib` field is at least a field a user might reasonably think of as a path. `name` and `version` are not: they are identity, they come from the plugin's own `znative.toml`, and they are interpolated into a directory name. A `..` in either one is a directory traversal that the plugin author controls and the installing user never sees.

A plugin was built whose entire manifest was:

```
[plugin]
name = "trav"
version = "../../../escaped"
```

Installing it succeeded, with a success message that contains the traversal in plain sight and is easy to read past:

```
$ zshrs -c "znative add path:$R/tree/traversal; echo rc=\$?"
traversal
znative: added trav@../../../escaped (script)
rc=0
```

(The bare `traversal` on the first line is the plugin's own `source` output — the install loads what it installed, so the code ran too.) The tree did not land in the store:

```
$ find $ZSHRS_HOME/pkg -maxdepth 3 -name 'escaped*' -o -maxdepth 3 -name 'trav*'
/private/tmp/znative-book2/home/pkg/escaped
/private/tmp/znative-book2/home/pkg/escaped/trav.plugin.zsh
/private/tmp/znative-book2/home/pkg/store/trav@..
```

`store/<name>@<version>/` with `version = ../../../../escaped` resolves to `pkg/escaped/`, three levels up from `store/trav@`. The empty `store/trav@..` directory is the residue of `create_dir_all` walking the prefix. `pkg/` is not a boundary either — the same field with more `..` reaches anywhere the user can write, and `store.rs:100–108` does the writing without looking.

Then the second half, which turns a write primitive into a delete primitive. `remove` computes the same path from the same recorded fields and calls `remove_dir_all` on it (`commands.rs:98–102`). A file was planted in the escaped directory to stand for unrelated content:

```
$ find $ZSHRS_HOME/pkg/escaped | sort
/private/tmp/znative-book2/home/pkg/escaped
/private/tmp/znative-book2/home/pkg/escaped/trav.plugin.zsh
/private/tmp/znative-book2/home/pkg/escaped/victim
/private/tmp/znative-book2/home/pkg/escaped/victim/important.txt
/private/tmp/znative-book2/home/pkg/escaped/znative.toml

$ zshrs -c 'znative remove trav'
znative: removed trav

$ find $ZSHRS_HOME/pkg/escaped
find: /private/tmp/znative-book2/home/pkg/escaped: No such file or directory
```

The planted file is gone. `znative remove <name>` recursively deleted a directory chosen by the plugin's manifest, including content that had nothing to do with the plugin. Chapter 29's threat model says a plugin you install is code you chose to run, and that remains true — but this path does not need the plugin's code to run at all. The traversal is executed by the manager, from a TOML string, during an `add`, and again during a `remove` that a user might perform precisely because they decided not to trust the thing.

Why the same string is safe in the other direction

It is worth being exact about which fields are dangerous, because most of the index is not. `source` is a label, matched with `==` (`commands.rs:280`) and translated back into a spec by a prefix strip (`commands.rs:327–334`); it never becomes a path directly. `kind` is matched against two literals (`commands.rs:340–371`). `integrity` is printed and nothing else, as Chapter 68 established. The three fields that reach the filesystem are exactly the three this chapter opened with, and a validator would therefore be three checks, not an audit of the schema.

The parser has the opposite problem in the other direction

There is a mirror-image case where a perfectly ordinary path is rejected because the spec grammar reads it as something else. `split_ref` splits on the last `@` that follows the last `/` (`resolver.rs:101–111`), which is what keeps `git@github.com:o/r.git` intact. A directory whose `name` contains an `@` is not so lucky:

```
$ zshrs -c "znative add path:$R/tree/weird@name"
znative: path /private/tmp/znative-book2/tree/weird: No such file or directory (os error
  2)
```

The `@name` was taken as a git ref, the remaining `path:` prefix pointed at a directory that does not exist, and the diagnostic names a path the user never typed. Chapter 67 reads `split_ref` in full; the point here is the

asymmetry. Local paths are checked hard enough to reject a legitimate one on a parsing technicality and not checked at all once they are inside the index — the strictness is applied at the boundary where the user is present and dropped at the boundary where they are not.

What a fix costs

Nothing in this chapter requires a redesign. `package_dir` could reject a `name` or `version` containing `/`, `..`, or a leading `.`; `load_entry` could reject an absolute `lib` and any `lib` whose canonicalised form is not under the store directory; `load` could compare the name the host returns against the name in the row and refuse the mismatch. Those are three predicates over strings the manager already holds, and none of them touches the startup cost this design exists to protect, because they are string tests rather than filesystem work. What they buy is that the index becomes a record again instead of a small language for addressing the filesystem — which is what every chapter before this one has assumed it already was.

Chapter 74 – A Promise With No Expiry Date

Chapter 21 sets out the rules for evolving the ABI and Chapter 40 describes the day it moves. Both treat the version number as an event to be managed. This chapter asks the longer question the endgame framing implies: what exactly has been promised to a stranger who compiles against `znative` today, how much of that promise is enforced by mechanism rather than by intent, and which parts of it are cheap to keep for decades and which are not.

The promise is one symbol and one integer

The whole contract, reduced to what a loader actually touches, is very small. On the plugin side there is exactly one exported symbol:

```
$ nm -gU $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib
00000000000000fa0 T _znative_init
$ nm -gU $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib | wc -l
1
```

One symbol, resolved by name from a constant the SDK exports (`INIT_SYMBOL`, `znative/src/lib.rs:87`). Everything else the plugin does reaches the shell through function pointers the *host* supplies, so the plugin exports no callable surface at all beyond its entry point. That is the property that makes this a durable contract: there is no symbol table to keep stable, no name mangling to preserve, no vtable layout shared across a version boundary. There is a function pointer table passed in, and a `PluginInfo` pointer passed back.

The integer is `ABI_VERSION` (`lib.rs:73`), currently `4`, compiled into both sides as a constant. Both sides check it: the plugin checks the host's before it touches the table (`lib.rs:477–480`), and the host checks the plugin's before it trusts the returned pointer (`plugin_host.rs:408–416`).

One of the two checks is unreachable in practice

The two-sided check is described in Chapter 15 as belt and braces. Measured, one of the two braces cannot fire for any plugin built the documented way.

A copy of the SDK was made with `ABI_VERSION` changed to `99`, and a plugin was built against it with the ordinary `declare_plugin!` macro. Loading it into the shipped host, which is version `4`:

```
$ zshrs -c "zmodload -R $R/plugin-v99/target/debug/libfutureplug.dylib; echo rc=\$?"
zshrs:zmodload:1: `./libfutureplug.dylib`: plugin init failed (ABI mismatch or error)
rc=1
```

The message is `plugin_host.rs:402`, not `plugin_host.rs:412`. The reason is order: the macro's generated `znative_init` compares the *host's* `abi_version` against its own compiled-in constant first and returns null on mismatch (`lib.rs:477–480`), so the host never receives a `PluginInfo` to inspect and its own comparison at `plugin_host.rs:408` is skipped. The host's message — the one that would have said `ABI version 99 != host 4` — is reachable only from a hand-written init that reports a version it does not implement.

This is not a defect, but it has a consequence for the promise. The legible half of the diagnostic lives in the half of the check that does not run. What a user sees when their plugin is a version behind is `plugin init failed (ABI mismatch or error)` — a message that cannot distinguish “you compiled against v3” from “the plugin’s init returned null because its own setup failed.” Over a decade of ABI movement, that is the message the majority of skew reports will quote, and it names neither version.

What “versioned” buys and what it does not

The version gate converts a layout disagreement into a refusal. It does that completely: every field in `HostApi` is `#[repr(C)]`, the table is passed by pointer, and a plugin that agrees on the integer agrees on every offset. The cost is that the integer is the *only* granularity. There are no feature flags, no capability query, no “which callbacks does this host implement” call. A plugin that needs only `print` and a plugin that needs `comp_dispatch` are gated identically, and both stop loading on the same day.

The table has grown three times — v2 added `register_completion`, v3 added the function pair, v4 added the three completion-router entries (`lib.rs:67–72`) — and each growth invalidated every previously compiled plugin, including the ones that used none of the new fields. Chapter 52 counts the result from the other end: six of the twelve callbacks have no caller anywhere in the shipped fleet. So the fleet has paid four version bumps for capabilities half of it does not use.

An append-only discipline plus a size field would have made those bumps non-breaking: a plugin that knows fields 0–7 can safely ignore fields 8–12 in a table it was handed, provided it can tell how many fields it was handed. The current table has no length, so a plugin has no way to distinguish “the host is older and this field does not exist” from “the host is newer.” That is the single structural decision that determines whether the next twenty years of ABI evolution costs a recompile of every plugin per bump or none of them, and it is a `u32` in the struct.

The extension slot that was reserved and never used

There is one field in the table that exists for exactly this problem and has never carried a value:

```
/// Reserved for the host; opaque to plugins. Currently null.
pub ctx: *mut c_void,
```

`lib.rs:118–119`, and the host fills it in accordingly:

```
abi_version: ABI_VERSION,
ctx: std::ptr::null_mut(),
```

`plugin_host.rs:343–344`. A `*mut c_void` in slot two is the conventional escape hatch for a C ABI: it is where a host puts a pointer to whatever it needs later without disturbing the fields around it. What it cannot do is solve the compatibility problem, because a plugin has no way to know what the pointer points at without agreeing on a type — which is another version number, on a second object, with the same recompile cost.

The field that *would* solve it is the one the table does not have: a `size` or `field_count` immediately after `abi_version`, filled by the host with the number of fields it actually populated. With it, a plugin compiled against v3 loaded into a v7 host can check that the fields it knows about are present and ignore the rest, and a plugin compiled against v7 loaded into a v3 host can detect the missing callbacks and degrade instead of failing. Without it, every plugin is welded to one host version, which is where the fleet is today: Chapter 51 counts four version numbers that do not agree, and this is the mechanism that makes the disagreement matter.

What has to hold for thirty years

Reading the contract against the durability bar the project sets for itself, the parts divide cleanly.

Element	Load-bearing for compatibility	Cost to keep stable
<code>INIT_SYMBOL</code> name (lib.rs:87)	total — a rename orphans every compiled plugin	free; it is a string constant
<code>#[repr(C)]</code> on <code>HostApi / PluginInfo</code>	total	free; it is an attribute
Field <i>order</i> in <code>HostApi</code> (lib.rs:113–189)	total	free if append-only, catastrophic if reordered
<code>BuiltinFn / CompFn</code> signature (lib.rs:82–100)	total	free; both are <code>(host, argc, argv) -> c_int</code>
<code>ABI_VERSION</code> value	total, and it is the thing that moves	one recompile per plugin per bump
String ownership rules (<code>free_cstring</code>)	high — a rule change leaks or double-frees	free; documented at lib.rs:130–138
The SDK's zero dependencies (znative/Cargo.toml)	high — a transitive dep is a bit-rot risk	free; the crate has none

The last row is the one most easily lost and the cheapest to keep. The SDK declares an empty `[dependencies]` section with a comment saying so; a plugin author's build graph for the ABI half of their crate is one crate deep. That is the property that makes “still builds in 2035” a plausible claim rather than a hope, and it survives only as long as nobody adds a convenience dependency to the SDK.

What the promise does not cover

Three things a reader may assume are versioned are not.

The *host's behaviour* is not versioned. `eval` runs shell code, and what a given fragment does depends on the shell's semantics, not on `ABI_VERSION`. A plugin that shells out through `eval` is coupled to the shell's language, and the ABI number will not move when that language does.

The *artifact* is not versioned. The store records `lib = "libhello.dylib"` — a filename with no target triple, no ABI tag, and no build metadata. Two plugins compiled against two different SDK versions produce byte-identical index rows. Chapter 75 takes that up as an architecture question; here it is a compatibility question, and the answer is that the index cannot tell you what a stored library was compiled against. The only way to find out is to load it and read the error.

The *plugin's own version string* is not checked against anything. `PluginInfo` carries it ([lib.rs:203](#)), the host stores it and prints it in `zmodload -R` listings ([plugin_host.rs:628](#)), and it is compared to nothing — including to the `version` the index recorded from [znative.toml](#). The two are independent strings that happen to usually agree; when they do not, as Chapter 73's experiment showed, the listing and the index simply disagree.

The honest summary

The contract is one symbol, one integer, and a table whose layout is frozen by that integer. It is small enough to keep for decades, the SDK's dependency graph is short enough that building it in 2035 is a reasonable expectation, and the parts that would be expensive to preserve are exactly the parts that are already free. What it lacks is granularity: no length field, no capability bits, and a skew diagnostic that names neither version. Those are the three things a version 5 should carry, and Chapter 62 already argues for the third on separate evidence.

Chapter 75 – What Cross-Architecture Would Actually Cost

Chapter 24 states the cross-architecture model — ship source, build locally — and Chapter 71 reads the two suffix rules that implement its library-shape half. Both chapters are about what happens on one machine. This chapter asks what would have to change for a store to be *shared* across machines of different architectures, which is the case Chapter 44 names as the one deployment where the per-machine assumption stops holding. The answer is four specific changes, and the evidence for each is a measurement rather than an argument.

The store key: two components, no target

```
pub fn package_dir(&self, name: &str, version: &str) -> PathBuf {
    self.store_dir().join(format!("{}", name, version))
}
```

`store.rs:73–75`. On a scratch store the result is exactly what it says:

```
$ ls -l $ZSHRS_HOME/pkg/store/  
hello@0.1.0  
sp@2.0.0
```

A macOS arm64 machine and a Linux x86-64 machine sharing that directory would both claim `hello@0.1.0`. Whichever installed last owns the contents, and the other one loads a library it cannot open. The failure is not silent — `dlopen` refuses a foreign object — but it is not attributed either. What the user sees is a load error for a plugin they installed successfully, on a store that `znative list` reports as healthy.

A target-keyed store is a one-line change to the format string (`{@{}-{} with std::env::consts::ARCH and OS, or the full triple), and it costs one directory per machine class rather than one per plugin. What it buys is that the failure becomes impossible rather than diagnosable.`

The index row records a filename that means different things per host

The `lib` field for the plugin above:

```
$ znative info hello | grep '^lib'
lib      libhello.dylib
```

The name encodes the *platform* (`.dylib`) and nothing else — no architecture, no ABI version, no build host. And the selection rule that produced it is per-target: `find_cdylib` matches `std::env::consts::DLL_SUFFIX` and `DLL_PREFIX` (`commands.rs:425–435`), so on Linux the same store directory yields `None` and the install fails with `no cdylib after build` (`commands.rs:57`) even though a loadable-looking file is sitting there. Chapter 71 reads the mismatch between that rule and the permissive detection rule; the cross-machine consequence is that the index cannot be shared even when the store could be, because a single `lib` string cannot be right on two platforms at once.

A shareable index needs `lib` to become a map from target to filename, which in turn means the schema version at `store.rs:117–118` has to move — the first real use that field would ever get.

The artifact is arch-stamped and its identity points at the build tree

The built library is exactly as portable as it looks:

```
$ file $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib
...: Mach-O 64-bit dynamically linked shared library arm64
$ lipo -archs $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib
arm64
```

Single-architecture, as `cargo build --release` produces by default. A fat binary would satisfy two machines from one file, and nothing in `prepare_native` (`commands.rs:379–422`) asks for one — the build is a bare `cargo build --release` with no target flags, so the artifact is whatever the build host is.

There is a second, less obvious detail in the same file:

```
$ otool -L $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib
/private/tmp/znative-book2/tree/plugin-hello/target/release/deps/libhello.dylib (...)
/usr/lib/libSystem.B.dylib (...)
```

The first line is the library's own install name, and it is an absolute path into the *build tree* — a directory that has nothing to do with the store and that will not exist on any other machine. `znative` loads by explicit path (`commands.rs:342–343`), so this does not break anything today. It does mean the stored artifact is not self-describing: a tool that resolved it the ordinary way, by its recorded identity, would look for it under a developer's `target/` directory. Any future work that relocates artifacts, mirrors them, or hands them to a system loader will meet this line first.

The copy semantics differ per platform, from one `cfg` in the standard library

`install_dir` copies the staged tree into the store with `std::fs::copy` per file (`store.rs:232`). What that call preserves is not the same on the two targets the project supports.

Measured on macOS, source and store agree to the nanosecond:

```
$ stat -f '%N mtime=%Fm' $R/tree/plugin-script/script.plugin.zsh \
  $ZSHRS_HOME/pkg/store/sp@2.0.0/script.plugin.zsh
.../tree/plugin-script/script.plugin.zsh mtime=1786319549.630334892
.../store/sp@2.0.0/script.plugin.zsh mtime=1786319549.630334892
```

The reason is in the standard library, which has two implementations of `copy` selected by `cfg`. The Apple one uses `fcopyfile` with `COPYFILE_ALL = COPYFILE_METADATA | COPYFILE_DATA` (`library/std/src/sys/fs/unix.rs:2403–2405`, flags applied at :2449); the non-Apple one opens the destination and sets permissions only (`unix.rs:2392–2393`, via `open_to_and_set_permissions`). Both from the toolchain in use here, `rustc 1.97.1`.

So the same `znative add` produces store files with the upstream timestamps on macOS and with the install timestamp on Linux. Chapter 68 already notes that timestamps are outside the integrity hash, so the hash agrees across the two; what differs is every consumer that keys on `mtime`, which Chapter 76 takes up. For the cross-architecture question the point is narrower: a store copied between machines carries timestamps that mean one thing on the machine that wrote it and another on the machine that reads it, and no field records which.

The four changes, priced

Change	Where	Cost
Target in the store key	<code>store.rs:73–75</code>	one format string; one store directory per machine class
<code>lib</code> as a per-target map	<code>store.rs:139–141</code> + <code>commands.rs:56</code>	schema bump at <code>store.rs:117–118</code> ; migration for existing indexes
Explicit <code>--target</code> , or fat artifacts	<code>commands.rs:397–409</code>	a build flag; cross toolchains on the building host
Record the build target in the row	<code>store.rs:127–148</code>	one string field; makes the mismatch diagnosable rather than fatal

The fourth is the cheapest and it is the one with the largest diagnostic return, because it converts “this plugin does not load and the store looks fine” into a message that can name the two triples. The first three are what a genuinely shared store would need, and none of them is required by the design as it stands, because the design’s answer to a shared store is that you should not have one. That answer is defensible. What is not visible today is *whether you have one*, and the fourth row is what would make it visible.

Chapter 76 – The Store and the Caches That Never Meet It

`$ZSHRS_HOME` is a single directory holding several unrelated systems: the package store this book is about, an rkyv bytecode shard, an rkyv autoload shard, a SQLite mirror, a history database, and the daemon's state. Sharing a parent directory invites the assumption that they are integrated. This chapter tests that assumption and finds it false in three specific ways, each of which is worth knowing before reasoning about what a plugin load costs.

What is actually in the directory after a session of installs

The scratch home used for this book's second pass received two script plugin installs, several native installs and reinstalls, an `update`, a `gc`, a `clean`, and repeated `znative load` runs of both kinds. Afterwards:

```
$ ls -l $ZSHRS_HOME
artifacts
images
pkg
plugins.db
plugins.db-shm
plugins.db-wal
replay
snapshots
zshrs_history
zshrs_history.db
zshrs-daemon.toml
zshrs-recorder.toml
zshrs.log
zshrs.toml

$ find $ZSHRS_HOME -maxdepth 1 -name '*.rkyv' | wc -l
0
```

No rkyv shard was created by any of it. Both bytecode caches this shell has — `autoloads.rkyv` and `scripts.rkyv` — were untouched by every plugin operation performed.

The bytecode cache does not follow `$ZSHRS_HOME`

The first reason is an address mismatch. Four of the five state locations resolve through `ZSHRS_HOME`:

Component	Resolution	Honors <code>ZSHRS_HOME</code>
package store	<code>store.rs:33–44</code>	yes
autoload shard	<code>autoload_cache.rs:424–433</code>	yes
compsys mirror	<code>compsys/cache.rs:34–41</code>	yes

Component	Resolution	Honors <code>ZSHRS_HOME</code>
plugin mirror	<code>plugin_cache.rs:1042–1048</code>	yes
script bytecode shard	<code>script_cache.rs:426–430</code>	no

The fifth is hardcoded:

```
pub fn default_cache_path() -> PathBuf {
    dirs::home_dir()
        .unwrap_or_else(|| PathBuf::from("/tmp"))
        .join(".zshrs/scripts.rkyv")
}
```

`script_cache.rs:426–430`. The shell's own diagnostic confirms the split at runtime — run with `ZSHRS_HOME` pointed at the scratch tree, the doctor reports the SQLite mirror from the scratch directory and the bytecode shard from the real home:

```
scripts:      1 entries, 1.7KB  /Users/wizard/.zshrs/scripts.rkyv
plugins.db:   0 plugins, 0 functions, 4.0KB  mirror
```

Chapter 44 treats `$ZSHRS_HOME` as the knob that relocates a store. It relocates four of the five things under it. A user who moves `$ZSHRS_HOME` to a different filesystem moves their plugins and leaves their script bytecode behind, and nothing reports the split.

Sourcing a plugin does not populate the bytecode cache

The second reason is a path mismatch, and it is the one that matters for load cost. The bytecode cache is consulted and written by `execute_script_file` (`vm_helper.rs:2376–2435`), keyed on the canonicalised path plus mtime. A script plugin's load does not go through it. `load_entry` synthesises a fragment and evaluates it:

```
for f in &p.source_files {
    let path = dir.join(f);
    code.push_str(&format!("{}", shquote(&path.to_string_lossy())));
}
```

`commands.rs:357–360`. Two runs against isolated `HOME` directories separate the two paths cleanly. Running the store's file as a script argument:

```
$ env HOME=$R/fakehome ZSHRS_HOME=$R/home zshrs
~ $ZSHRS_HOME/pkg/store/sp@2.0.0/script.plugin.zsh
$ find $R/fakehome -maxdepth 2
/private/tmp/znative-book2/fakehome
/private/tmp/znative-book2/fakehome/.zshrs
/private/tmp/znative-book2/fakehome/.zshrs/scripts.rkyv
/private/tmp/znative-book2/fakehome/.zshrs/scripts.rkyv.lock
```

and the shard's key is the store path itself:

```
$ strings $R/fakehome/.zshrs/scripts.rkyv | grep -o '/private/tmp/znative-book2[^ ]*'
/private/tmp/znative-book2/home/pkg/store/sp@2.0.0/script.plugin.zsh
```

Sourcing the same file — which is what `znative load` does — against a fresh `HOME`:

```
$ env HOME=$R/fakehome2 ZSHRS_HOME=$R/home zshrs -c "source
~ $ZSHRS_HOME/pkg/store/sp@2.0.0/script.plugin.zsh; sp_func"
sp_func v2
$ find $R/fakehome2 -maxdepth 2
/private/tmp/znative-book2/fakehome2
```

Nothing. The plugin ran, defined its function, and left no cache entry. Every `znative load` of a script plugin therefore pays a full lex/parse/compile of every file in `source_files`, on every shell start, forever. That is the cost Chapter 28 attributes to the script path, and this is the mechanism: not that the cache misses, but that the cache is not on this path at all.

The autoload shard is not on this path either

The other rkyv shard caches compiled autoload function bodies (`autoload_cache.rs:1–23`), and a script plugin's `fpath` entry is exactly the kind of directory it exists for. Loading the plugin, autoloading one of its functions, and running it:

```
$ zshrs -c 'znative load sp; autoload -Uz sp_auto; sp_auto; print -l $fpath | head -2'
sp_auto v1
/private/tmp/znative-book2/home/pkg/store/sp@2.0.0/functions
...
```

The function resolved out of the store directory and ran. The shard:

```
$ find $ZSHRS_HOME -maxdepth 1 -name '*.rkyv' | wc -l
0
```

Still nothing, and this shard *does* honour `ZSHRS_HOME` (`autoload_cache.rs:424–433`), so this is not the addressing problem of the previous section — it is that the shard is populated on a different trigger than an ad-hoc `autoload` of a function found through a plugin's `fpath`. The practical reading for a plugin author is Chapter 42's ordering rule seen from the cost side: the completion and autoload machinery discovers your plugin's `functions/` directory correctly, and nothing about that discovery is cached across shells today.

The side-effect cache is defined and unwired

The third reason is that the cache designed for exactly this workload has no callers. `plugin_cache` describes itself as the store for “side effects of `source/.`”, keyed on (`canonical_path`, `mtime_secs`, `mtime_nsecs`) (`plugin_cache.rs:1–21`), with a snapshot/diff/replay trio to capture and reapply what a sourced file did. After every load performed for this chapter, its tables are empty:

```
$ for t in $(sqlite3 $ZSHRS_HOME/plugins.db "select name from sqlite_master where
  type='table';"); do
  echo "$t: $(sqlite3 $ZSHRS_HOME/plugins.db "select count(*) from $t;")"; done
plugins: 0
plugin_functions: 0
plugin_aliases: 0
...
compaudit_cache: 0
```

Fourteen tables, zero rows in each. The reason is not a miss:

```
$ grep -rn "snapshot_state\|diff_state\|replay_plugin_delta" --include='*.rs' src/ \
| grep -v 'src/extensions/plugin_cache.rs'
```

— no output. The capture and replay entry points (`plugin_cache.rs:1364`, `:1439`, `:1675`) and the store/check pair (`:472`, `:283`) have no call site outside the module that defines them. The live uses of the opened handle are the doctor's statistics, `dbview`, and `compaudit_cached` (`ext_builtins.rs:573`, `:763`, `:973`, `:2156`). So the side-effect cache exists as machinery and is not engaged by any `source`.

That is a fact about the shell rather than about `znative`, and it is stated here because the store is where it would be observed: the file that would be cached lives at a stable store path, the manager sources it on every startup, and the cache built for that pattern is not connected.

Where the two systems would have to agree

If the bytecode cache were ever put on the `source` path, the store's copy semantics become load-bearing, because the cache key is `(path, mtime)` and the store controls both halves.

The path half is stable by construction: `store/<name>@<version>/` is reused verbatim when the same version is reinstalled, since `install_dir` deletes and recreates that exact directory (`store.rs:100–108`). So an `update` that reinstalls an unchanged version produces the same key.

The mtime half is the platform-dependent one from Chapter 75. On macOS the store copy inherits the upstream file's timestamp, so a reinstall of identical content produces an identical key and a warm cache; on Linux the copy takes the install time, so the same operation produces a new key and a cold one. Identical command, identical store contents, opposite cache behaviour, decided by `library/std/src/sys/fs/unix.rs:2403` versus `:2392`. Any future integration between the store and the bytecode cache has to record its own fingerprint rather than inherit the filesystem's, or it inherits that divergence with it.

Chapter 77 – Uninstall and gc, Measured

Chapter 27 covers living with the store, Chapter 34 covers what `remove` cannot undo, and Appendix M gives the reconciliation matrix. What none of them does is run the commands and read the output. This chapter does, on the scratch store, and three of the results are not what the prose predicts.

`remove` deletes two things and unloads at most one

The implementation is eighteen lines (`commands.rs:89–106`) and does exactly three things: drop the index row, attempt an unload if the row says `native`, and `remove_dir_all` the store directory. The middle step is conditional and its result is discarded:

```
if entry.kind == "native" {
    let _ = crate::plugin_host::unload(name);
}
```

`commands.rs:95–97`. Two consequences follow that the prose chapters do not state. A script plugin never attempts an unload at all, which is correct — there is nothing to unload — but it means `remove` has no code path that touches the running shell for the kind of plugin whose effects on the running shell are the hardest to reverse. And a native unload that fails is invisible: Chapter 73’s experiment produced a `remove` that reported success while the plugin stayed loaded and `zmodload -R` kept listing it.

The script case, measured in one session:

```
$ zshrs -c 'znative load plug-script; znative remove plug-script; \
           whence -w sp_func; alias sp-hi; echo "SP_LOADED=$SP_LOADED"; \
           print -l $fpath | grep plug-script'
znative: removed plug-script
sp_func: function
sp-hi='echo script-plugin-alias'
SP_LOADED=1
/private/tmp/znative-book2/home/pkg/store/plug-script@0.0.0/functions
```

The function, the alias, and the exported variable survive, as Chapter 34 describes. The fourth line is the sharper one: `$fpath` still contains a directory that `remove` deleted two commands earlier. Chapter 34 predicts that state; this is it, printed.

The index after removing the only plugin is worth recording too, because it is what a subsequent `load` parses:

```
# znative – auto-generated. Do not edit.
version = 1
package = []
```

An empty array rather than an absent key, which round-trips through `InstalledIndex::load_from` (`store.rs:159–169`) without special-casing.

gc removes what the index does not name, and reports in units it usually cannot express

Two versions of the same plugin were installed by bumping `version` in the manifest between installs, which leaves the older store directory unreferenced:

```
$ ls -l $ZSHRS_HOME/pkg/store/
hello@0.1.0
sp@1.0.0
sp@2.0.0
$ znative gc --dry-run
znative gc: would remove sp@1.0.0 (0 KB)
znative gc: would free 0 KB total
$ znative gc
znative gc: removed sp@1.0.0 (0 KB)
znative gc: freed 0 KB total
$ ls -l $ZSHRS_HOME/pkg/store/
hello@0.1.0
sp@2.0.0
```

The orphan went, the pinned version stayed, and the dry run's prediction matched the real run exactly. The reported size did not match anything, because the arithmetic is integer:

```
(bytes + 512) / 1024
```

`commands.rs:169`. The tree it collected:

```
$ find $ZSHRS_HOME/pkg/store/sp@2.0.0 -type f -exec stat -f '%z %N' {} \;
39 .../znative.toml
18 .../functions/sp_auto
91 .../script.plugin.zsh
```

148 bytes, which rounds to 0 KB. Script plugins are routinely this size, so the common case for `gc` output is a correct list of directories with a size column reading zero. The number is not wrong; the unit is too coarse for the thing being measured. A native plugin's store copy is a different scale entirely — the store directory for the example plugin in this book carries a 385,744-byte `cdylib` — so `gc` reports meaningfully for exactly the kind of plugin whose orphans matter and uselessly for the kind that accumulates most often.

clean empties three directories, two of which never had anything in them

```
$ ls -l $ZSHRS_HOME/pkg
bin
cache
git
installed.toml
store
$ znative clean
znative clean: cleared 0 KB of scratch
$ ls -l $ZSHRS_HOME/pkg
installed.toml
store
$ znative add path:$R/tree/plugin-script >/dev/null && ls -l $ZSHRS_HOME/pkg
bin
cache
git
installed.toml
store
```

The three directories are removed and recreated by the next install's `ensure_layout` (`store.rs:78–89`), so `clean` is safe and idempotent. Two of the three, though, have no writer anywhere in the manager:

```
$ grep -rn "cache_dir()" --include='*.rs' src/extensions/pkg/
src/extensions/pkg/store.rs:81:         self.cache_dir(),
src/extensions/pkg/commands.rs:218:     for d in [store.git_dir(), store.cache_dir(),
  ~ store.bin_dir()] {
$ grep -rn "bin_dir()" --include='*.rs' src/extensions/pkg/
src/extensions/pkg/store.rs:83:         self.bin_dir(),
src/extensions/pkg/commands.rs:218:     for d in [store.git_dir(), store.cache_dir(),
  ~ store.bin_dir()] {
```

Each appears exactly twice: once where it is created and once where it is deleted. `cache/` is documented as “download scratch” (`store.rs:9`) and nothing downloads — the only fetch in the manager is `git clone` (`resolver.rs:171–194`), which writes to `git/`. `bin/` is documented as “launcher symlinks (future)” (`store.rs:12`). Half of `clean`'s work is removing empty directories so that the next install can create them again.

The clone cache is empty until an install fails

`gc`'s second branch clears `git/` unconditionally (`commands.rs:182–201`), which reads as routine house-keeping and is in fact a failure detector. A successful install of a git source deletes its own clone:

```
if staged.source.starts_with("github:") || staged.source.starts_with("git+") {
    let _ = std::fs::remove_dir_all(&staged.dir);
}
```

`commands.rs:70–73`. Measured against a local repository, so the whole exercise stays offline:

```
$ znative add git+file:///private/tmp/znative-book2/gitsrc
znative: added gitplug@1.0.0 (script)
$ ls -l $ZSHRS_HOME/pkg/git | wc -l
0
```

The clone is gone the moment the store copy exists. A source pinned with `@REF` behaves the same and records the pin in its label, which is what makes `update` reproducible (Chapter 72):

```
$ znative add git+file:///private/tmp/znative-book2/gitsrc@v1.0.0
znative: added gitplug@1.0.0 (script)
$ grep source $ZSHRS_HOME/pkg/installed.toml | grep gitsrc
source = "git+file:///private/tmp/znative-book2/gitsrc@v1.0.0"
```

Now a git source that clones successfully and then fails detection — a repository with a `README.md` and nothing else:

```
$ znative add git+file:///private/tmp/znative-book2/gitempty
znative: could not determine plugin kind: no znative.toml, no *.plugin.zsh, no functions/,
  and no Rust cdylib/Cargo.toml
$ ls -l $ZSHRS_HOME/pkg/git
gitempty
```

The clone survives, because the cleanup at `commands.rs:70–73` sits after the copy and the failure came before it. So a non-empty `git/` means “at least one install got as far as cloning and then did not finish,” and `gc` is what clears it:

```
$ znative gc --dry-run
znative gc: would remove trav@.. (0 KB)
znative gc: would clear git cache (29 KB)
znative gc: would free 29 KB total
$ znative gc
znative gc: removed trav@.. (0 KB)
znative gc: cleared git cache (29 KB)
znative gc: freed 29 KB total
```

Two details in that output. The first line collected `store/trav@..`, the empty directory Chapter 73’s traversal left behind — unpinned, inside `store/`, so `gc` reaches it even though the tree it was a prefix of landed elsewhere. And the size column is a byte sum, `dir_size` walking file lengths (`commands.rs:127–141`), not an allocation figure: `du -sk` reports 128 KB for the same clone because a git object store is thousands of small files, each occupying a block. Both numbers are correct about different questions; `gc`’s is the one that tells you how much content is there.

The three states `gc` cannot reach

`gc` scans one directory — `store.rs:52–54`’s `store/` — and removes entries whose `name@version` is not pinned by the index (`commands.rs:159–180`). Three kinds of debris fall outside that scan.

An **orphaned row** — an index entry whose upstream renamed itself, so the row points at a directory nobody will load again. Chapter 72 describes how it arises. `gc` cannot remove it because `gc` removes directories, and this one is named.

An **escaped tree** — anything Chapter 73’s traversal put outside `store/`. The scan does not look there, and the pinned-set test would not match if it did.

A **half-written install** — `install_dir` clears the destination and copies into it (`store.rs:100–108`) with no temporary directory and no atomic rename, so an interrupted install leaves a partial tree at a real store path. Here `gc` is the right tool and it works, because the index write happens after the copy (`commands.rs:66–68`), so a partial tree is unpinned. Appendix M’s “interrupted install” row is accurate. It is also the one repair path in the manager that depends on an ordering nothing enforces, and the manager has no locking of any kind — a `grep` for `flock` or `\.lock` across `src/extensions/pkg/` returns nothing, which Chapter 69 reads in full. Two shells, one installing and one collecting, can interleave those steps in an order where `gc` deletes a tree the other is about to record.

The measured summary

Command	Removes	Leaves
<code>remove</code> (native)	index row, store dir, registrations <i>if the names match</i>	the mapped library if they do not; anything the plugin wrote outside the store
<code>remove</code> (script)	index row, store dir	every function, alias, option, hook, and variable the source defined; a dangling <code>\$fpath</code> entry
<code>gc</code>	unpinned <code>store/</code> dirs, the whole <code>git/</code> clone cache	orphaned rows, escaped trees, everything outside <code>store/</code>
<code>clean</code>	<code>git/</code> , <code>cache/</code> , <code>bin/</code>	the store and the index, untouched by design

The pattern across all four rows is the same one Chapter 34 states for `remove` alone: these commands manage *files the manager wrote*, accurately, and they do not manage the shell. Reading them as an uninstall in the system-package sense is the mistake, and the `fpath` line above is the cheapest way to see it.

Chapter 78 — What a Registry Would Have to Provide

Chapter 31 places `znative` in the landscape and Chapter 46 lists the deliberate omissions, of which a registry is one. Those chapters argue the position. This one takes the opposite tack and asks the engineering question: if the decision were reversed tomorrow, what would have to be built? The answer is useful even to a reader who agrees with the omission, because it makes visible which parts of the current design are registry-shaped already and which parts would have to change.

What resolution does today

The whole of source resolution is a mapping from a string to a git clone. Four forms are accepted (`resolver.rs:138–168`), three of them produce a URL and one produces a local directory, and the fetch is:

```
let mut cmd = Command::new("git");
cmd.arg("clone").arg("--depth").arg("1");
if let Some(r) = git_ref {
    cmd.arg("--branch").arg(r);
}
```

`resolver.rs:172–176`. There is no version negotiation, because there are no versions in the request — `@REF` is passed to git verbatim, and git decides what it means. There is no dependency graph, because a plugin cannot declare a dependency: the manifest schema has three tables and none of them has a dependency field (`manifest.rs:34–83`). There is no index to consult, no metadata to download, and no artifact to fetch: the only network operation the manager performs is `git clone`.

That is the baseline. Everything below is what a registry would have to add to it.

The name authority

Today a plugin's name comes from its own manifest, and the store keys on it:

```
let name = manifest
    .as_ref()
    .map(|m| m.plugin.name.clone())
    .filter(|n| !n.is_empty())
    .unwrap_or_else(|| staged.name.clone());
```

`commands.rs:19–23`. Two unrelated repositories can both call themselves `forgit`, and the second one to install wins the store key and replaces the first's index row (`store.rs:194–200`). Chapter 63 counts the fleet's command-name collisions in the shell's namespace; this is the same problem one level up, in the store's namespace.

A registry's first job is to make a name mean one thing. That requires an authority that can refuse a registration, which requires accounts, which requires an identity system — and the moment that exists, the trust model of Chapter 29 changes shape, because “the source you named” acquires a second meaning. This is the step that cannot be built incrementally: a name authority is either binding or it is decoration.

Version resolution, which does not exist at any layer

The `version` field is a string that the plugin asserts and the manager interpolates into a directory name. It is never parsed, compared, or ordered. `update` re-resolves the recorded label and reinstalls whatever it finds (`commands.rs:309–324`); it performs no comparison because it has nothing to compare — there is no way to ask a source what versions it has.

A registry would have to supply, in order: a version grammar, an ordering over it, a request language (`^1.2`, `~1.2.3`, or whatever), a resolver that maps a request to a concrete version, and a record of what was resolved so the result is reproducible. That last item is a lockfile, which Chapter 46 lists as a deliberate omission of the global model — so a registry does not add one feature, it adds a feature that reintroduces the file the design removed.

It is worth noting how far the current code is from needing any of it: `version` is compared with `==` in exactly one place, inside the store-key format string, and nowhere else in the manager.

Provenance, which the current design explicitly declines

Chapter 29 states why there is no signing: a signature needs an authority. A registry *is* an authority, so the argument that closes the question today opens it again. The mechanical requirements are a key distribution channel, a signature over something stable, and a verification point in the install pipeline.

The second of those is where the current design would have to change most. The integrity hash covers the store copy after the `.git/` and `target/` exclusions and after a native build has already run (`commands.rs:42–43`), which makes it a hash of a *derived* artifact — as Chapter 71 notes, its value differs between architectures for the same source. A signature has to cover something the publisher produced, not something the installer derived, so a registry needs a second fingerprint taken upstream of the build. That is a new field, a new computation point, and a verification step in a pipeline that currently has none.

Artifact hosting, and the two empty directories

A registry that serves only source is a slower `git clone` with a namespace. The value most package registries deliver is prebuilt artifacts, and that is where the store layout is already shaped for something it does not do:

```
$ZSHRS_HOME/pkg/
store/  name@version/  # one extracted copy per (name, version)
cache/                                     # download scratch
git/                                         # bare/working git clones
bin/                                         # launcher symlinks (future)
```

`store.rs:6–13`. As Chapter 77 measures, `cache/` and `bin/` are created and deleted and never written — `cache_dir()` and `bin_dir()` appear exactly twice each across the whole manager, once in `ensure_layout` and once in `clean`. The download scratch has no downloader. The layout anticipates a fetch-and-extract flow that was never built, which is a useful signal: adding artifact hosting would not require a new store shape, only a writer for the directory that is already there.

Serving artifacts also forces the Chapter 75 problem to the surface immediately. An artifact is per-target; the store key and the `lib` field are not. A registry that served binaries would have to fix the target-keying first, or serve exactly one platform.

Revocation, mirroring, and the offline case

Three smaller requirements round out the list, and each one has a concrete anchor in the current design.

Revocation needs something to revoke against. The index records `source`, `version`, and `integrity` per plugin (`store.rs:127–148`), which is enough to answer “do I have the affected version” *if* something reads it. There is no verb that reads it in bulk and no verb that reads it for verification at all, as Chapter 68 establishes. A yank notification with no `verify` and no `audit` on the receiving side is a notification with nowhere to land.

Mirroring is the easy one and is already expressible: `git+URL` accepts any reachable URL (`resolver.rs:139–141`), and “any URL” means anything git will clone, including a path. An entirely of-line install, from a bare local repository with no network in the loop:

```
$ znative add git+file:///private/tmp/znative-book2/gitsrc@v1.0.0
znative: added gitplug@1.0.0 (script)
$ grep source $ZSHRS_HOME/pkg/installed.toml | grep gitsrc
source = "git+file:///private/tmp/znative-book2/gitsrc@v1.0.0"
```

The label round-trips through `source_to_spec` unchanged (`commands.rs:327–334`), so `update` re-fetches from the mirror and honours the tag. An internal mirror is therefore a spec change and not a code change — which is worth stating precisely because it is the one registry-adjacent capability an organisation usually wants first, and it is already there.

The offline case is the one a registry must not break. The current design’s strongest operational property is that `znative load` after the first install touches no network at all — it reads `installed.toml` and the store (`commands.rs:265–306`). Every registry feature above adds a network dependency to *install*, and none of them may add one to *load*, or the property that makes the `.zshrc` idiom viable is gone. That constraint is what makes the current design’s answer coherent: the manager pushed every network operation into an explicit, user-initiated command, and a registry is a large amount of machinery whose entire benefit lands on the side of the line the design deliberately keeps small.

The summary a decision would need

Registry capability	What exists today	What it would take
unique names	manifest self-assertion (<code>commands.rs:19–23</code>)	an authority that can refuse a registration
version selection	a string in a directory name	grammar, ordering, request syntax, resolver, lockfile
provenance	none, by doctrine (Chapter 29)	keys, an upstream fingerprint, a verification step
prebuilt artifacts	an empty <code>cache/</code> and an empty <code>bin/</code>	a downloader, plus target-keyed store paths (Chapter 75)
revocation	three index fields nothing reads	a <code>verify/audit</code> verb first
mirroring	<code>git+URL</code>	nothing
offline load	already guaranteed	must survive all of the above

Six of the seven rows are new subsystems. The seventh is the constraint they all have to respect. That ratio is the actual argument for the omission, and it is stronger stated as engineering cost than as doctrine.

Chapter 79 – The Failure Taxonomy by Durable State

Chapter 25 catalogues the failures a user meets and Chapter 66 censuses every error string in the manager with its construction site. Both organise by *message*. This chapter organises by the question a user actually has after a failure: what is on disk now, what is in this process now, and which command fixes it. The messages below were produced by running the failing commands against the scratch store; the state columns were verified by inspecting the store, the index, and the process afterwards.

The exit convention, confirmed

Every failure the manager reports converges on one shape — a single line on stderr, prefixed, with status 1 ([builtin.rs:92–98](#)):

```
$ zshrs -c 'znative info ghost; echo rc=$?'
znative: ghost is not installed
rc=1
$ zshrs -c 'znative frobnicate'
znative: unknown command 'frobnicate'
usage: znative <command> [args]
```

The host's failures come out through the `zmodload` builtin instead and carry the shell's own prefix, also with status 1:

```
$ zshrs -c "zmodload -R $R/notalib.dylib; echo rc=\$?"
zshrs:zmodload:1: cannot load `/private/tmp/znative-book2/notalib.dylib`: dlopen(...) (slice
↳ is not valid mach-o file)
rc=1
$ zshrs -c "zmodload -uR ghost; echo rc=\$?"
zshrs:zmodload:1: plugin `ghost` not loaded
rc=1
```

Failures that leave nothing

These are the ones that fail before touching the store, and they are the majority by count. The pipeline's ordering is what makes it so: resolve, load the manifest, detect the kind, build, and only then copy ([commands.rs:13–43](#)).

Command	Message	State after
znative add path:<missing>	path /.../nope-dir: No such file or directory (os error 2)	nothing; resolve failed first

Command	Message	State after
<code>znative add <empty dir></code>	could not determine plugin kind: no znative.toml, no *.plugin.zsh, no functions/, and no Rust cdylib/Cargo.toml	nothing; detection precedes the copy
<code>znative add "weird spec"</code>	unrecognized source 'weird spec': expected owner/repo, github:owner/repo, git+URL, or a local path	nothing
<code>znative info\ remove\ load\ update ghost</code>	ghost is not installed	nothing; index untouched
<code>zmodload -R <not a dylib></code>	cannot load ...: dlopen(...)	nothing; <code>dlopen</code> failed
<code>zmodload -R <cdylib, no init></code>	not a zshrs plugin (no znative_init)	nothing; symbol lookup failed

The `dlopen` failures deserve one note: the library is closed on the error path because `libloading::Library` is dropped when `load` returns `Err` (`plugin_host.rs:377–389`), so a failed load leaves no mapping behind.

Failures that leave a partial store and no index row

One class of failure happens after the copy and before the index write. The window is four lines wide (`commands.rs:42–68`), and the only failure inside it is a native plugin whose store copy contains no host-shaped library:

```
znative: hello: no cdylib after build
```

`commands.rs:57`. The store directory exists, fully populated, and no row names it. This is the state Appendix M calls “interrupted install,” and `gc` is the repair — verified in Chapter 77, where an unpinned directory was collected correctly.

Failures that leave a clone

There is a second post-fetch, pre-copy window, and it is the wider one: everything between the clone and the store copy — manifest parse, kind detection, and the whole native build — happens with a working tree sitting in `pkg/git/<name>/`, and the cleanup that removes it runs only after a successful install (`commands.rs:70–73`). Any failure in between leaves it:

```
$ znative add git+file:///private/tmp/znative-book2/gitempty
znative: could not determine plugin kind: no znative.toml, no *.plugin.zsh, no functions/,
  and no Rust cdylib/Cargo.toml
$ ls -l $ZSHRS_HOME/pkg/git
gitempty
```

The state is harmless — nothing loads out of `git/`, and the next install of the same source clears the directory before re-cloning (`resolver.rs:59–62`) — but it is durable, it accumulates one directory per distinct failing

source, and a git clone is the largest thing the manager writes. `znative clean` and `znative gc` both clear it; nothing else does.

Failures that leave a complete, correct install

This is the class the message shape hides, and Chapter 72 reads its main instance in full. Restated here as state rather than as narrative, because it is the failure most likely to be misread:

```
$ zshrs -c 'znative load hello; znative update hello; echo "exit: $?"'
znative: plugin `hello` already loaded
exit: 1
```

Everything durable succeeded. The proof is the next shell, which loads the store as it now stands:

```
$ zshrs -c 'znative load hello; rhello y'
HELLO-V3, y - from a native Rust plugin (...)
```

The source had been edited to print `HELLO-V3` before the failing update. So a command that exited 1 replaced the store copy, recomputed the hash, and rewrote the index row; the only step that failed was the in-process load, which could not have succeeded. The repair is not a repair — it is a new shell, or `zmodload -uR <name>` followed by `znative load <name>`.

The same class has a second member with a wider blast radius. `update` with no argument iterates with `add(&spec)?` (`commands.rs:316–322`), so the first failure aborts the run:

```
$ zshrs -c 'znative load hello; znative update'
znative: plugin `hello` already loaded
$ grep -n 'sp_func' $ZSHRS_HOME/pkg/store/plugin-script@0.0.0/script.plugin.zsh
2:sp_func() { echo "sp_func v1"; }
$ grep -n 'sp_func' $R/tree/plugin-script/script.plugin.zsh
2:sp_func() { echo "sp_func v2"; }
```

The index is sorted by name (`store.rs:173`), `hello` precedes `plugin-script`, the first target failed, and the second was never attempted — its store copy is still the old content while its source has moved on. One loaded native plugin is enough to make `znative update` silently skip every plugin alphabetically after it, and the message names only the first.

Failures that report success

Two paths complete with status 0 and leave state that does not match what the user asked for. Both have been read elsewhere in this pass; they belong in a taxonomy of failure because status 0 is what the user sees.

The first is the stale-artifact reinstall of Chapter 71. With a previously staged `lib*.dylib` at the root of a `path:` tree, `prepare_native` returns immediately (`commands.rs:379–382`) and the build never runs. Measured, the difference is visible only in the timing:

```
first install, builds:      3.220s total
second install, no build:   0.319s total
```

both printing the same success line. The store afterwards is internally inconsistent — its `src/` has the new code and its library does not:

```
$ grep -c 'HELLO-V2' $ZSHRS_HOME/pkg/store/hello@0.1.0/src/lib.rs
1
$ strings $ZSHRS_HOME/pkg/store/hello@0.1.0/libhello.dylib | grep -c 'HELLO-V2'
0
```

And the integrity hash moved — from `sha256-2336924a...` to `sha256-a2abf736...` — because the source file

changed, which is a precise illustration of what the hash covers: the recorded fingerprint tracked a file that is not the file that gets loaded.

The second is the manifest traversal of Chapter 73, which reports `znative: added trav@../../../escaped (script)` with status 0 and writes outside the store.

Failures that are silent

One path drops content with no diagnostic at any status. `copy_dir_filtered` handles a symlink by reading through it and writing the bytes, and swallows the error when that read fails (`store.rs:225–230`). A plugin tree carrying three symlinks — one to a directory, one to a file, one dangling — installs cleanly, and the store afterwards contains:

```
$ find $ZSHRS_HOME/pkg/store -type l | wc -l
0
```

The directory symlink and the dangling symlink are simply absent; the file symlink became a regular file with the target's content. Chapter 68 reads the consequence for the integrity grammar. The taxonomy entry is that this is the only failure in the manager with no message, no status, and no record — the install is reported as successful and the store is missing something the source had.

There is a mirror-image case in the same branch worth recording, because it moves content in the other direction. A symlink pointing *outside* the plugin tree has its target's content copied *into* the store:

```
$ ln -s $R/outside-secret.txt $R/tree/plugin-links/pulled.txt
$ znative add path:$R/tree/plugin-links >/dev/null
$ cat $ZSHRS_HOME/pkg/store/plugin-links@0.0.0/pulled.txt
secret-outside-the-plugin-tree
```

The store is documented as holding the plugin's loadable subset. A link in the source tree is sufficient to put any file the installing user can read into it, under a name the plugin chose, hashed into the integrity field as if it were part of the plugin.

The taxonomy, condensed

Class	Example	Status	Durable state	Repair
pre-copy	bad spec, missing path, unknown kind	1	none	fix the input
post-clone, pre-copy	detection or build failure on a git source	1	a clone under <code>pkg/git/</code>	<code>znative clean</code> or <code>gc</code>
post-copy, pre-index	<code>no cdylib after build</code>	1	unpinned store dir	<code>znative gc</code>
post-index, load refused	<code>plugin ... already loaded</code>	1	complete install	new shell, or unload + load
aborted multi-target	<code>update</code> with no argument	1	first target done, rest untouched	rerun after the blocker
stale artifact	<code>path:</code> reinstall with a staged library	0	store's source and library disagree	<code>rm -f lib*.dylib</code> then reinstall

Class	Example	Status	Durable state	Repair
escaped write	manifest <code>version</code> with <code>..</code>	0	tree outside <code>store/</code>	<code>znative remove</code> , which deletes it — and more
silent drop	directory or dangling symlink	0	store missing source content	none available

The three rows with status 0 are the ones a user cannot act on, because nothing told them there was anything to act on. That is the argument for the three string-level checks Chapter 73 prices, and for the one diagnostic `copy_dir_filtered` does not emit.

Chapter 80 – Two Managers From One Port

`znative` has a sibling. `zmax` — the editor in the same fleet — carries a native plugin SDK and a package manager that were ported from the ones this book describes, and reading the two side by side is the most direct evidence available for which parts of this design are essential and which are incidental. A design’s portability across a different host is a fact about the design, and this one has been tested exactly once.

The lineage, stated by the source

The `zmax` manager does not merely resemble `znative`; parts of it are the same file. Its resolver’s header says so:

```
#!/ Ported from zshrs's `pkg/resolver.rs` (unchanged – source resolution is
#!/ plugin-kind-agnostic).
```

`zmax-term/src/commands/plugmgr/resolver.rs:2–3`. Dified against the original, the claim holds:

```
$ diff zshrs/src/extensions/pkg/resolver.rs \
      zmax/zmax-term/src/commands/plugmgr/resolver.rs | grep -c '^[<>]'
22
```

Twenty-two changed lines across 285, and every one of them is a doc comment naming a different tool or a different default path — except a single body change, which rewrites `if git_ref.is_some() { ... git_ref.unwrap() }` as `if let Some(r)`. Source resolution transferred to a text editor without a semantic edit. `znative` itself was ported from the stryke dependency manager (Chapter 32), so this is the second hop of the same lineage.

The sizes

Component	zshrs	zmax
manager	1,513 lines (<code>src/extensions/pkg/*.rs</code>)	1,385 lines (<code>plugmgr/*.rs</code>)
host	699 lines (<code>plugin_host.rs</code>)	430 lines (<code>commands/plugin.rs</code>)
SDK	548 lines (<code>znative/src/lib.rs</code>)	331 lines (<code>zmax-native/src/lib.rs</code>)
<code>HostApi</code>	14 fields, 12 callbacks	9 fields, 7 callbacks
<code>ABI_VERSION</code>	4	1
manager tests	7	9

The manager is nearly the same size on both sides; the host and the SDK are not, and the difference is entirely the completion system. Five of `znative`’s twelve callbacks exist to serve compsys — `register_completion`,

`register_compf``n`, `comp_dispatch`, `empty_command_hash`, and the function pair that a completion override needs. An editor has no compsys, so those disappear, and with them roughly a third of the host.

What the editor dropped, and what that reveals

The script kind is gone. `zmax`'s index records `kind` as a field whose only value is `"native"` — its own comment says “`zmax` has a single plugin kind” (`plugmgr/store.rs:126`), and its `mod.rs:8` explains why: there is “no shell to `source` into.” Everything Chapter 43 says about one kind per plugin, and everything Chapter 34 says about the script path being irreversible, applies to exactly half of `znative` and to none of its sibling. The two-kind design is a consequence of the host being a shell, not of the manager being a manager.

`println!` is gone. Every `zmax` manager verb returns `PkgResult<String>` instead of printing — `pub fn add(spec: &str) -> PkgResult<String>` (`plugmgr/commands.rs:20`) — and the caller routes the string to the editor's status line (`typed.rs:57583–57585`). The comment at the call site states the constraint: “a `println!` here would corrupt the TUI, so `pkg` never prints.” `znative`'s `commands.rs` prints directly, in twenty places. Which of the two is better factored is not in question; the interesting part is that the shell's freedom to print is the reason its manager never had to separate computing a result from reporting it.

`has_package` is gone (present at `store.rs:91–94` in `znative`, absent in the port), which is a small confirmation of Chapter 52's finding about unused surface: an unused helper does not survive a port.

What the editor renamed, and the naming collision it inherited

`zmax` exposes the manager under `:plugin`, alongside the raw host verbs, and the result is a namespace where two verbs mean different things than they do here:

zmax verb	Meaning	znative equivalent
<code>:plugin add\\ install\\ i</code>	install from a source	<code>znative add</code>
<code>:plugin get\\ ensure</code>	install-on-first-use, then load	<code>znative load <spec></code>
<code>:plugin sync</code>	load everything in the index	<code>znative load</code>
<code>:plugin registry\\ installed</code>	list the index	<code>znative list</code>
<code>:plugin load</code>	dlopen a path	<code>zmodload -R <path></code>
<code>:plugin unload</code>	drop a loaded plugin	<code>zmodload -uR <name></code>

`typed.rs:57576–57690`. In `znative`, `load` is the manager verb and the raw `dlopen` lives on a different builtin entirely; in `zmax` the two share a prefix and `load` went to the raw one, so the manager's equivalent had to be renamed to `get`. The shell's separation of `znative` from `zmodload -R` — two builtins, two audiences — turns out to be doing more work than it appears: it is what keeps the manager's most-used verb from colliding with the host's.

The three defects travelled with the port

The sharpest finding from reading the two together is that everything this pass found in `znative`'s install path is present, unchanged, in the port.

The integrity hash is computed once at `plugmgr/commands.rs:48`, stored, printed by `info` at `:218`, and compared by nothing — the same one-call-site shape Chapter 68 establishes for the original.

`load_entry` joins the recorded `lib` onto the store directory (`plugmgr/commands.rs:320–322`), so an absolute `lib` escapes the store exactly as Chapter 73 demonstrates here.

The build stages its output into the source tree (`plugmgr/commands.rs:371–372`, `stage cdylib`), and `prepare_native` short-circuits on a library already at the root (`:332–334`), reproducing the stale-binary reinstall of Chapter 71.

And `add` writes the index before it loads (`:62` then `:69`), while the host refuses a duplicate name (`commands/plugin.rs:310–312`) — so `zmax`'s `plugin update` on a loaded plugin reports failure after a successful install, precisely as Chapter 72 describes for `znative`.

Four behaviours, four ports, zero divergence. That is what a faithful port buys and what it costs: the sibling inherited the design's correctness and its defects with equal fidelity, and a fix in one is now a fix that has to be made twice.

What the comparison settles

Three questions this book raises elsewhere get an empirical answer from having a second implementation.

Is the store layout shell-specific? No. `store/`, `cache/`, `git/`, `bin/`, and `installed.toml` transferred unchanged, including the two directories that have no writer — `zmax` created and cleans an empty `cache/` and an empty `bin/` too. The layout is a property of the design, and so is its unused half.

Is the two-kind design essential? No. It is the shell's half of the manager, and it deleted cleanly.

Is the ABI shape general? Yes, within one fleet. The same one-symbol entry point, the same `#[repr(C)]` table passed by pointer, the same trampoline macro, the same two-sided version check, and the same `free_cstring` ownership rule appear in `zmax-native` with a different callback list and a different symbol name (`zmax_native_init`, `zmax-native/src/lib.rs:57`). What varies between the two is exactly the set of things the host can do; what does not vary is the mechanism for letting a stranger call them.

Appendix A – Command Reference

Command	Aliases	What it does
<code>znative load [SOURCE...]</code>	<code>source</code>	Load a plugin. If SOURCE is present in the store, load with zero network; if absent, install then load. With no argument , load everything already in the store. The verb a <code>.zshrc</code> calls.
<code>znative add SOURCE...</code>	<code>install, i</code>	Resolve, install into the store, record in the index, and load — without needing a <code>.zshrc</code> line. The interactive install.
<code>znative remove NAME...</code>	<code>rm, uninstall</code>	Unload a native plugin (<code>zmodload -uR</code>), delete its store copy, and drop its index row.
<code>znative list</code>	<code>ls</code>	Print one line per installed plugin: name, version, kind, source. Reads the global index.
<code>znative info NAME</code>	<code>show</code>	Show the full record for one plugin: store path, integrity hash, and the lib (native) or source files + <code>fpath</code> (script).
<code>znative update [NAME]</code>	<code>upgrade, up</code>	Re-resolve and reinstall from the recorded source, respecting any pin. One named plugin, or all with no argument.
<code>znative gc [--dry-run]</code>	<code>-n (for --dry-run)</code>	Remove <code>store/<name>@<version>/</code> dirs the index no longer pins (orphans from <code>update</code>) and clear the <code>git/</code> clone cache. <code>--dry-run</code> lists without deleting.
<code>znative clean</code>	—	Clear the scratch directories (<code>git/</code> , <code>cache/</code> , <code>bin/</code>); the store and index are untouched.
<code>znative help</code>	<code>-h, --help</code>	Usage.

Error contract: failures print `znative: <reason>` on stderr and return non-zero. `znative` needs `git` on `PATH` for remote sources and `cargo` for native plugins that ship as source.

Appendix B – Source-Spec Grammar

Form	Classified as	Network	Notes
<code>owner/repo</code>	GitHub repository	yes	The common form; bare two-part slug, no scheme.
<code>github:owner/repo</code>	GitHub repository	yes	The same, said explicitly.
<code>git+URL</code> , or a URL ending <code>.git</code> or containing <code>://</code>	arbitrary git URL	yes	Any git host — GitLab, self-hosted, mirror.
<code>path:DIR</code>	local directory	no	Install from disk; the development / offline path.
absolute <code>/ ./ / ../ / ~</code> path	local directory	no	Same as <code>path:</code> , recognised by shape.

Any remote form may carry an `@ref` suffix to pin a tag, branch, or commit: `owner/repo@v1.2.0`, `github:owner/repo@main`, `git+https://host/x.git@<sha>`. The pin is recorded and honoured by `znative update`.

Clone strategy: shallow by default (`--depth 1 --branch REF`); full-clone fallback only when a shallow branch clone cannot reach a requested commit.

Appendix C – Store Layout

Root: `$ZSHRS_HOME/pkg/` (default `~/.zshrs/pkg/`). One store per user; global only.

Path	Role
<code>store/<name>@<version>/</code>	The installed plugin tree, stripped of <code>.git/</code> and <code>target/</code> .
<code>installed.toml</code>	The global index and source of truth — one <code>[[package]]</code> record per plugin.
<code>git/</code>	Internal scratch — remote clones land here, then copy to <code>store/</code> .
<code>cache/</code>	Internal scratch — fetch caches.
<code>bin/</code>	Internal scratch — helper binaries.

Each `[[package]]` record carries: `name`, `version`, `source` (the recorded spec, pin included), `kind`, `integrity` (`sha256-<hex>`), and either `lib` (the `cdylib` for a native plugin) or `source_files` + `fpath` (for a script plugin):

```
[[package]]
name = "revolver"
version = "0.2.0"
source = "github:MenkeTechnologies/zshrs-revolver"
kind = "native"
integrity = "sha256-..."
lib = "librevolver.dylib"
# script plugins record instead:
# source_files = ["...plugin.zsh"]
# fpath = ["functions"]
```

There is no per-project manifest and no lockfile. The `sha256-<hex>` hash in the index is the per-record integrity check. `znative gc` removes `store/` dirs the index no longer names; `znative clean` empties the three scratch dirs.

Appendix D — znative.toml Schema

An optional manifest at a plugin's root. Overrides auto-detection where present.

```
[plugin]
name      = "my-plugin"
version    = "1.0.0"
description = "one-line description"

# exactly one of the two kind tables:

[native]
lib      = "myplugin"    # lib stem: loads libmyplugin.{dylib,so}
build    = true          # default true when a Cargo.toml is present

# - or -

[script]
source    = ["my-plugin.zsh", "extra.zsh"] # files to source, in order
fpath     = ["functions"]                 # dirs to prepend to $fpath
```

- **[plugin]** — identity: **name** and **version** set the **store/<name>@<version>/** key; **description** is the human-readable line.
- **[native]** — a compiled plugin. **lib** names the **cdylib** stem to load; **build** (default **true** with a **Cargo.toml**) says whether to **cargo build --release** before loading.
- **[script]** — a script plugin. **source** lists files to **source**; **fpath** lists directories to prepend to **\$fpath**.

Auto-detection (Chapter 7) covers the common cases; the manifest is the explicit override for unusual layouts or pinned metadata.

Appendix E — Host API Reference

The callback table a handler receives as `Host`, plus the `Args` decoder. All of it is `#[repr(C)]` across the boundary and provided by the `znative` crate.

Method	Signature	Purpose
<code>print</code>	<code>host.print(&str)</code>	Write to the shell's stdout.
<code>eval</code>	<code>host.eval(&str) -> i32</code>	Run shell code; return its exit status.
<code>getvar</code>	<code>host.getvar(&str) -> Option<String></code>	Read a shell scalar parameter.
<code>setvar</code>	<code>host.setvar(&str, &str)</code>	Set a shell scalar parameter.
<code>getfunction</code>	<code>host.getfunction(&str) -> Option<String></code>	Read a function's deparsed body (<code>\${functions[name]}</code>) — ABI v3.
<code>addfunction</code>	<code>host.addfunction(&str, &str) -> bool</code>	Define/replace a function (<code>functions[name]=body</code>) — ABI v3.
<code>register_builtin</code>	<code>host.register_builtin(&str, f)</code>	Register a command handler at runtime.
<code>add_match</code>	<code>host.add_match(&str)</code>	Emit one completion candidate.
<code>install_completion</code>	<code>host.install_completion(&str, gen)</code>	Wire a native completion into compsys.

`Args`: `.name()` → `argv[0]`; `.rest()` → the arguments as `&[String]`; `.to_vec()` → the whole vector. A handler returns the command's exit status (`c_int`, `0` = success), like a shell builtin.

`getfunction/addfunction` are the only structured access to shell *functions* (`getvar` is scalars only; `eval` returns just a status). Round-tripping a body through `addfunction` then `getfunction` normalises it through the shell's own parser and pretty-printer — deparse-as-a-service.

Appendix F — zmodload -R Reference

The low-level plugin-loading primitive under `znative`. Loading and unloading a plugin by hand, without the package manager.

Command	What it does
<code>zmodload -R <path>...</code>	Load each <code>cdylib</code> : <code>dlopen</code> , resolve <code>znative_init</code> , register its builtins.
<code>zmodload -R</code>	List loaded plugins — <code>name version path</code> , one per line.
<code>zmodload -uR <name>...</code>	Unload each plugin by its registered name; purges the registry before <code>dlclose</code> .
<code>zmodload -A -R <name></code>	The C-zsh meaning: remove a module alias. Unchanged — full zsh parity.

Notes: `-R` without `-A` is the zshrs extension (in C zsh, `-R` is only meaningful with `-A`). Loading a name already loaded is refused — unload first. The single exported symbol every plugin provides is `znative_init`, generated by `declare_plugin!`. Plugin commands resolve after real builtins and functions, before `PATH`.

Appendix G – Porting Cheat-Sheet

The zsh-construct → Rust-plugin mapping, condensed from Chapter 16.

zsh plugin construct	Rust plugin equivalent
<code>foo() { ... }</code>	a builtin <code>fn foo(&Host, &Args) -> c_int</code>
<code>alias g='foo'</code>	register under the alias: <code>"g" => foo</code>
<code>compdef _foo foo+_foo()</code>	a <code>completions: { "g" => gen }</code> entry
<code>\$1 \$2 "\$@"</code>	<code>args.rest()</code> (<code>&[String]</code>)
<code>\$VAR</code>	<code>host.getvar("VAR")</code>
<code>echo / print</code> calling <code>git/fzf/fd/rg</code>	<code>host.print(...)</code> , or a subprocess inheriting stdout <code>std::process::Command</code>
shell code you keep	<code>host.eval("...")</code> Rust <code>str</code> methods and iterators
<code>sed/awk/grep</code>	

Gotchas: register under the alias name, not the internal function name; keep the same runtime deps (`git`, `fzf`, ...) — `Command::new` fails cleanly if absent; inherit the tty for interactive children (`fzf`, `less`, editors); no leading underscore on a builtin name (zsh treats `_*` as autoloadable completers); do not over-port — shell out to `git/fzf/pagers`, port only the glue.

Appendix H – The Doctrine

`znative` obeys the same shell doctrine as the rest of zshrs. The relevant rules for a package manager:

- **No startup banner, no version stripe, no first-run message.** Installing on first shell start is silent to the terminal.
- **No init/cache progress to the terminal.** Clone, build, and store work do not narrate themselves on screen; the shell's prompt is not delayed by chatter.
- **No deprecation nags, no “did you mean,” no tips.** The tool does not editorial on your choices.
- **Errors on stderr, terse.** `znative: <reason>`, non-zero exit. State the error, exit — no friendly explanation, no “run `–help` for more.”
- **First paint equals full functionality.** The native path is not deferred behind a “loading in the background” trick; when a plugin is loaded, it is loaded — the same doctrine that forbids instant-prompt fakery elsewhere in zshrs applies to how plugins come up here.

The manager is small on purpose, silent on purpose, and additive on purpose. It extends the plugin workflow every zsh user already has to compiled native code, and it does so without a second tool, a second mental model, or a single line of chatter the user did not ask for.

Appendix I — Inside the `znative` Crate

The ABI crate is small enough to hold in your head, and worth doing so, because it is the entire contract between a stranger's plugin and the shell. Everything below is `#[repr(C)]` where it crosses the boundary and safe-Rust convenience where it does not.

The two constants

```
pub const ABI_VERSION: u32 = 3;
pub const INIT_SYMBOL: &[u8] = b"znative_init\0";
```

`ABI_VERSION` is the version gate (Chapter 15). `INIT_SYMBOL` is the one symbol the host resolves with `dlsym` after `dlopen` — the NUL-terminated name `znative_init` — and its value being a `&[u8]` with an explicit trailing `\0` is the small detail that lets the host pass it straight to `dlsym` without re-allocating.

The two function-pointer types

```
pub type BuiltinFn =
    extern "C" fn(host: *const HostApi, argc: usize, argv: *const *const c_char) -> c_int;

pub type InitFn =
    extern "C" fn(host: *const HostApi) -> *const PluginInfo;
```

`BuiltinFn` is a command handler at the raw C level: `argv` is NUL-terminated C strings, `argv[0]` the command name, valid only for the duration of the call (copy anything you keep). It returns an exit status, `0` for success. `InitFn` is `znative_init`'s type: called exactly once right after load, it registers the plugin's builtins and returns a pointer to a `'static PluginInfo` (or null on failure). A plugin author writes neither signature by hand — `declare_plugin!` emits both — but they are the shape of the boundary.

HostApi – the callback table

```
#[repr(C)]
pub struct HostApi {
    pub abi_version: u32,           // checked by the plugin before trusting the rest
    pub ctx: *mut c_void,          // reserved for the host; opaque to plugins
    pub register_builtin: extern "C" fn(*const HostApi, *const c_char, BuiltinFn) ->
        c_int,
    pub print: extern "C" fn(*const HostApi, *const c_char),
    pub eval: extern "C" fn(*const HostApi, *const c_char) -> c_int,
    pub getvar: extern "C" fn(*const HostApi, *const c_char) -> *mut c_char,
    pub setvar: extern "C" fn(*const HostApi, *const c_char, *const c_char) ->
        c_int,
    pub free_cstring: extern "C" fn(*const HostApi, *mut c_char),
    pub register_completion: extern "C" fn(*const HostApi, *const c_char, *const c_char)
        -> c_int, // v2
    pub getfunction: extern "C" fn(*const HostApi, *const c_char) -> *mut c_char,
        // v3
    pub addfunction: extern "C" fn(*const HostApi, *const c_char, *const c_char) ->
        c_int, // v3
}
```

Every field is a C-ABI function pointer *into* `zshrs`. One instance lives for the whole process, and a plugin may store the `*const HostApi` it is given and call through it from any builtin. The `abi_version`-first layout is deliberate: it is the field the plugin checks before it trusts any of the pointers after it. `getvar` and `getfunction` return freshly allocated C strings the caller must release with `free_cstring` (or null for “unset”/“no such function”); the `Host` wrapper hides that. The version comments mark exactly when the surface grew: `register_completion` at v2, `getfunction/addfunction` at v3.

PluginInfo – what a plugin returns

```
#[repr(C)]
pub struct PluginInfo {
    pub abi_version: u32,           // redundant with the host-side check, on purpose
    pub name: *const c_char,       // used by `zmodload -R` listing and `-uR <name>`
    pub version: *const c_char,    // informational
}
```

The strings are `'static` (string literals from the macro). The redundant `abi_version` here is the plugin’s half of the two-sided version check: the host can reject a plugin that lied about its ABI. `PluginInfo` is only ever pointed at `'static` data with no interior mutability, so the crate marks it `Sync` to let the macro place it in a `static`.

The wrappers, and the macro

`Host` is a safe wrapper over `*const HostApi` (`Host::from_raw` is the one `unsafe` seam, asserting the pointer is the valid table the host passed in); its methods — `print`, `eval`, `getvar`, `setvar`, `register_builtin`, `add_match`, `install_completion`, `getfunction`, `addfunction` — are the ergonomic surface Chapter 13 documents, none of which crosses the ABI itself. `Args` decodes the raw `argc/argv` into `.name()/.rest()/.to_vec()`. And `declare_plugin!` is the macro that ties it together: it emits the `#[no_mangle] extern "C" fn znative_init`, builds the `'static PluginInfo`, checks the incoming `HostApi.abi_version`, and registers each `builtins/completions` entry — so the plugin author writes handlers and a declaration, and never an `extern "C" function`, a raw pointer, or a version check by hand.

Appendix J – The Build Decision

The one-glance version of Chapter 23: given a resolved tree, what `znative` does about compilation. The rows are checked top to bottom.

Tree has	build flag	Action
prebuilt <code>lib*.{dylib,so}</code> at root	(ignored)	use the library as-is; no compile
<code>Cargo.toml</code> (<code>cdylib</code>), no prebuilt lib	<code>true</code> (default with <code>Cargo.toml</code>)	<code>cargo build --release</code> , hoist the lib to the root, then store-copy error : “no prebuilt cdylib and build is disabled”
<code>Cargo.toml</code> , no prebuilt lib	<code>false</code>	error : nothing to load and nothing to build
neither a library nor a <code>Cargo.toml</code>	—	

Facts that hold across every native build:

- **Once, at install** — the build runs in `add/load`'s install branch or in `update`, never on a plain `load`. Every subsequent load is a `dlopen`, not a compile.
- **--release, not debug** — the optimizing compile is paid a single time at install and repaid on every load; a plugin is kept, not thrown away, so it is built to be fast to run rather than fast to build.
- **The prerequisite is a Rust toolchain on the installing host**, and only for the native-from-source case; prebuilt-library plugins and every script plugin need none.
- **Compile chatter goes to `~/.cache/zshrs/`**, per doctrine — the terminal stays clean on a first install; only a failure surfaces, as `znative: <reason>` on stderr.

Appendix K — Cross-Architecture Rules

The one-glance version of Chapter 24. zshrs runs on macOS `aarch64`, Linux `x86_64`, and Linux `aarch64`; a `cdylib` is per-target; the store is per-machine.

Question	Answer
What does a plugin <i>ship</i> ?	Rust source + <code>Cargo.toml</code> — architecture-neutral
Where is the <code>cdylib</code> produced?	on the installing host, by its toolchain, for its target
Is a store shared across machines?	no — one store per user per machine under that host's <code>\$ZSHRS_HOME/pkg/</code>
How is a foreign binary kept out?	by the <code>lib*.dylib</code> (macOS) / <code>lib*.so</code> (Linux) shape — the wrong-OS binary is simply not found, and detection falls through to a source build
The one unrescuable case	a repo shipping only a prebuilt single-arch library and no <code>Cargo.toml</code> — loads only on the matching arch, clean error elsewhere
The author's rule	ship source, not a binary — source installs on every architecture because every host builds it for itself

The design carries no per-target binary registry on purpose: local build makes every host self-sufficient, including an offline one, with no trusted binary cache anywhere in the loop.

Appendix L – Trust and Integrity at a Glance

The one-glance version of Chapter 29. Three mechanisms are easy to conflate; they answer three different questions, and only two of the three exist.

Mechanism	Answers	Does not answer
Choosing the source (owner/repo , URL, path)	do I trust this plugin's author enough to run their code?	— this is <i>your</i> judgement, made at install, same as sourcing any .zsh whether <i>this</i> store still holds them
sha256-<hex> integrity hash (Chapter 6)	do two stores hold the same bytes for the same name@version ? (comparison, computed once at install)	— nothing recomputes it, and there is no verify verb (Chapters 68, 73) — or who wrote it
ABI_VERSION check (Chapter 15)	will this plugin avoid corrupting the process by layout mismatch? (safety)	whether the plugin is benign — a hostile plugin compiles against the right version too

There is **no** signing, sandbox, or allow-list, by doctrine: **znative** does not interpose a trust ceremony between a user and a plugin they asked for, and it declines to imply an authenticity guarantee it cannot deliver without becoming the registry it deliberately is not. Loading native code is neither more nor less trusted than sourcing a script — in both cases the trusted thing is the source you named.

Appendix M – Reconciliation Matrix

The one-glance version of Chapter 26. The index (`installed.toml`) is the authority; the store is its content cache. Every disagreement resolves the same direction and has a plain repair.

State	What it means	Repair
index row, no <code>store/</code> dir	content deleted out from under a live record	<code>znative update</code> (reinstall from recorded source) or <code>znative remove</code> (drop the row)
<code>store/</code> dir, no index row	an orphan — usually a prior version an <code>update</code> left behind	<code>znative gc</code> (reclaims only unpinned dirs; can never remove a live one)
interrupted install	died before the record was written	leaves at most an unreferenced dir → <code>gc</code> ; never a live row on a partial tree
store bytes altered after install	tampering or corruption	not detected — the recorded <code>sha256-<hex></code> is never recompared (Chapter 68); <code>znative update</code> reinstalls from the source if you suspect it

There is no `fsck` verb and no repair database, because the index already *is* the database and `update/remove/gc` already *are* the repair tools — the reason the command surface stays as small as Chapter 4 describes.

Appendix N — HostApi Field Reference

The complete table, in declaration order. Layout is frozen by `ABI_VERSION`; the order below is the memory order, and a plugin compiled against a different order reads the wrong fields. Definitions at `znative/src/lib.rs:113-189`; host implementations at `zshrs/src/extensions/plugin_host.rs`.

#	Field	ABI v	Type
0	<code>abi_version</code>	1	<code>u32</code>
1	<code>ctx</code>	1	<code>*mut c_void</code> — reserved, host sets null
2	<code>register_builtin</code>	1	<code>fn(host, *const c_char, BuiltinFn) -> c_int</code>
3	<code>print</code>	1	<code>fn(host, *const c_char)</code>
4	<code>eval</code>	1	<code>fn(host, *const c_char) -> c_int</code>
5	<code>getvar</code>	1	<code>fn(host, *const c_char) -> *mut c_char</code>
6	<code>setvar</code>	1	<code>fn(host, *const c_char, *const c_char) -> c_int</code>
7	<code>free_cstring</code>	1	<code>fn(host, *mut c_char)</code>
8	<code>register_completion</code>	2	<code>fn(host, *const c_char, *const c_char) -> c_int</code>
9	<code>getfunction</code>	3	<code>fn(host, *const c_char) -> *mut c_char</code>
10	<code>addfunction</code>	3	<code>fn(host, *const c_char, *const c_char) -> c_int</code>
11	<code>register_compfn</code>	4	<code>fn(host, *const c_char, CompFn) -> c_int</code>

#	Field	ABI v	Type
12	<code>comp_dispatch</code>	4	<code>fn(host, *const c_char, usize, *const *const c_char) -> c_int</code>
13	<code>empty_command_hash</code>	4	<code>fn(host)</code>

Semantics, ownership, and the `Host` wrapper for each:

Field	Returns	Ownership	<code>Host</code> method
<code>register_builtin</code>	0 = ok	<code>name</code> copied by host	<code>register_builtin(&str, BuiltinFn) -> bool</code>
<code>print</code>	—	<code>text</code> borrowed for the call	<code>print(&str)</code> , and <code>add_match(&str)</code>
<code>eval</code>	exit status	<code>code</code> borrowed for the call	<code>eval(&str) -> i32</code>
<code>getvar</code>	string or null	transferred to plugin ; free with <code>free_cstring</code>	<code>getvar(&str) -> Option<String></code>
<code>setvar</code>	0 = ok	both borrowed	<code>setvar(&str, &str) -> bool</code>
<code>free_cstring</code>	—	consumes the pointer	— (called internally by the wrappers)
<code>register_completion</code>	0 = ok	both borrowed; staged, wired at first completion	<code>install_completion(&str, &str) -> bool</code>
<code>getfunction</code>	deparsed body or null	transferred to plugin	<code>getfunction(&str) -> Option<String></code>
<code>addfunction</code>	0 = ok, non-zero = parse failure	both borrowed	<code>addfunction(&str, &str) -> bool</code>
<code>register_compfn</code>	0 = ok	<code>name</code> copied by host	<code>register_compfn(&str, CompFn) -> bool</code>
<code>comp_dispatch</code>	the function's status	<code>argv</code> borrowed for the call	<code>comp_dispatch(&str, &[&str]) -> i32</code>
<code>empty_command_hash</code>	—	—	<code>empty_command_hash()</code>

Two counts that differ and are easy to conflate: the table has **twelve callbacks**; `Host` exposes **thirteen methods**, because `add_match` is convenience over `print` with no field of its own.

Fleet usage, measured across all eight shipped plugins (Chapter 52): `print` 7 repositories, `getvar` 6, `eval` 3, `add_match` 3; `register_builtin` and `install_completion` used indirectly by every plugin

through `declare_plugin!`; `setvar`, `getfunction`, `addfunction`, `register_compfn`, `comp_dispatch`, and `empty_command_hash` used by none.

Appendix 0 – The Fleet at a Glance

Eight sibling repositories, each installed with `znative load MenkeTechnologies/<repo>`.

Repository	LOC	Tests	Version	Extra deps	Builtins	Completions	Uses <code>eval</code>
<code>zshrs-git-fuzzy</code>	868	6	0.1.1	—	2	—	no
<code>zshrs-fasd</code>	783	7	0.1.0	<code>regex</code>	10	—	yes
<code>zshrs-forgit</code>	599	2	0.1.1	—	9	—	no
<code>zshrs-zsh-z</code>	491	7	0.1.1	—	1	1	yes
<code>zshrs-revolver</code>	403	3	0.2.1	<code>libc</code>	1	—	no
<code>zshrs-git-repos</code>	315	2	0.1.1	—	1	—	yes
<code>zshrs-reveal</code>	305	7	0.1.0	—	1	—	no
<code>zshrs-kubectl-completion</code>	141	5	0.1.1	—	0	1	no
total	3,905	39			25	2	3

Shared properties across all eight:

- `Cargo.toml` declares `znative = "0.12"`.
- `Cargo.lock` is committed and resolves `znative` to `0.12.22`.
- `.github/workflows/ci.yml` is **byte-identical** in all eight, runs `fmt`, `clippy`, `doc`, `build`, and `test`, and passes `--locked`.
- The `declare_plugin!` version literal matches the crate version — true in all eight as of the most recent commit in each, after a fixing pass across six of them.
- No plugin contains a `catch_unwind`.
- No plugin contains an ABI-level test; all 39 tests are unit tests of the plugin's own logic.

The one technique each is the only plugin to use:

Repository	The move nobody else makes
<code>zshrs-git-fuzzy</code>	<code>dladdr</code> itself, writes a 0755 shim that re- <code>zmodloads</code> its own dylib; hand-rolled HTTP POST over <code>TcpStream</code> to fzf's <code>--listen</code> port; a 500 ms watcher comparing an FNV-1a hash of <code>git status --porcelain</code>
<code>zshrs-fasd</code>	the only plugin with a transitive dependency tree, the only one that evals user-supplied text (<code>-e</code>), and the only one depending on a specific zsh expansion flag (<code>\${(z)...</code>) holding up inside zshrs
<code>zshrs-forgit</code>	nine builtins in one dylib and zero calls to <code>host.eval</code> — pure subprocess orchestration
<code>zshrs-zsh-z</code>	atomic tmp+rename datafile writes, and <code>fmt_rank</code> printing whole numbers as integers so the file stays byte-compatible with the shell original's <code>\$((...))</code> output
<code>zshrs-revolver</code>	resident <code>static</code> state across separate builtin calls; zero subprocesses; <code>TIOCGWINSZ</code> instead of <code>tput cols</code> ; writes to <code>/dev/tty</code> with a stderr fallback; a 55-entry frame table
<code>zshrs-git-repos</code>	a bounded 16-way worker pool over an <code>Arc<Mutex<...>></code> queue, inside a builtin
<code>zshrs-reveal</code>	detached fire-and-forget <code>.spawn()</code> , and runtime OS dispatch that detects WSL by reading <code>/proc/version</code> for <code>microsoft</code>
<code>zshrs-kubectl-completion</code>	zero user-facing builtins — a <code>completions:</code> block and nothing else — delegating to cobra's <code>kubectl _complete</code>

Appendix P – Reproducing the Measurements

Every count in the chapters added to this edition came from a command. Paths assume the `zshrs` checkout and the eight plugin repositories are siblings.

The SDK crate.

```
$ wc -l zshrs/znative/Cargo.toml zshrs/znative/README.md zshrs/znative/src/lib.rs
  21 zshrs/znative/Cargo.toml
 134 zshrs/znative/README.md
 548 zshrs/znative/src/lib.rs
 703 total
```

Zero dependencies is the empty `[dependencies]` table at the bottom of the manifest. One exported symbol — note that the `grep` reports two hits, because the attribute is also named inside the `declare_plugin!` doc comment (`lib.rs:431`, prose describing what the macro expands to); the real one is inside the macro expansion at `lib.rs:469`:

```
$ grep -n '#\[no_mangle\]' zshrs/znative/src/lib.rs
431:/// native completions it provides. Expands to the '#\[no_mangle] extern "C"
469:      #\[no_mangle]
```

The host.

```
$ wc -l zshrs/src/extensions/plugin_host.rs
 699
$ grep -c 'extern "C" fn host_' zshrs/src/extensions/plugin_host.rs
 12
$ grep -c 'OnceLock' zshrs/src/extensions/plugin_host.rs
 11
```

Eleven lines mention `OnceLock`: the `use` at the top, the nine registry accessors (`plugin_host.rs:60-125`), and the leaked host table's `static API` (`plugin_host.rs:340`). The registries are the nine of Chapter 48's table.

No tests, no panic guard.

```
$ grep -c 'cfg(test)' zshrs/znative/src/lib.rs zshrs/src/extensions/plugin_host.rs
zshrs/znative/src/lib.rs:0
zshrs/src/extensions/plugin_host.rs:0

$ grep -rn 'catch_unwind' zshrs/src/extensions/ zshrs/znative/
zshrs/src/extensions/worker.rs:23:...
zshrs/src/extensions/worker.rs:136:...
zshrs/src/extensions/worker.rs:137:...
```

Three hits, all in the worker pool, none on the plugin path.

The example plugins.

```
$ ls -d zshrs/examples/plugin-*/ | wc -l
11
$ find zshrs/examples/plugin-*/ -name '*.rs' | xargs wc -l | tail -1
3871 total
```

The fleet.

```
$ for r in zshrs-git-fuzzy zshrs-fasd zshrs-forgit zshrs-zsh-z \
  zshrs-revolver zshrs-git-repos zshrs-reveal zshrs-kubectrl-completion; do
  printf '%-26s %5s %3s\n' "$r" \
    "$(find $r/src -name '*.rs' | xargs wc -l | tail -1 | awk '{print $1}')" \
    "$(grep -c '#\[test\]' $r/src/lib.rs)"
done
```

Builtin counts come from the `declare_plugin!` invocation:

```
$ perl -0777 -ne 'if (/builtins:\s*\{(.*)\}/s) { my @m = ($1 =~ /\s*=>/g);
  print scalar(@m) } else { print 0 }' <repo>/src/lib.rs
```

CI identity:

```
$ md5 -q zshrs-*/.github/workflows/ci.yml | sort | uniq -c
8 fc36e8b0ac1ed33275a5a4fbd2d3c256
```

(Use `md5sum` on Linux; the point is one hash for eight files.)

Lockfile resolution:

```
$ for r in zshrs-*/; do grep -A1 'name = "znative"' "$r/Cargo.lock" | grep version; done
```

Callback usage across the fleet — the source of Chapter 52's table:

```
$ for f in print getvar eval setvar getfunction addfunction \
  register_compfn comp_dispatch empty_command_hash add_match; do
  printf '%-20s %5s\n' "$f" \
    "$(grep -rl "\.$f(" zshrs-*/src/*.rs 2>/dev/null | wc -l)"
done
```

Direct writes that bypass `host.print`:

```
$ grep -rnoE 'println!|eprintln!|print!|eprint!' zshrs-git-fuzzy/src/lib.rs | wc -l
7
$ grep -rnoE 'println!|eprintln!|print!|eprint!' zshrs-revolver/src/lib.rs | wc -l
4
```

The other six repositories return zero.

The ABI's history. The four version bumps, by commit:

```
$ cd zshrs 66 for c in 4a181954a8 b4388c5e08 f9804b895c 01516458ec; do
  git log --format='%h %ad %s' --date=short -1 $c
done
4a181954a8 2026-07-17 plugins: host native (Rust) plugins via stable C-ABI dylibs
  ~ (zmodload -R)
b4388c5e08 2026-07-17 plugins: native (Rust) completions via declare_plugin! completions:
  ~ + zmodload -R
f9804b895c 2026-07-17 plugin ABI v3: getfunction / addfunction (deparse-as-a-service)
01516458ec 2026-07-18 plugin command names
```

and the SDK crate rename:

```
$ git log --format='%h %ad %s' --date=short -1 fc38701af7
fc38701af7 2026-07-17 rename plugin SDK crate: zshrs-plugin -> znative (matches the
  ~ znative package manager)
```

The four version numbers of Chapter 51. The workspace version is `zshrs/Cargo.toml`'s `[workspace.package] version`; the path-dependency pin is in `[workspace.dependencies]`; the plugin-author requirement is in `znative/README.md` and in the crate's own doc comment (`lib.rs:38`); the resolved version is in each plugin's `Cargo.lock`.

A note on this book's own build. `znative/scripts/book_pdf.sh` derives the cover's edition string from `SRC_ROOT/Cargo.toml` with

```
awk -F'"' '/^version = "[0-9]/ {print $2; exit}'
```

That pattern cannot read `znative`'s version, because `znative/Cargo.toml` declares `version.workspace = true` rather than a literal — the version lives in the workspace root's `[workspace.package]` table. Pointing `SRC_ROOT` at the vendored crate therefore yields an empty version string rather than the right one. This edition is consequently built by passing the resolved workspace version explicitly:

```
$ EDITION=v0.12.28 bash znative/scripts/book_pdf.sh
```

which is what makes the cover agree with the book's own claim, in two places, that the edition string tracks the `znative` crate version.

What is not measured anywhere in this book. No timing numbers, no benchmark results, no throughput comparisons between the native and script paths. The arguments about cost in Chapter 28 and Chapter 20 are structural — a `dlopen` of an mmap'd dylib versus a parse of shell text, a thread versus a fork — and the book states them as structural rather than dressing them in numbers that were not taken. Where a number appears, one of the commands above produced it.

Colophon

This book is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

The three architecture figures — the install-and-load flow, the content-addressed store's layout on disk, and the ABI boundary between host and plugin — are drawn as SVG in `docs/diagrams/` and converted to vector PDF by the PDF build, so they stay sharp at any zoom and carry the same palette as the rest of the book rather than being screenshots of something else.

Every line number, count and transcript in these pages was read from the `zshrs` checkout at the commit named in the front matter by running the command that produces it, and Appendix P collects those commands in one place. Where the tool deliberately omits something other package managers have — a lockfile, a per-project manifest, a global registry — the book says so and says why rather than treating the omission as an oversight.

Reproducing the Book's Claims

The book is written to be checkable, and Appendix P is the full list. These are the commands behind its central claims. Run them from the root of the `zshrs` checkout; the fleet commands additionally assume the plugin repositories sit beside it.

The revision every `lib.rs:NNN` and `plugin_host.rs:NNN` citation was read at:

```
git rev-parse HEAD
```

The two sides of the boundary, measured. The SDK crate is deliberately small and has an empty `[dependencies]` table; the host is the other half of the same table:

```
wc -l znative/Cargo.toml znative/README.md znative/src/lib.rs
wc -l src/extensions/plugin_host.rs
```

The exported symbol, and the callbacks the host publishes across it. The `no_mangle` grep reports two matches: the attribute inside the macro expansion, and prose in the doc comment describing what the macro expands to.

```
grep -n '#\[no_mangle\]' znative/src/lib.rs
grep -c 'extern "C" fn host_' src/extensions/plugin_host.rs
```

The claim that nothing on either side of the boundary catches a panic, and that neither file carries a test module:

```
grep -rn 'catch_unwind' src/extensions/ znative/
grep -c 'cfg(test)' znative/src/lib.rs src/extensions/plugin_host.rs
```

The example tree, whose size and staleness the audit chapters measure:

```
ls -ld examples/plugin-*/ | wc -l
find examples/plugin-*/ -name '*.rs' | xargs wc -l | tail -1
```

The fleet chapters need the eight plugin repositories checked out beside the shell; Appendix P prints the loops that produce their line counts, their builtin counts, their CI identity and the callbacks each one actually calls.

Also from MenkeTechnologies

znative is one piece of a stack that is documented the same way — from source, with the numbers cited to the tools that produce them. Companion volumes cover the shell this manager is built into, the editor that received a port of this same ABI, the bytecode machine underneath the language fleet, and the language ports built on it.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.